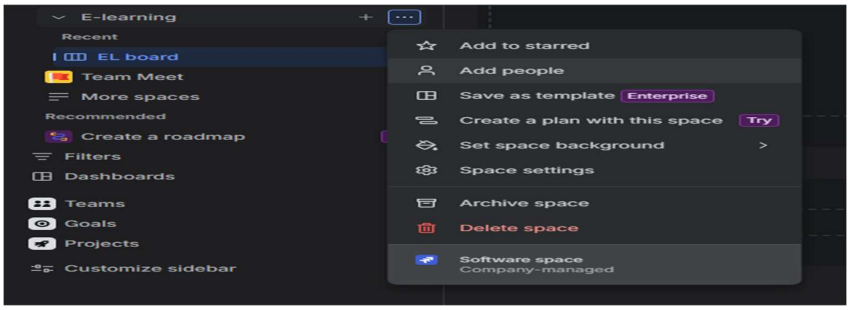
Practical No 2.
Team Creation & Role Assignment

A. Add Team Members

Step 1: Open E-learning → Click on the three dots (...).

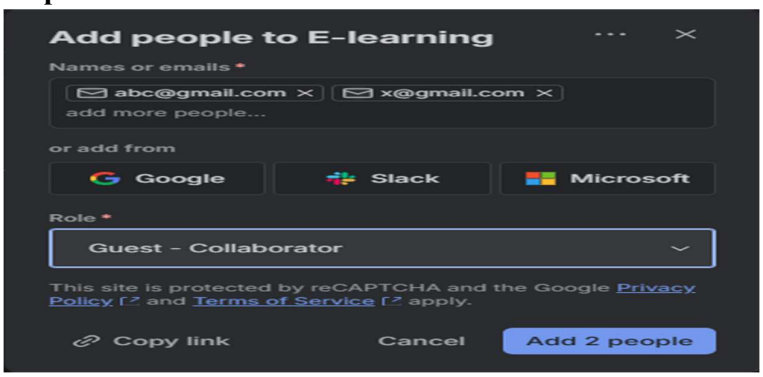
Step 2: Click on Add people.



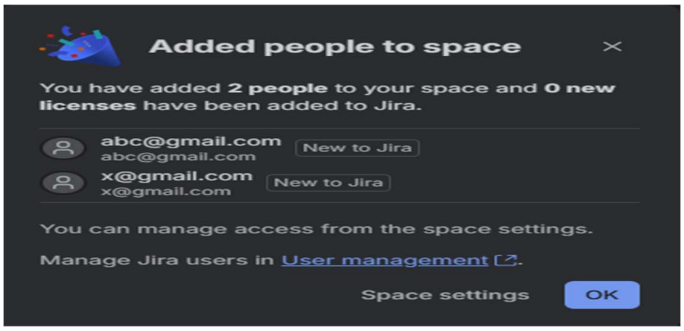
Step 3: Enter the name or email address.

Step 4: Select the required role.

Step 5: Click on Add.



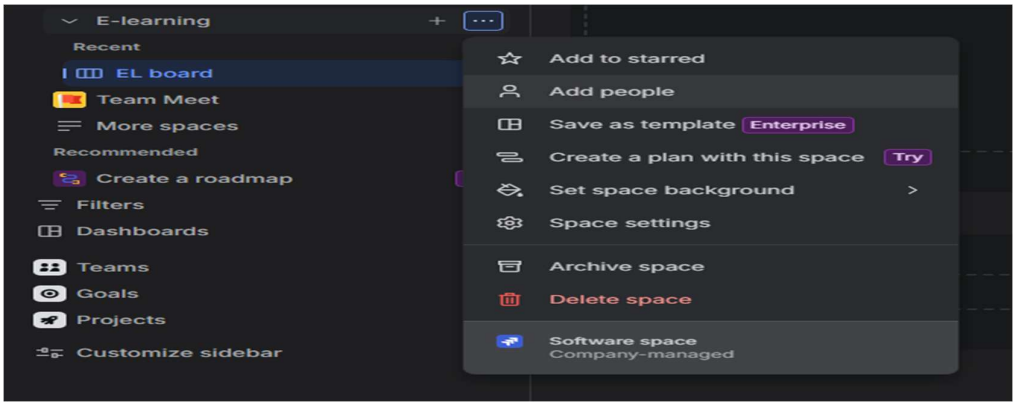
Step 6: Click on OK.



B. Assign Roles

Step 1: Open E-learning → Click on the three dots (...).

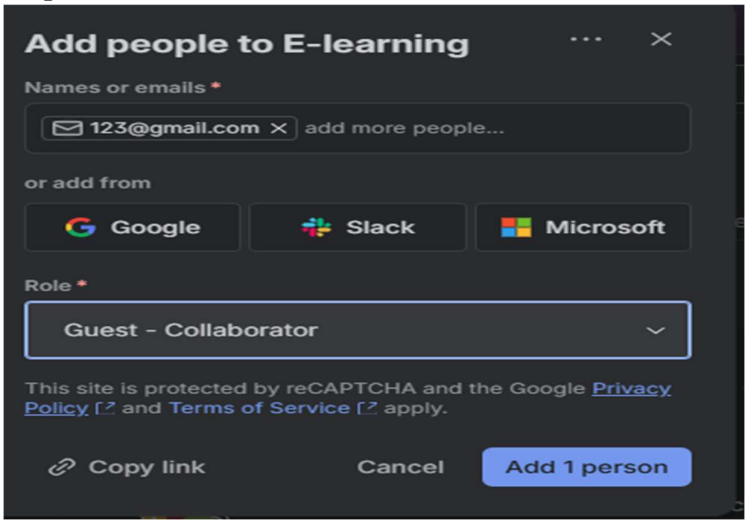
Step 2: Click on Add people.



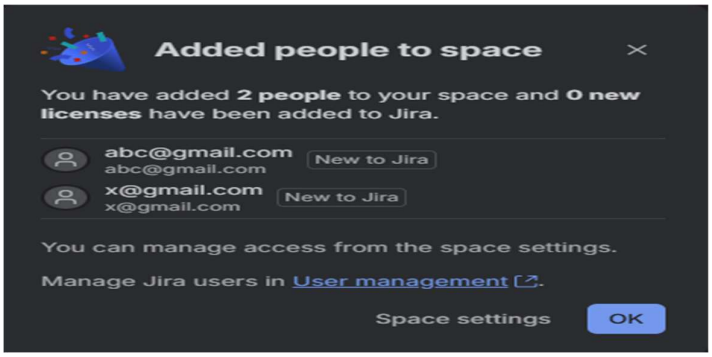
Step 3: Enter the name or email address.

Step 4: Select the required role.

Step 5: Click on Add.



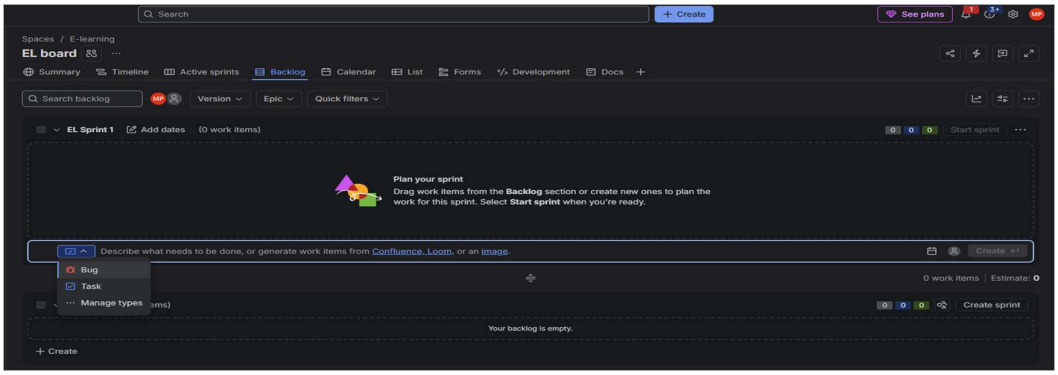
Step 5: Click on OK.



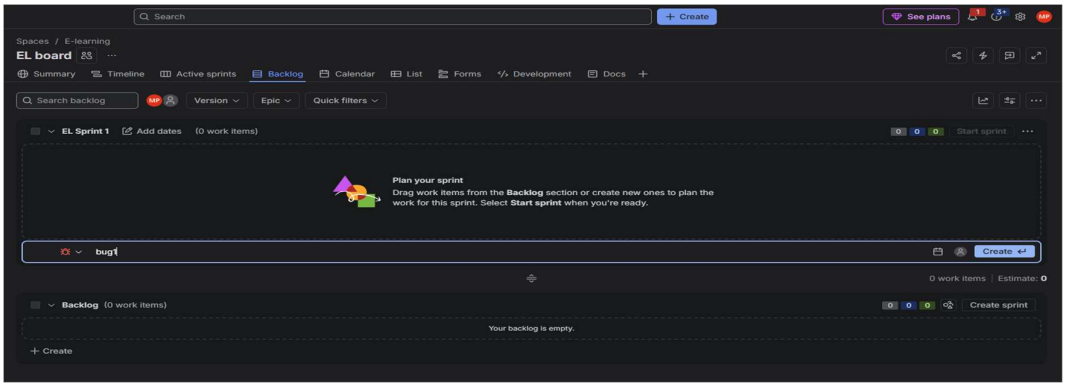
Practical No 3.
Issue Types & Configuration

A. Create and configure Issue Types (Epic, Story, Task, Bug)

Step 1: Click on Backlog. → Click on Create (+).

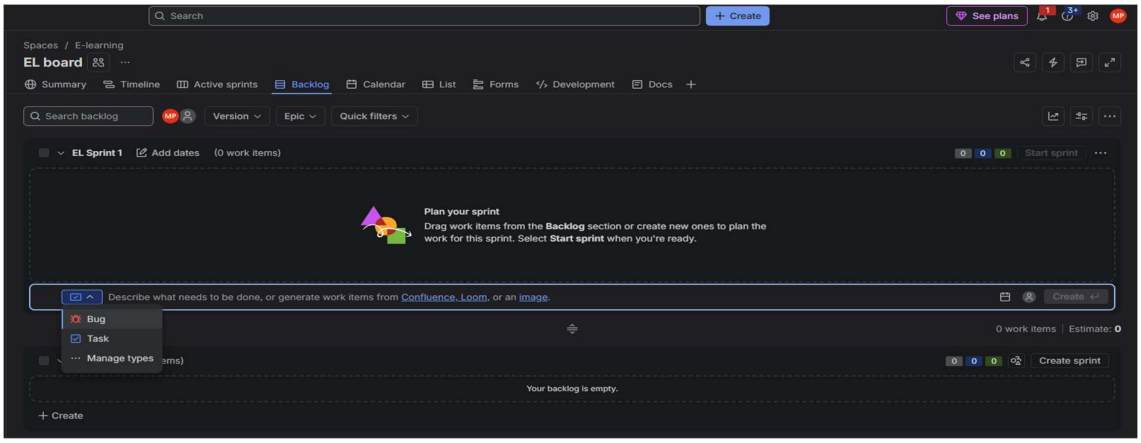


Step 2: Select Bug. → Enter the Bug Description.

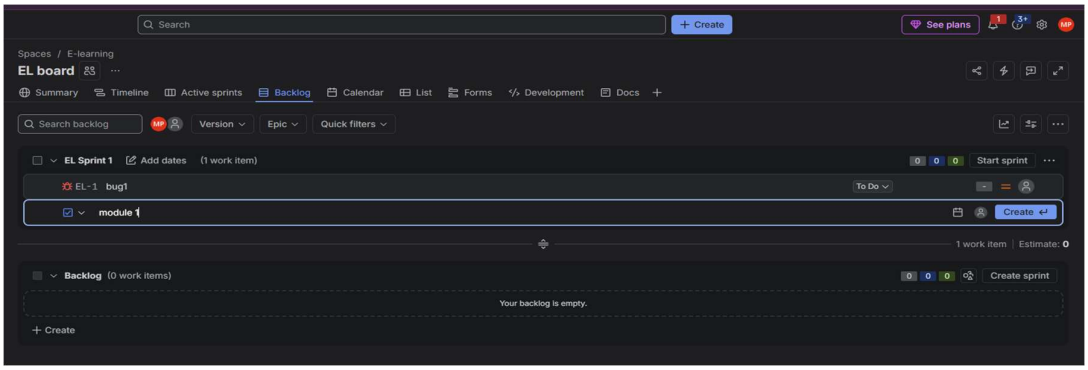


Step 3: Click on Create.

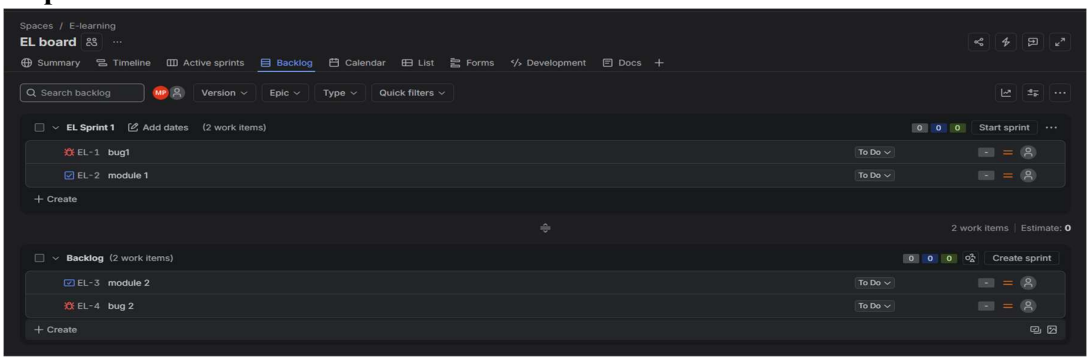
Step 4: Click on Create (+) again. → Select Task.



Step 5: Enter the Task Description.



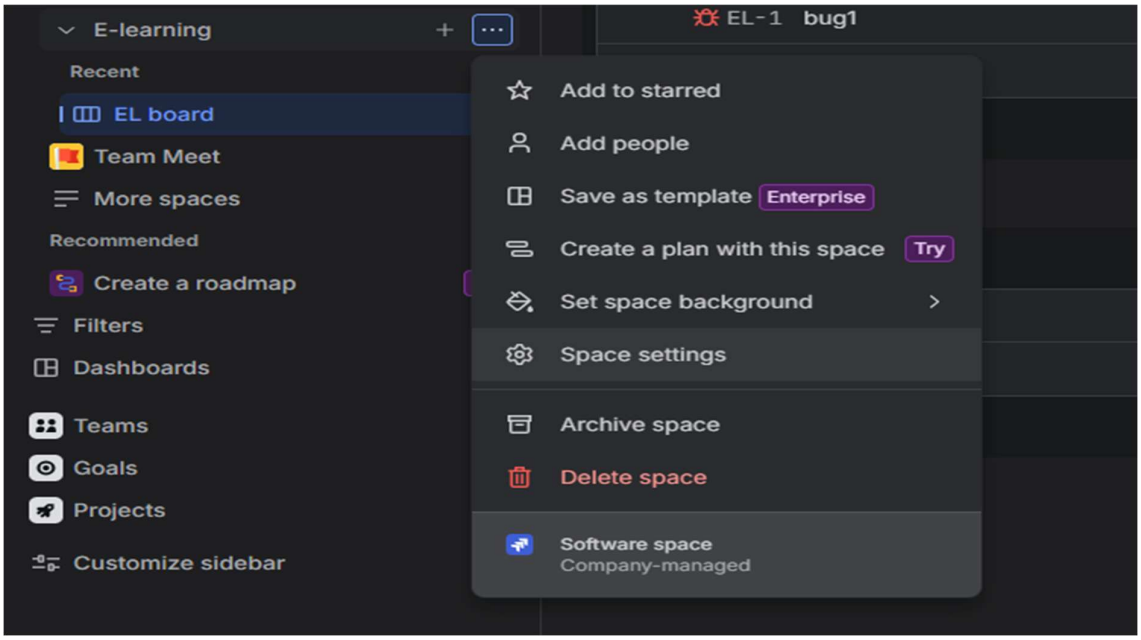
Step 6: Click on Create.



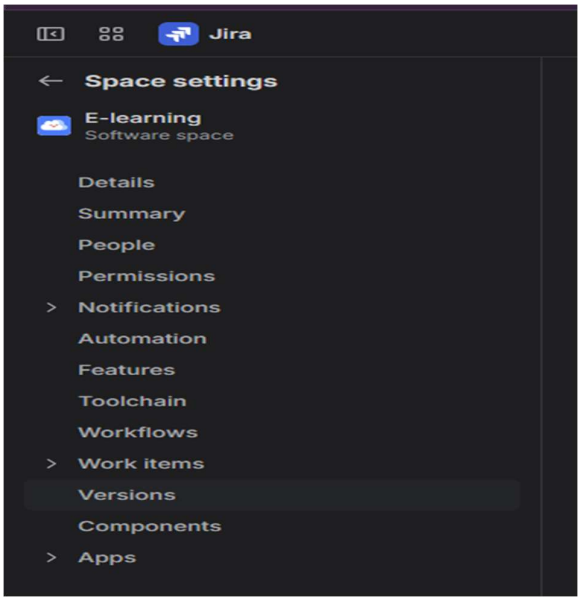
B. Create Components and Versions.

Step 1: Open E-learning → Click on the three dots (...).

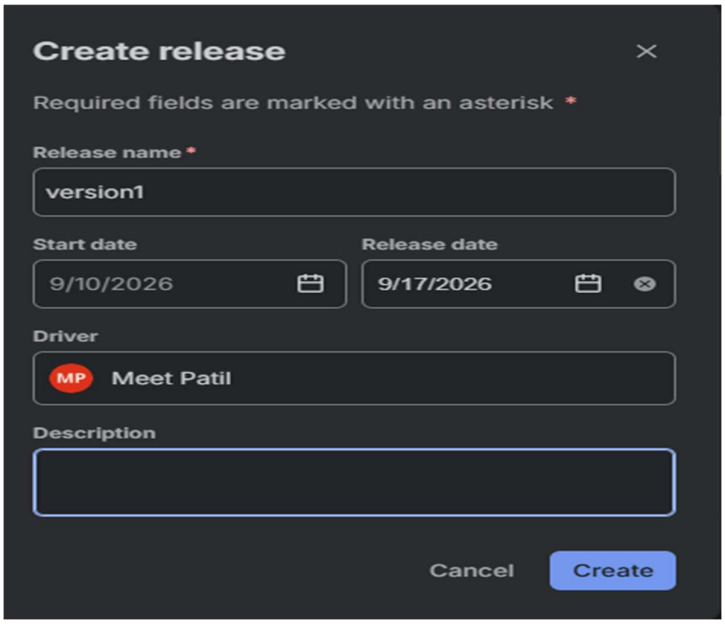
Step 2: Click on Space settings.



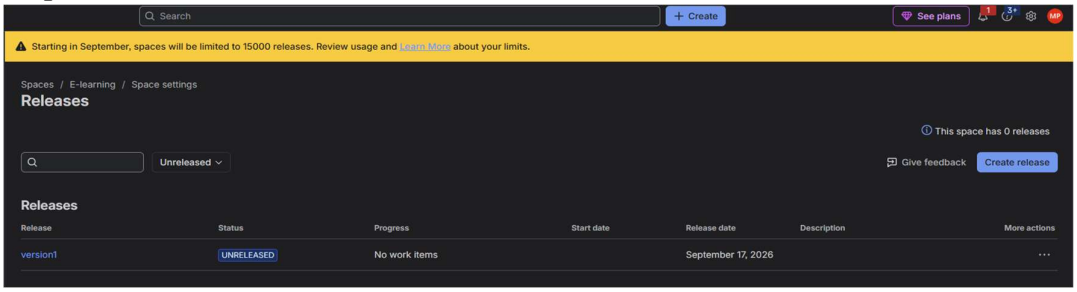
Step 3: Click on Versions.



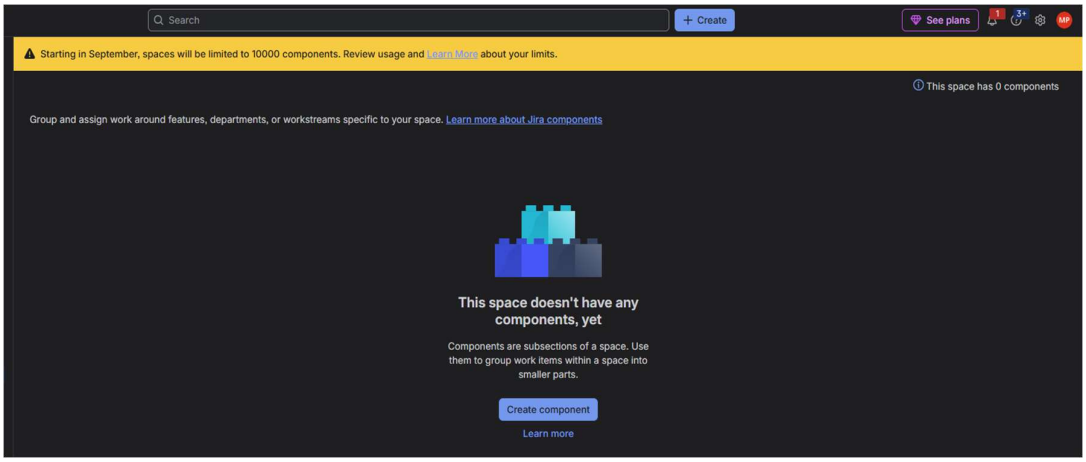
Step 4: Enter the Release Name, Start Date and Release Date.



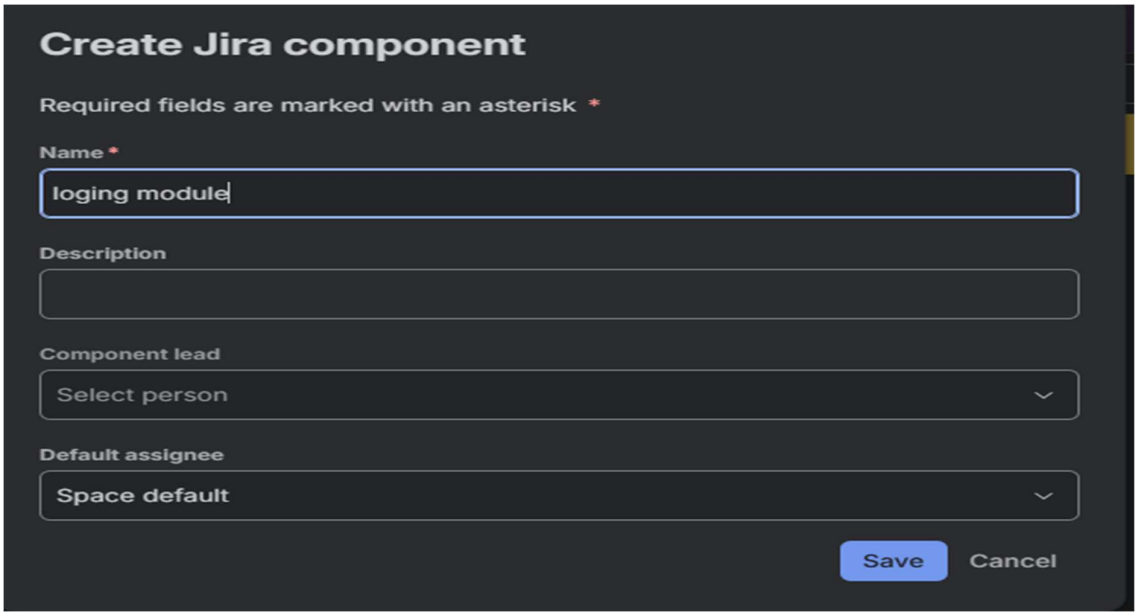
Step 5: Click on Create.



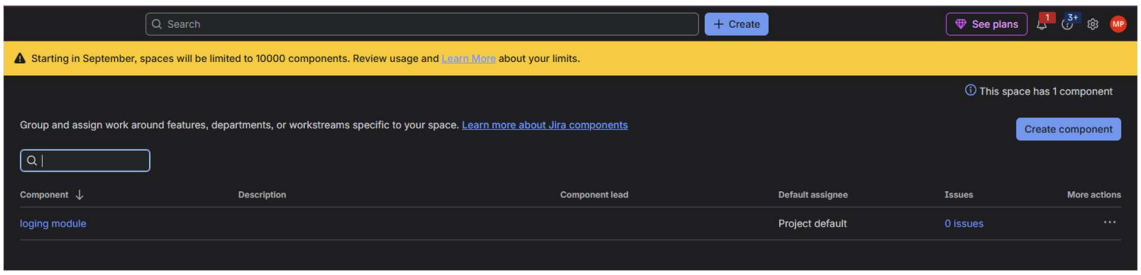
Step 6: Click on Components. → Click on Create component.



Step 7: Enter the Component Name.



Step 8: Click on Save.

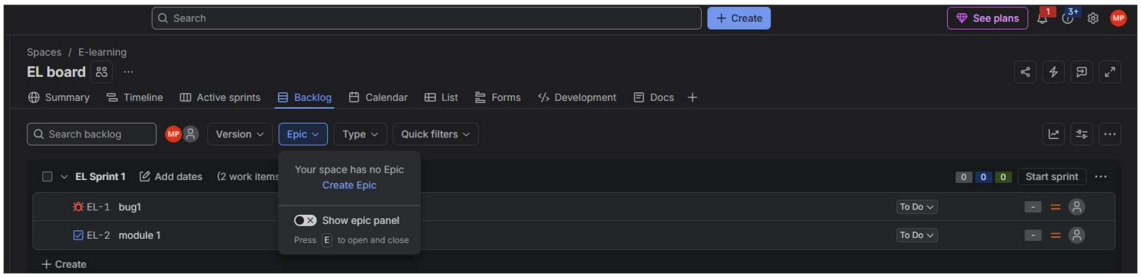


Practical No 4.
Issue Management

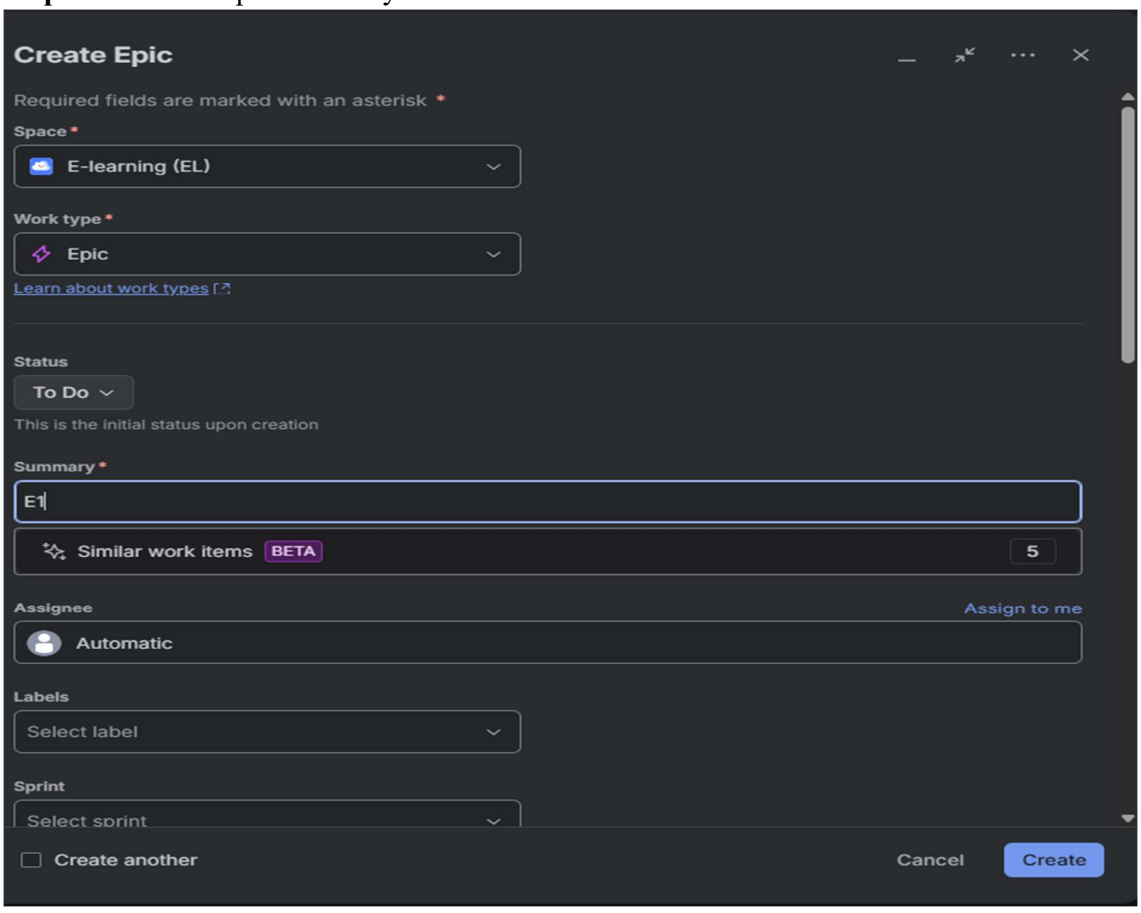
A. Create issues under an Epic.

Step 1: Open Backlog. → Click on Epic.

Step 2: Click on Create Epic.



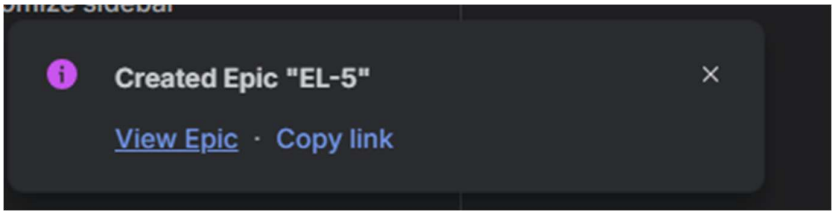
Step 3: Enter the Epic Summary.



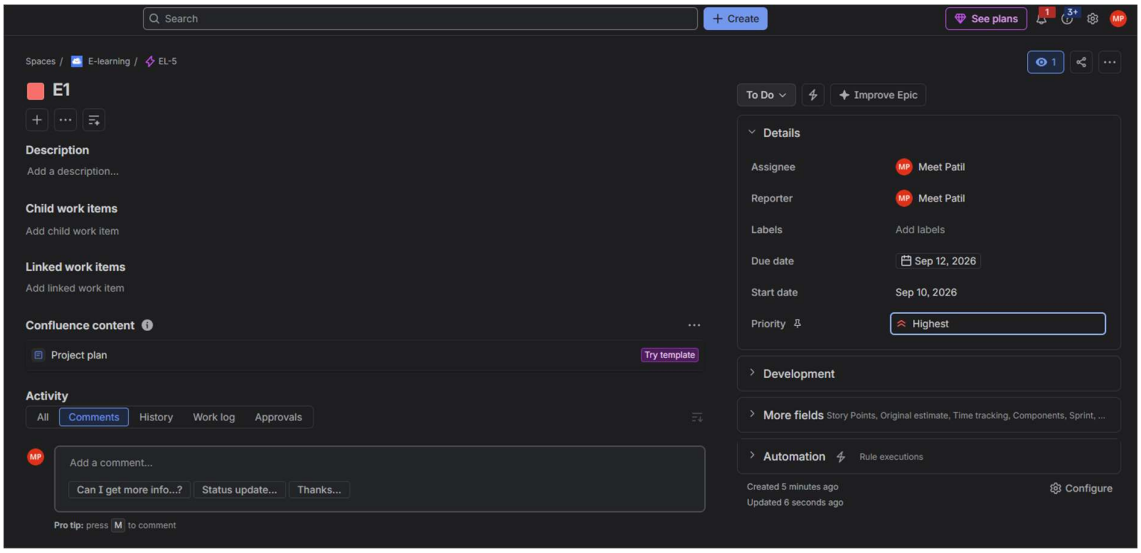
Step 4: Click on Create.

B. Assign issues to users and set Priority & Due Date.

Step 1: Click on View Epic.



Step 2: Enter the required Epic Information.



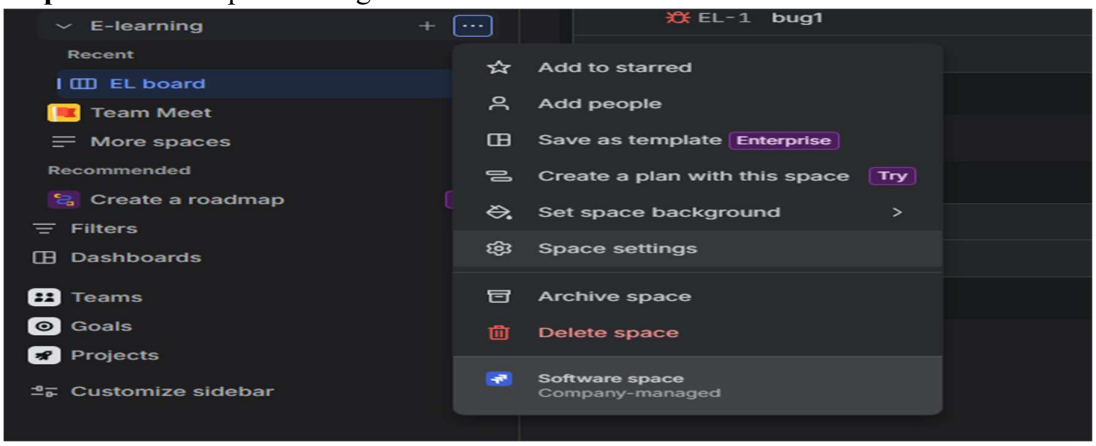
Step 3: Save the information.

Practical No. 5
Workflow Customization

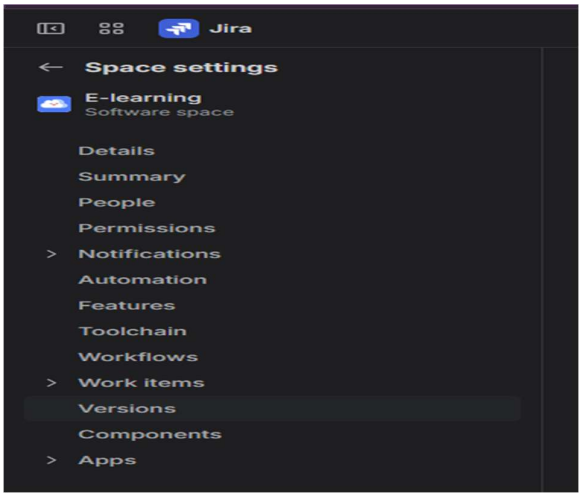
A. Design a custom workflow

Step 1: Go to E-learning. → Click on the three dots (...).

Step 1: Click on Space settings.

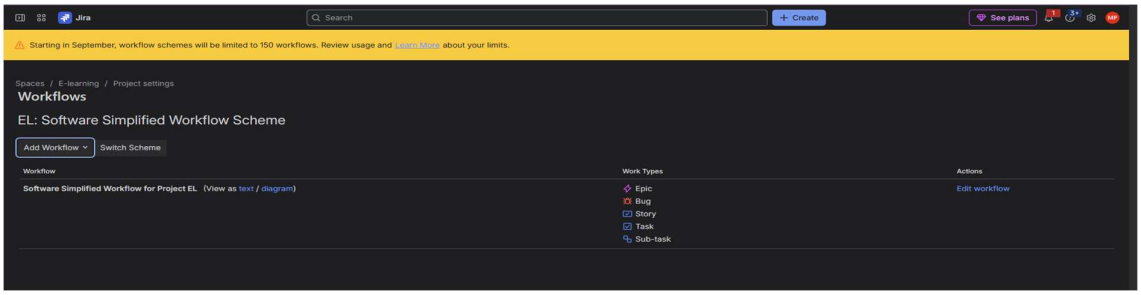


Step 2: Click on Workflows.



Step 3: Click on Add workflow.

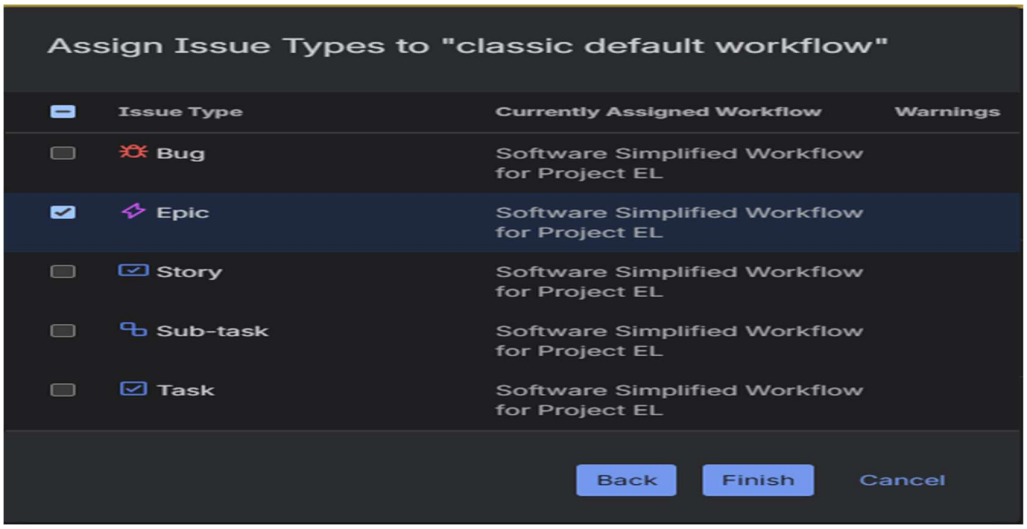
Step 4: Select Add to existing workflow.



Step 5: Select Classic Default Workflow. → Click on Next.



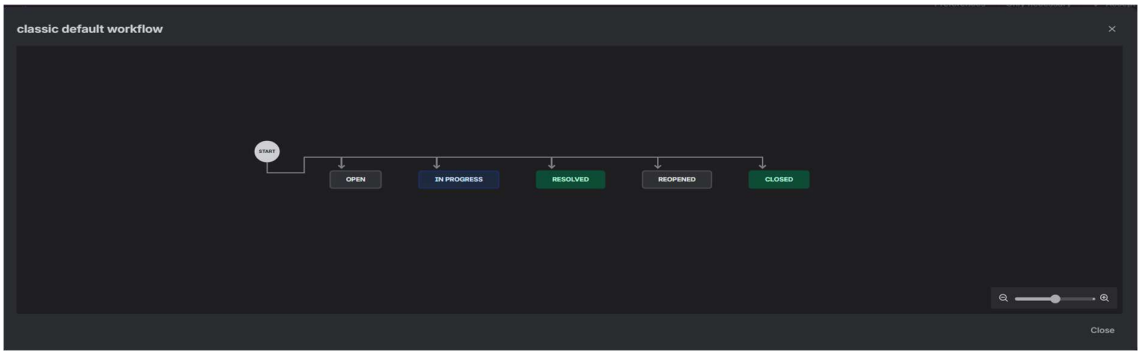
Step 6: Select the required Epic.



Step 7: Click on Next.

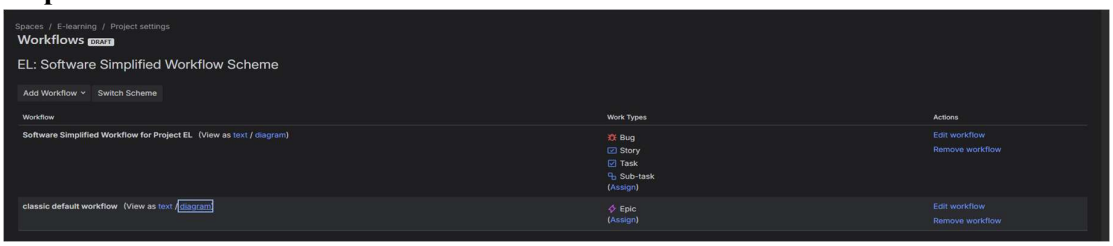


Step 8: Click on Diagram.

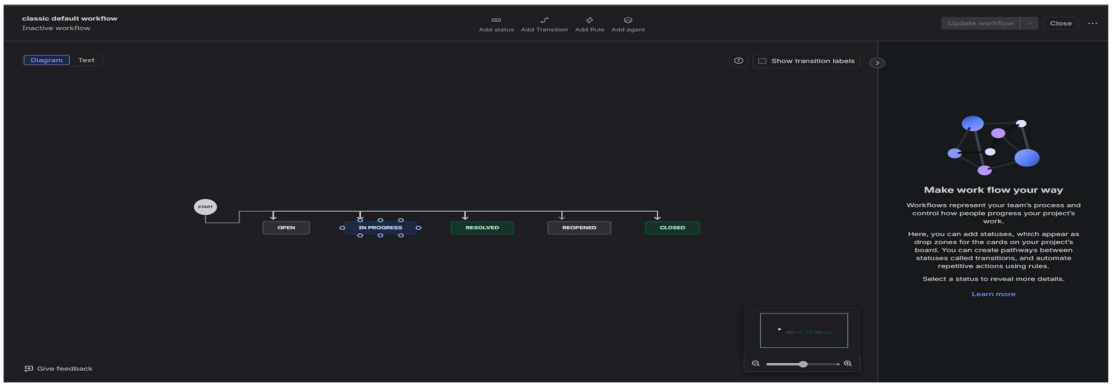


B. Add transitions and conditions.

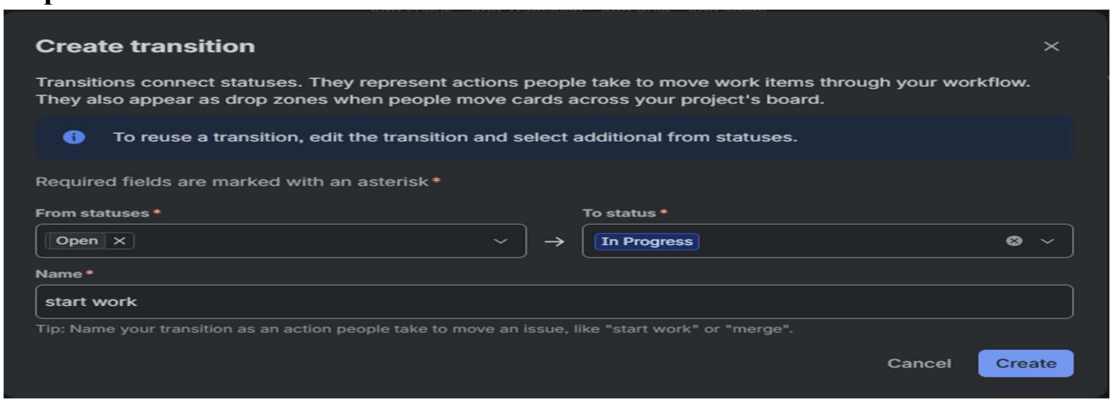
Step 1: Click on Edit workflow.



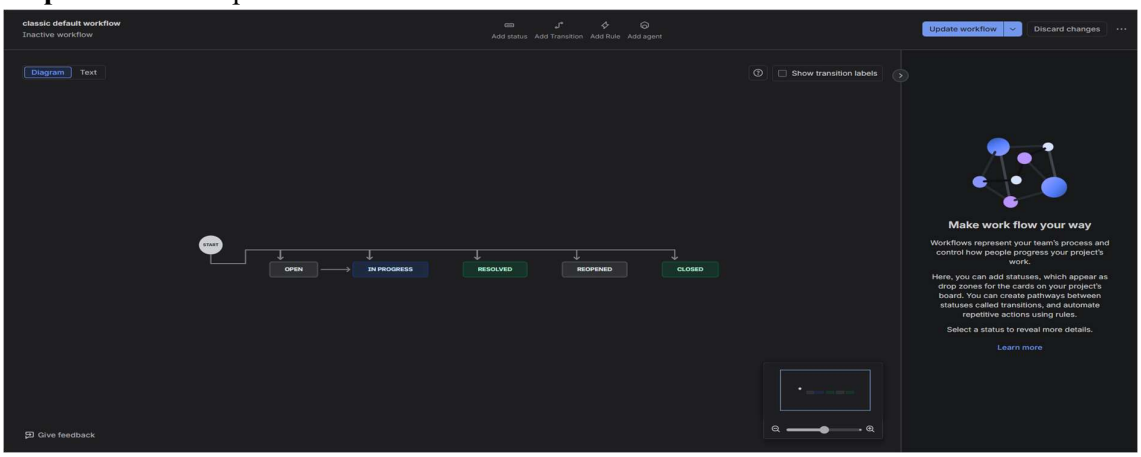
Step 2: Click on Add transition.



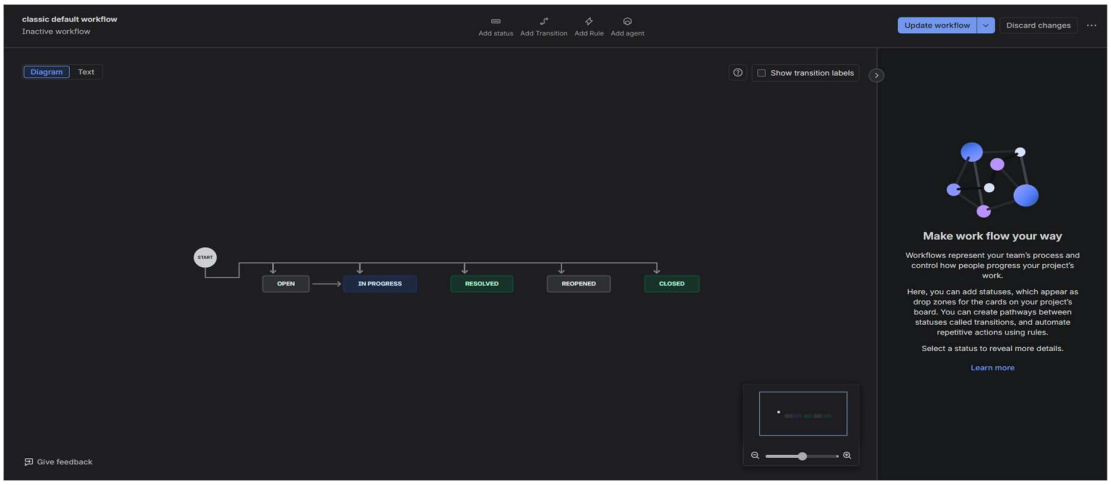
Step 3: Enter the Transition Name. → Click on Create.



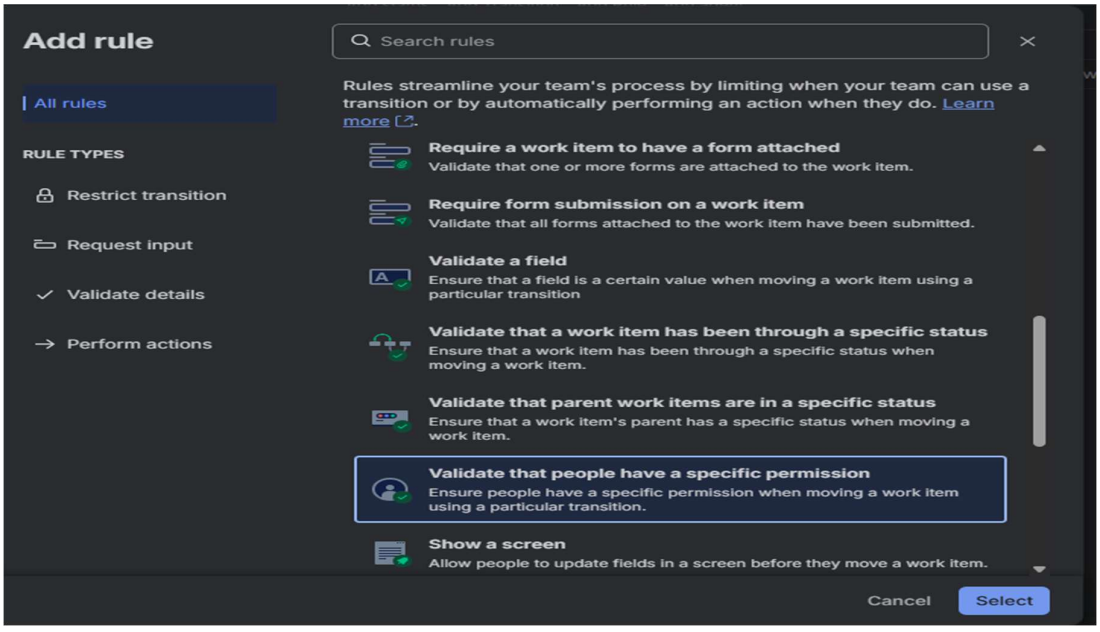
Step 4: Click on Update workflow.



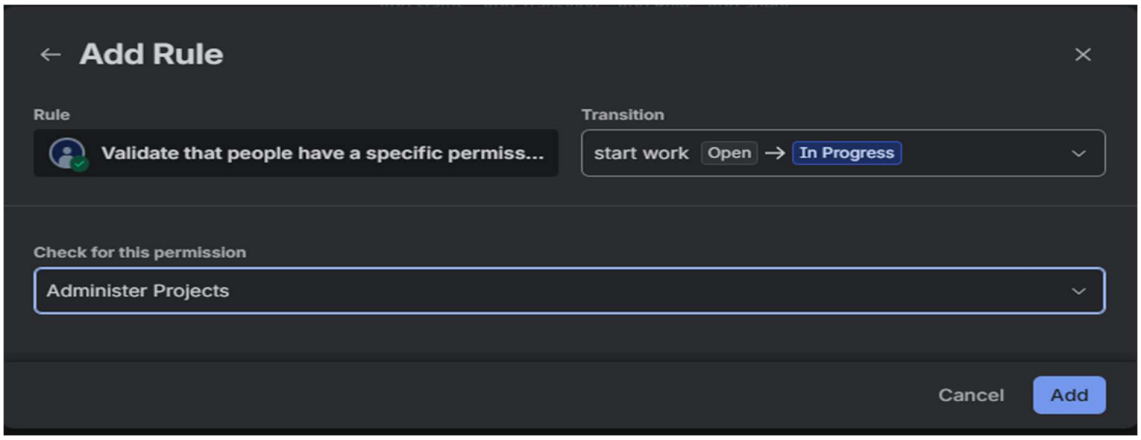
Step 5: Click on Add rule.



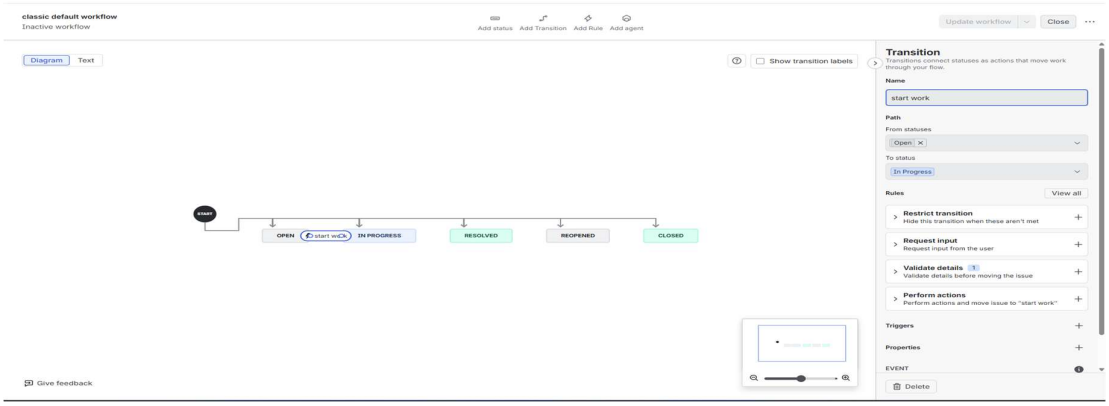
Step 6: Select Information. And Click on Select.



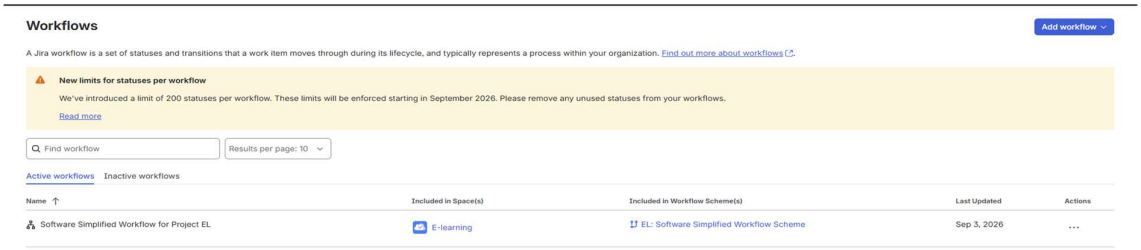
Step 7: Click on Add.



Step 8: Click on Update workflow.



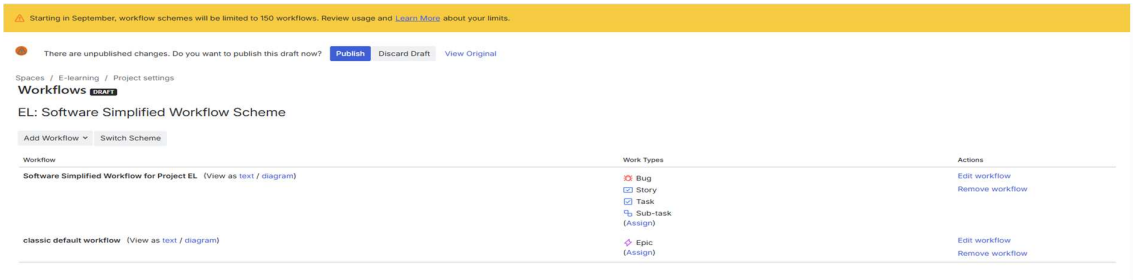
Step 9: Click on Close.



C. Publish and associate workflow with a project.

Step 1: Click on Publish.

Step 2: Select Associate.



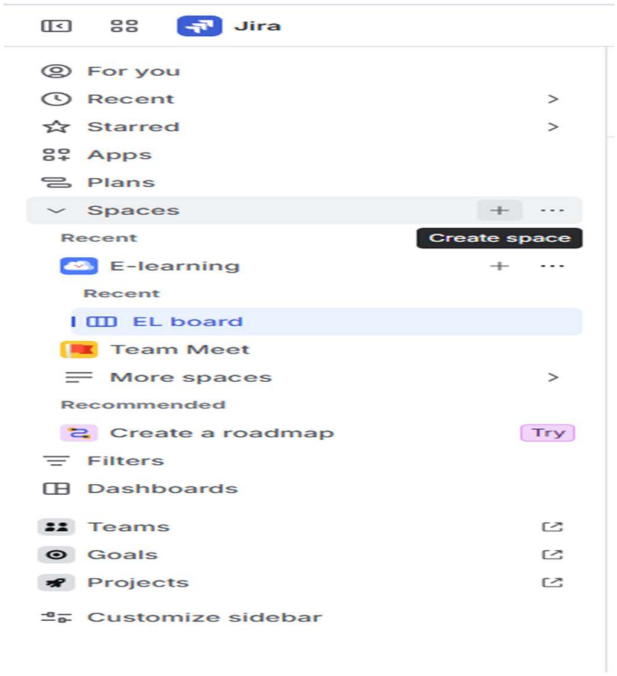
Step 3: Click on Update workflow.



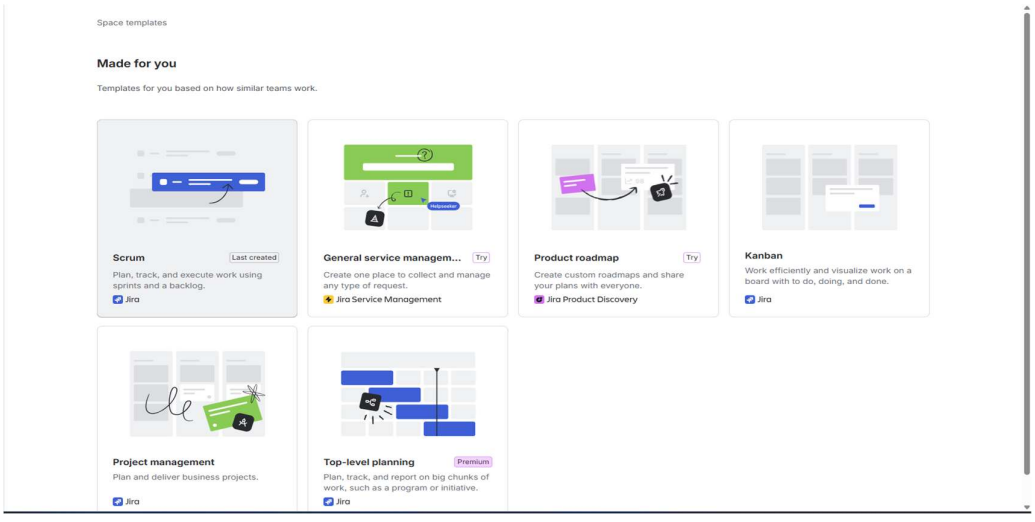
Practical No 6.
Scrum Board & Sprint Management

A. Create and configure Scrum Board.

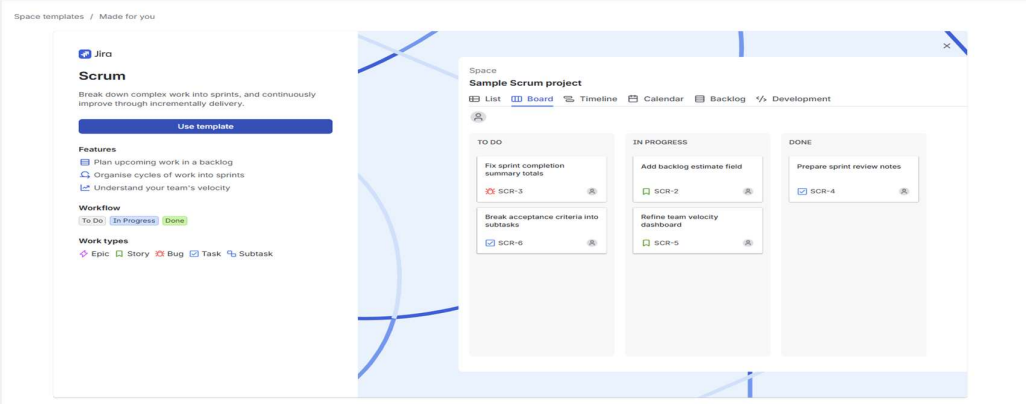
Step 1: Click on Spaces. → Click on the (+) Add sign.



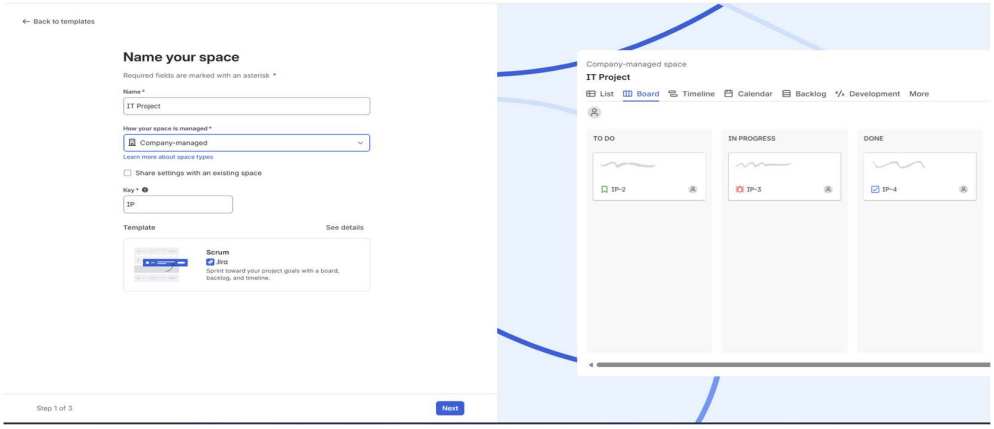
Step 2: Select Scrum.



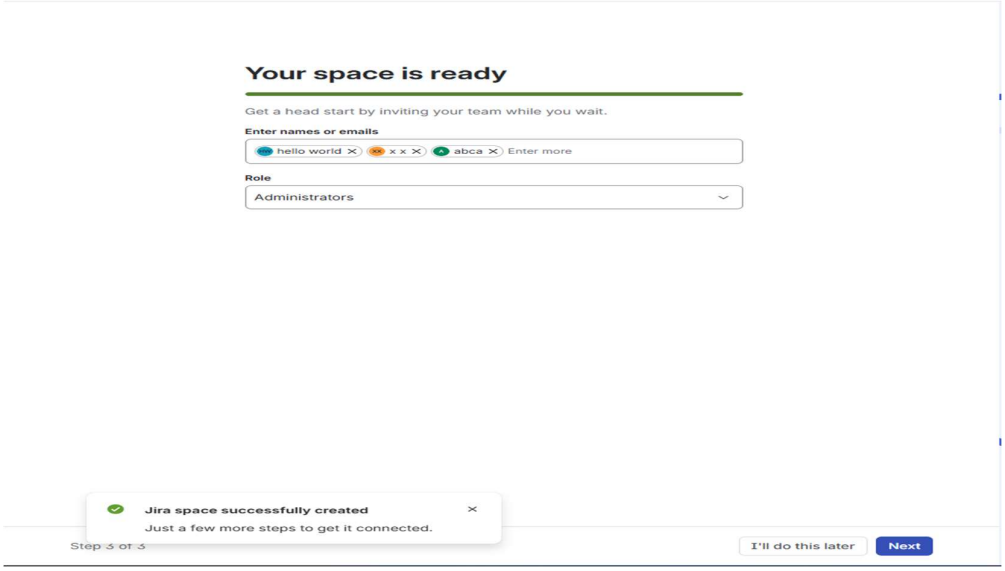
Step 3: Select the Use Template.



Step 4: Enter the Space Name. → Click on Next.

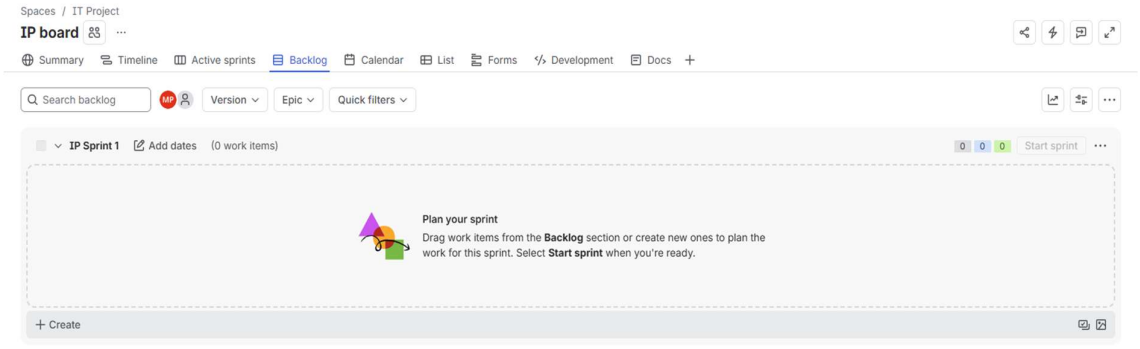


Step 5: Click on Next.

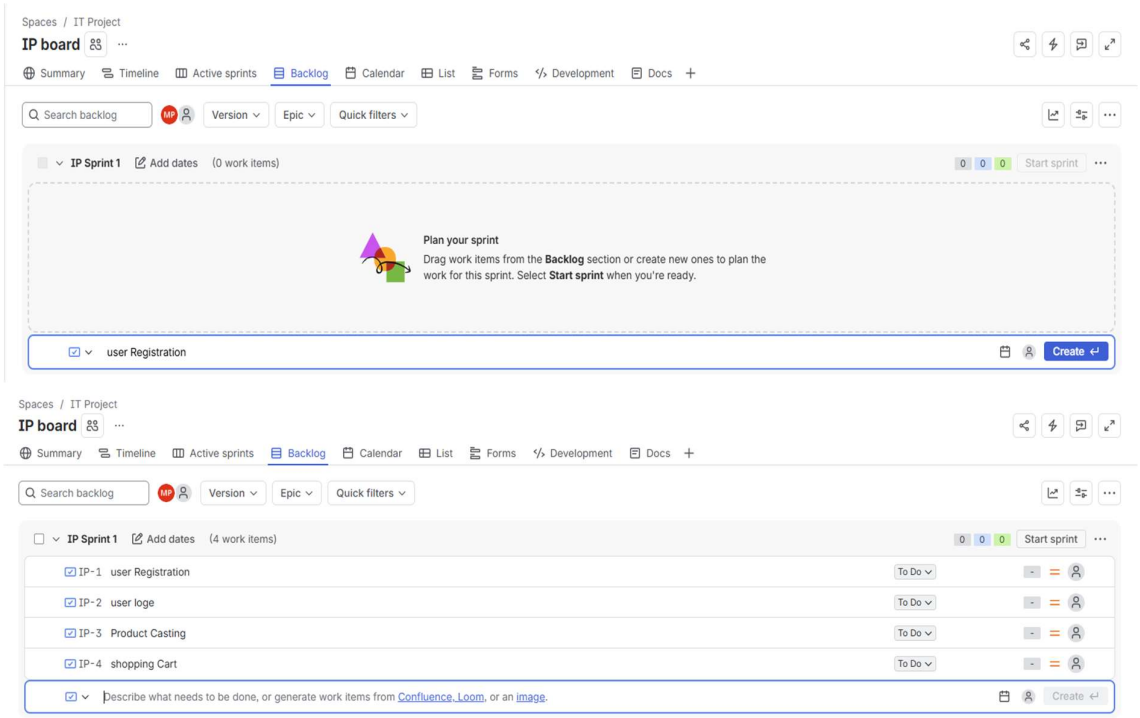


B. Create and start a Sprint.

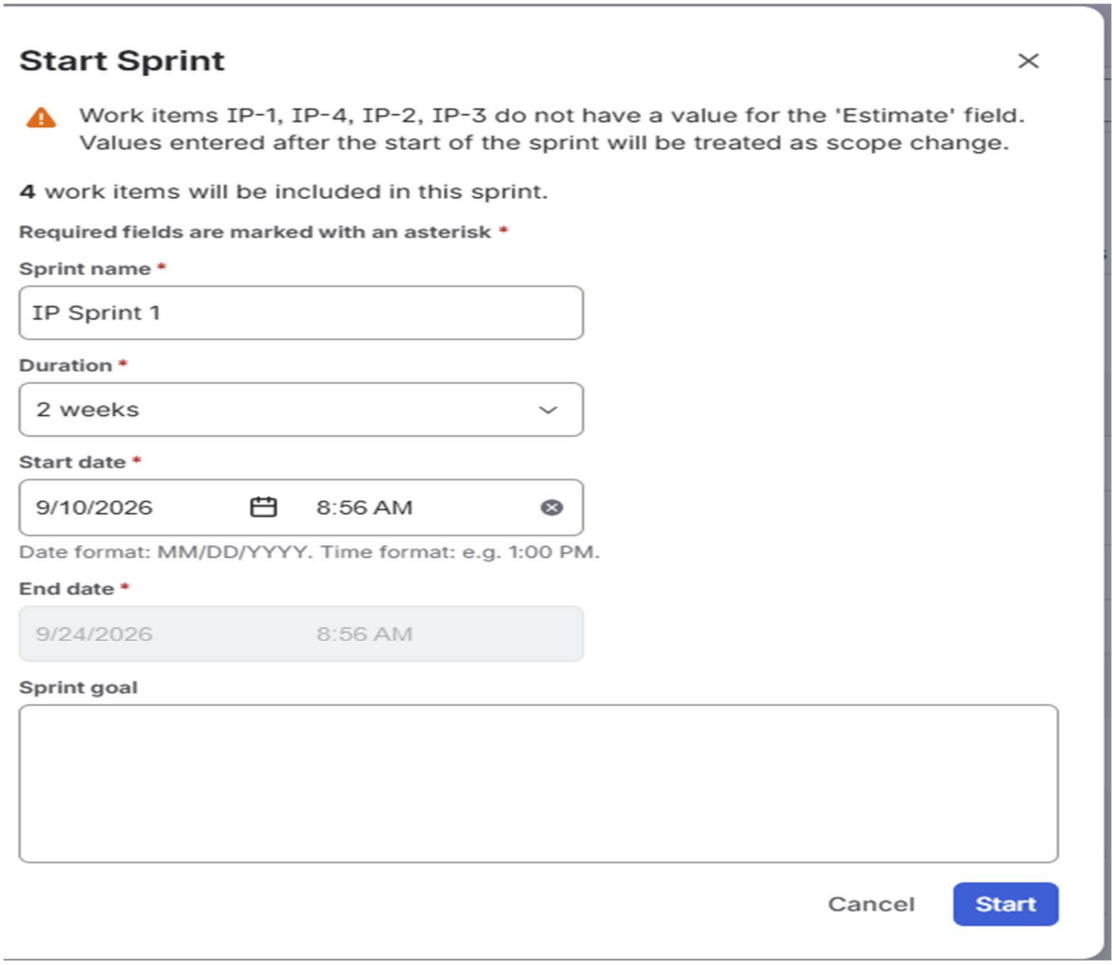
Step 1: Open the Sprint page. → Click on Create.



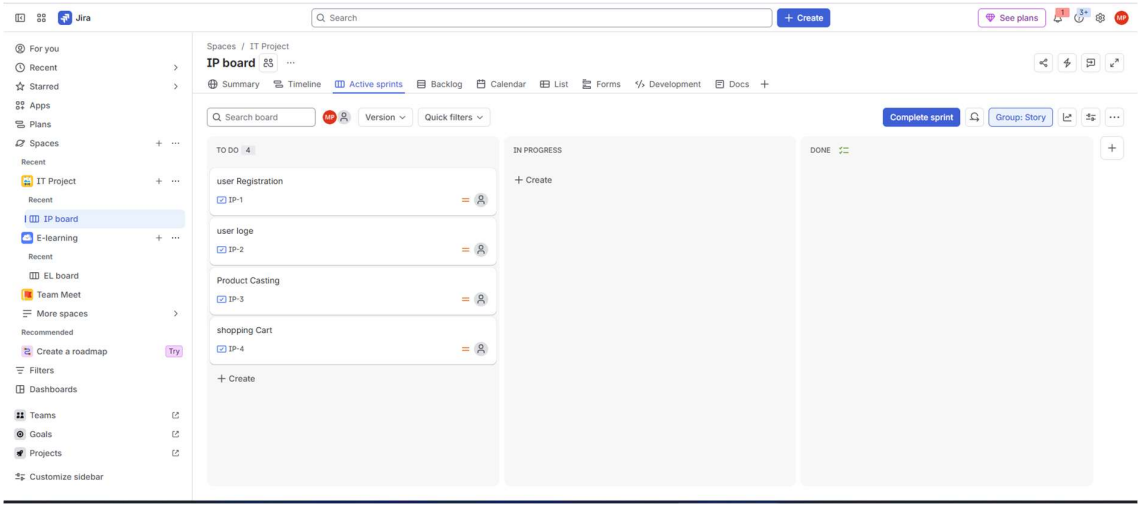
Step 2: Enter the Sprint Description. → Click on Create.



Step 3: Click on Start Sprint.

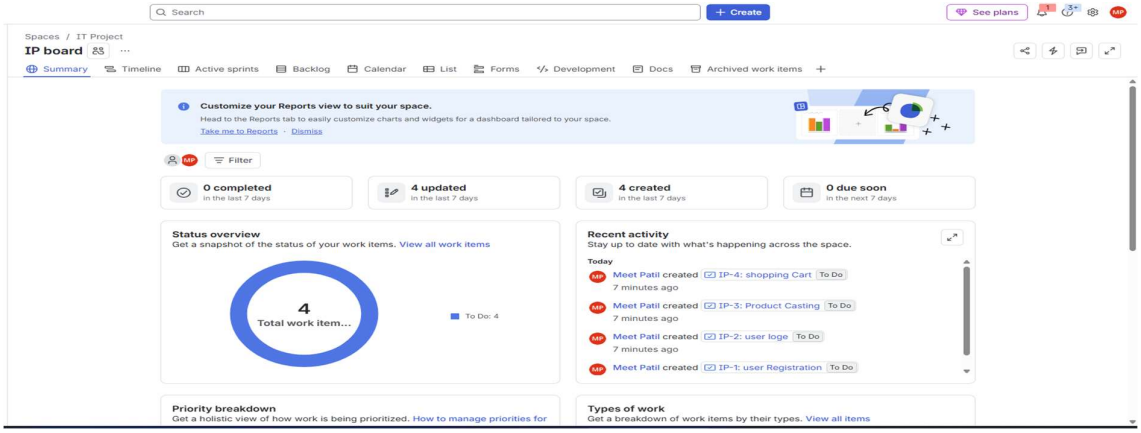


Step 4: Click on Start.

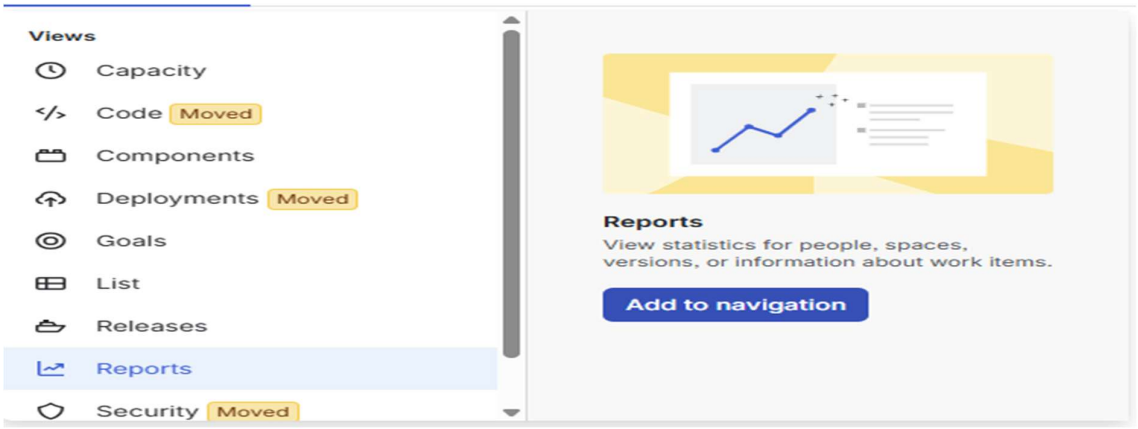


C. Complete Sprint and generate Sprint Report

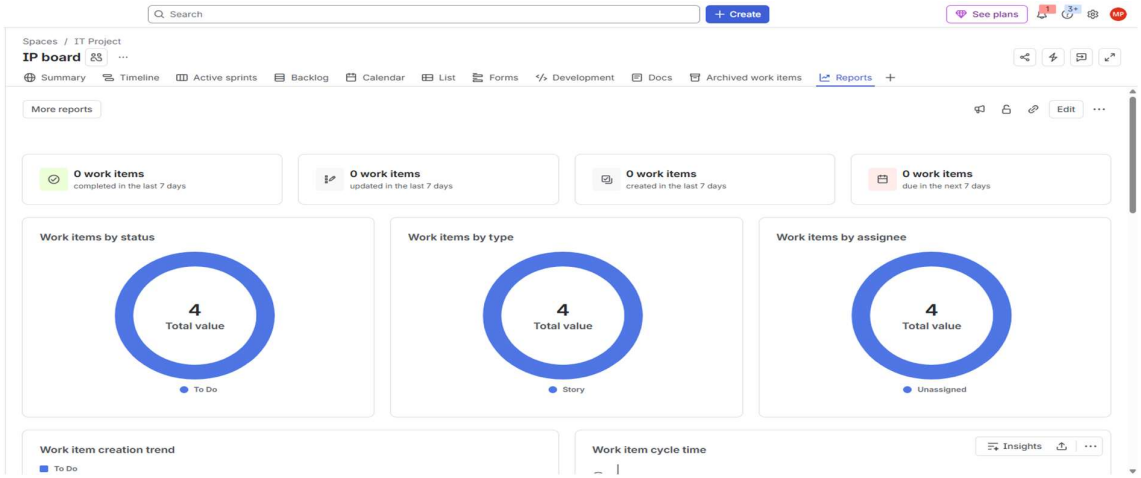
Step 1: Open the Summary page. → Click on the (+) Add sign.



Step 2: Select Report.



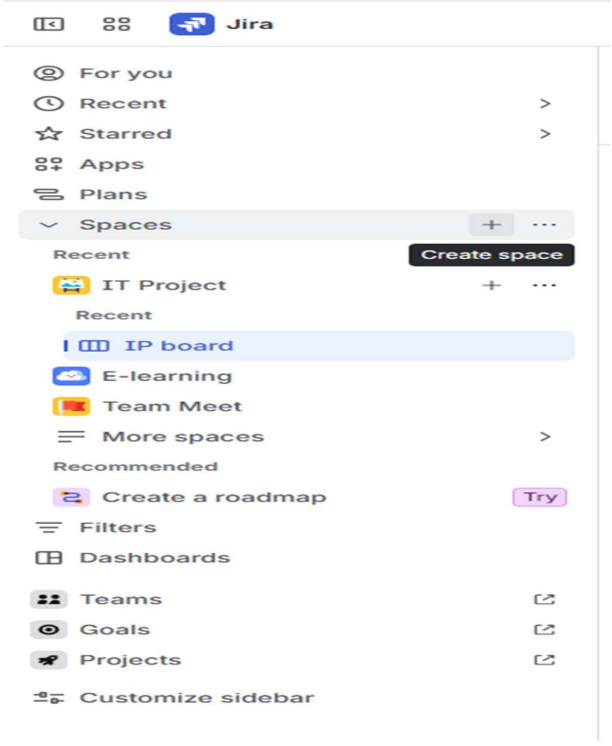
Step 3: Click on Add to Navigation.



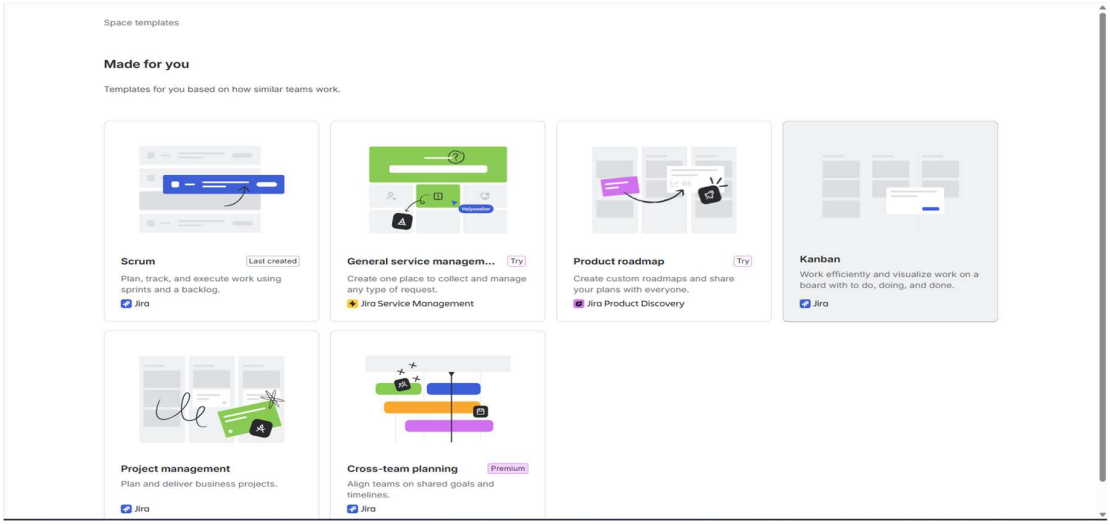
Practical No. 7
Kanban Board Management

A. Create a Company-managed Kanban Project.

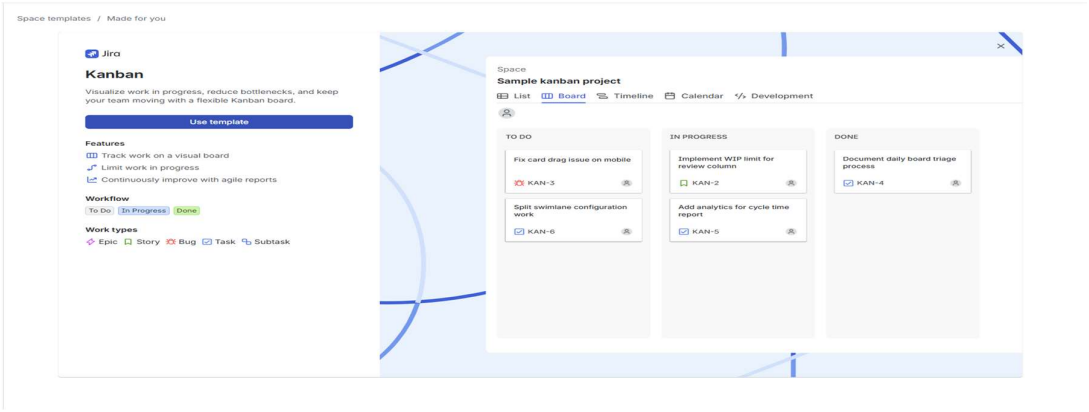
Step 1: Click on Spaces. → Click on the (+) Add sign.



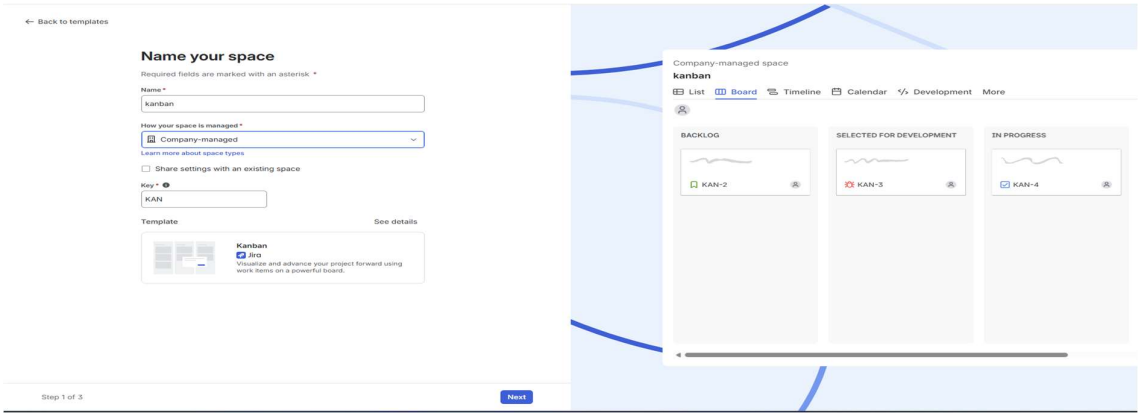
Step 2: Select Kanban.



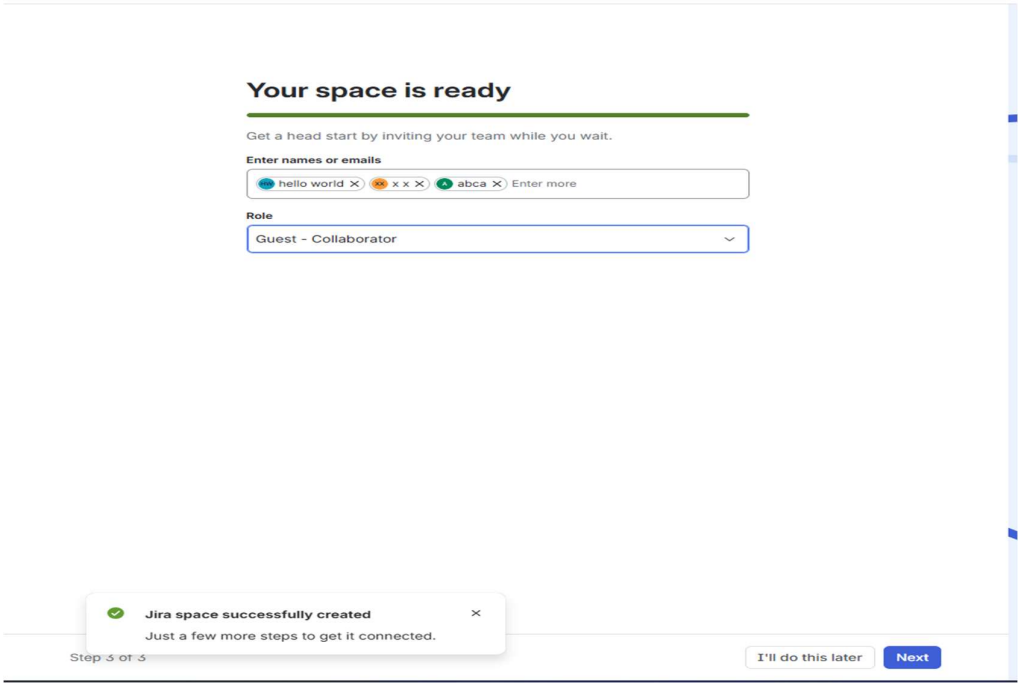
Step 3: Select the Use Template.



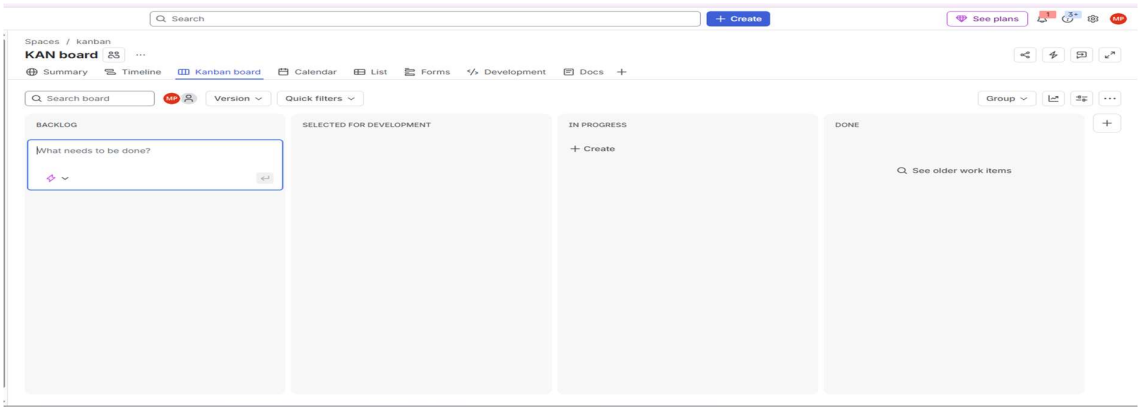
Step 4: Enter the Space Name. → Click on Next.



Step 5: Select Administrators.

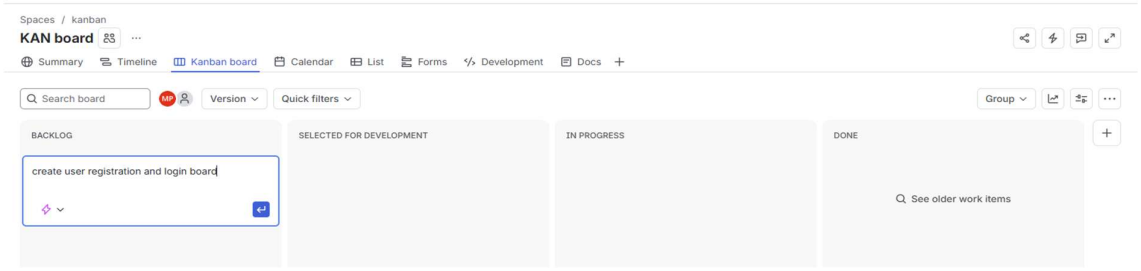


Step 6: Click on Next.

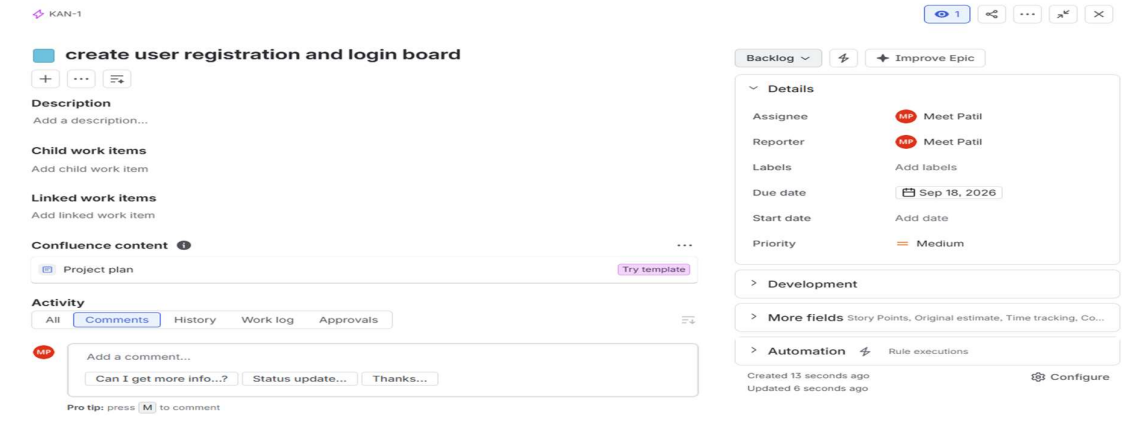


B. Configure Kanban Board

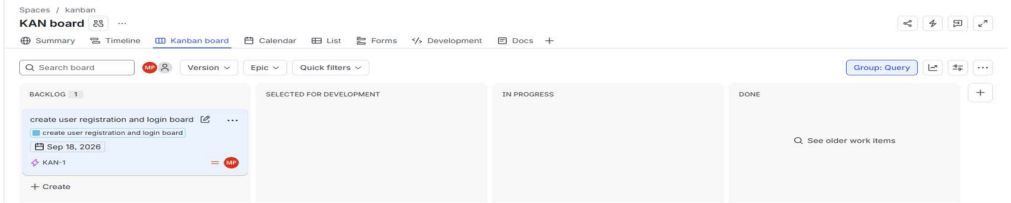
Step 1: Open the Backlog.



Step 2: Enter the Item Name. And Click on Create.



Step 3: Close the created item.

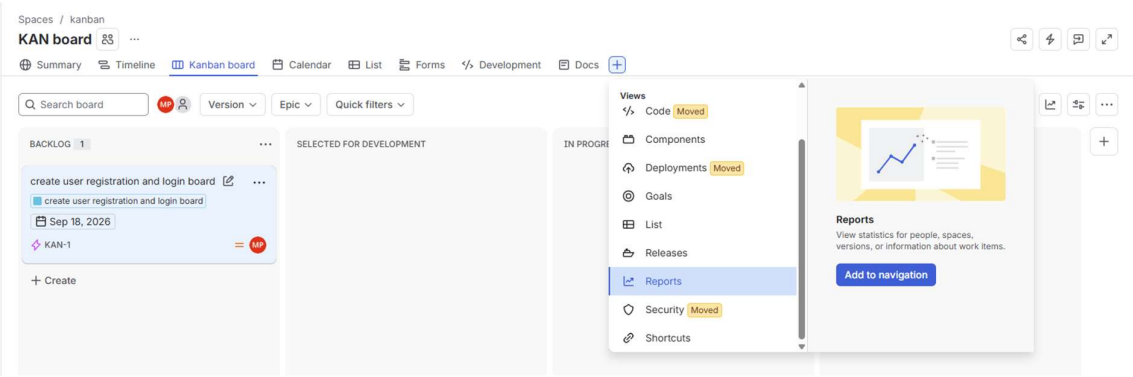


C. Track issue progress and generate Control Chart

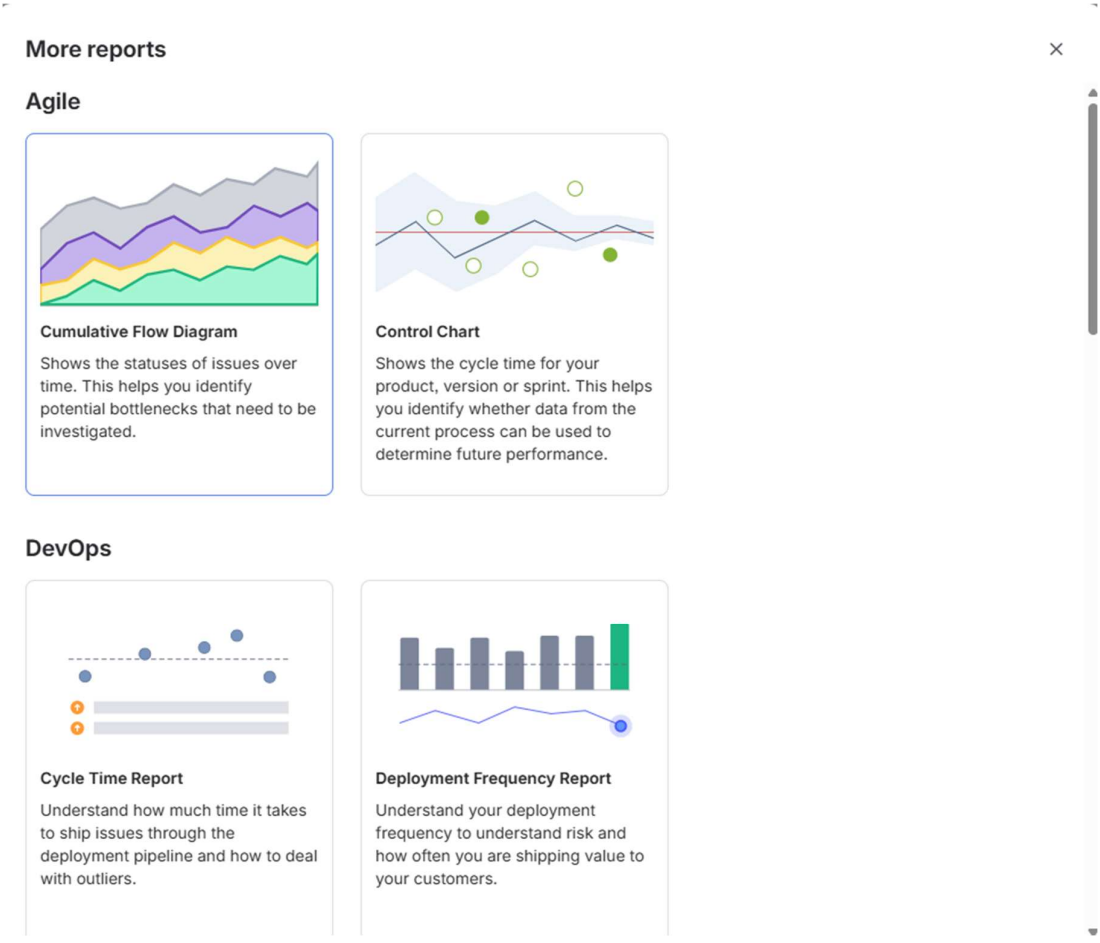
Step 1: Click on the Kanban Board.

Step 2: Click on the (+) Add sign. → Select Reports.

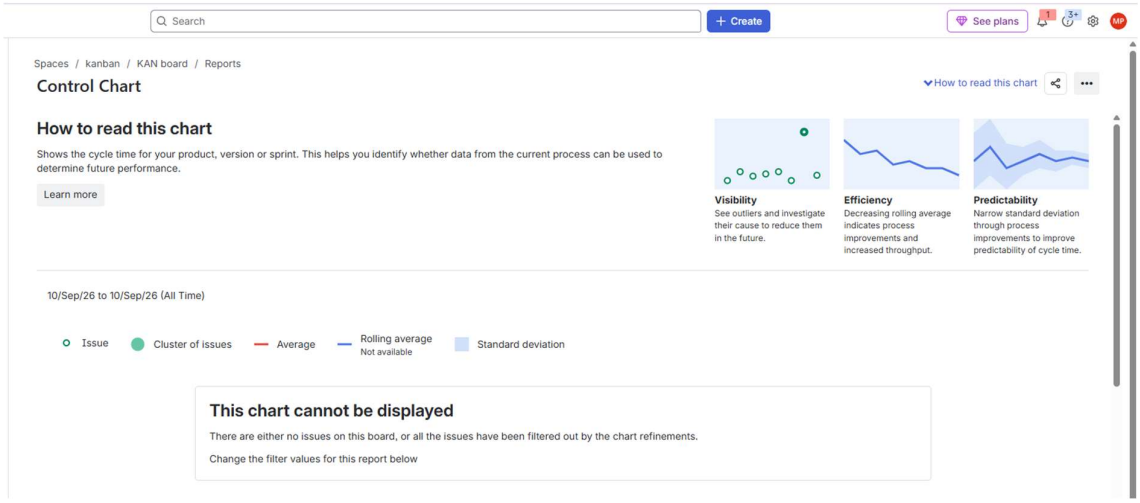
Step 3: Click on Add to Navigation.



Step 4: Click on More Reports



Step 5: In Agile Select the Cumulative Flow Diagram.



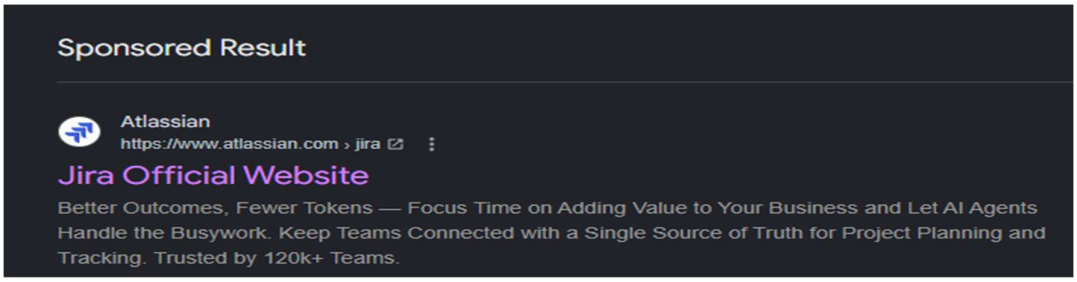
Module 2

Practical No. 1

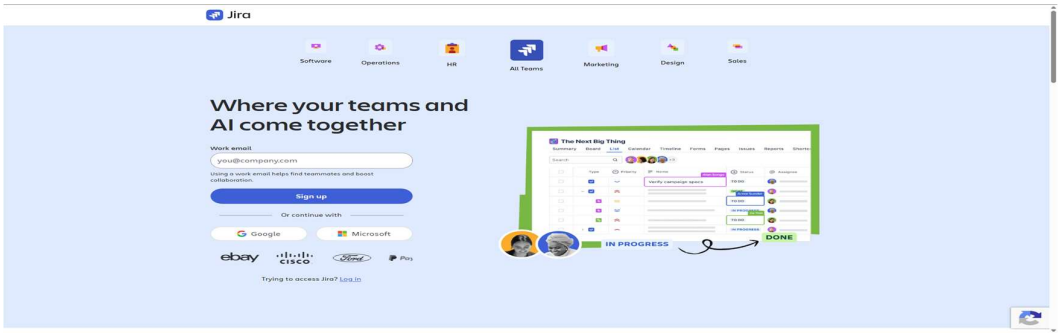
Aim:- Jira Setup & Project Configuration, Advanced Configuration, Reporting & Governance

A. Create a Jira Cloud account and explore the dashboard.

Step 1: Open the Jira website.

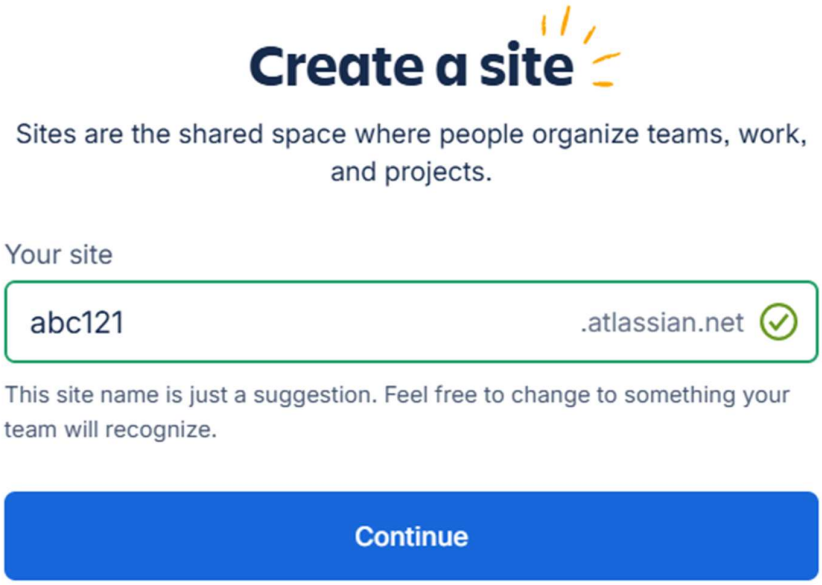


Step 2: Click on **Get it free** or **Sign up**.

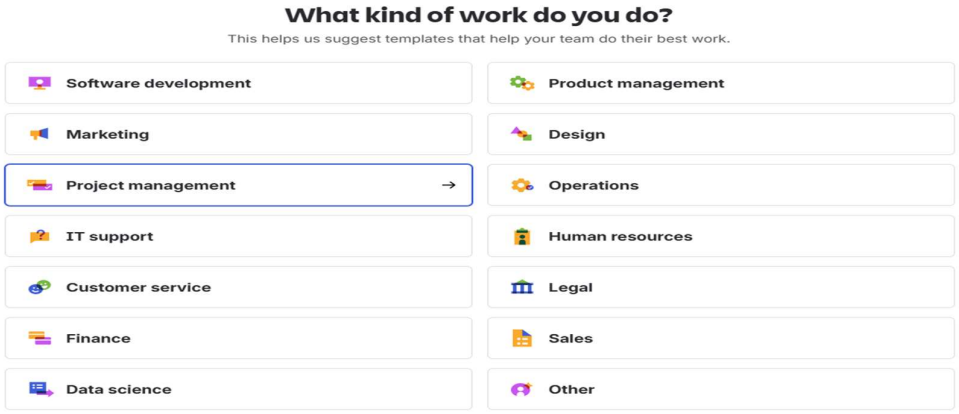


Step 3: Enter your email name → Click on the **Continue** button.

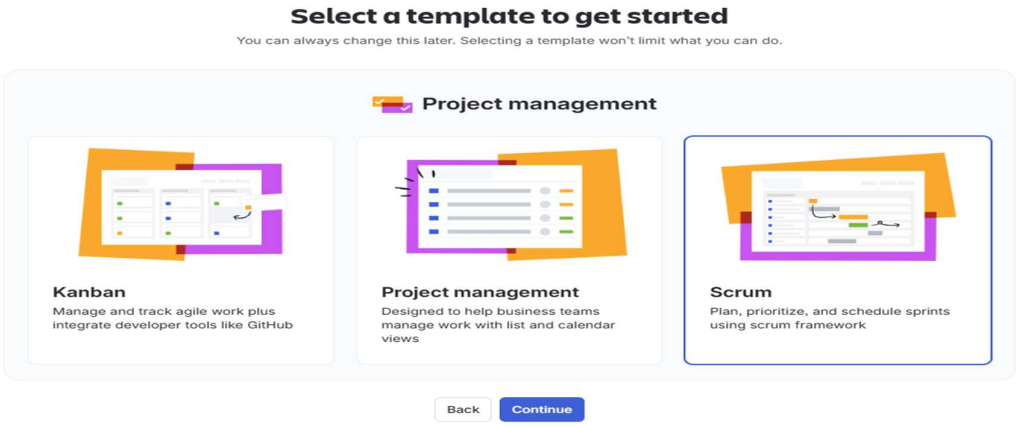
Step 4: Verify your email and complete the account setup.



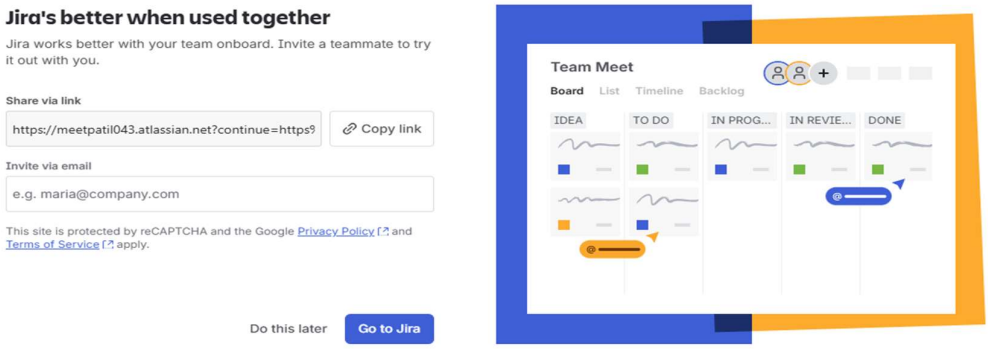
Step 5: Select **Project management** as the type of work.



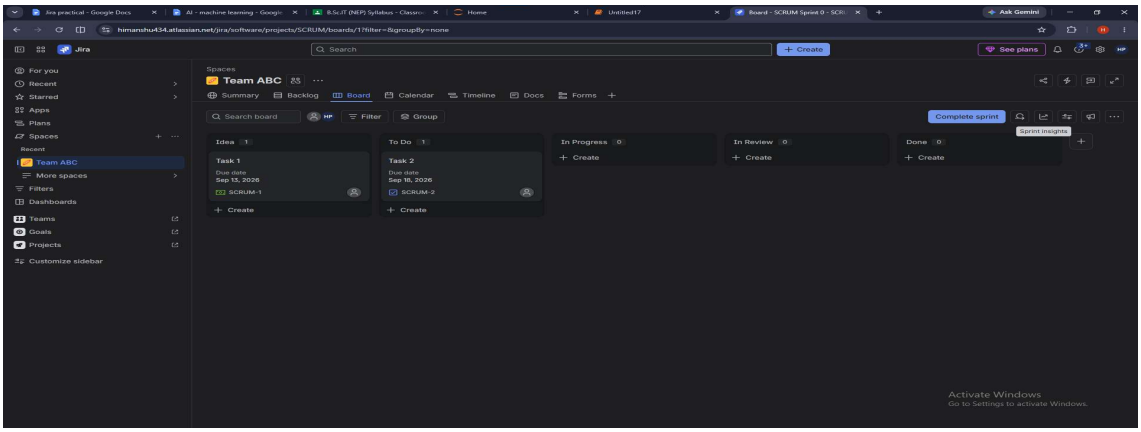
Step 6: Select the **Scrum** template → Click on the **Continue** button.



Step 7: Enter the **Space Name** → Click on the **Get started** button.

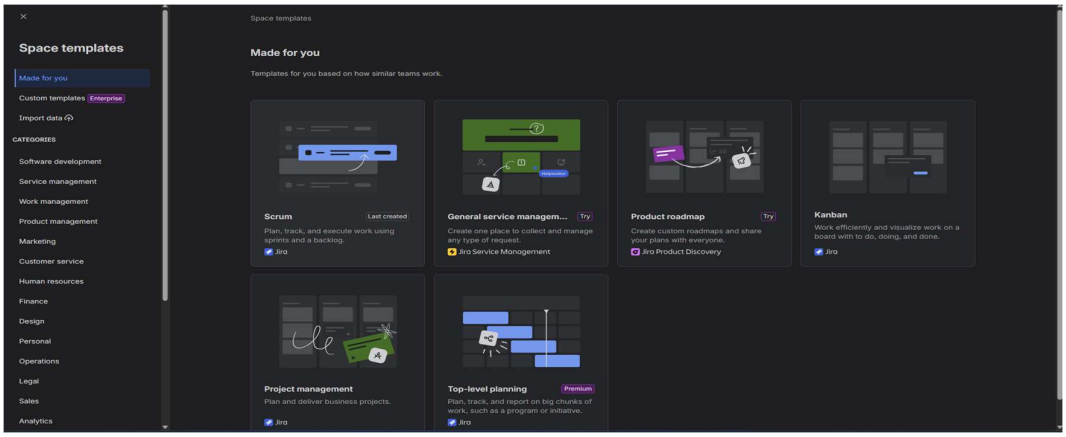


Step 8: The **Jira Dashboard** will open.

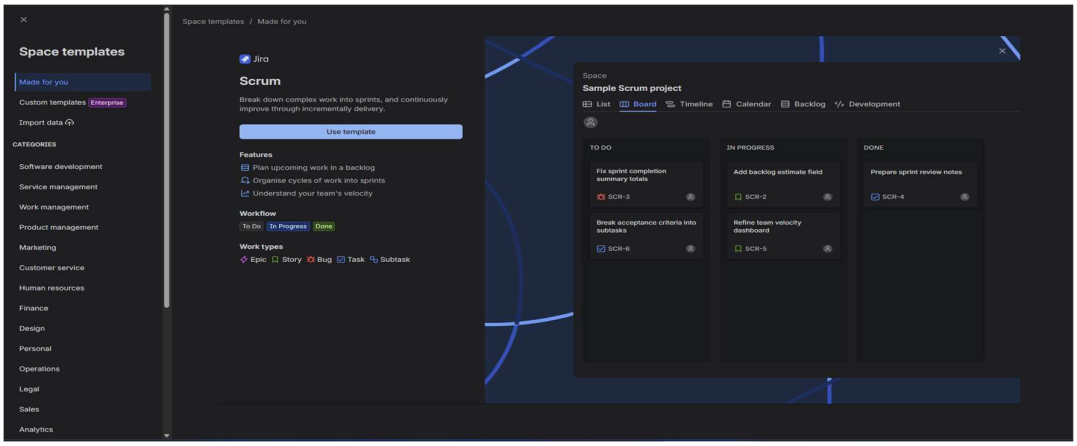


B. Create a Company-managed Scrum/Kanban Project.

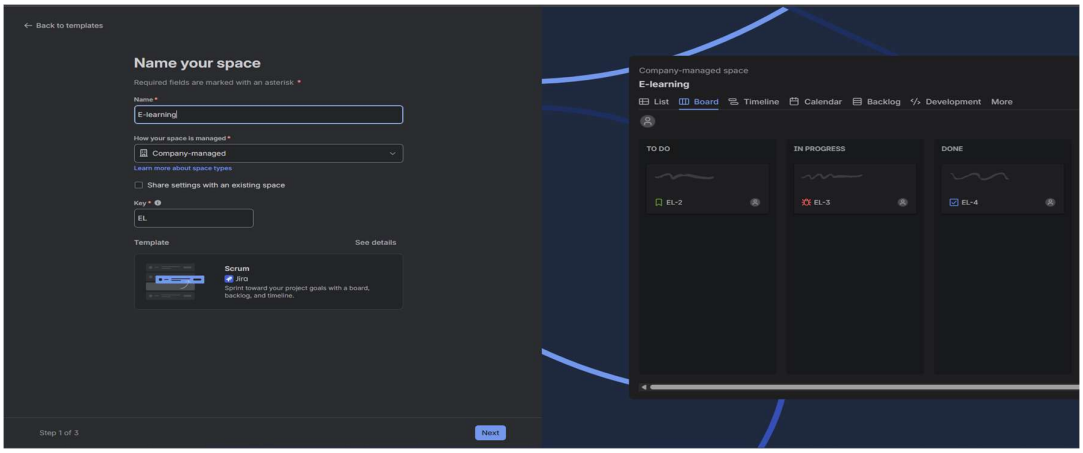
Step 1: Click on **Projects** from the navigation menu and Select **Scrum**.



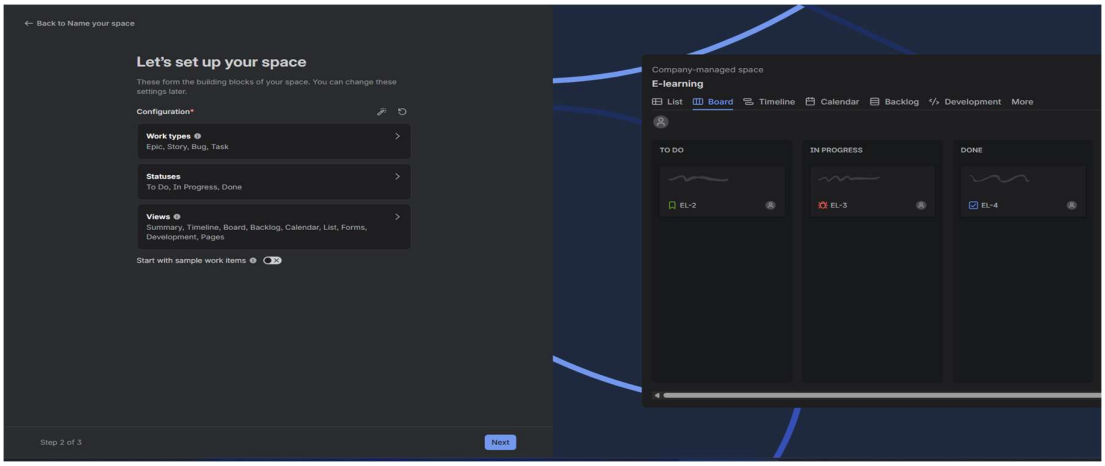
Step 2: Select Company-managed project.



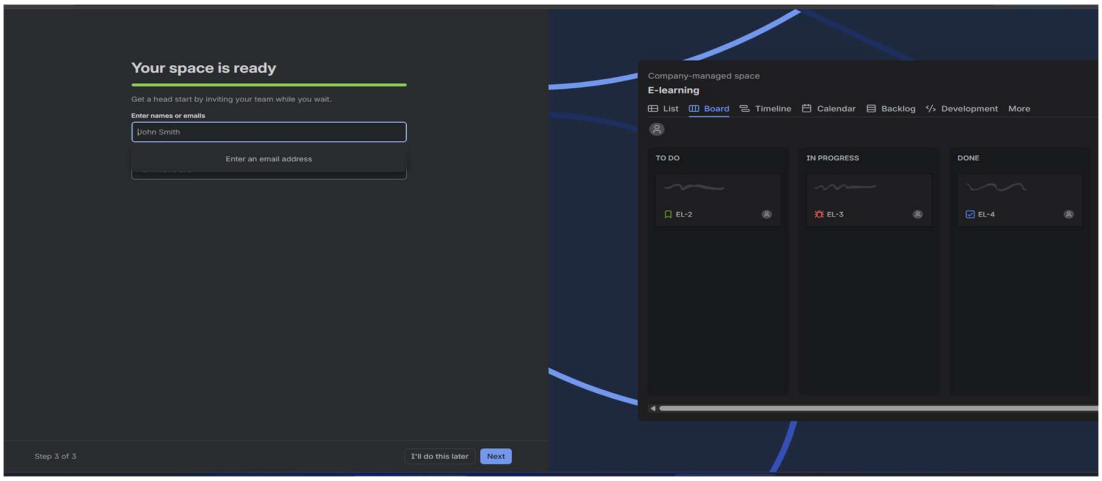
Step 3: Enter the Project Name and click Next.



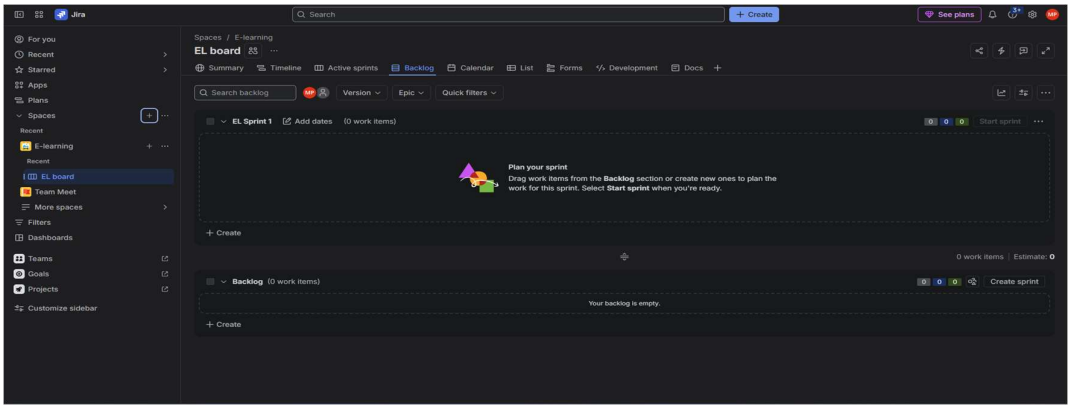
Step 4: Click the Next button.



Step 5: Click the Next button.



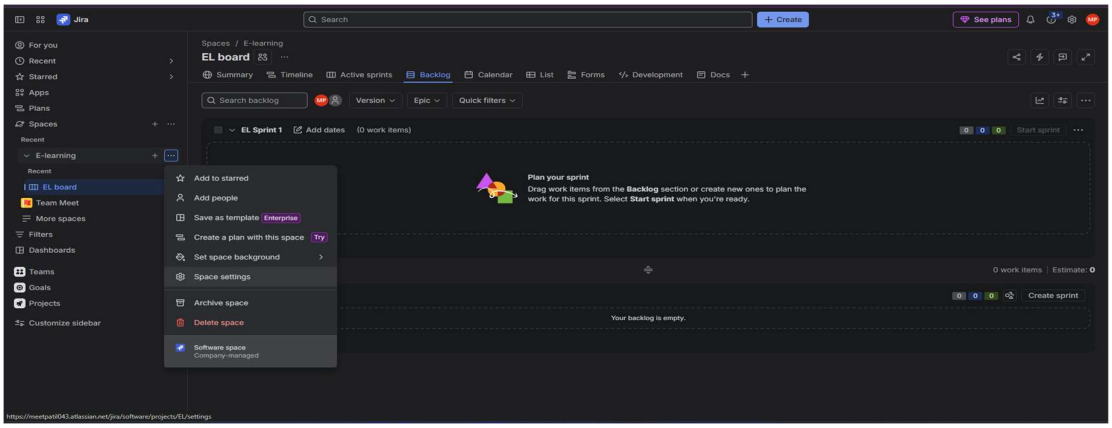
Step 6: The project is created successfully.



C. Configure project details

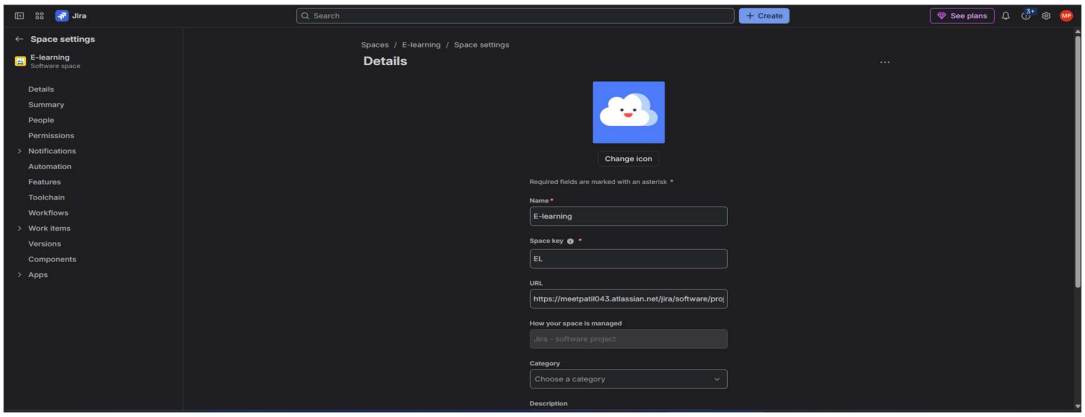
Step 1: Open the newly created project.

Step 2: Go to Project settings.



Step 3: Configure the project name and other basic details.

Step 4: Review the project settings and save the changes.



Problem Solving by Searching(Informed Search)

Practical No.3(A)

Aim:- Given an initial configuration of the 8-puzzle and a goal configuration, write a Python program to find the solution using Greedy Best-First Search with the Manhattan Distance heuristic.

Code:-

```
import heapq

GOAL_STATE = (1, 2, 3,
              4, 5, 6,
              7, 8, 0)

GOAL_POSITIONS = {tile: (i // 3, i % 3) for i, tile in enumerate(GOAL_STATE)}


def get_manhattan_distance(state):
    distance = 0
    for i, tile in enumerate(state):
        if tile != 0:  # Skip the blank tile
            current_row, current_col = i // 3, i % 3
            goal_row, goal_col = GOAL_POSITIONS[tile]
            distance += abs(current_row - goal_row) + abs(current_col - goal_col)
    return distance


def get_neighbors(state):
    neighbors = []
    blank_idx = state.index(0)
    row, col = blank_idx // 3, blank_idx % 3

    moves = [(-1, 0), (1, 0), (0, -1), (0, 1)]  # Up, Down, Left, Right

    for dr, dc in moves:
        new_row, new_col = row + dr, col + dc
        if 0 <= new_row < 3 and 0 <= new_col < 3:
            new_blank_idx = new_row * 3 + new_col

            # Convert to list to swap tiles, then back to tuple
```

```
            new_state = list(state)
            new_state[blank_idx], new_state[new_blank_idx] = new_state[new_blank_idx],
new_state[blank_idx]
            neighbors.append(tuple(new_state))

    return neighbors


def greedy_best_first_search(start_state):
    open_list = []
    heapq.heappush(open_list, (get_manhattan_distance(start_state), start_state, [start_state]))

    visited = set()
    visited.add(start_state)

    while open_list:
        _, current_state, path = heapq.heappop(open_list)

        # Check if goal is reached
        if current_state == GOAL_STATE:
            return path

        for neighbor in get_neighbors(current_state):
            if neighbor not in visited:
                visited.add(neighbor)
                h_value = get_manhattan_distance(neighbor)
                heapq.heappush(open_list, (h_value, neighbor, path + [neighbor]))

    return None


def print_state(state):
    for i in range(0, 9, 3):
        print(list(state[i:i + 3]))
    print()


if __name__ == "__main__":
    initial_configuration = (1, 2, 3,
                             0, 4, 6,
                             7, 5, 8)

    print("Initial State:")
    print_state(initial_configuration)
```

```
solution_path = greedy_best_first_search(initial_configuration)

if solution_path:
    print(f"Solution found in {len(solution_path) - 1} moves!\n")

    for step, state in enumerate(solution_path):
        print(f"Step {step}:")
        print_state(state)
else:
    print("No solution could be found.")
```

Output:-

```
Initial State:
[1, 2, 3]
[0, 4, 6]
[7, 5, 8]

Solution found in 3 moves!

Step 0:
[1, 2, 3]
[0, 4, 6]
[7, 5, 8]

Step 1:
[1, 2, 3]
[4, 0, 6]
[7, 5, 8]

Step 2:
[1, 2, 3]
[4, 5, 6]
[7, 0, 8]

Step 3:
[1, 2, 3]
[4, 5, 6]
[7, 8, 0]
```

Practical No.3(B)

Aim:- Given a weighted graph representing citie and distances between them. Write a python program to find the shortest path from Arad to Bucharest using A* search with heuristic function(straight-line distance to destination).

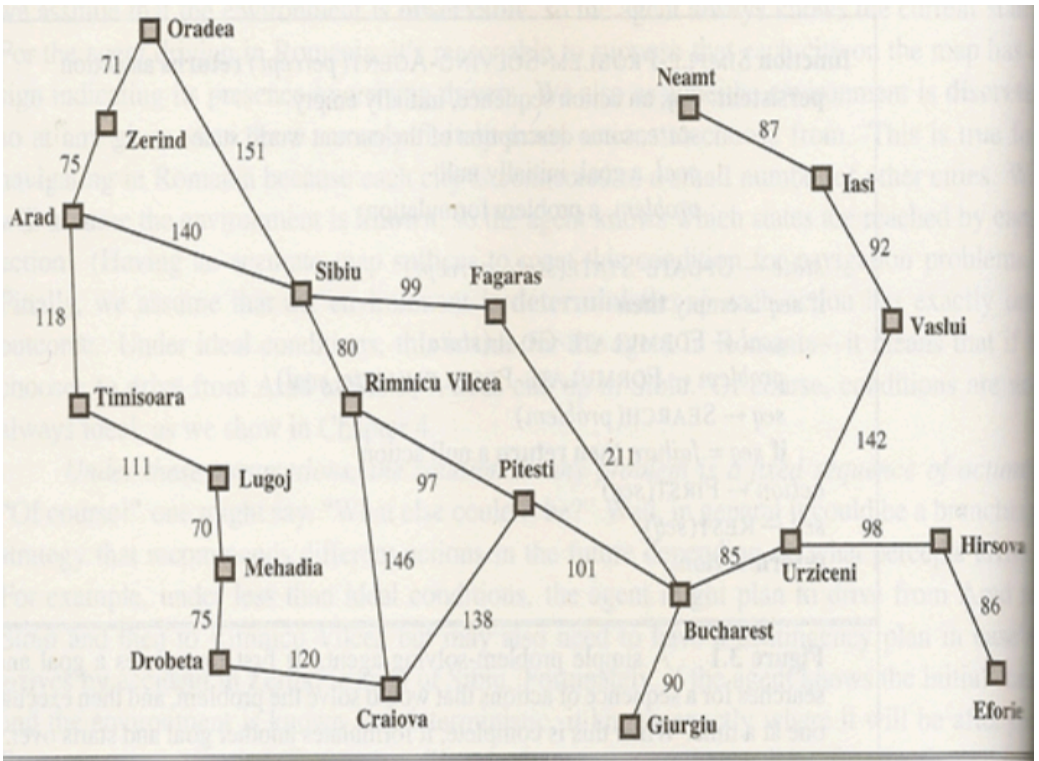


Figure 3.2 A simplified road map of part of Romania.

Code:-

```
import heapq

# 1. Graph representation extracted directly from the map image:
romania_map = {
    'Arad': {'Zerind': 75, 'Sibiu': 140, 'Timisoara': 118},
    'Zerind': {'Arad': 75, 'Oradea': 71},
    'Oradea': {'Zerind': 71, 'Sibiu': 151},
    'Sibiu': {'Arad': 140, 'Oradea': 151, 'Fagaras': 99, 'Rimnicu Vilcea': 80},
    'Timisoara': {'Arad': 118, 'Lugoj': 111},
    'Lugoj': {'Timisoara': 111, 'Mehadia': 70},
    'Mehadia': {'Lugoj': 70, 'Drobeta': 75},
    'Drobeta': {'Mehadia': 75, 'Craiova': 120},
    'Craiova': {'Drobeta': 120, 'Rimnicu Vilcea': 146, 'Pitesti': 138},
    'Rimnicu Vilcea': {'Sibiu': 80, 'Craiova': 146, 'Pitesti': 97},
```

```
    'Fagaras': {'Sibiu': 99, 'Bucharest': 211},
    'Pitesti': {'Rimnicu Vilcea': 97, 'Craiova': 138, 'Bucharest': 101},
    'Bucharest': {'Fagaras': 211, 'Pitesti': 101, 'Giurgiu': 90, 'Urziceni': 85},
    'Giurgiu': {'Bucharest': 90},
    'Urziceni': {'Bucharest': 85, 'Vaslui': 142, 'Hirsova': 98},
    'Hirsova': {'Urziceni': 98, 'Eforie': 86},
    'Eforie': {'Hirsova': 86},
    'Vaslui': {'Urziceni': 142, 'Iasi': 92},
    'Iasi': {'Vaslui': 92, 'Neamt': 87},
    'Neamt': {'Iasi': 87}
}


# 2. Straight-line distance heuristic h(n) to Bucharest
straight_line_to_bucharest = {
    'Arad': 366, 'Bucharest': 0, 'Craiova': 160, 'Drobeta': 242, 'Eforie': 161,
    'Fagaras': 176, 'Giurgiu': 77, 'Hirsova': 151, 'Iasi': 226, 'Lugoj': 244,
    'Mehadia': 241, 'Neamt': 234, 'Oradea': 380, 'Pitesti': 100, 'Rimnicu Vilcea': 193,
    'Sibiu': 253, 'Timisoara': 329, 'Urziceni': 80, 'Vaslui': 199, 'Zerind': 374
}



def a_star_search(graph, heuristics, start, goal):
    """Solves the shortest path problem using A* Search."""
    # Priority queue stores tuples of: (f_score, counter, g_score, current_city, path)
    counter = 0
    pq = []

    g_score = {start: 0}
    f_score = heuristics[start]

    # Push initial node
    heapq.heappush(pq, (f_score, counter, 0, start, [start]))

    while pq:
        f, _, current_g, current_city, path = heapq.heappop(pq)

        # If goal is reached, return the path and distance
        if current_city == goal:
            return path, current_g

        # Explore neighbors
        for neighbor, distance in graph[current_city].items():
```

```
            tentative_g = current_g + distance

            # If a shorter path to neighbor is found
            if neighbor not in g_score or tentative_g < g_score[neighbor]:
                g_score[neighbor] = tentative_g
                h = heuristics[neighbor]
                f_neighbor = tentative_g + h

                counter += 1
                heapq.heappush(pq, (f_neighbor, counter, tentative_g, neighbor, path + [neighbor]))

    return None, float('inf')

# --- Run Search ---
if __name__ == "__main__":
    start = 'Arad'
    goal = 'Bucharest'

    path, distance = a_star_search(romania_map, straight_line_to_bucharest, start, goal)

    print(f"Shortest Path: {' -> '.join(path)}")
    print(f"Total Distance: {distance} km")
```

Output:-

Shortest Path: Arad -> Sibiu -> Rimnicu Vilcea -> Pitesti -> Bucharest
Total Distance: 418 km

Practical No.4 (Baye's Rule Application)

Scenario:-

A medical test is used to detect a particular disease.
The probability that a randomly selected person has the disease is 1%.
If a person has the disease, the test returns positive with probability 99%.
If a person does not have the disease, the test still returns positive with probability 5% (false positive rate).
A person takes the test and receives a positive result.

Aim:- Write a Python program to compute the probability that the person actually hasthe disease given that the test result is positive.

Code:-

```
def calculate_disease_probability():

    prob_disease=0.01
    prob_no_disease=0.99

    true_positive_rate=0.99
    false_positive_rate=0.05

    #1.Calculate the total probability of the positive test result P(T+)
prob_positive_test=(true_positive_rate*prob_disease)+(false_positive_rate*prob_no_disease)

    #2.Apply Baye's Theorem to calculate P(D|T+)
    prob_disease_given_positive=(true_positive_rate*prob_disease)/prob_positive_test

    print(f"Total probability of testing positive P(T+):{prob_positive_test:.4f}")
    print(f"Probability of actually having the disease given a positive test P(D|T+):
{prob_disease_given_positive:.4f}")
    print(f"In Percentage:{prob_disease_given_positive * 100:.2f}%")

if __name__=="__main__":
    calculate_disease_probability()
```

Output:-

```
Total probability of testing positive P(T+):0.0594
Probability of actually having the disease given a positive test P(D|T+): 0.1667
In Percentage:16.67%
```

PRACTICAL NO.5

Aim:- Using a small dataset (e.g., weather or spam dataset), implement a Naive Bayes classifier using scikit-learn.

Code:-

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB

#1. Create a tiny dataset(Text, and Label: 1 for spam, 0 for ham)
corpus=[
    "Get rich quick! Click here now!", #Spam
    "Hey, are we still meeting for lunch today?", #Ham
    "Free cash prize! Claim your reward.", #Spam
    "Can you send me the project report?", #Ham
    "Urgent: Account suspended. Verification required.", #Spam
    "Don't forget to buy milk on your way home.", #Ham
    ]
labels=[1,0,1,0,1,0]

#2. Convert text into numerical vectors(Bag of words)
vectorizer=CountVectorizer()
X=vectorizer.fit_transform(corpus)
y=labels

#3. Train the Naive Bayes model(MultinomialNB is great for text)
model= MultinomialNB()
model.fit(X,y)

#4. Test the model with brand new emails
test_emails=[
    "Hey buddy, let's grab a coffee later",
    "CONGRATULATIONS! You won a free vacation! Click now.",
    ]

#Convert test text using the same vectorizer
X_test=vectorizer.transform(test_emails)
predictions=model.predict(X_test)
```

```
#5. Output the results
for email,pred in zip(test_emails,predictions):
    status="SPAM 🚨" if pred==1 else "HAM(Safe)✔"
    print(f"Email:'{email}'--> Predicted as: {status}")
```

Output:-

```
Email:'Hey buddy, let's grab a coffee later'--> Predicted as: HAM(Safe)✔
Email:'CONGRATULATIONS! You won a free vacation! Click now.'--> Predicted as: SPAM
🚨
```

PRACTICAL NO.6

Aim:- Using a simple example (Burglary–Alarm), create a Bayesian Network using pgmpy.

Code:-

```
import pgmpy
from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import VariableElimination

model=DiscreteBayesianNetwork([('Burglary','Alarm'),('Earthquake','Alarm')])
cpd_burglary=TabularCPD(variable='Burglary',variable_card=2,values=[[0.99],[0.01]])

#Prior Probability of Earthquake: P(Earthquake)
cpd_earthquake=TabularCPD(variable='Earthquake',variable_card=2,values=[[0.998],[0.002]
])

#B=0,E=0),(B=0,E=1),(B=1,E=0),(B=1,E=1)
cpd_alarm=TabularCPD(
   variable='Alarm',
   variable_card=2,
   values=[
      [0.999,0.71,0.06,0.05], #P(Alarm=False)
      [0.001,0.29,0.94,0.95] #P(Alarm=True)
      ],
   evidence=['Burglary','Earthquake'],
   evidence_card=[2,2]
)

#3. Add CPDs to the model and validate
model.add_cpds(cpd_burglary,cpd_earthquake,cpd_alarm)
assert model.check_model(),"Model defination is invalid!"

#4. Perform Exact inference using variable elimination
infer=VariableElimination(model)

#Compute posterior probability: P(Burglary | Alarm = True)
posterior=infer.query(variables=['Burglary'],evidence={'Alarm':1})
```

```
#Display the output
print("Posterior Probability distribution P(Burglary| Alarm = True):")
print(posterior)
```

Output:-

```
Posterior Probability distribution P(Burglary| Alarm = True):
+-------------+-----------------+
| Burglary    |   phi(Burglary) |
+=============+=================+
| Burglary(0) |          0.1425 |
+-------------+-----------------+
| Burglary(1) |          0.8575 |
+-------------+-----------------+
```

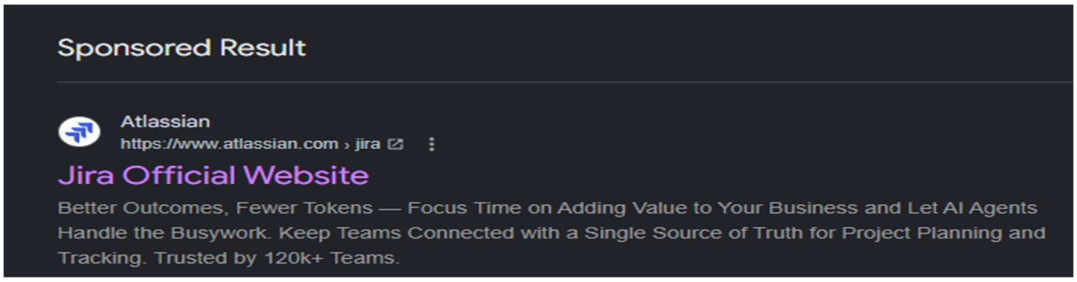
Module 2

Practical No. 1

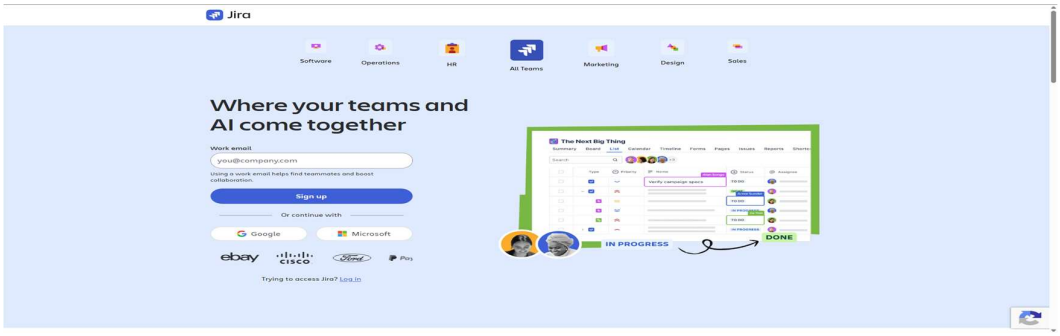
Aim:- Jira Setup & Project Configuration, Advanced Configuration, Reporting & Governance

A. Create a Jira Cloud account and explore the dashboard.

Step 1: Open the Jira website.

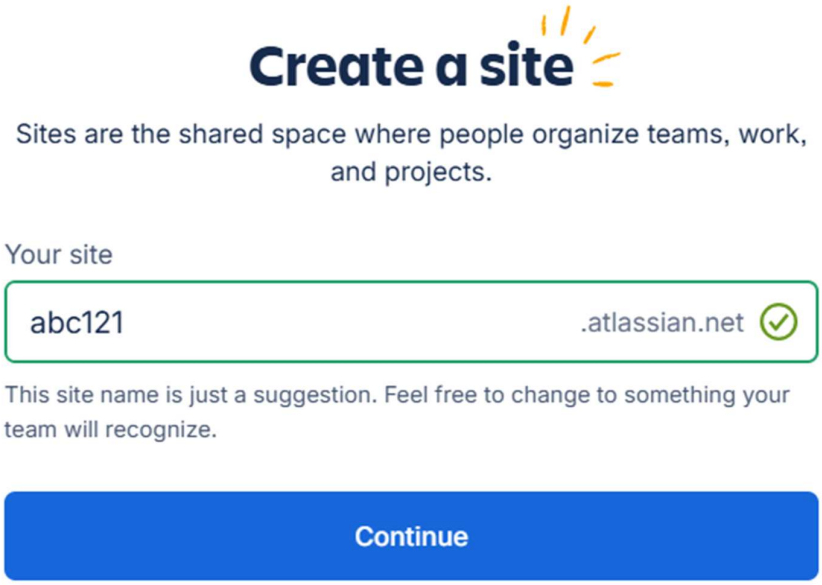


Step 2: Click on **Get it free** or **Sign up**.

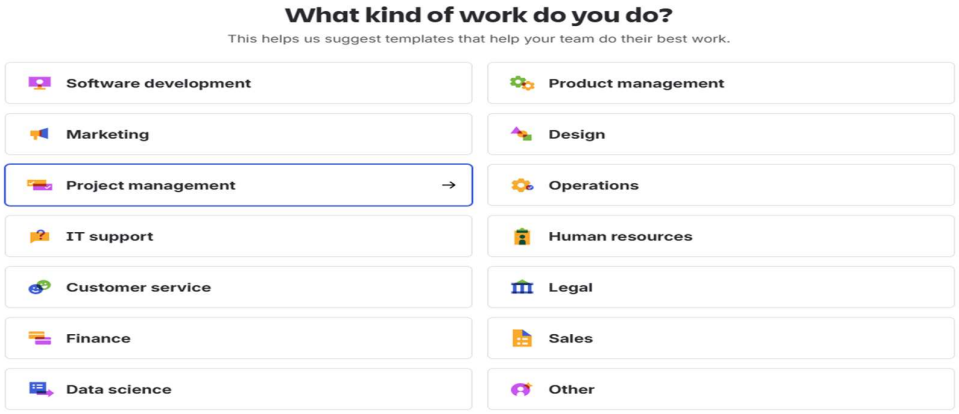


Step 3: Enter your email name → Click on the **Continue** button.

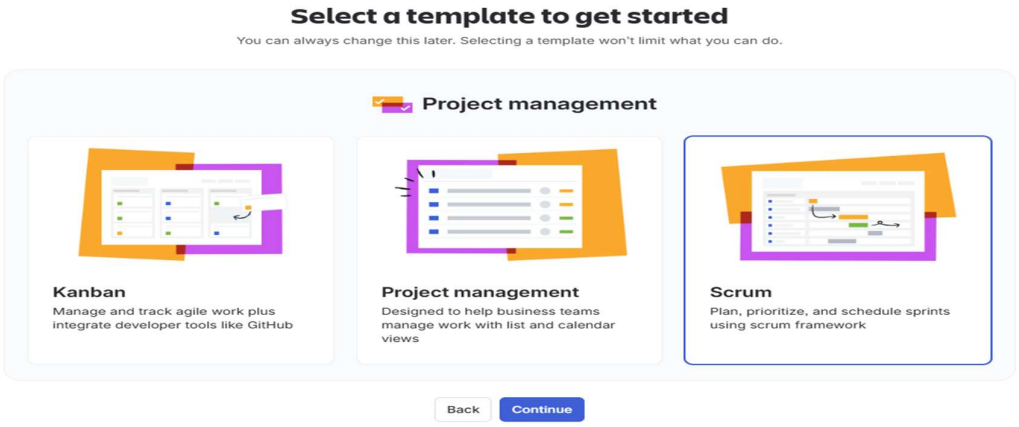
Step 4: Verify your email and complete the account setup.



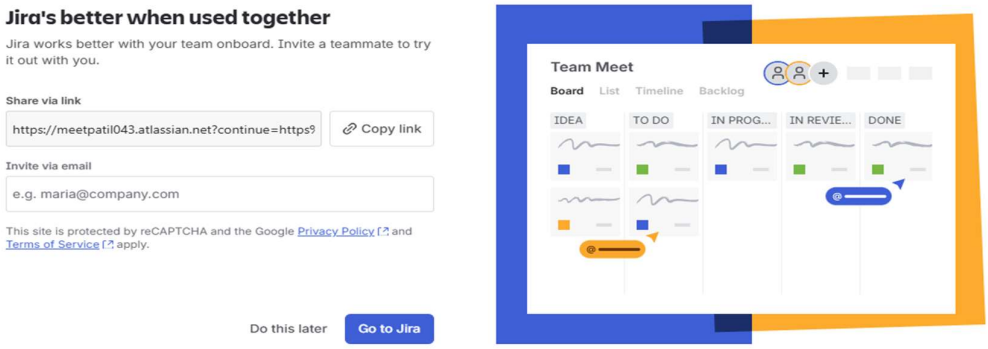
Step 5: Select **Project management** as the type of work.



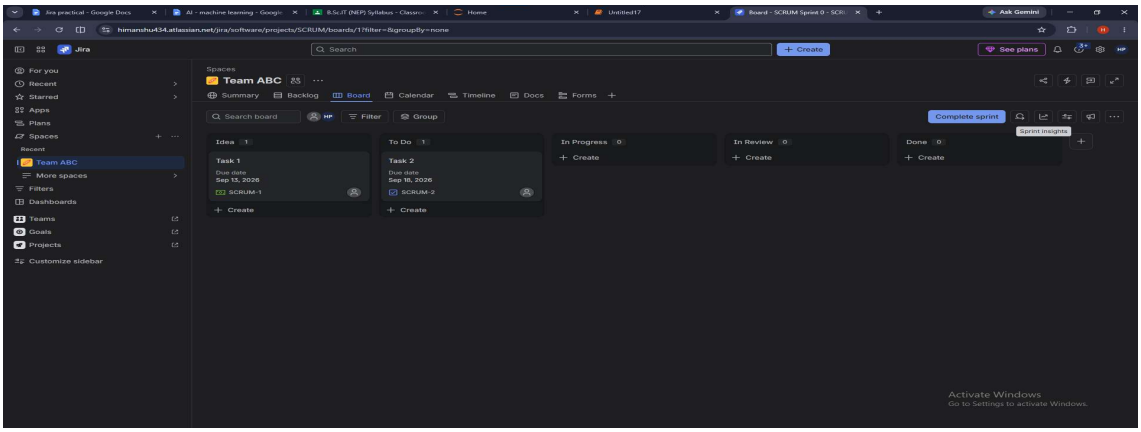
Step 6: Select the **Scrum** template → Click on the **Continue** button.



Step 7: Enter the **Space Name** → Click on the **Get started** button.

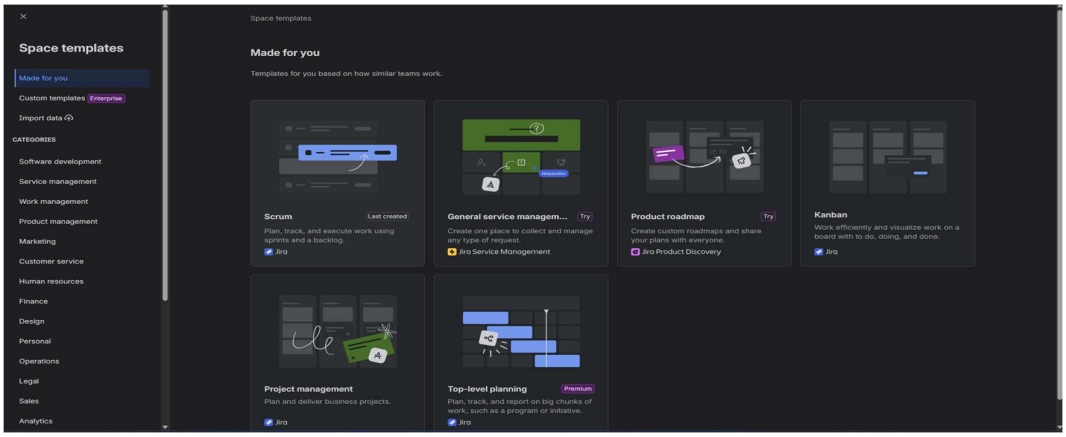


Step 8: The **Jira Dashboard** will open.

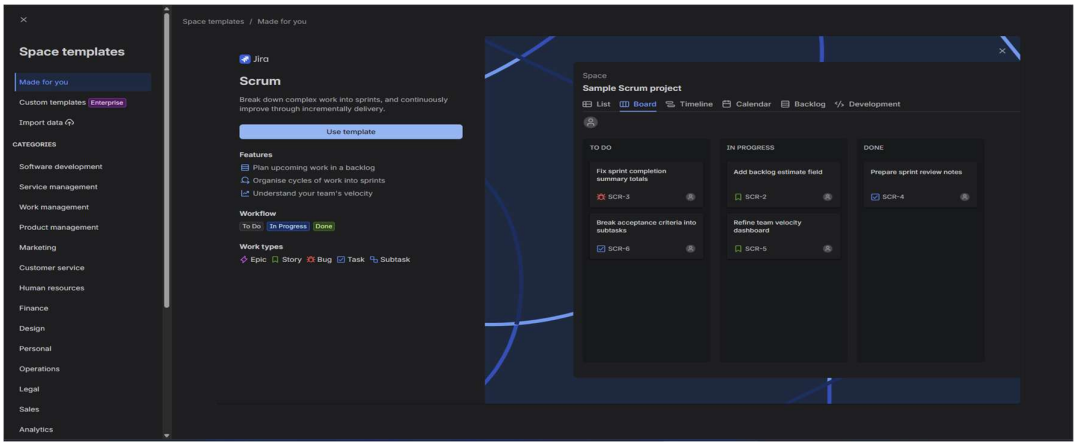


B. Create a Company-managed Scrum/Kanban Project.

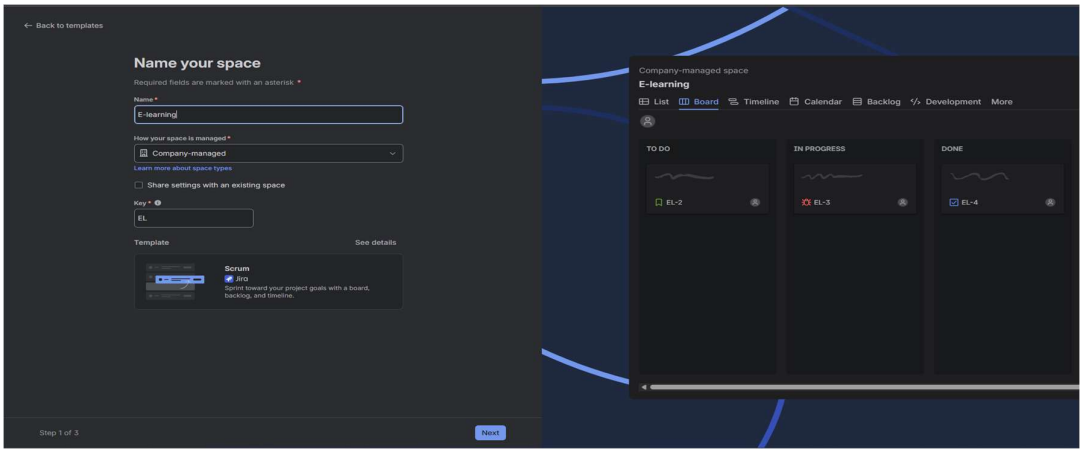
Step 1: Click on **Projects** from the navigation menu and Select **Scrum**.



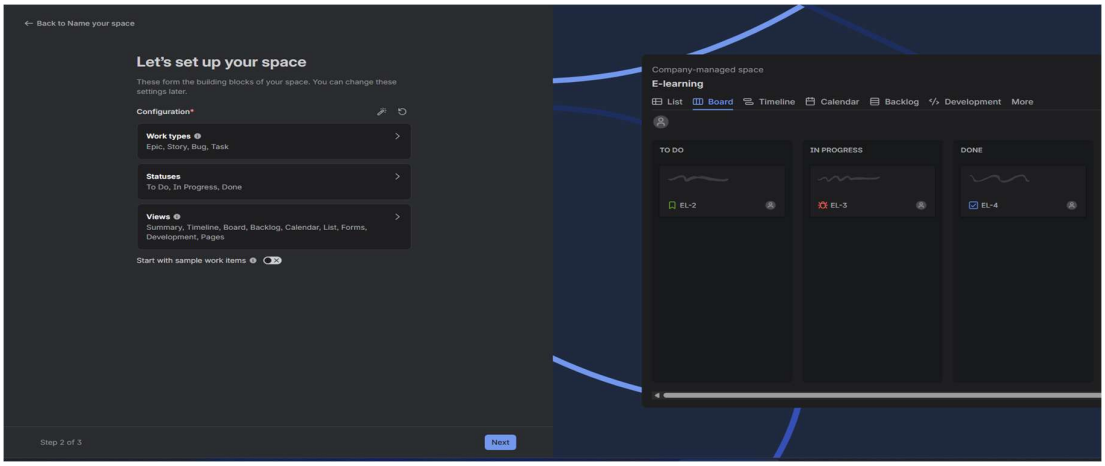
Step 2: Select Company-managed project.



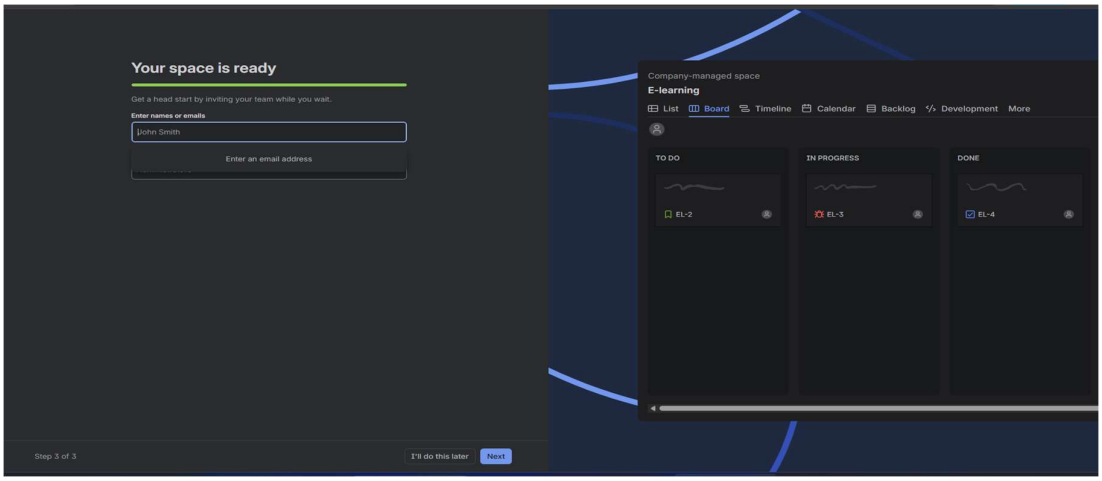
Step 3: Enter the Project Name and click Next.



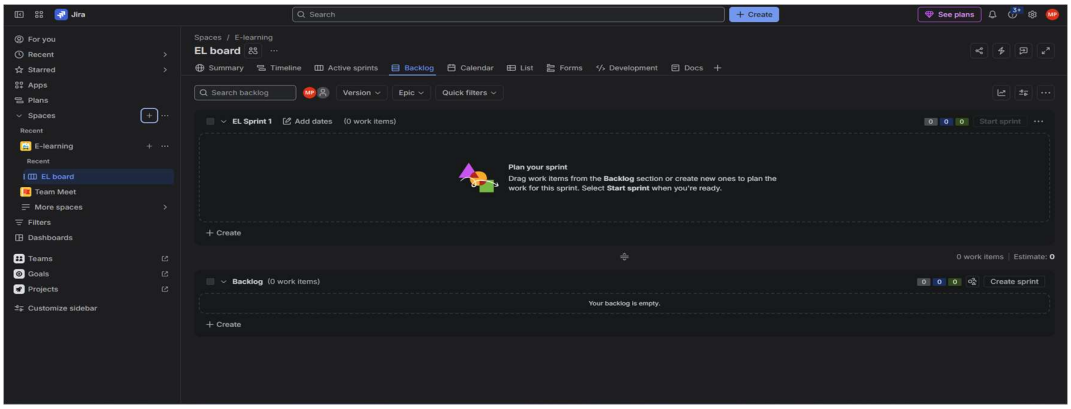
Step 4: Click the Next button.



Step 5: Click the Next button.



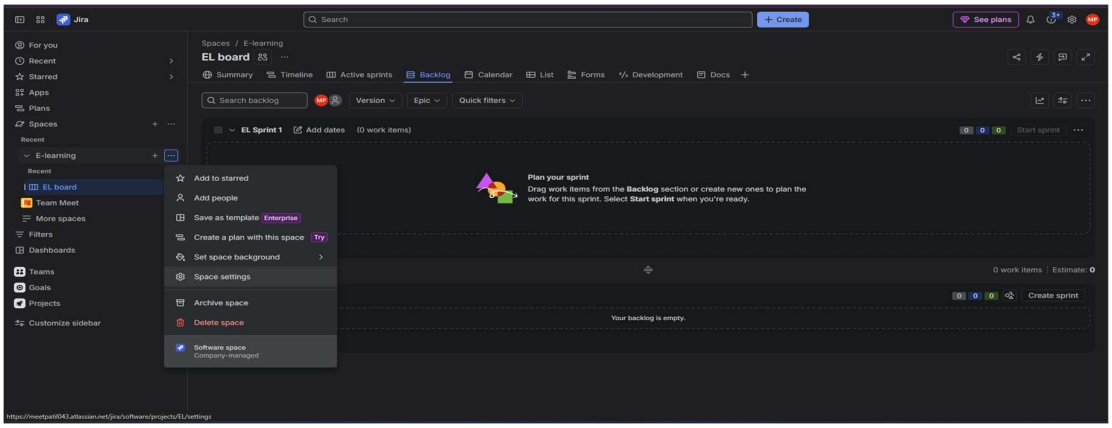
Step 6: The project is created successfully.



C. Configure project details

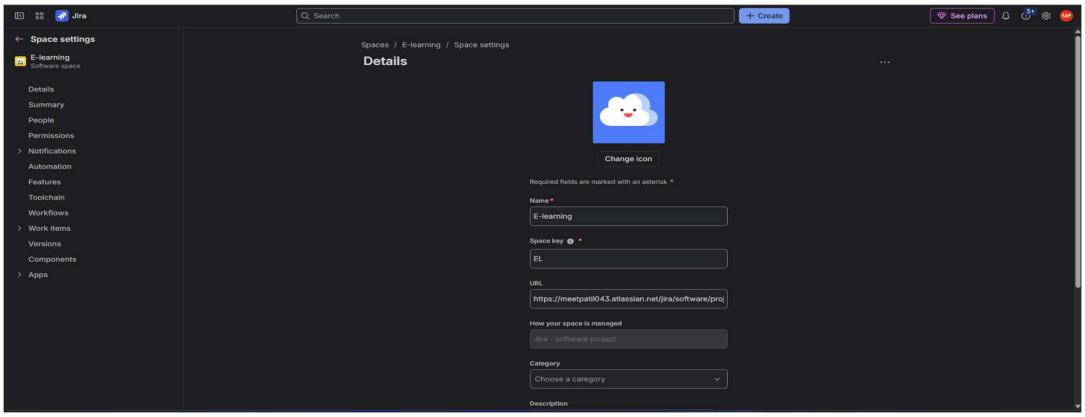
Step 1: Open the newly created project.

Step 2: Go to Project settings.



Step 3: Configure the project name and other basic details.

Step 4: Review the project settings and save the changes.

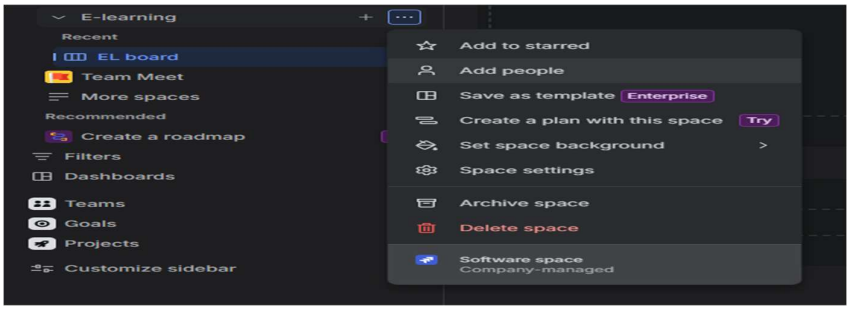


Practical No 2.
Team Creation & Role Assignment

A. Add Team Members

Step 1: Open E-learning → Click on the three dots (...).

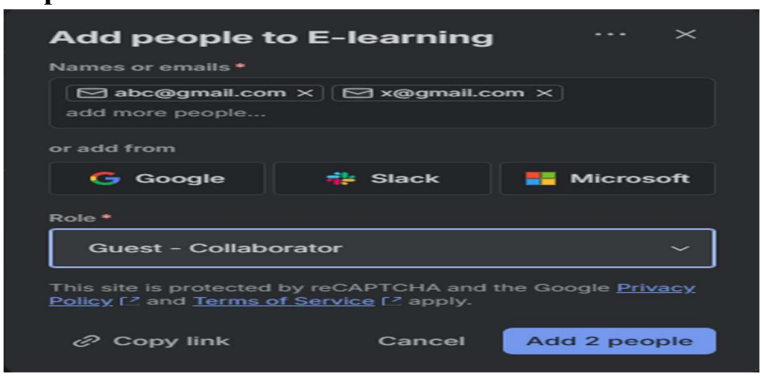
Step 2: Click on Add people.



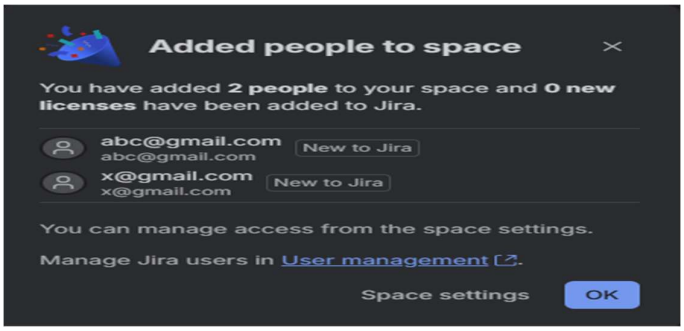
Step 3: Enter the name or email address.

Step 4: Select the required role.

Step 5: Click on Add.



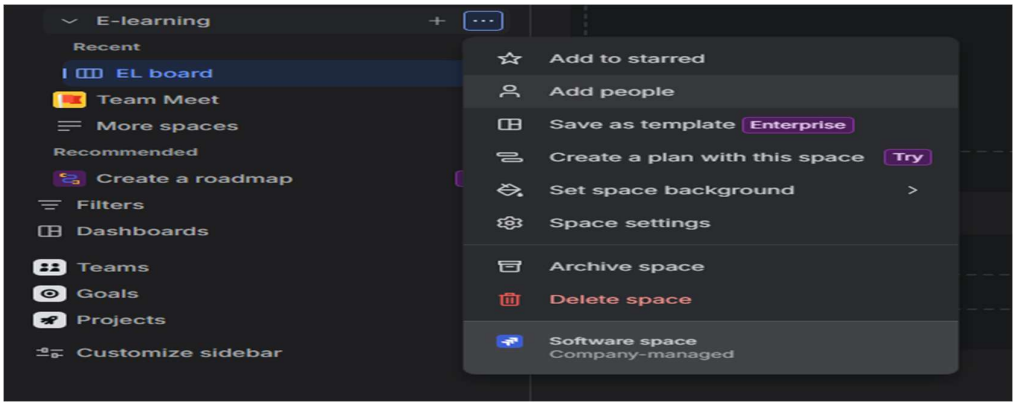
Step 6: Click on OK.



B. Assign Roles

Step 1: Open E-learning → Click on the three dots (...).

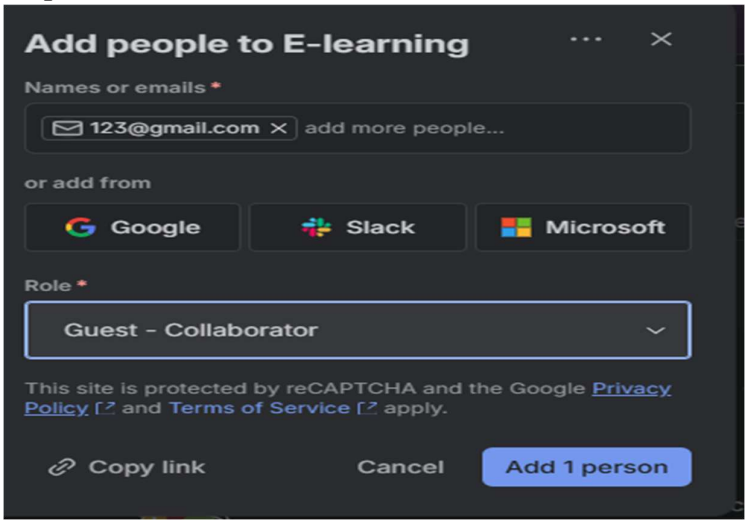
Step 2: Click on Add people.



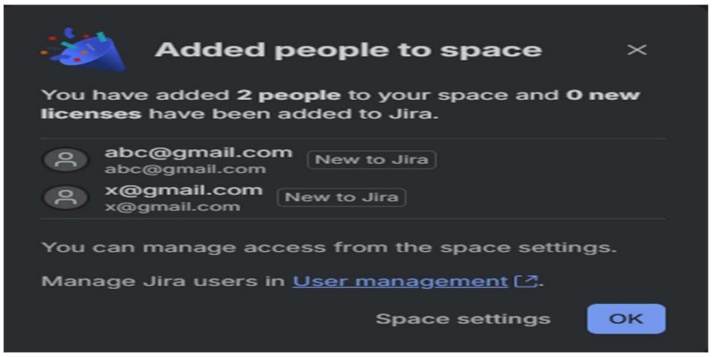
Step 3: Enter the name or email address.

Step 4: Select the required role.

Step 5: Click on Add.



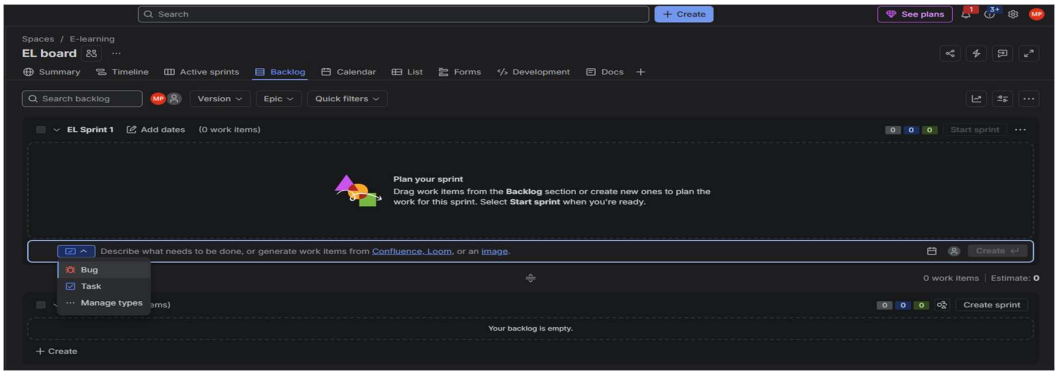
Step 5: Click on OK.



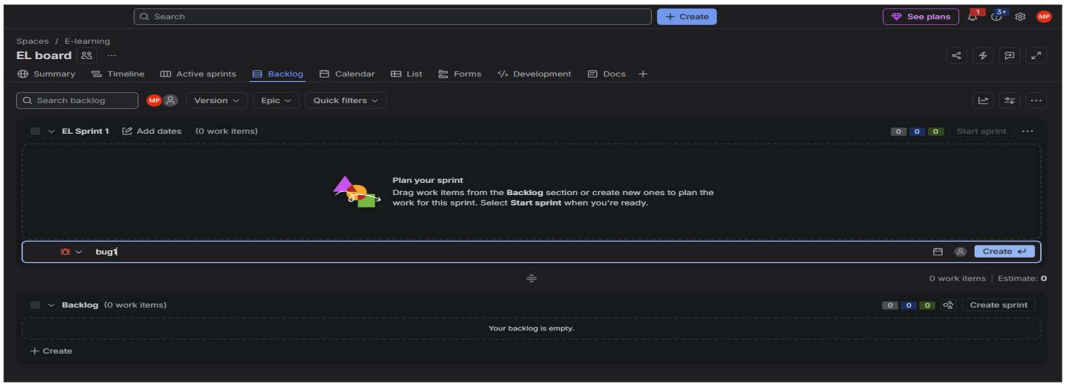
Practical No 3.
Issue Types & Configuration

A. Create and configure Issue Types (Epic, Story, Task, Bug)

Step 1: Click on Backlog. → Click on Create (+).

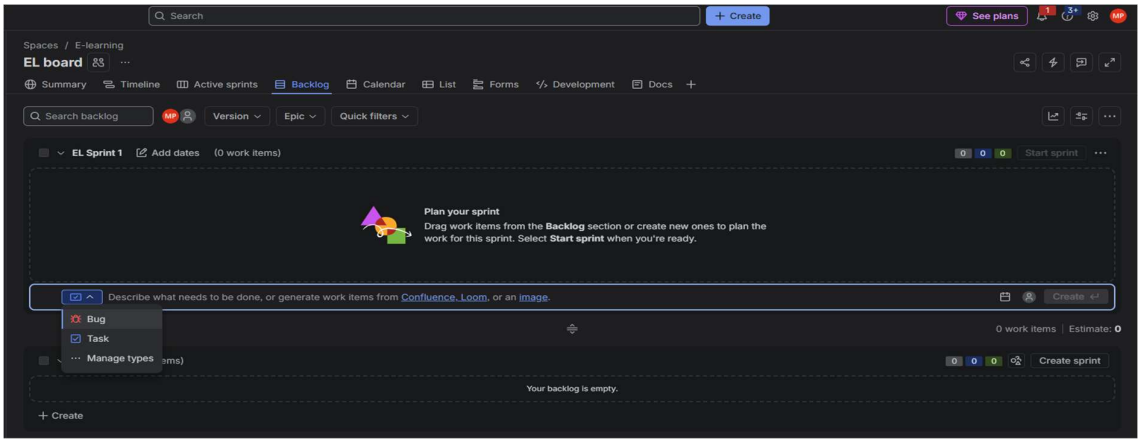


Step 2: Select Bug. → Enter the Bug Description.

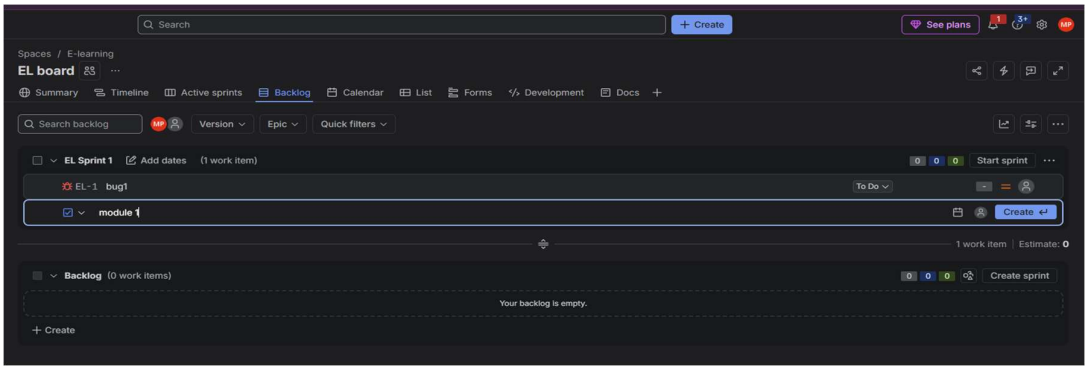


Step 3: Click on Create.

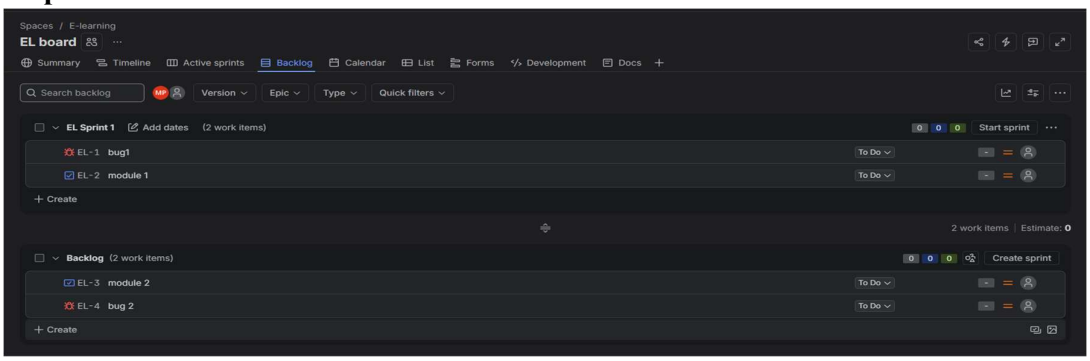
Step 4: Click on Create (+) again. → Select Task.



Step 5: Enter the Task Description.



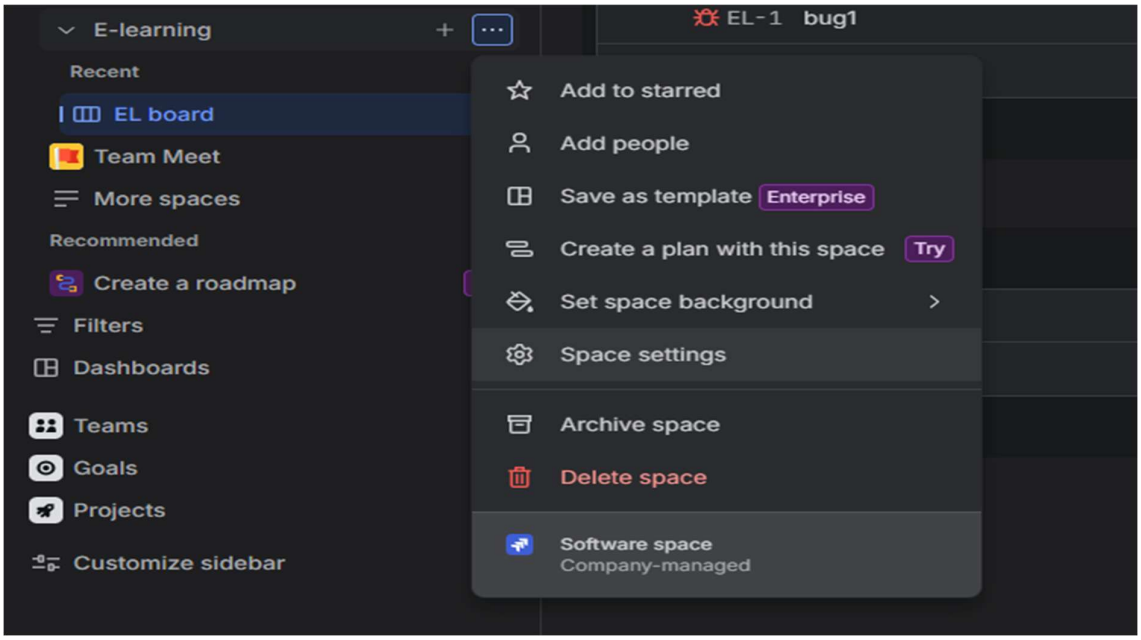
Step 6: Click on Create.



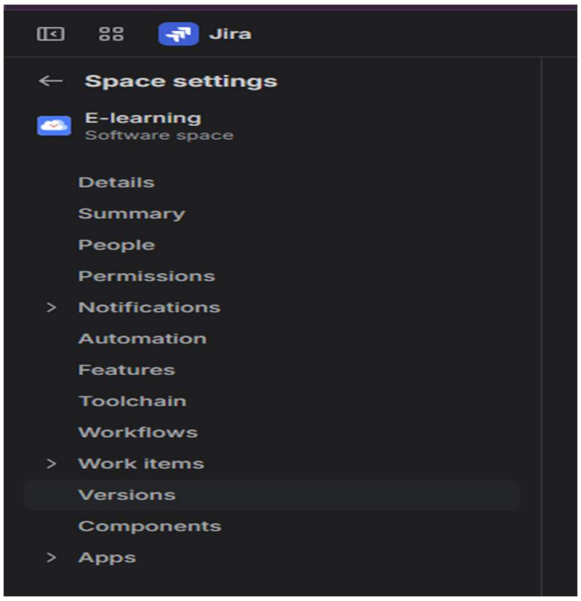
B. Create Components and Versions.

Step 1: Open E-learning → Click on the three dots (...).

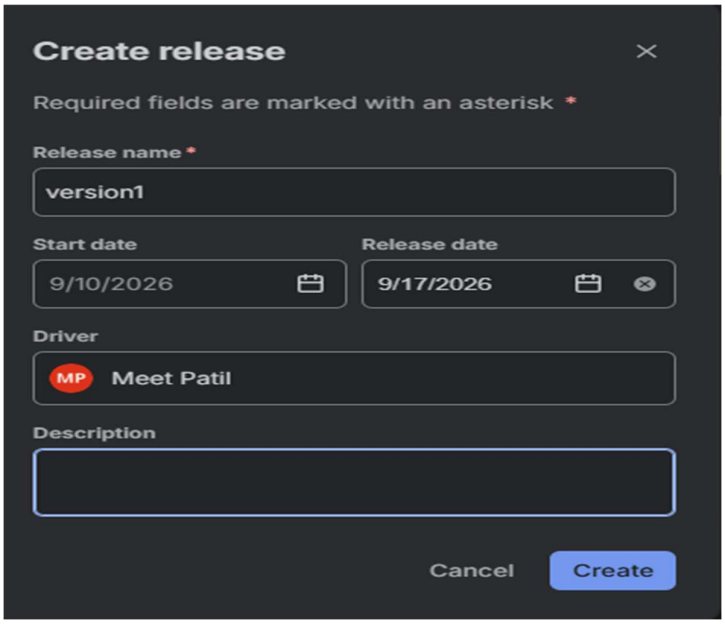
Step 2: Click on Space settings.



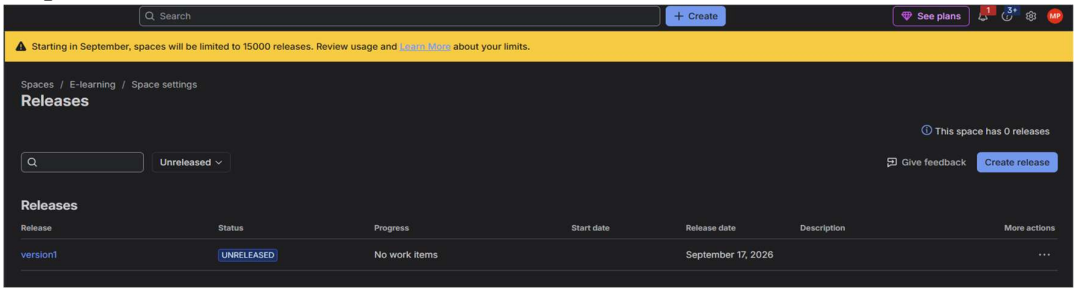
Step 3: Click on Versions.



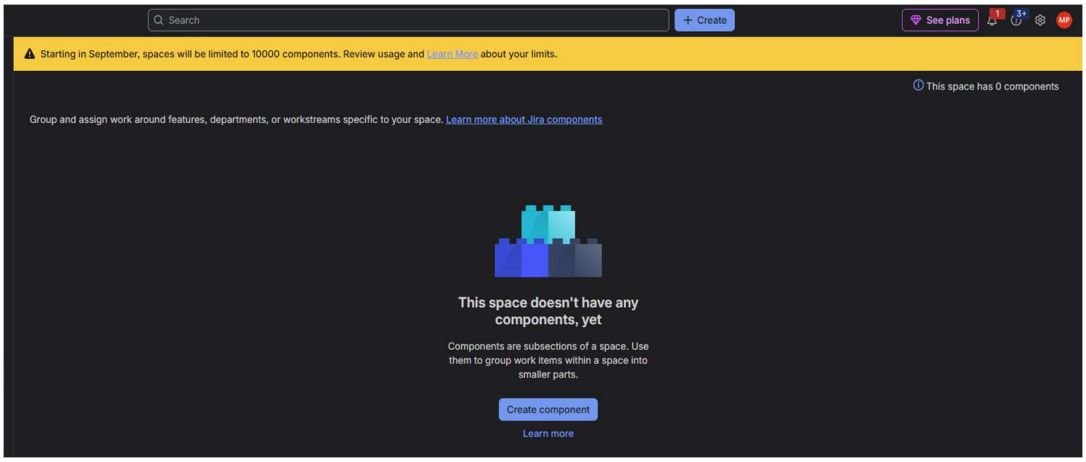
Step 4: Enter the Release Name, Start Date and Release Date.



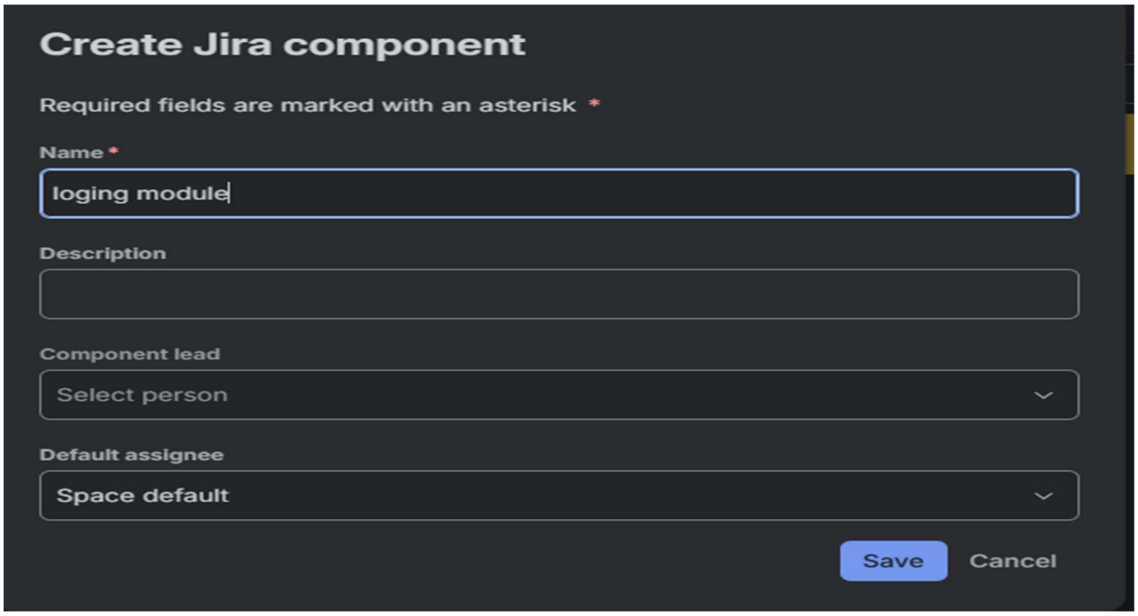
Step 5: Click on Create.



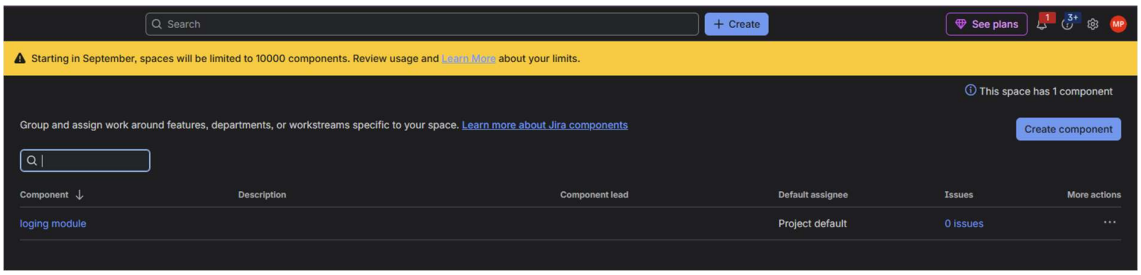
Step 6: Click on Components. → Click on Create component.



Step 7: Enter the Component Name.



Step 8: Click on Save.

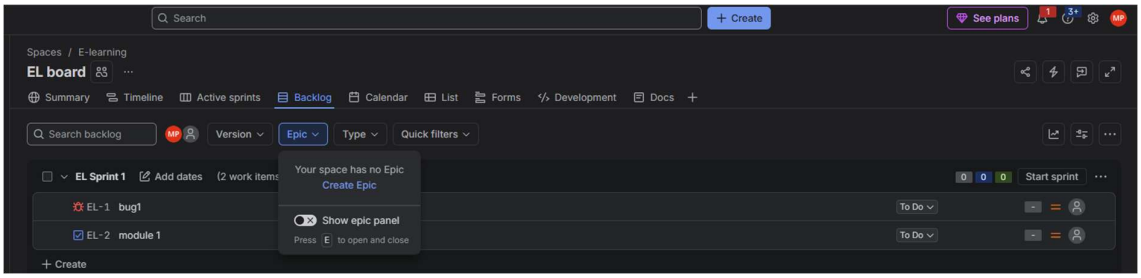


Practical No 4.
Issue Management

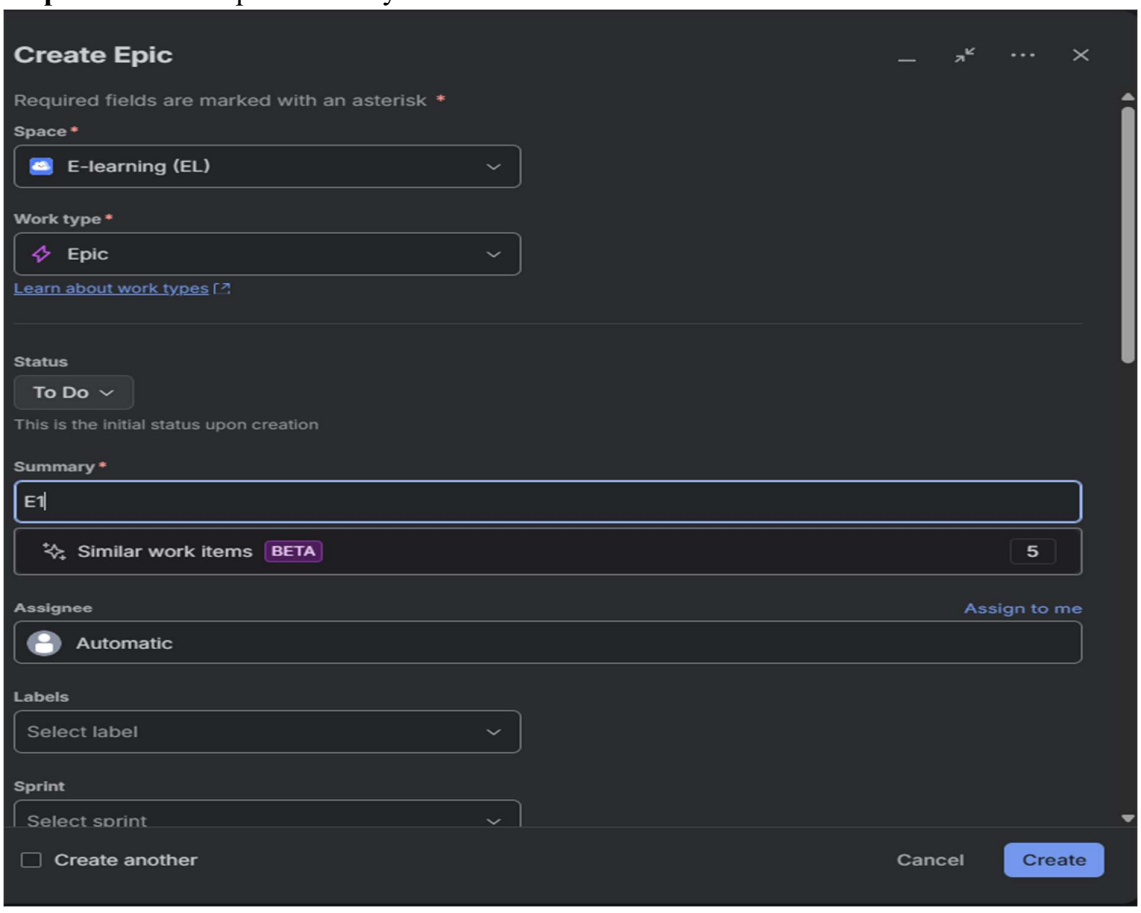
A. Create issues under an Epic.

Step 1: Open Backlog. → Click on Epic.

Step 2: Click on Create Epic.



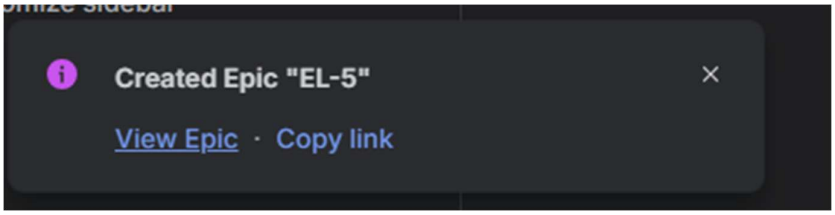
Step 3: Enter the Epic Summary.



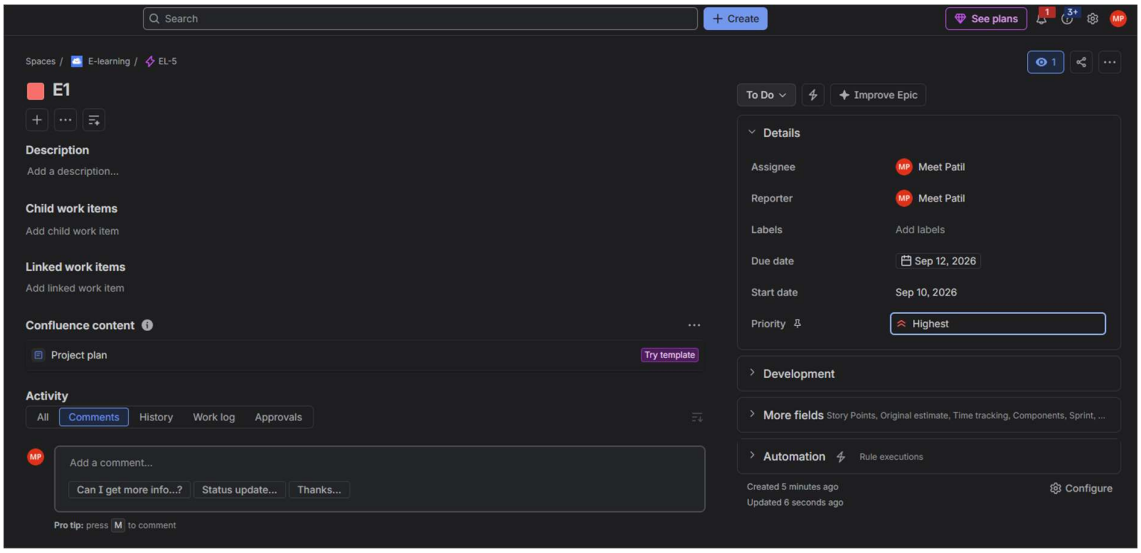
Step 4: Click on Create.

B. Assign issues to users and set Priority & Due Date.

Step 1: Click on View Epic.



Step 2: Enter the required Epic Information.



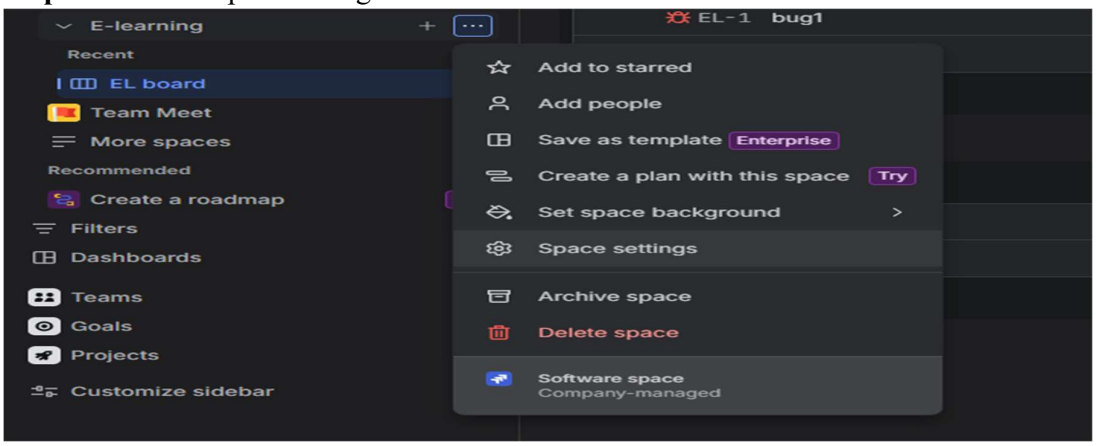
Step 3: Save the information.

Practical No. 5
Workflow Customization

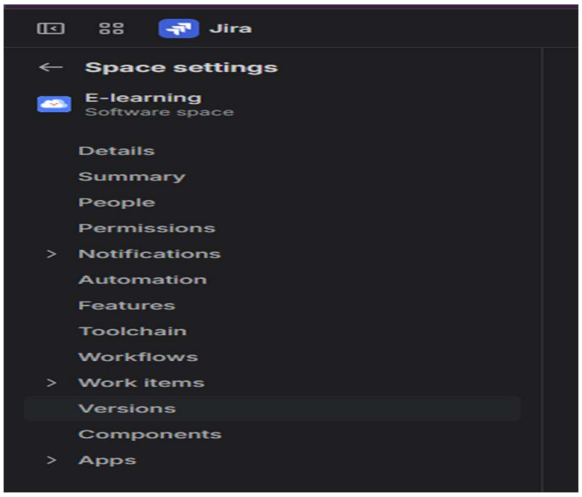
A. Design a custom workflow

Step 1: Go to E-learning. → Click on the three dots (...).

Step 1: Click on Space settings.

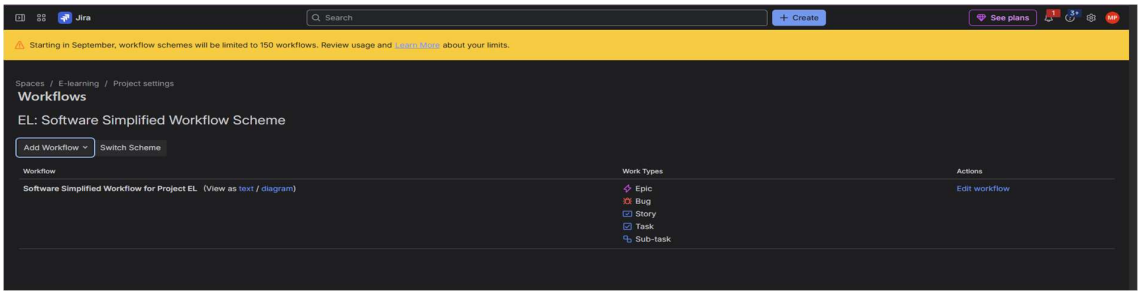


Step 2: Click on Workflows.



Step 3: Click on Add workflow.

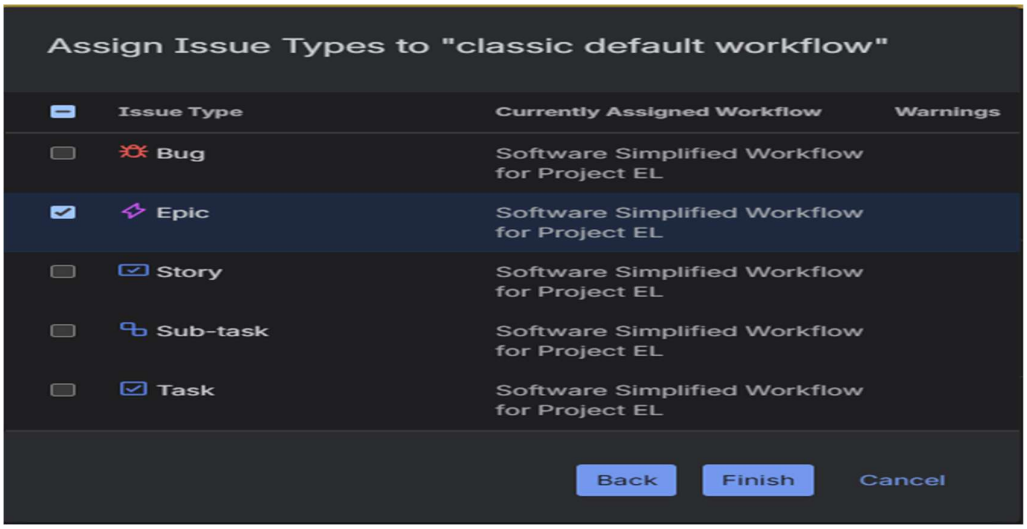
Step 4: Select Add to existing workflow.



Step 5: Select Classic Default Workflow. → Click on Next.



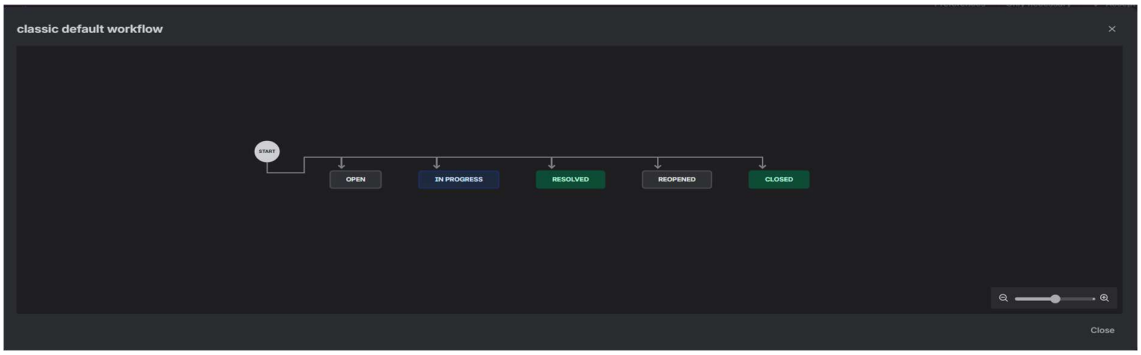
Step 6: Select the required Epic.



Step 7: Click on Next.

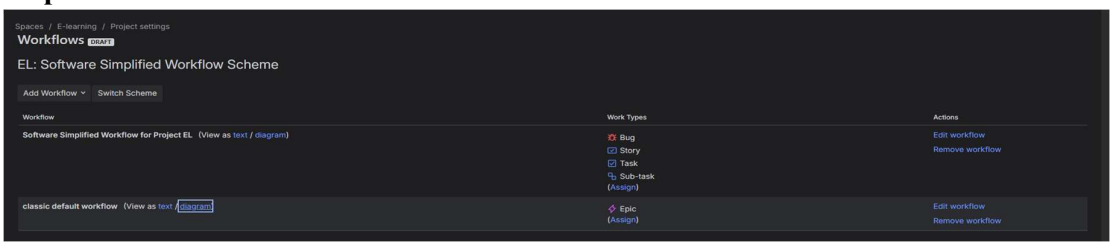


Step 8: Click on Diagram.

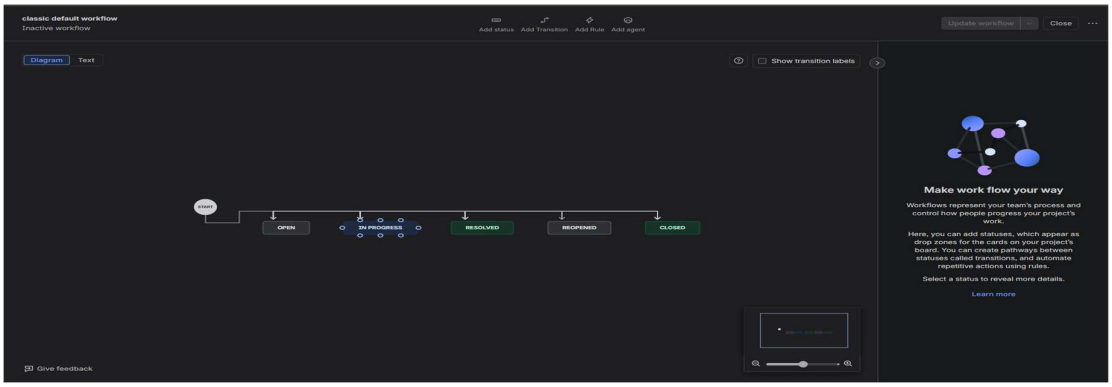


B. Add transitions and conditions.

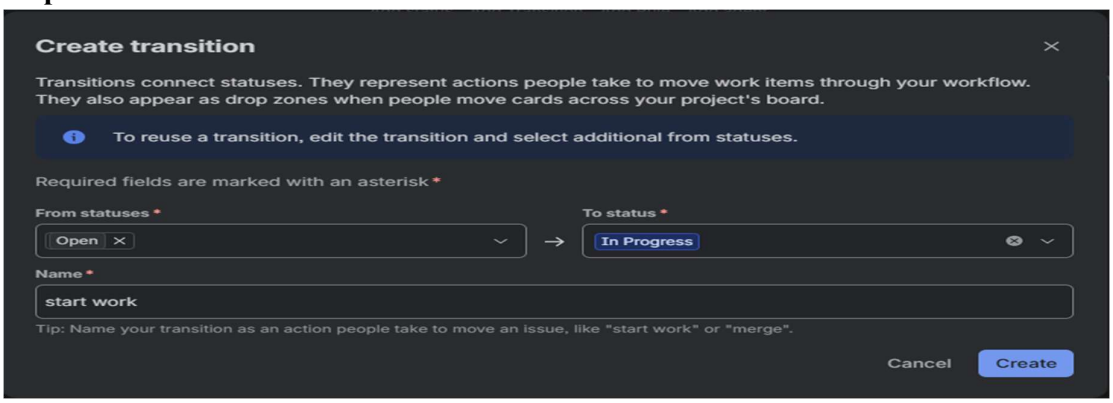
Step 1: Click on Edit workflow.



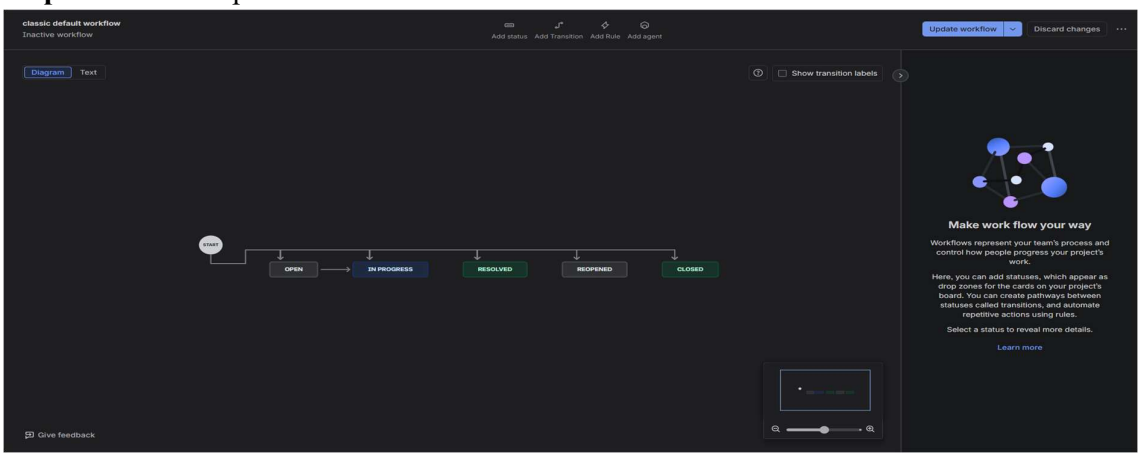
Step 2: Click on Add transition.



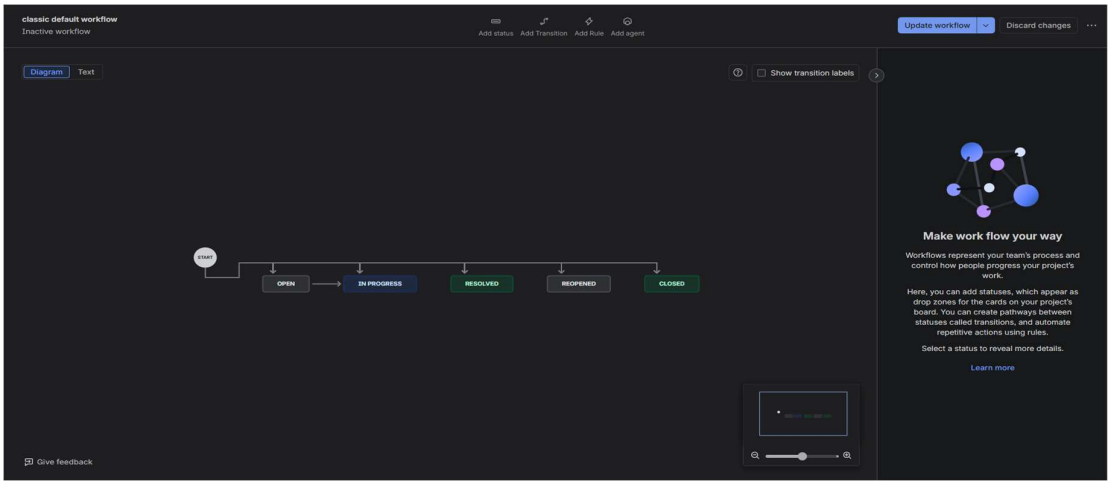
Step 3: Enter the Transition Name. → Click on Create.



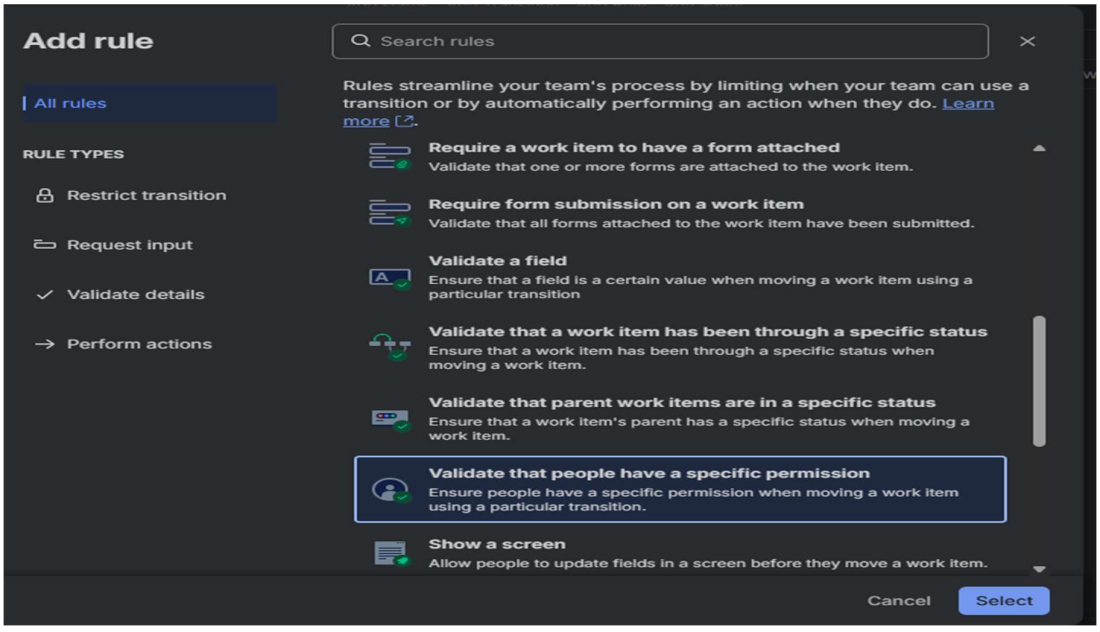
Step 4: Click on Update workflow.



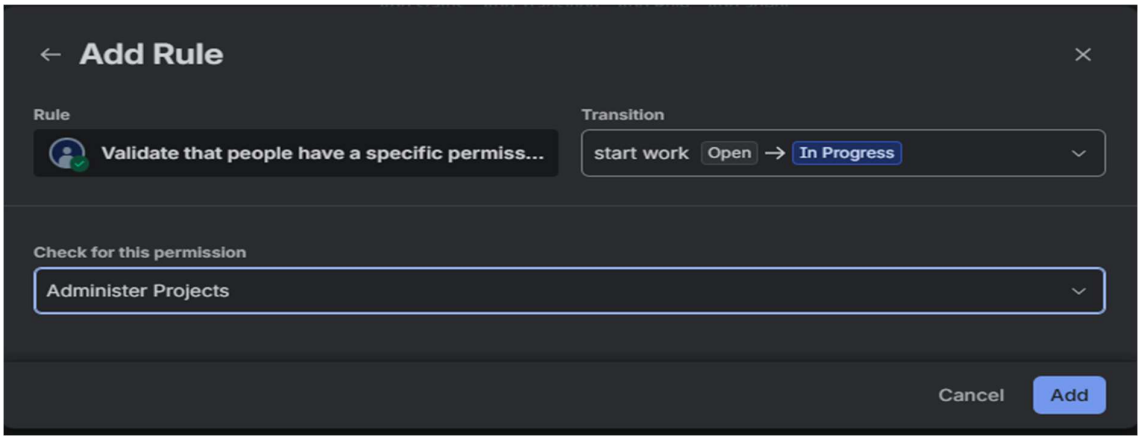
Step 5: Click on Add rule.



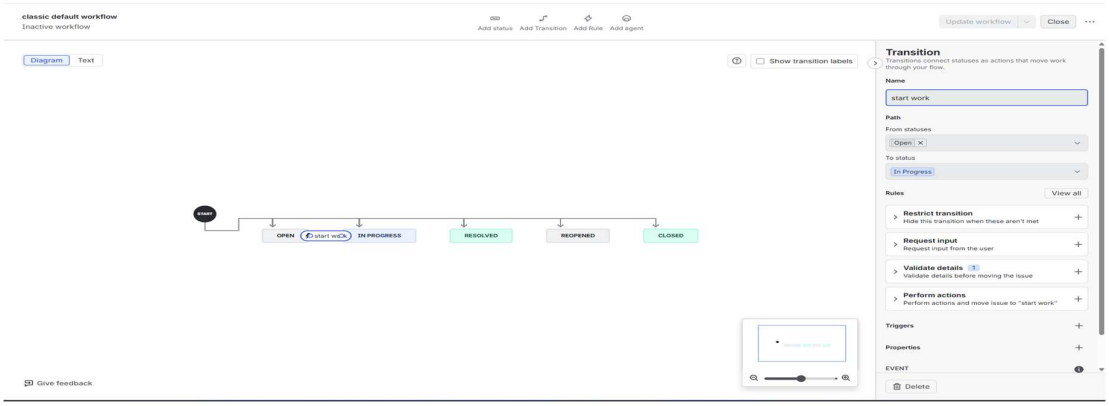
Step 6: Select Information. And Click on Select.



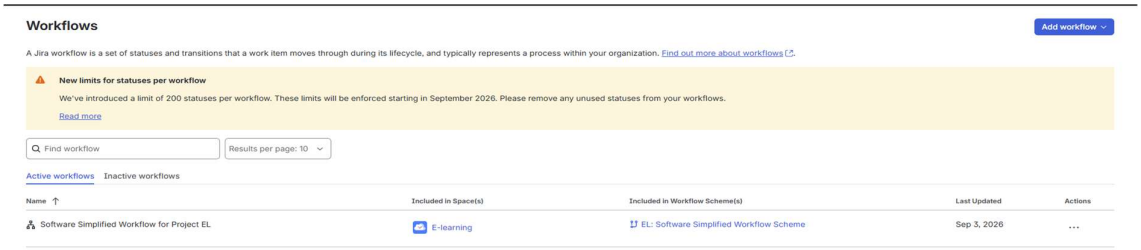
Step 7: Click on Add.



Step 8: Click on Update workflow.



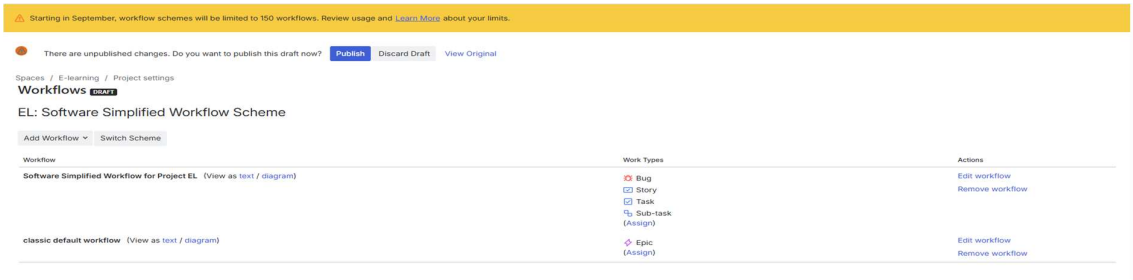
Step 9: Click on Close.



C. Publish and associate workflow with a project.

Step 1: Click on Publish.

Step 2: Select Associate.



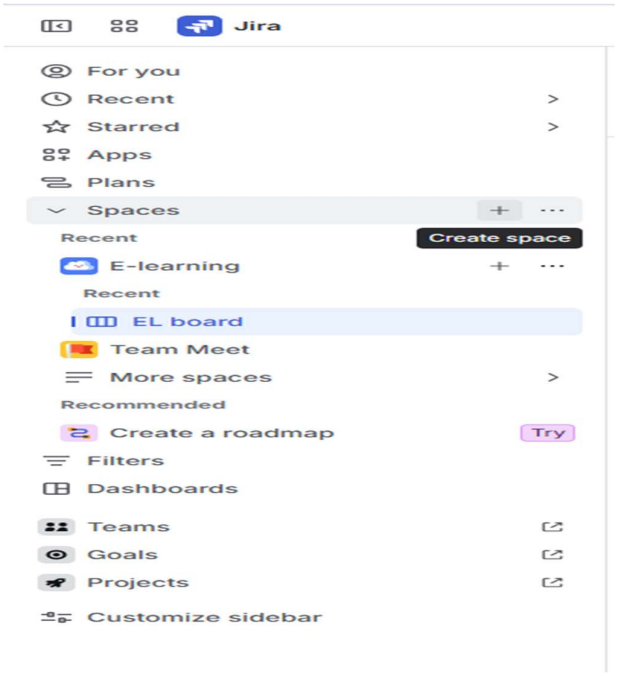
Step 3: Click on Update workflow.



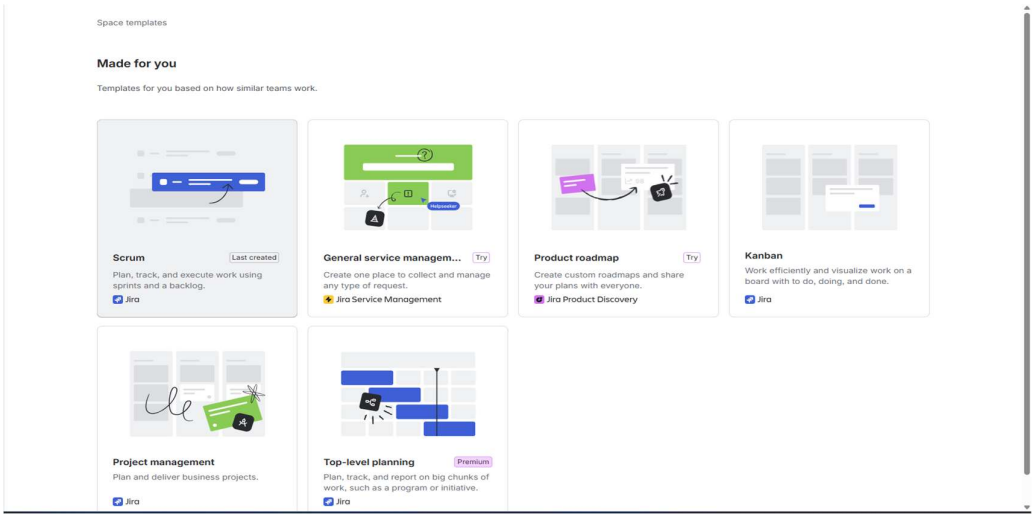
Practical No 6.
Scrum Board & Sprint Management

A. Create and configure Scrum Board.

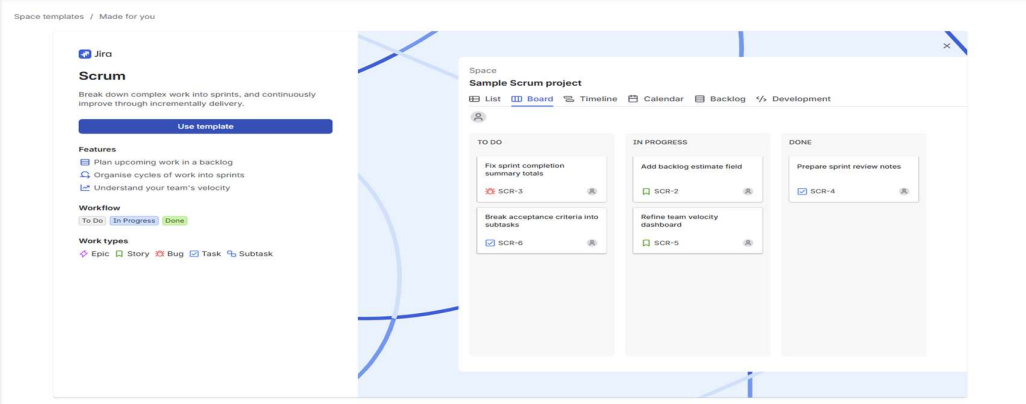
Step 1: Click on Spaces. → Click on the (+) Add sign.



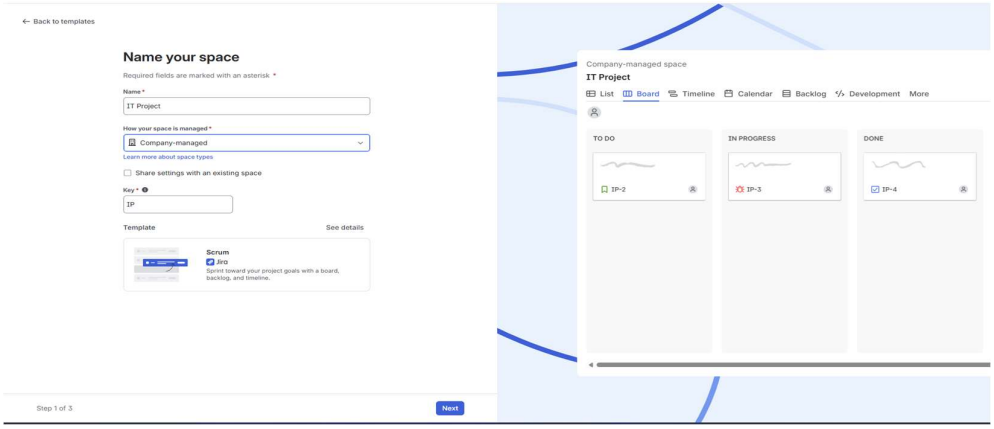
Step 2: Select Scrum.



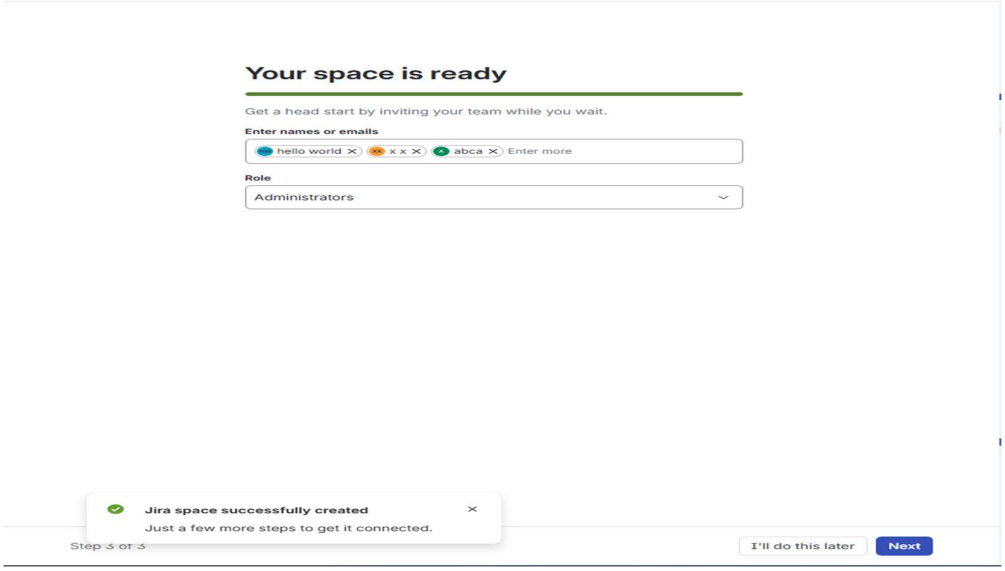
Step 3: Select the Use Template.



Step 4: Enter the Space Name. → Click on Next.

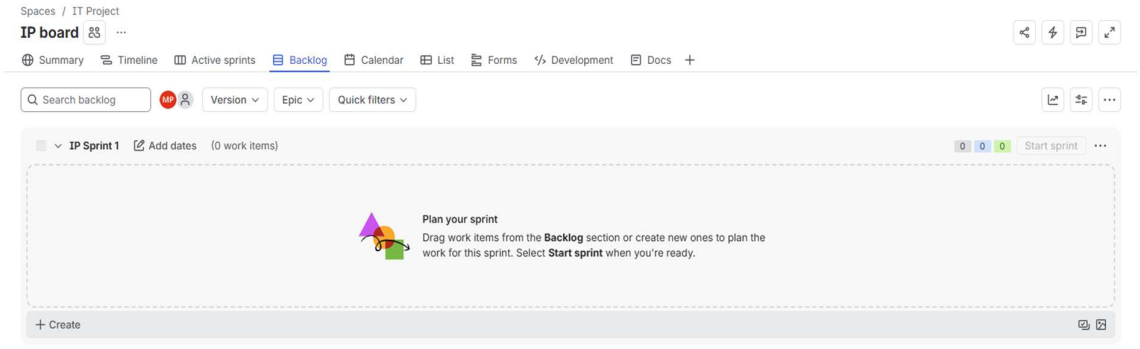


Step 5: Click on Next.

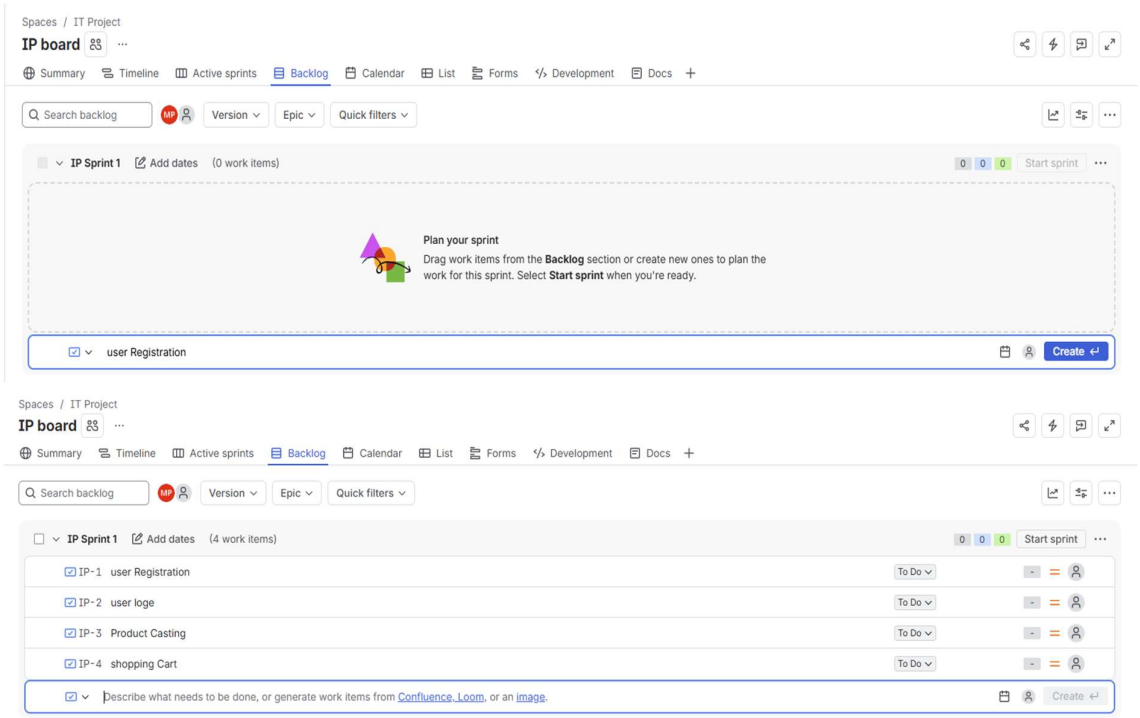


B. Create and start a Sprint.

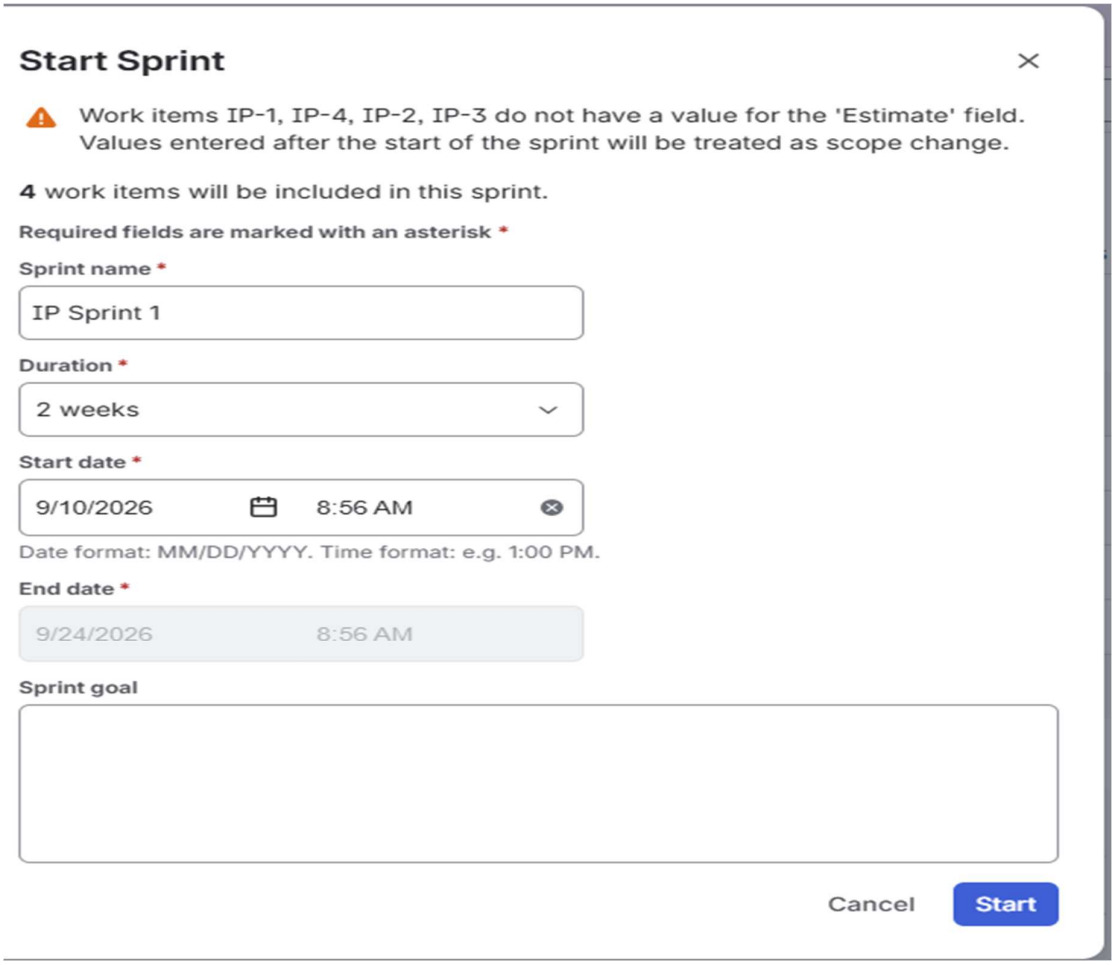
Step 1: Open the Sprint page. → Click on Create.



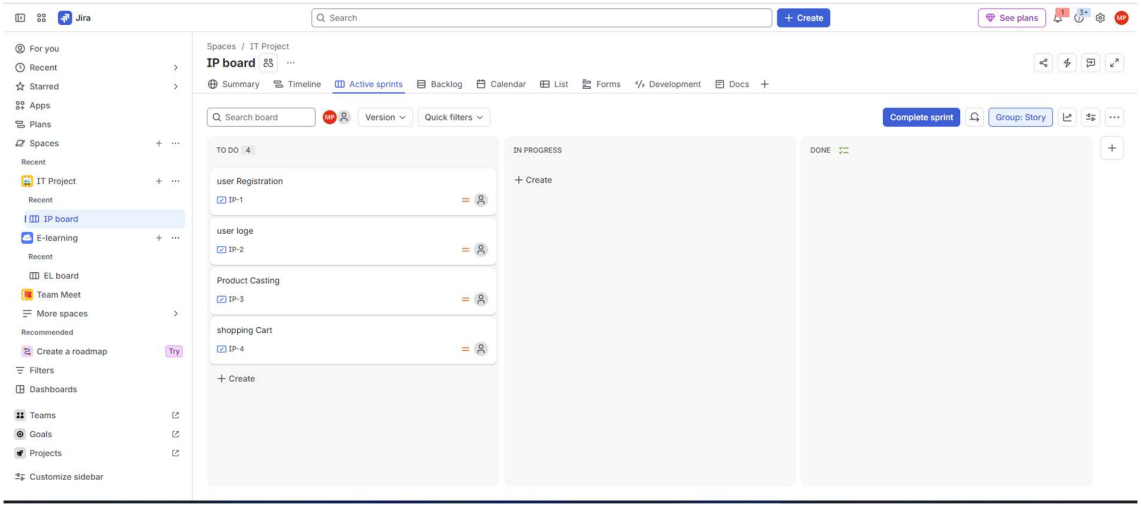
Step 2: Enter the Sprint Description. → Click on Create.



Step 3: Click on Start Sprint.

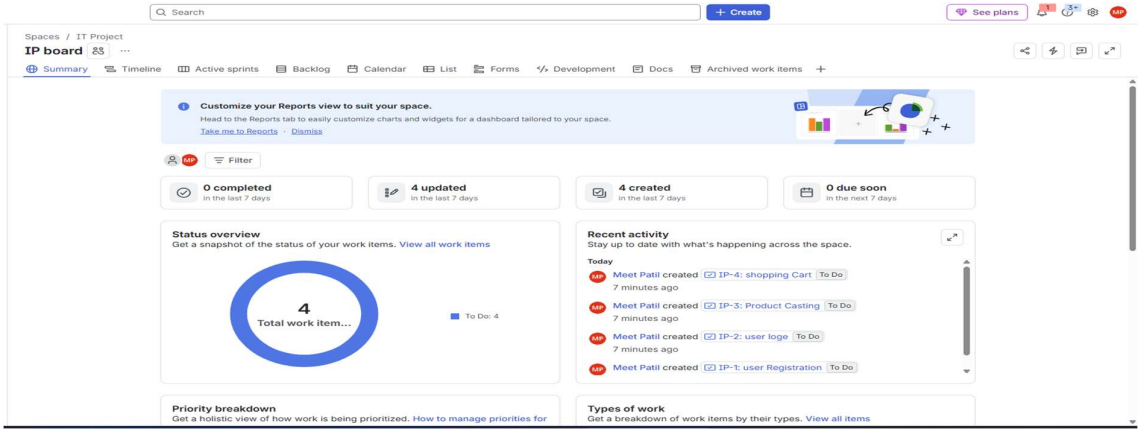


Step 4: Click on Start.

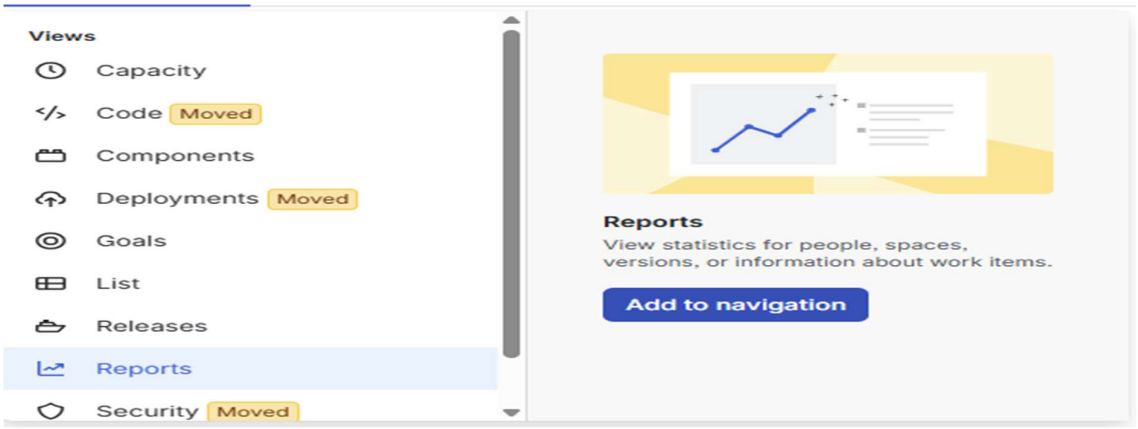


C. Complete Sprint and generate Sprint Report

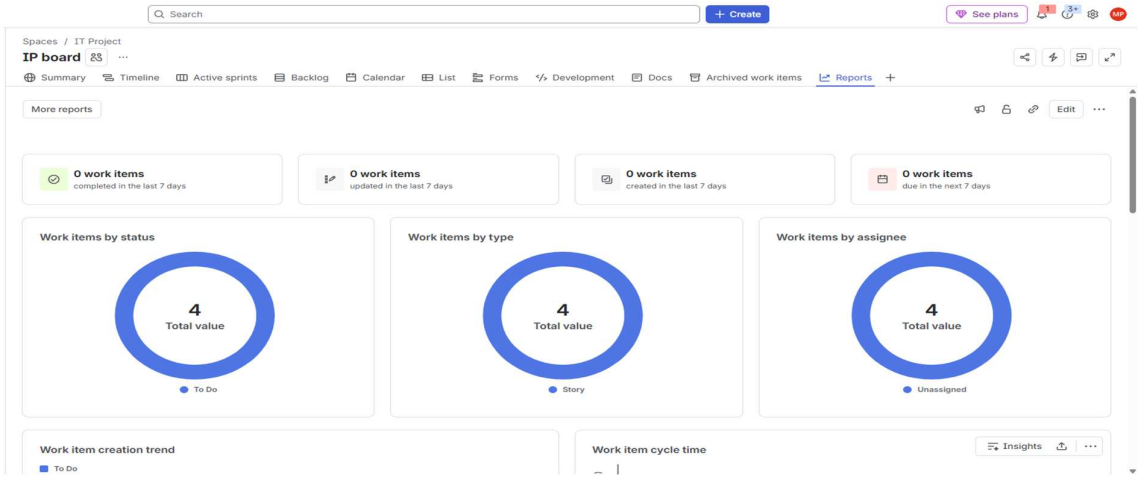
Step 1: Open the Summary page. → Click on the (+) Add sign.



Step 2: Select Report.



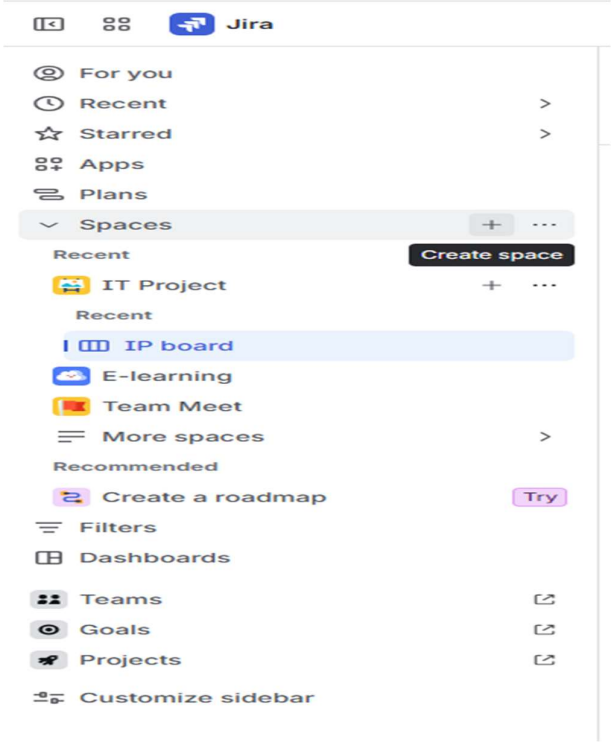
Step 3: Click on Add to Navigation.



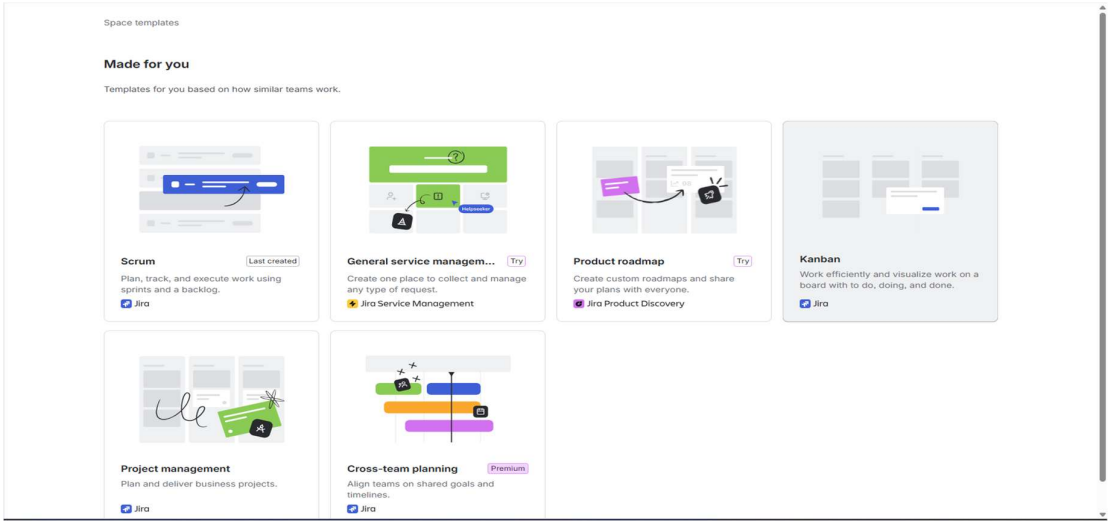
Practical No. 7
Kanban Board Management

A. Create a Company-managed Kanban Project.

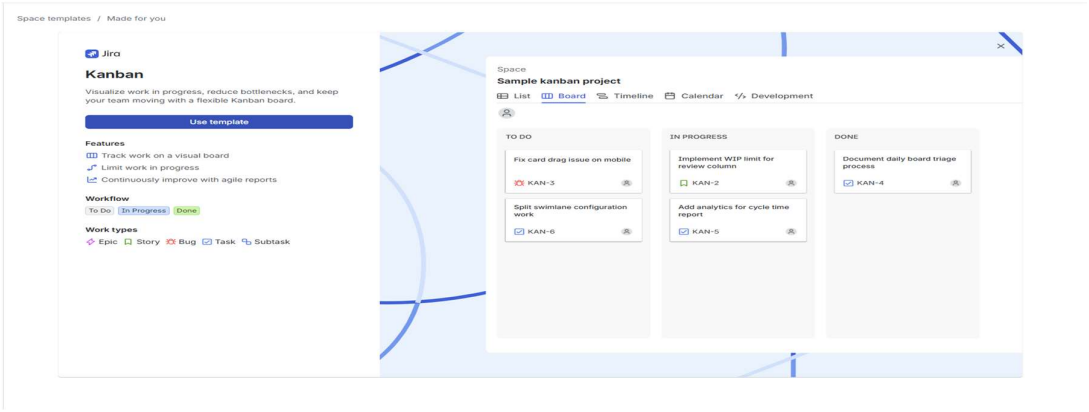
Step 1: Click on Spaces. → Click on the (+) Add sign.



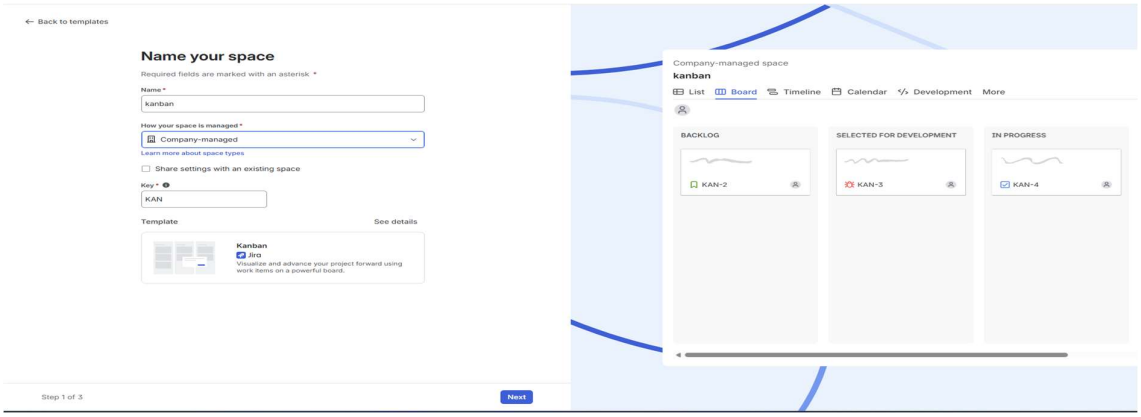
Step 2: Select Kanban.



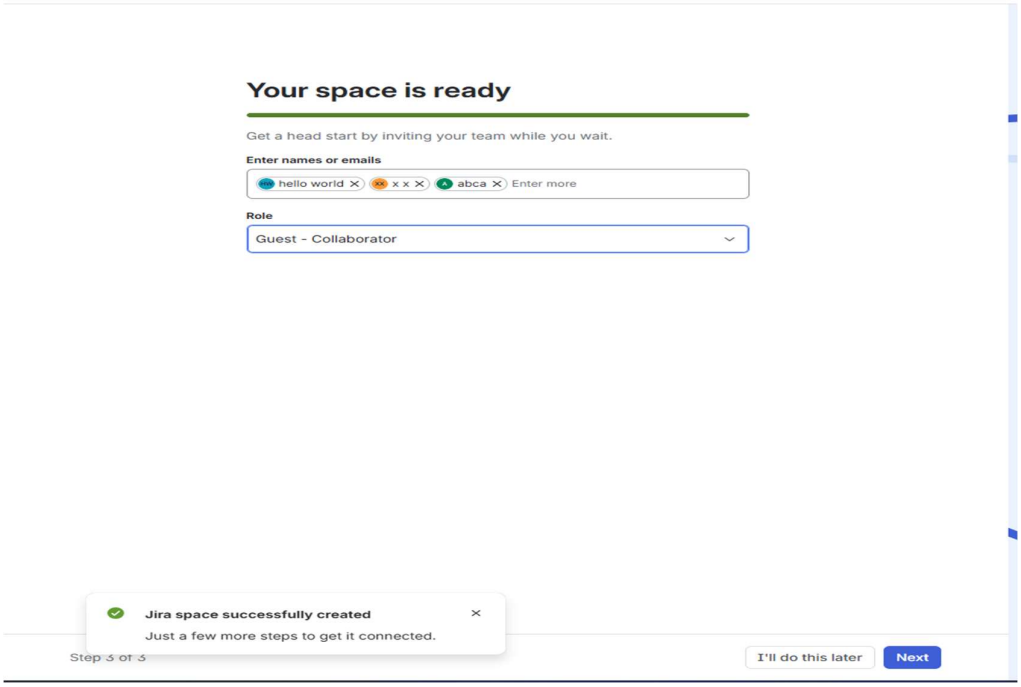
Step 3: Select the Use Template.



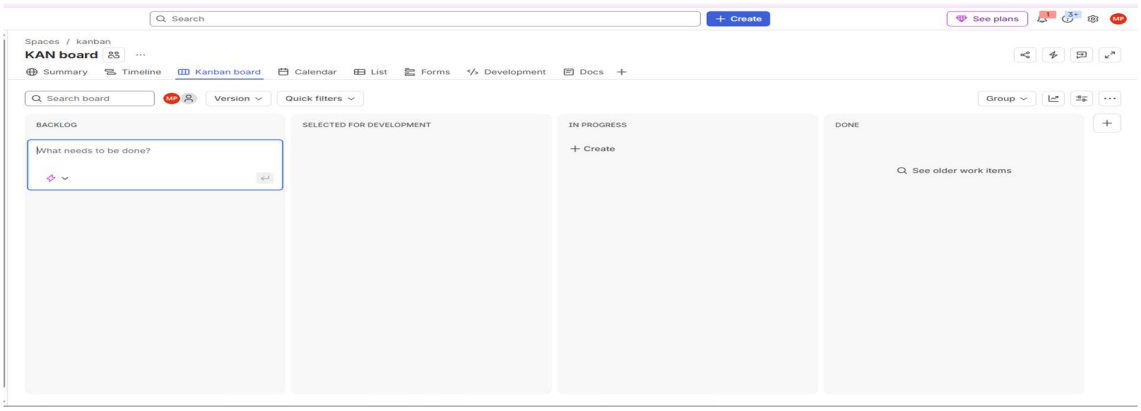
Step 4: Enter the Space Name. → Click on Next.



Step 5: Select Administrators.

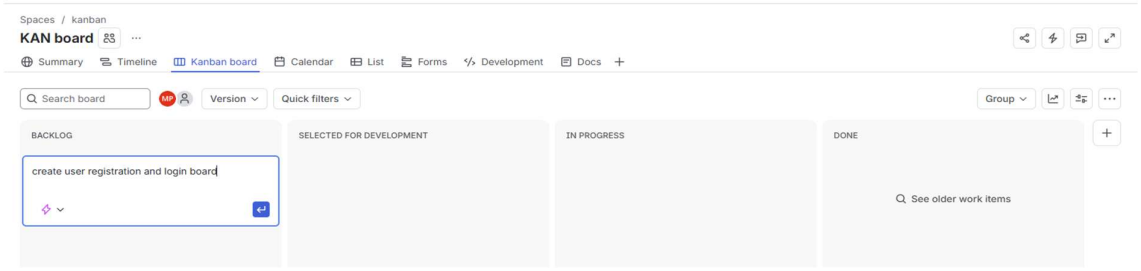


Step 6: Click on Next.

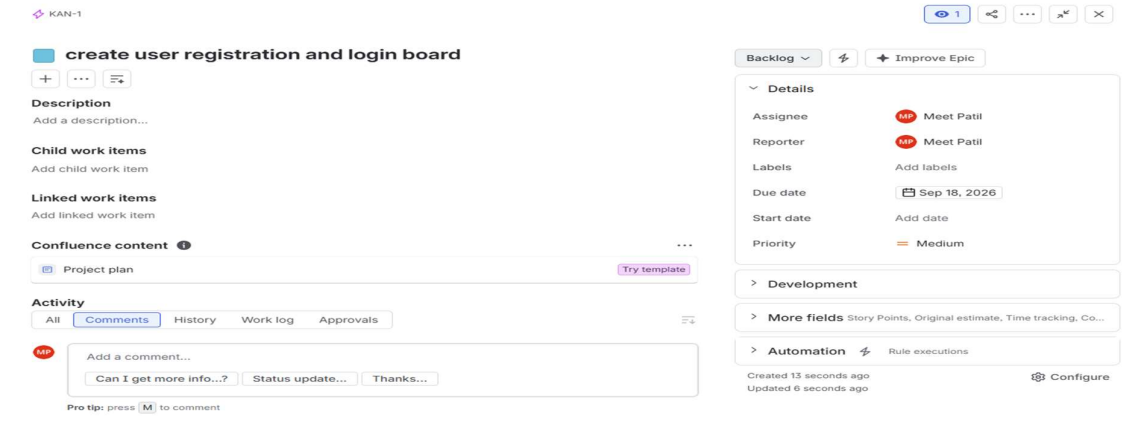


B. Configure Kanban Board

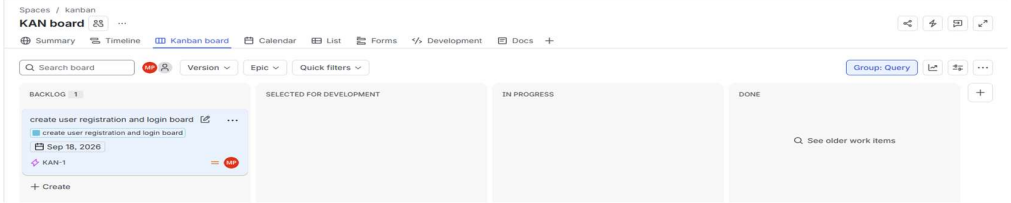
Step 1: Open the Backlog.



Step 2: Enter the Item Name. And Click on Create.



Step 3: Close the created item.

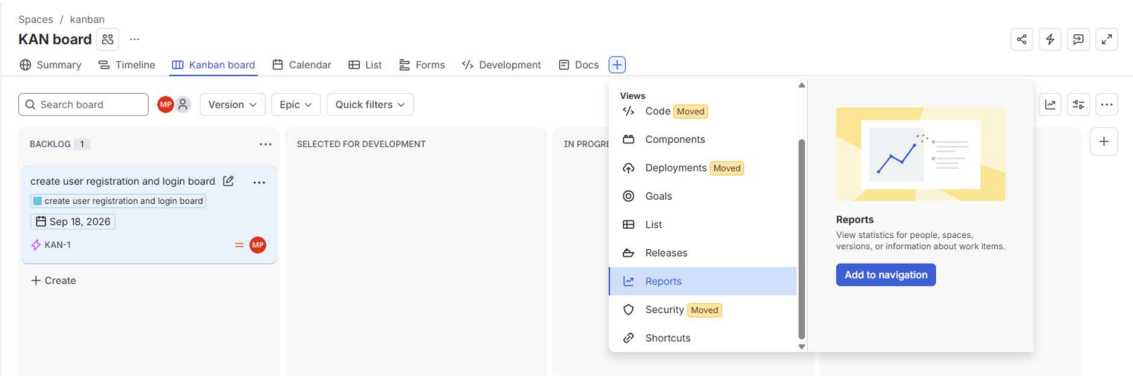


C. Track issue progress and generate Control Chart

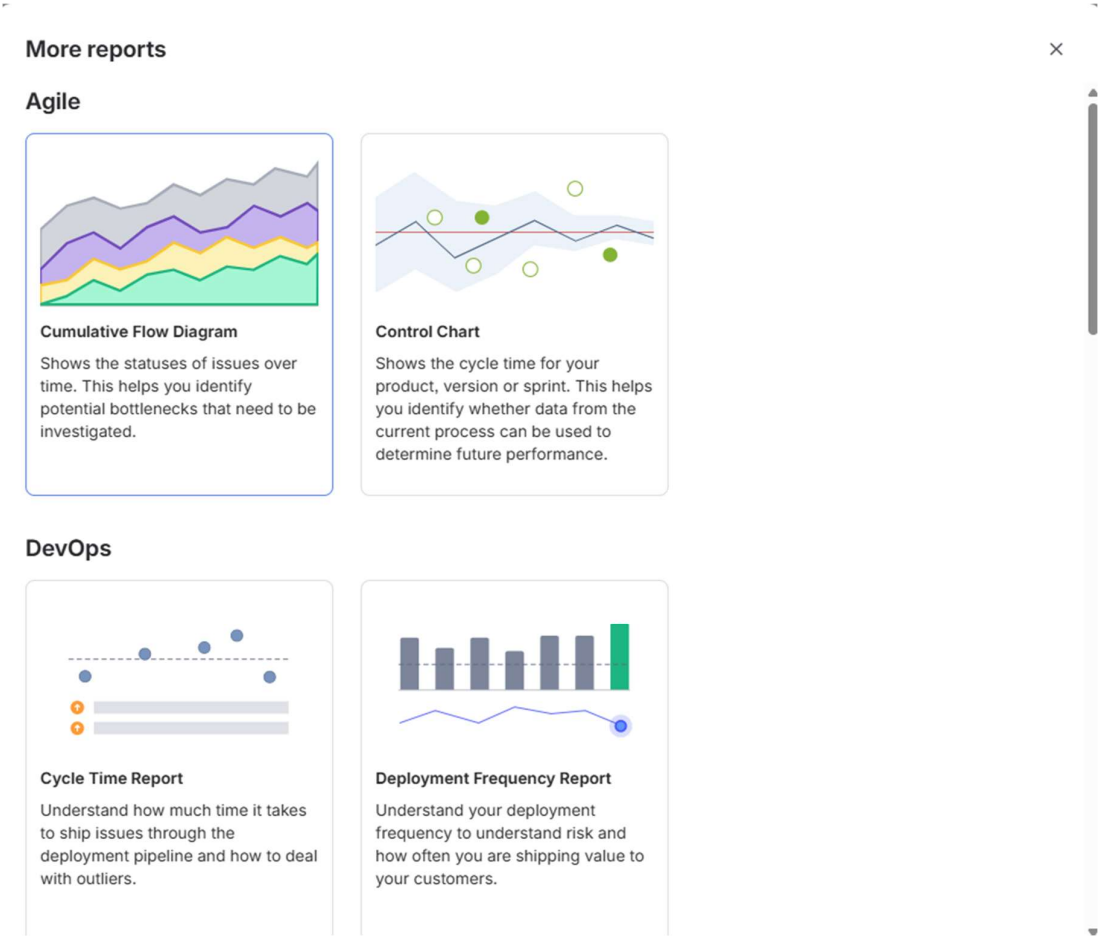
Step 1: Click on the Kanban Board.

Step 2: Click on the (+) Add sign. → Select Reports.

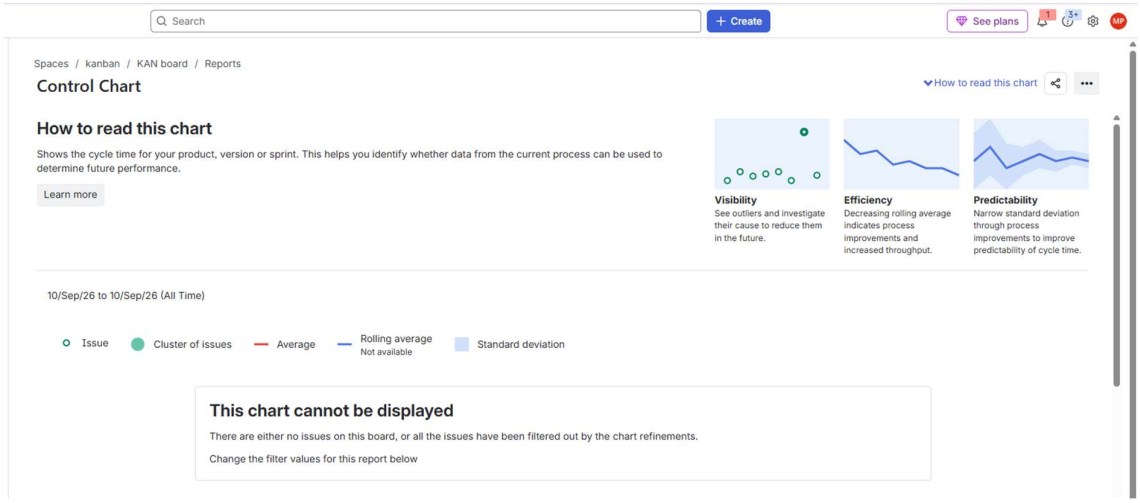
Step 3: Click on Add to Navigation.



Step 4: Click on More Reports



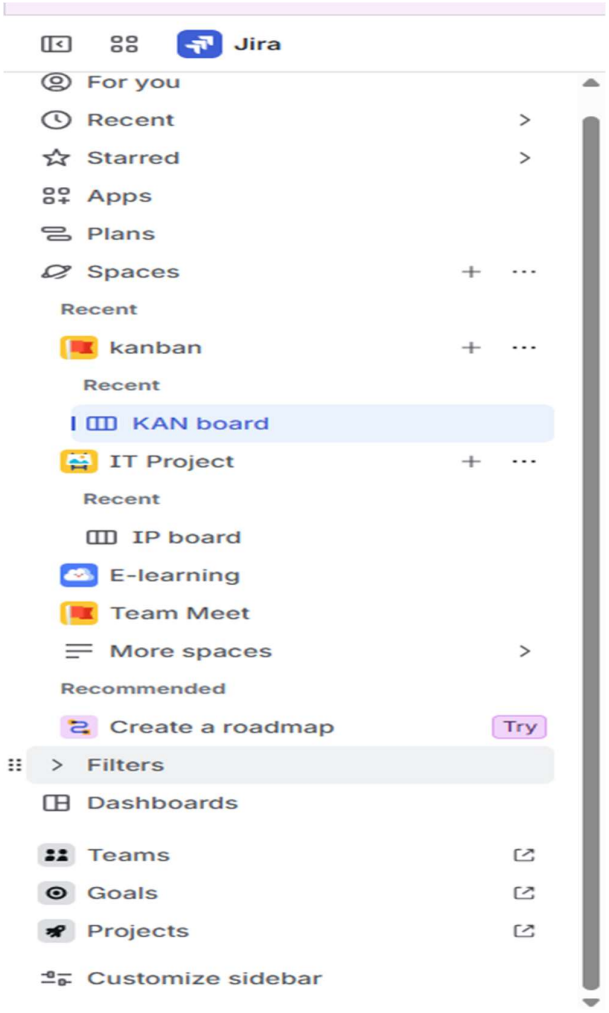
Step 5: In Agile Select the Cumulative Flow Diagram.



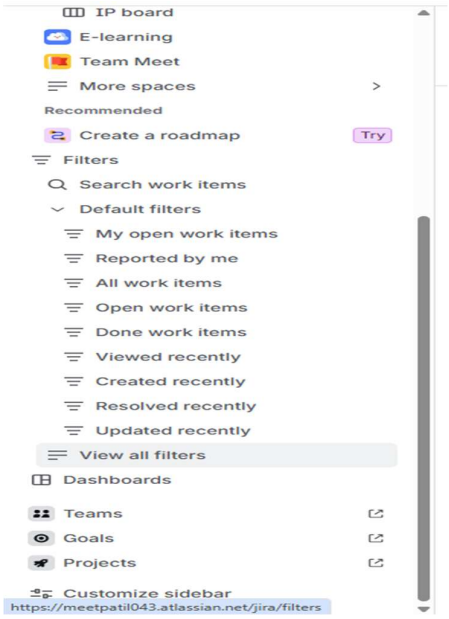
Practical No. 8
JQL & Filters

A. Write basic JQL queries (Status, Assignee, Priority).
B. Create and save filters.

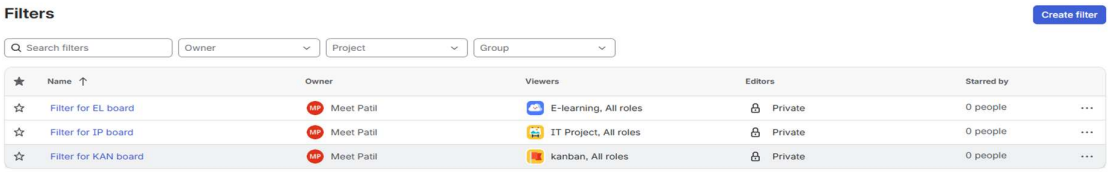
Step 1: Click on Filters.



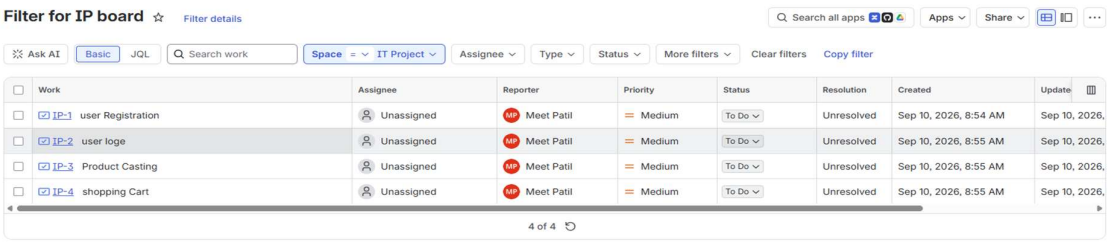
Step 2: Click on View all filters.



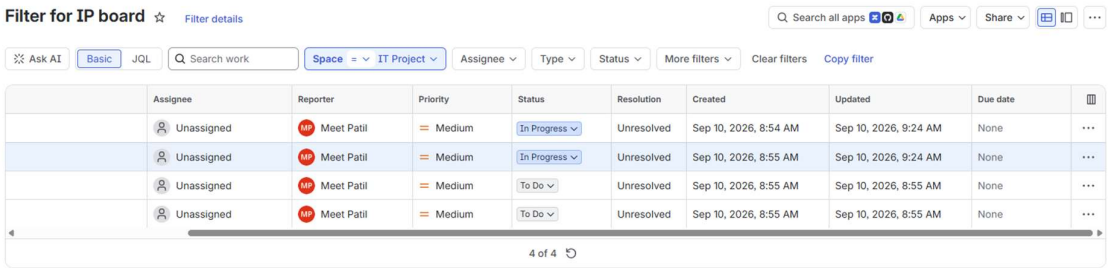
Step 3: Select the One Project.



Step 4: Click on IP-1.

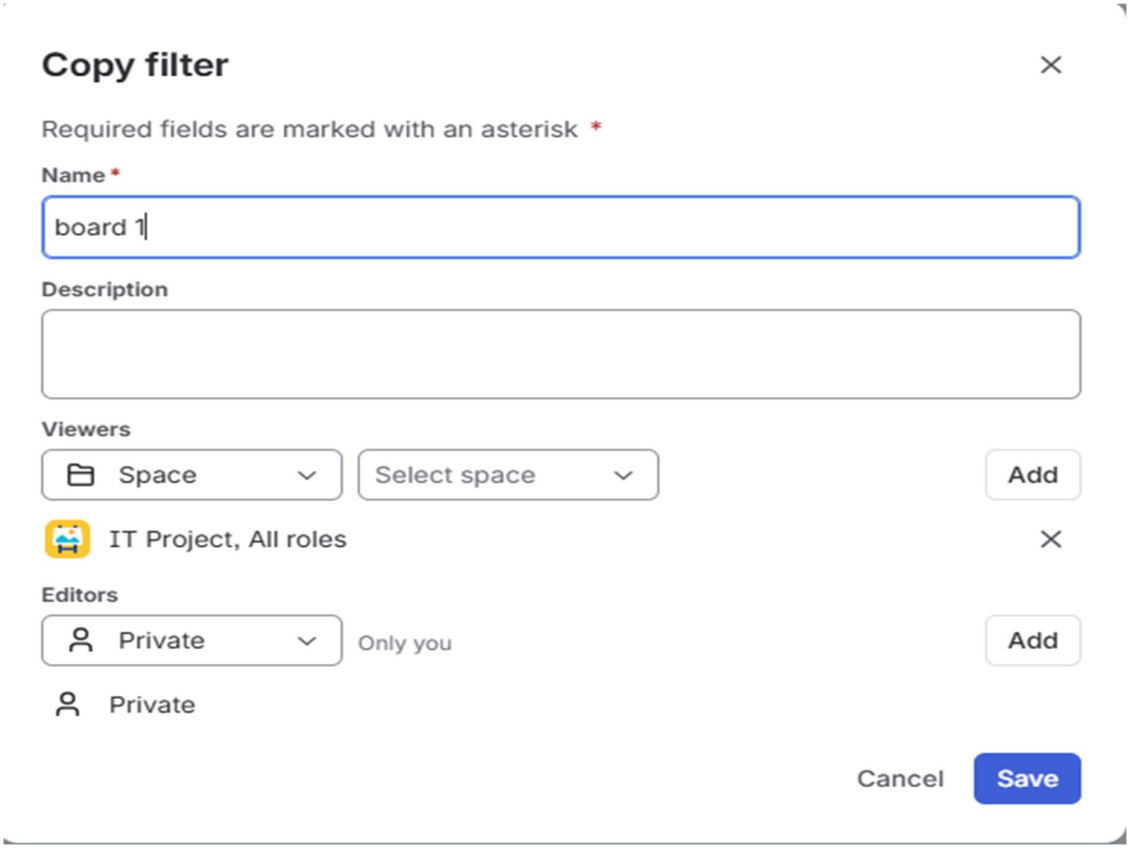


Step 5: Set the required Status.

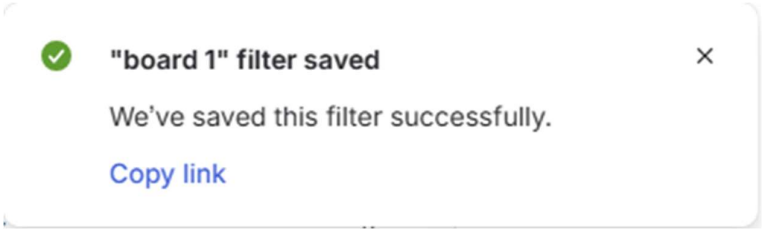


Step 6: Click on Copy Filter.

Step 7: Enter the Filter Name.



Step 8: Click on Save.

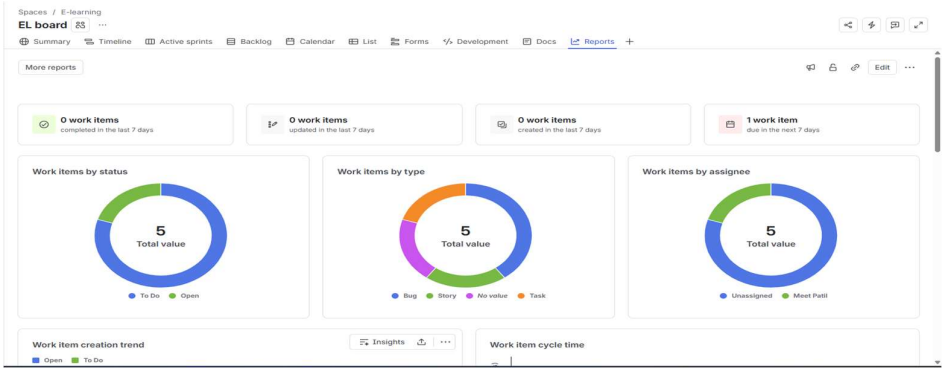


Practical No. 9
Reports & Dashboards

A. Generate Burndown Chart and Velocity Chart.

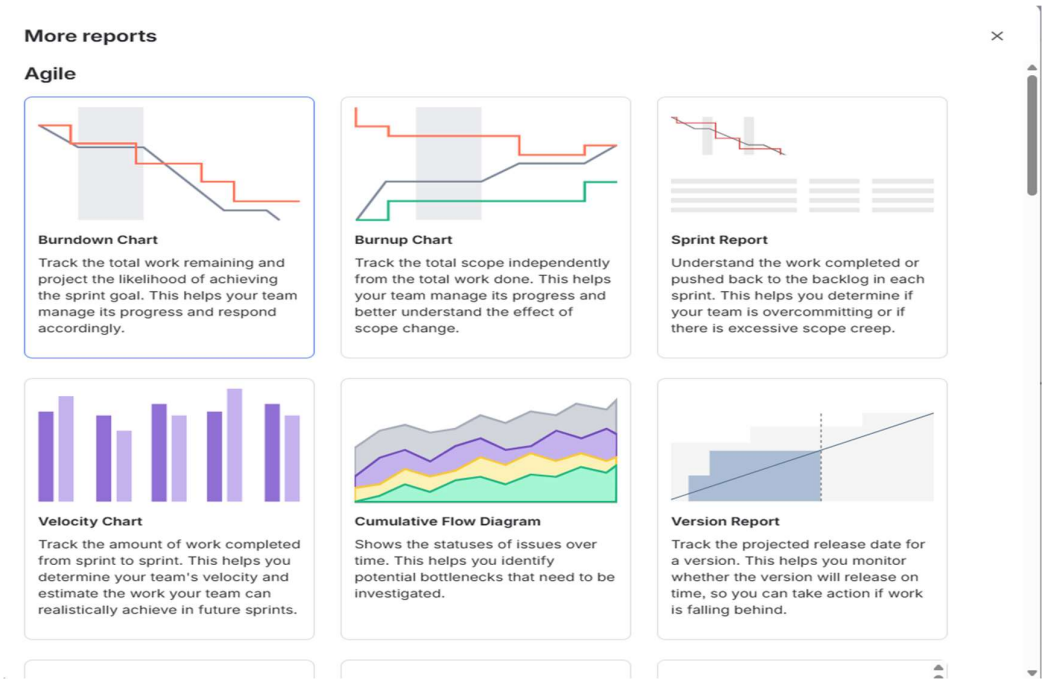
Step 1: Open the Board.

Step 2: Click on Reports. → Click on More Reports.



Step 3: Select Burndown Chart.

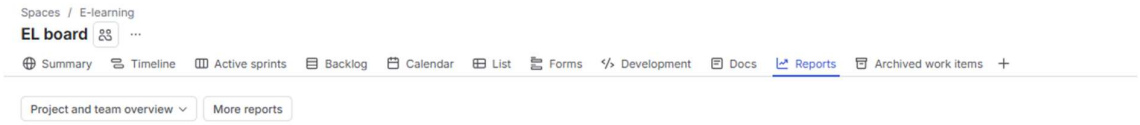
Step 4: View the Burndown Chart.



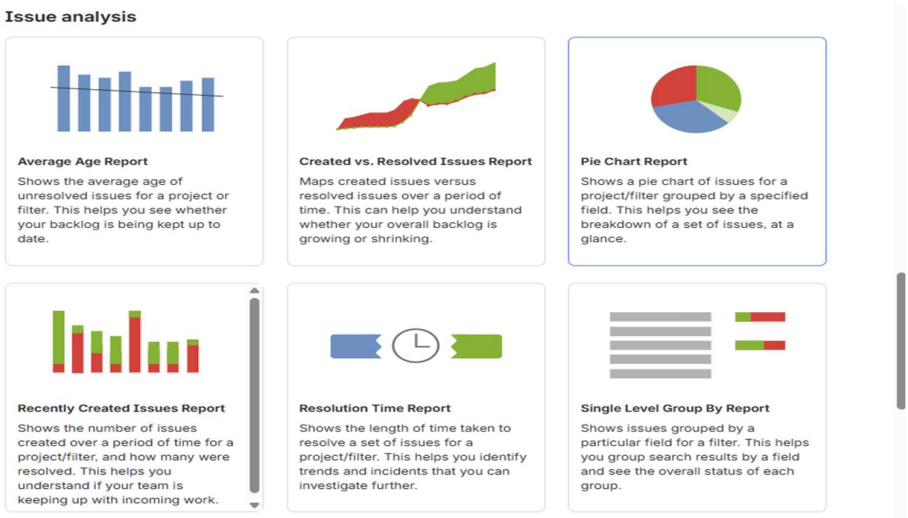
B. Create Pie Chart / Issue Statistics Report.

Step 1: Open the Jira Board.

Step 2: Click on Reports. → Click on More Reports.

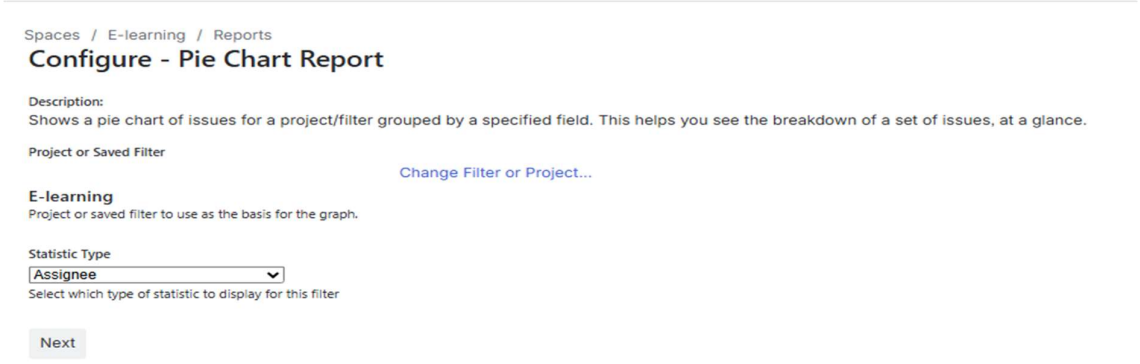


Step 3: In Issue Analysis Select Pie Chart Report.

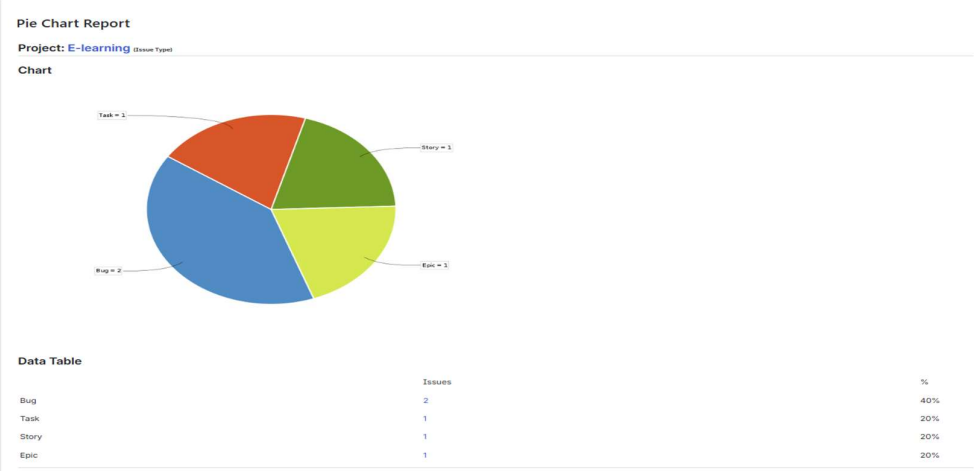


Step 4: Select the required Project.

Step 5: Select the required Statistic Type.



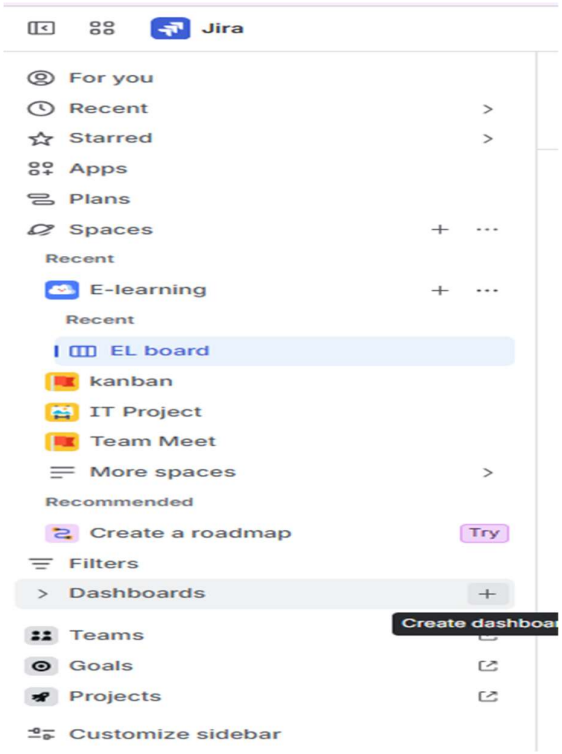
Step 6: View the generated Report.



C. Create a Custom Dashboard with gadgets.

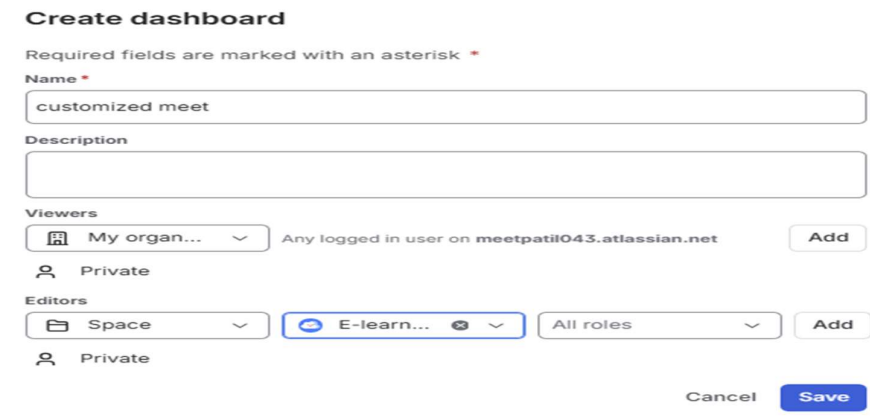
Step 1: Open the Jira Dashboard.

Step 2: Click on Create Dashboard.

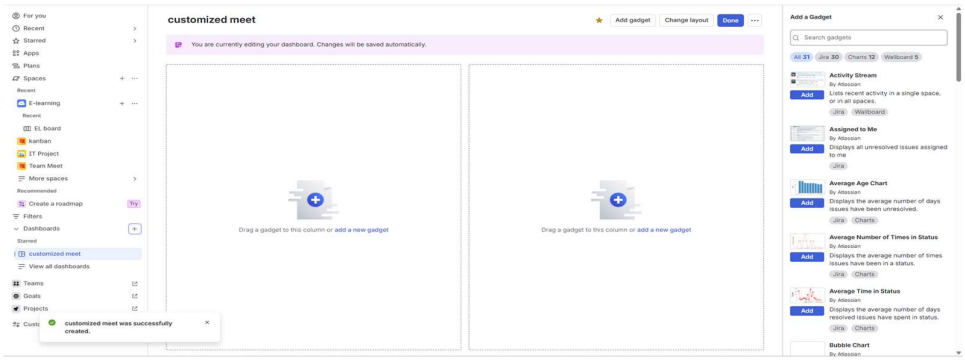


Step 3: Enter the Dashboard Name.

Step 4: Click on Save.



Step 5: Click on Add Gadget.

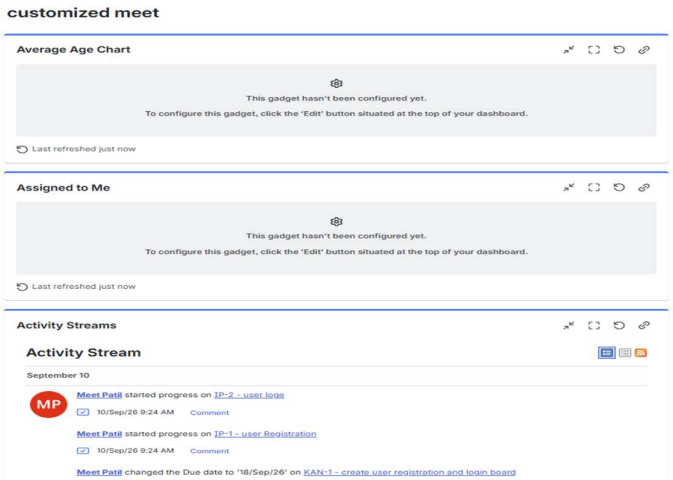


Step 6: Select the required Gadget.

Step 7: Configure the gadget settings.

Step 8: Click on Save.

Step 9: View the created Custom Dashboard.

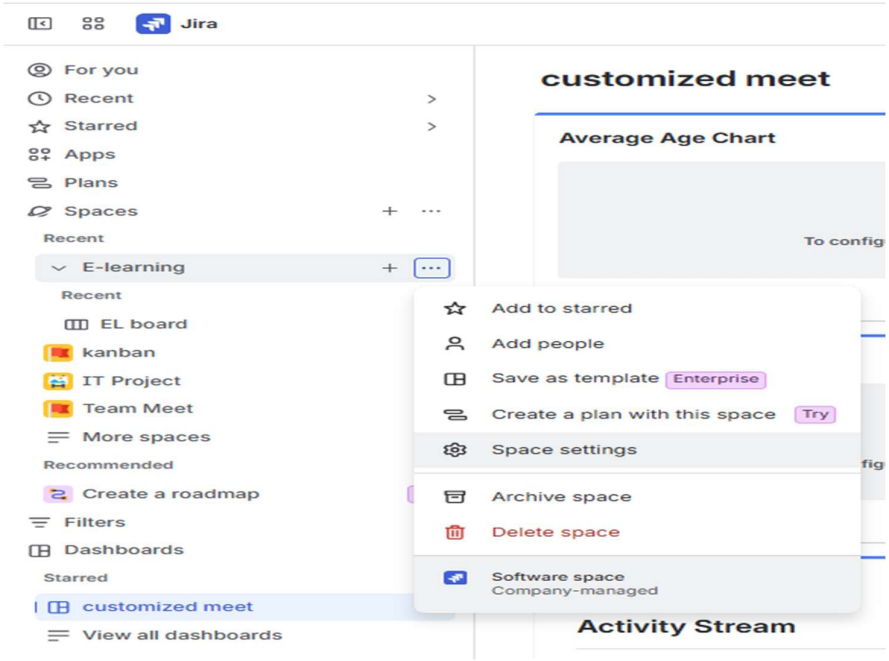


Practical No 10.
Permission & Security Management

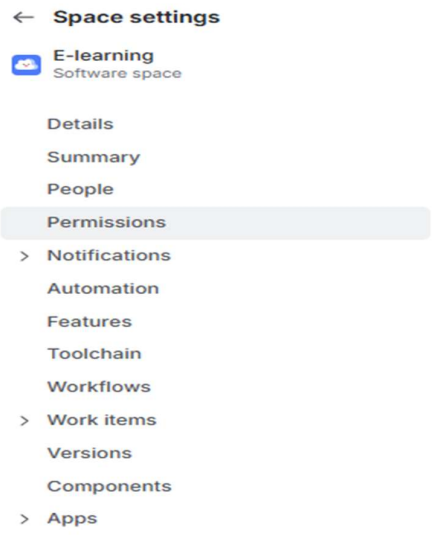
A. View and modify Project Permission Scheme.

Step 1: Open E-learning → Click on the three dots (...).

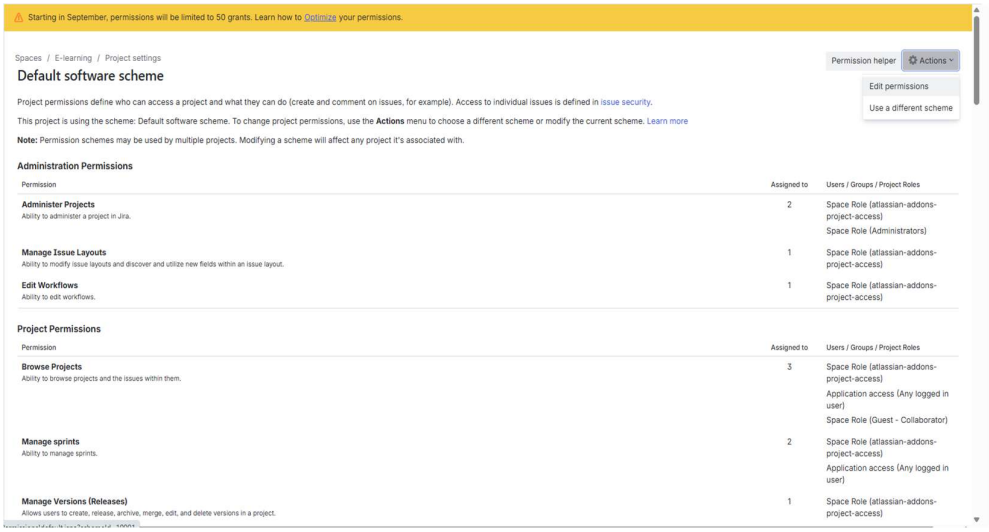
Step 2: Click on Space settings.



Step 3: Click on Permissions.



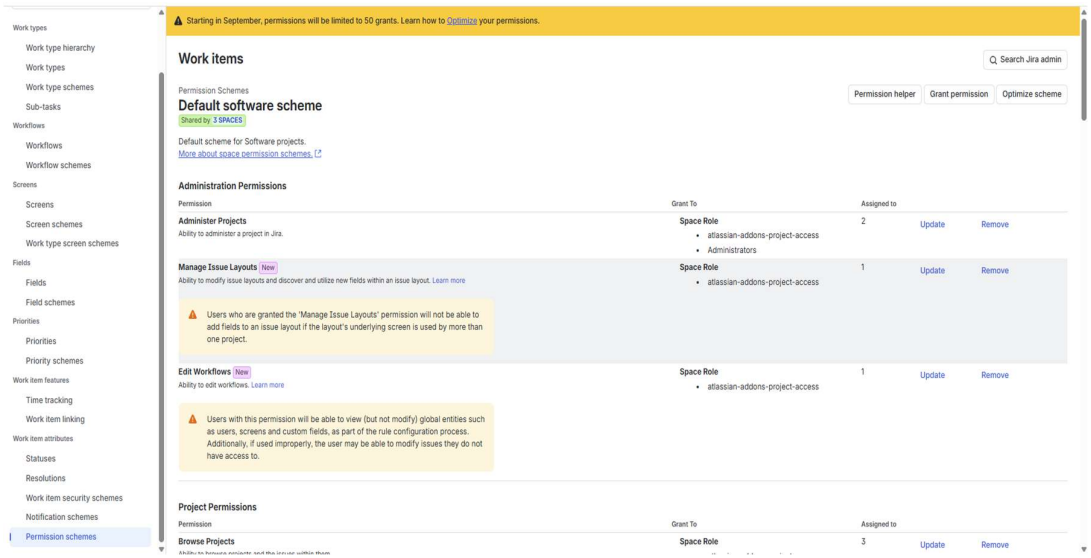
Step 4: Click on Actions or Edit Permissions.



Step 5: Select the required Permission.

Step 6: Add the required User, Group, or Project Role.

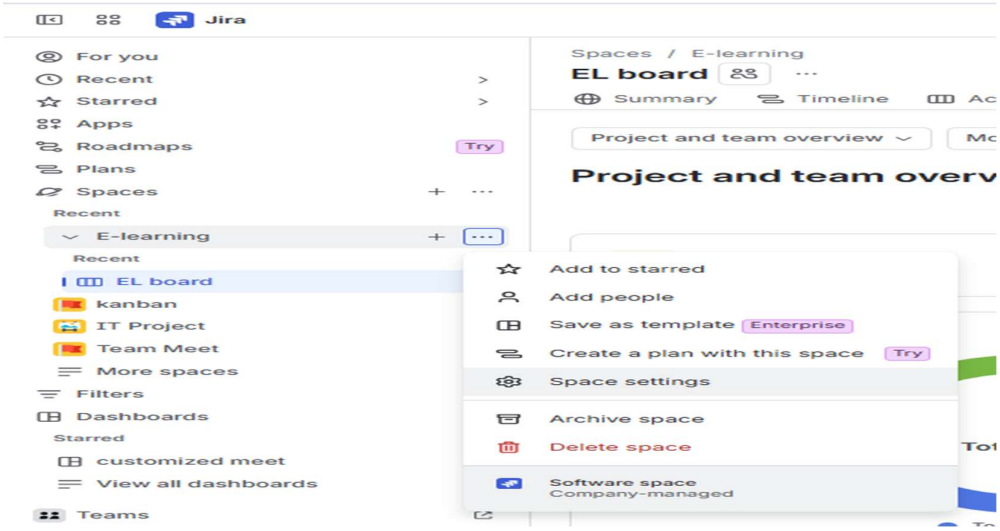
Step 7: Click on Save or Update.



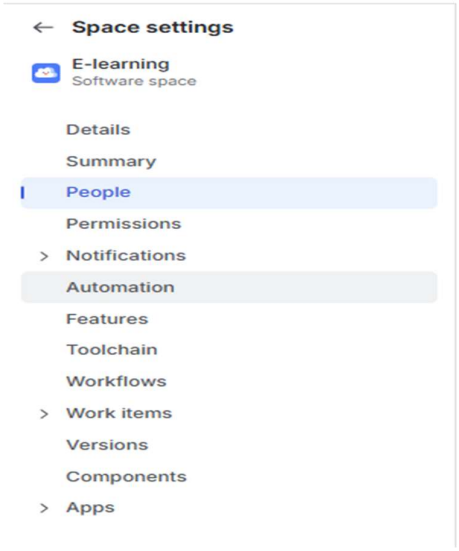
B. Configure Project Roles and assign permissions.

Step 1: Open E-learning → Click on the three dots (...).

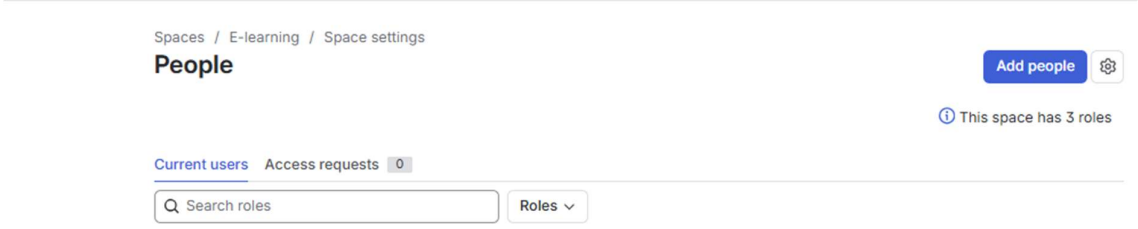
Step 2: Click on Space settings.



Step 3: Click on People or Project Roles.



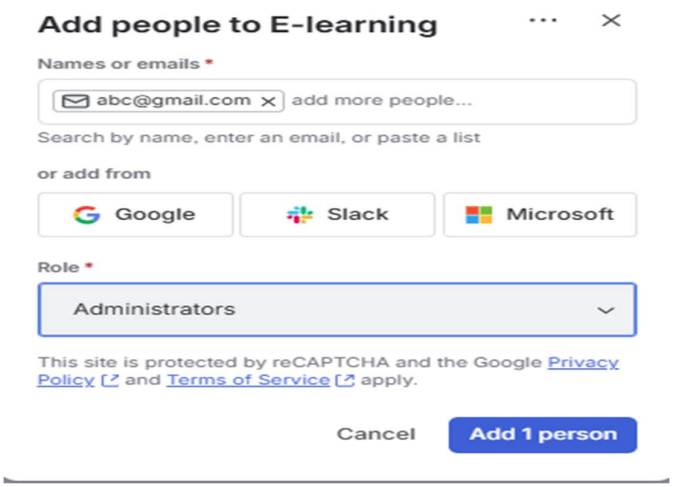
Step 4: Click on Add People.



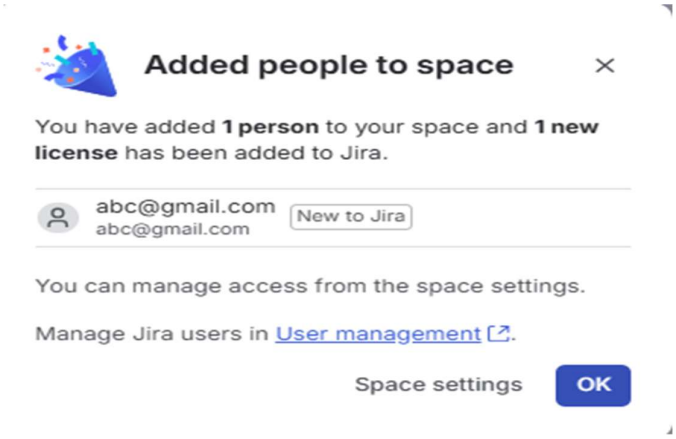
Step 5: Enter the User Name or Email Address.

Step 6: Select the required Project Role.

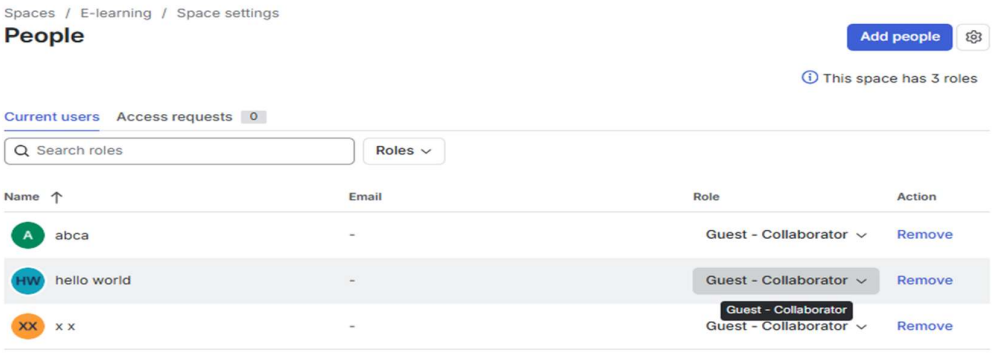
Step 7: Assign the role. → Click Add Person.



Step 8: Click on OK.



Step 9: Save the changes.

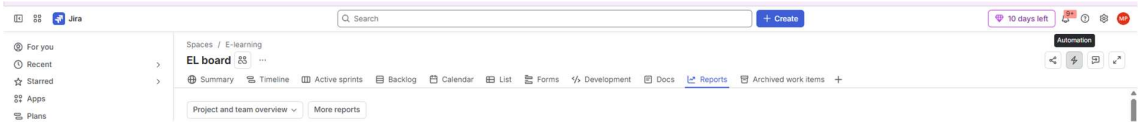


Practical No. 11
Jira Automation

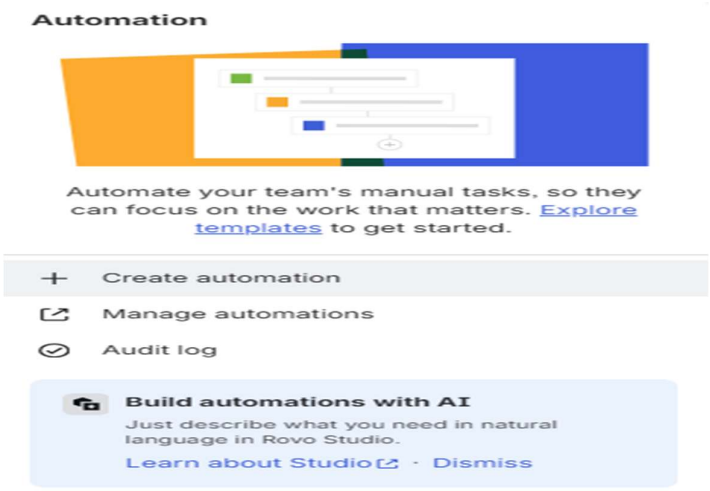
A. Create an Automation Rule (Trigger → Condition → Action).

Step 1: Open the Jira Project.

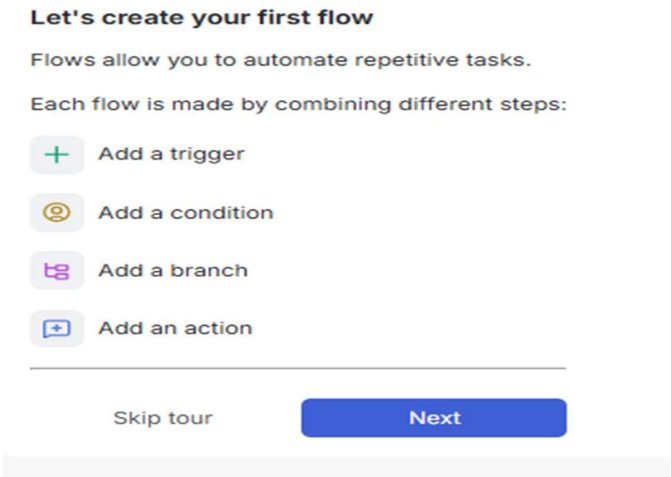
Step 2: Click on Automation.



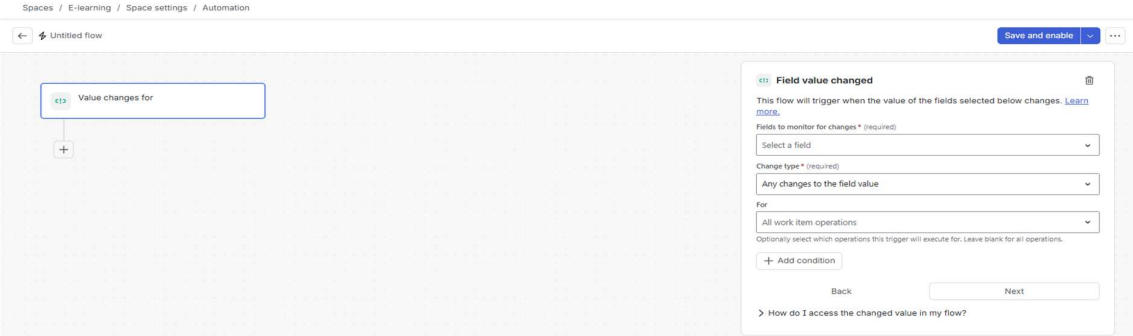
Step 3: Click on Create Automation.



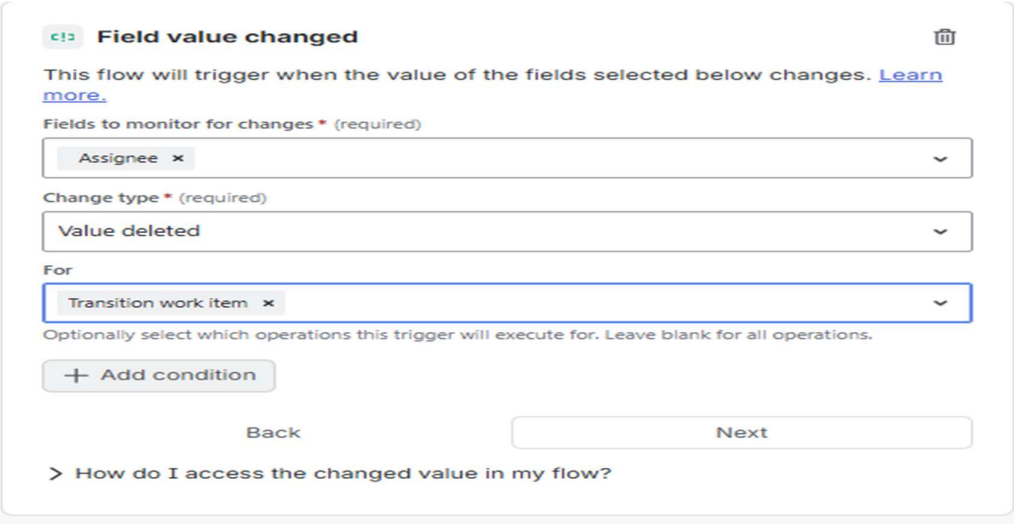
Step 4: Select the required Trigger.



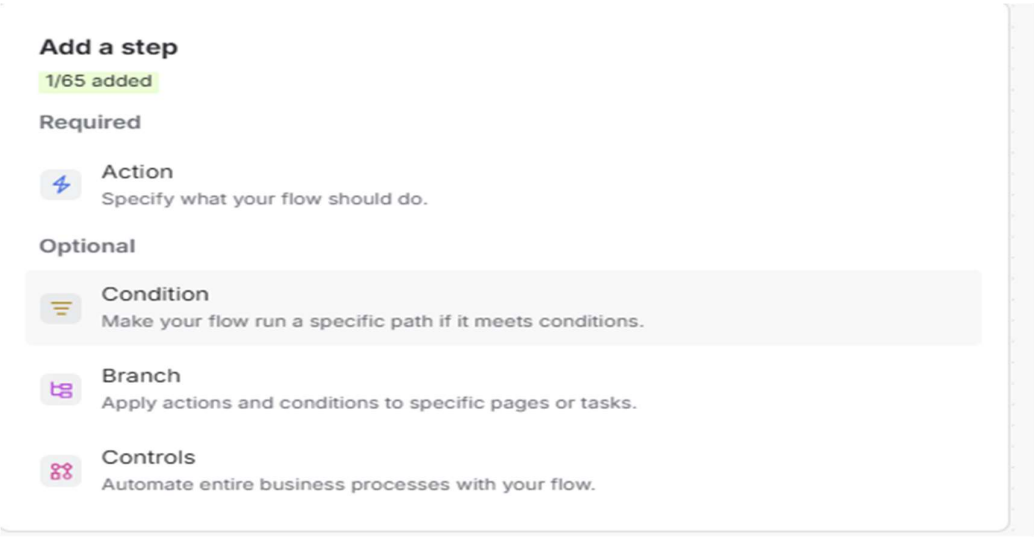
Step 5: Click Next.



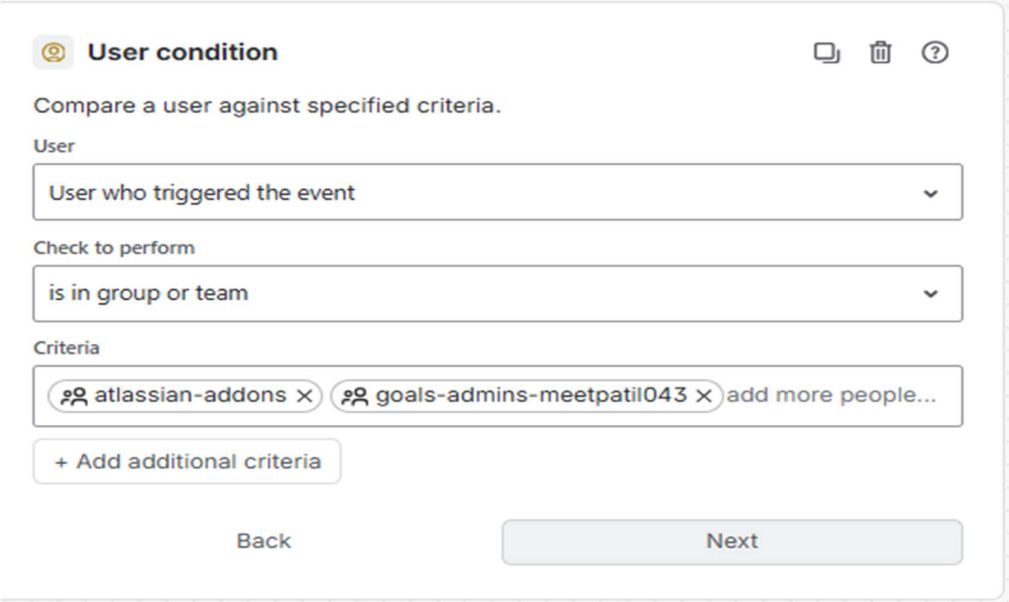
Step 6: Enter the required Fields and Click Next.



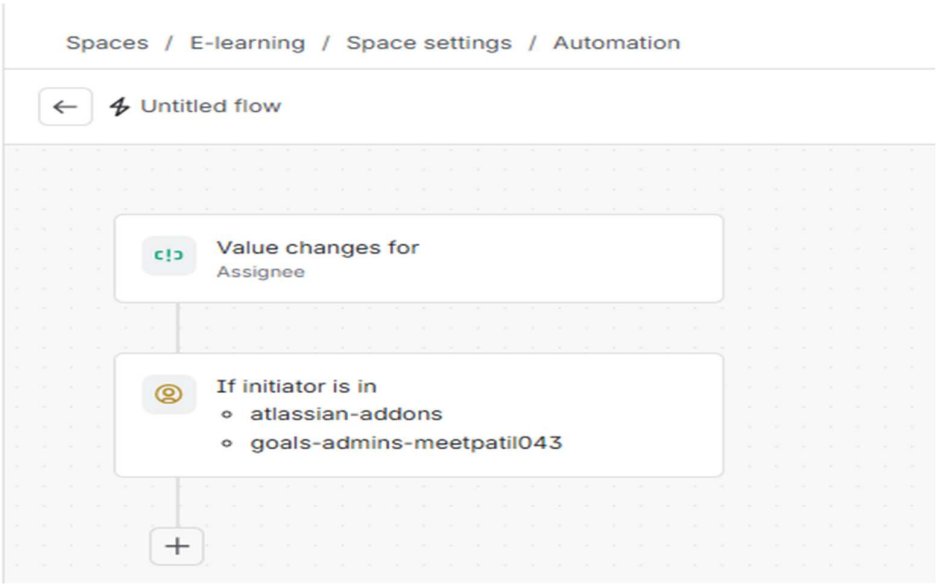
Step 7: Add a Condition



Step 8: Select the User Conditions.



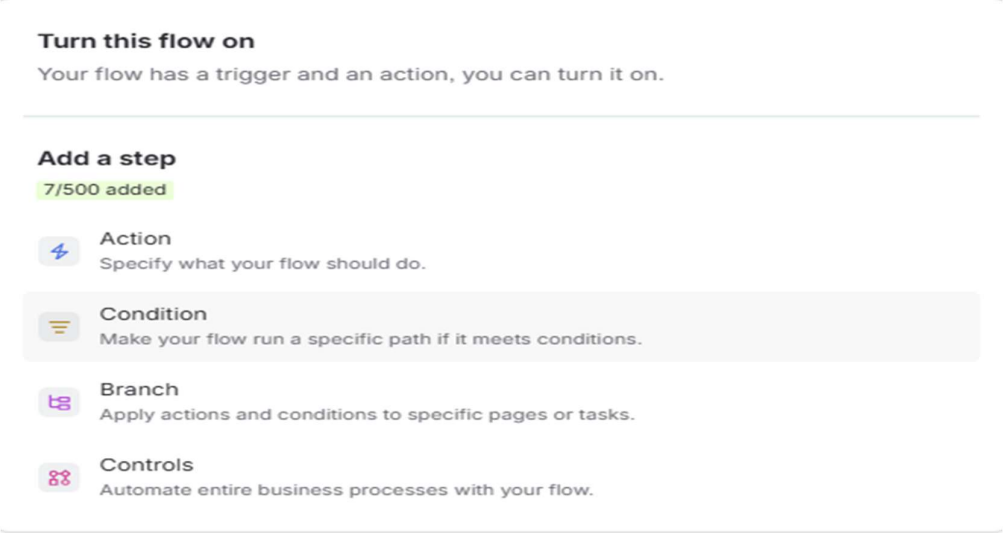
Step 9: Click on Next.



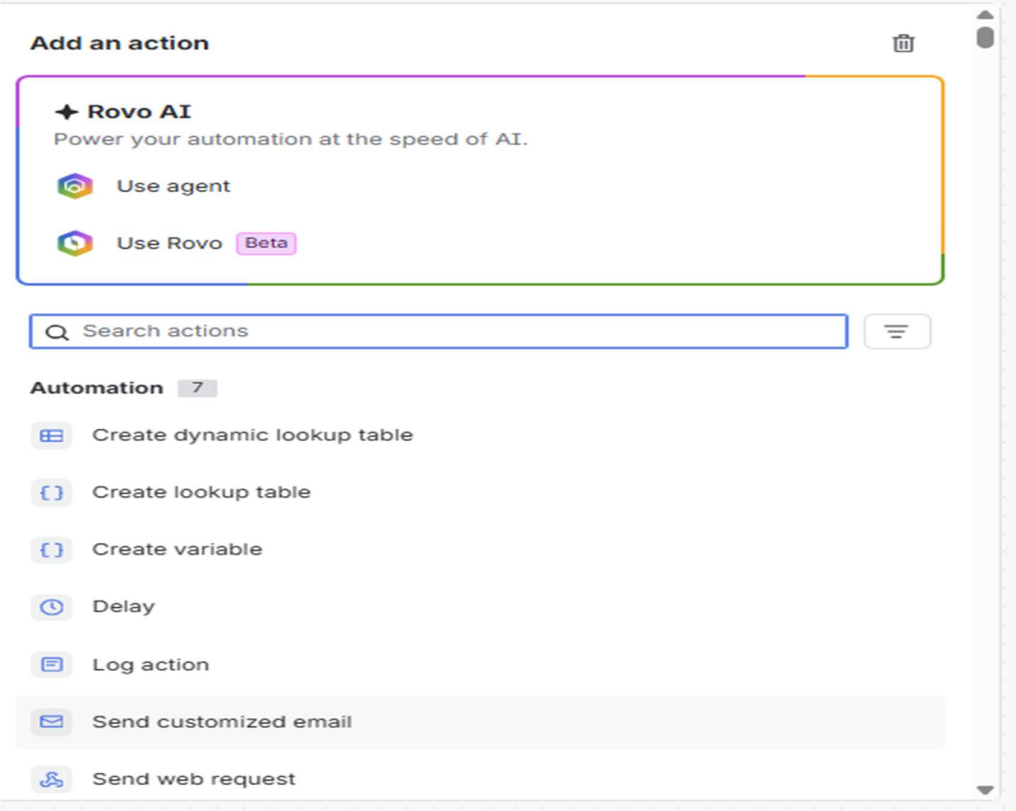
B. Configure rule (e.g., When issue is moved to Done → send email / auto-assign).

C. Test and enable automation rules.

Step 1: Add a Condition.

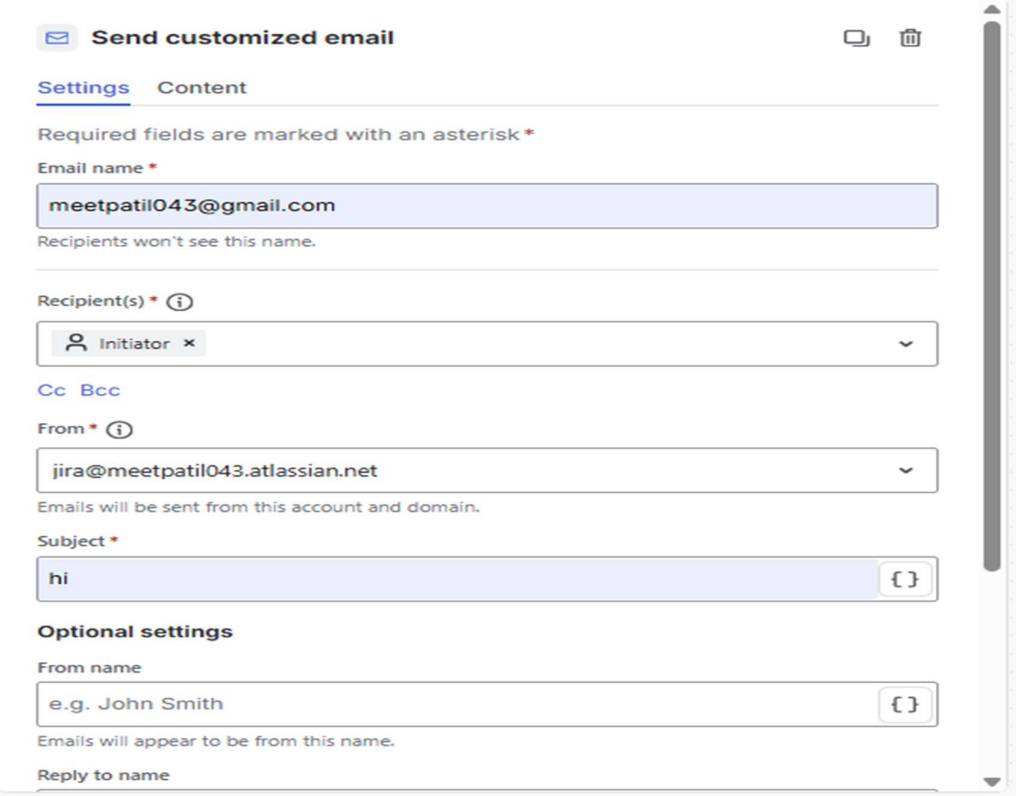


Step 2: In Automation Select the Send Customized Email.



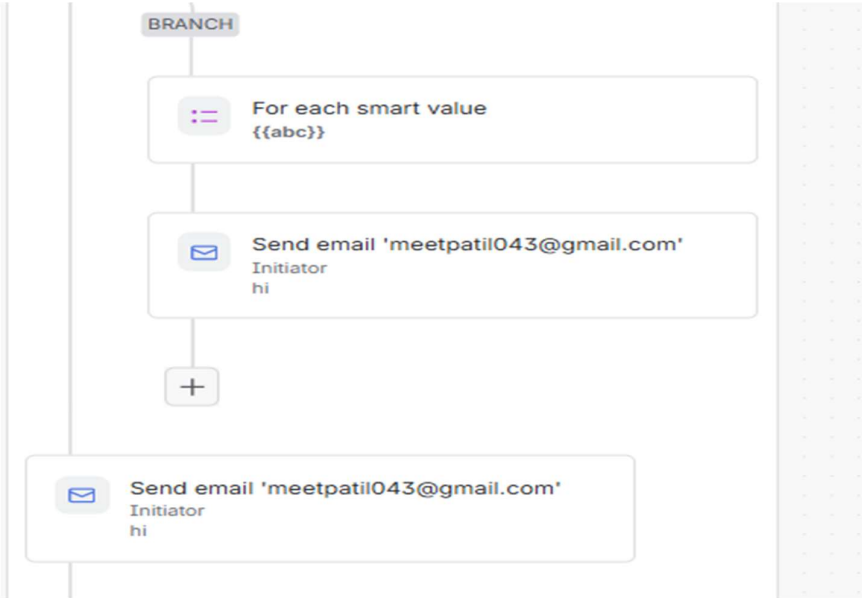
Step 3: In Settings Configure the details.

Step 4: Click on Save.



Step 5: Enable the Automation Rule.

Step 6: Save the changes.



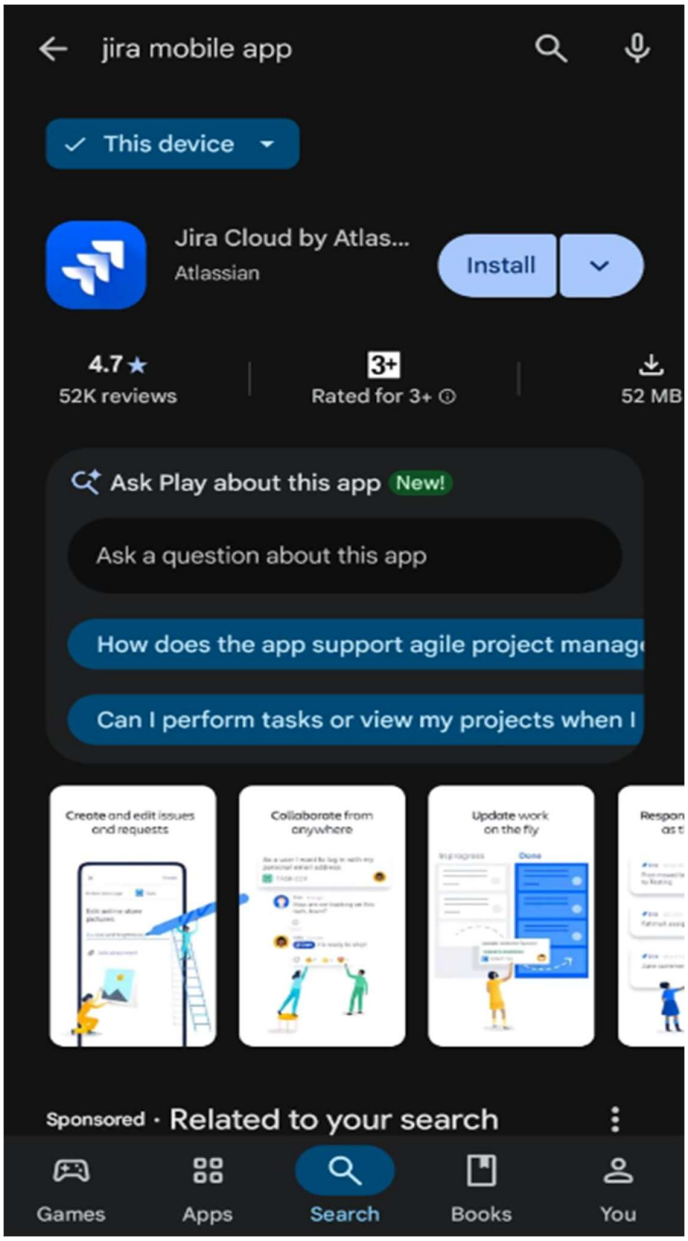
Practical No 12.
Introduction to Jira Mobile App

A. Install and configure the Jira Mobile App.

Step 1: Open the Google Play Store on your mobile phone.

Step 2: Search for Jira Cloud by Atlassian.

Step 3: Click on Install.



Step 4: Open the Jira Mobile App.



Step 5: Click on Log in.

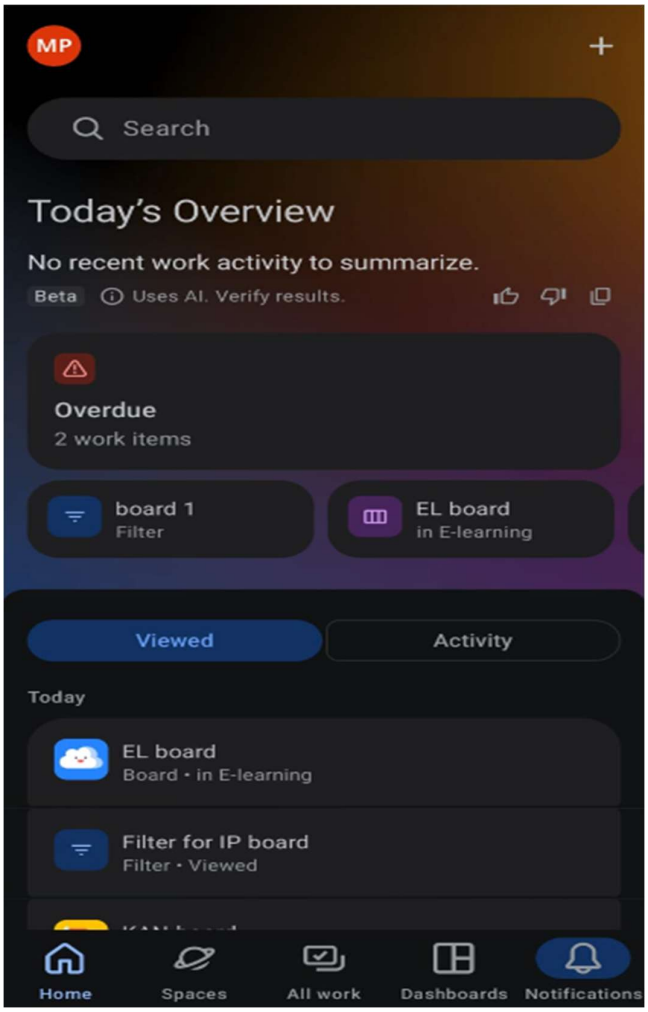
Step 6: Enter your Email Address.





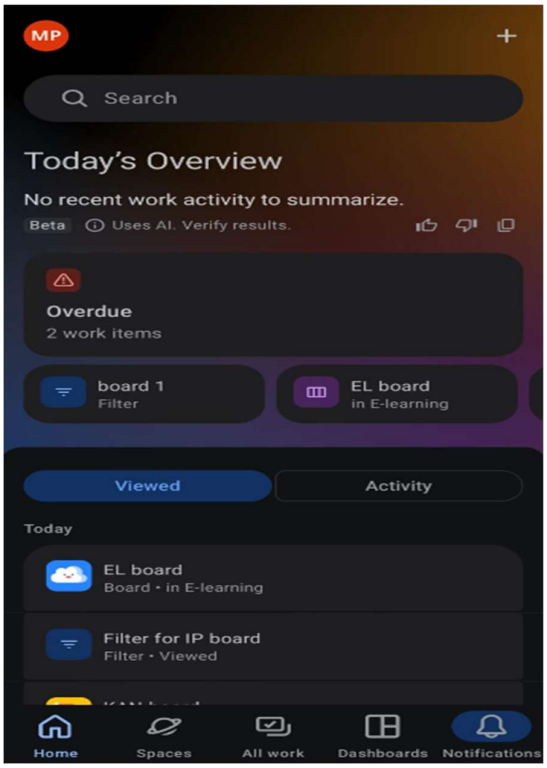
Step 7: Select the required Jira Workspace/Project.

Step 8: The Jira Mobile App is successfully configured.



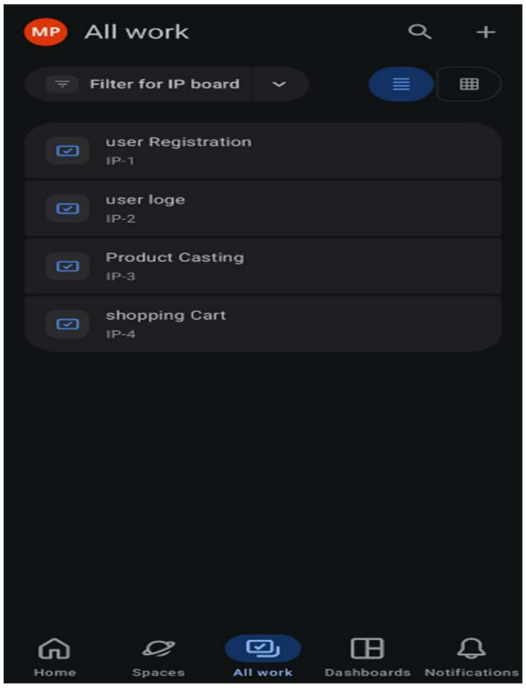
B. View and update issues from mobile (Status, Comment, Assign).

Step 1: Open the Jira mobile app.

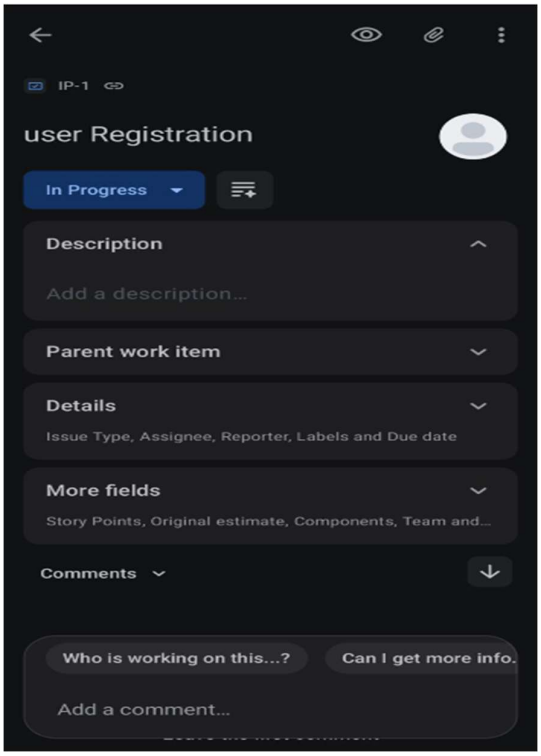


Step 2: Tap All work at the bottom.

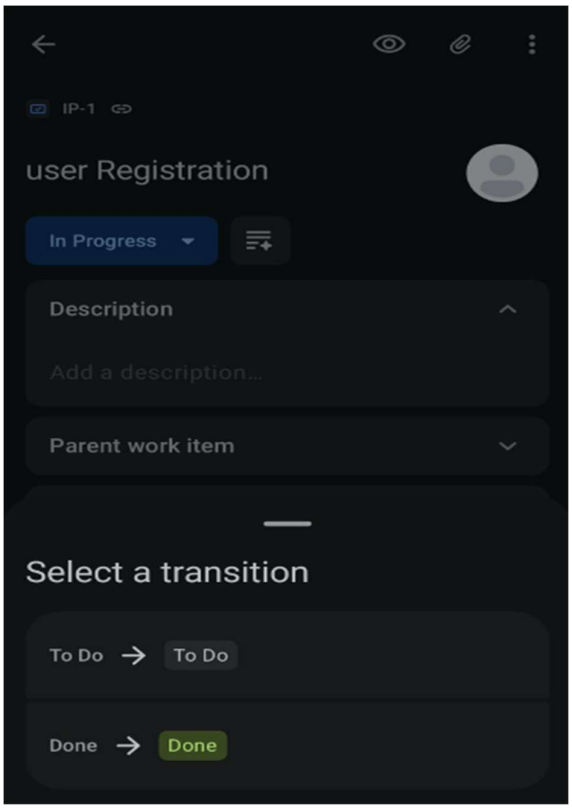
Step 3: Select Projects and choose your project.



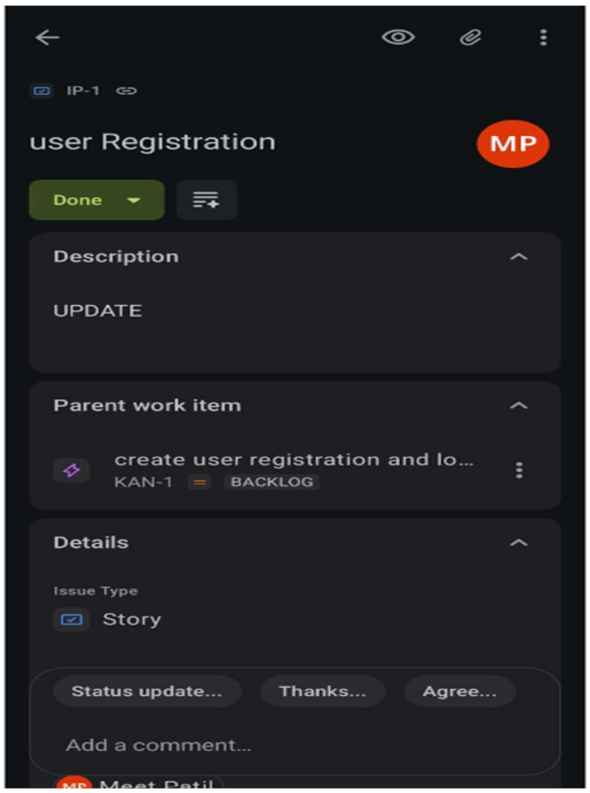
Step 4: Click the "In Progress".



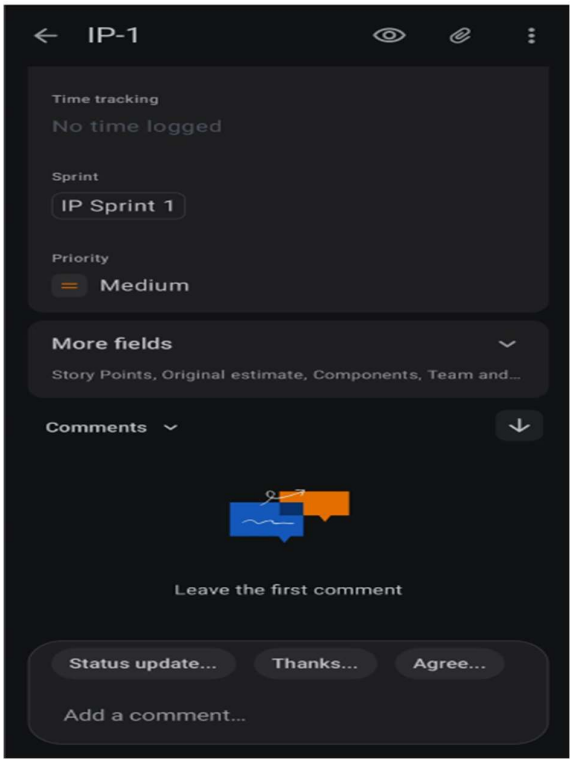
Step 5: Select another status, such as Done.



Step 6: Update the issue.

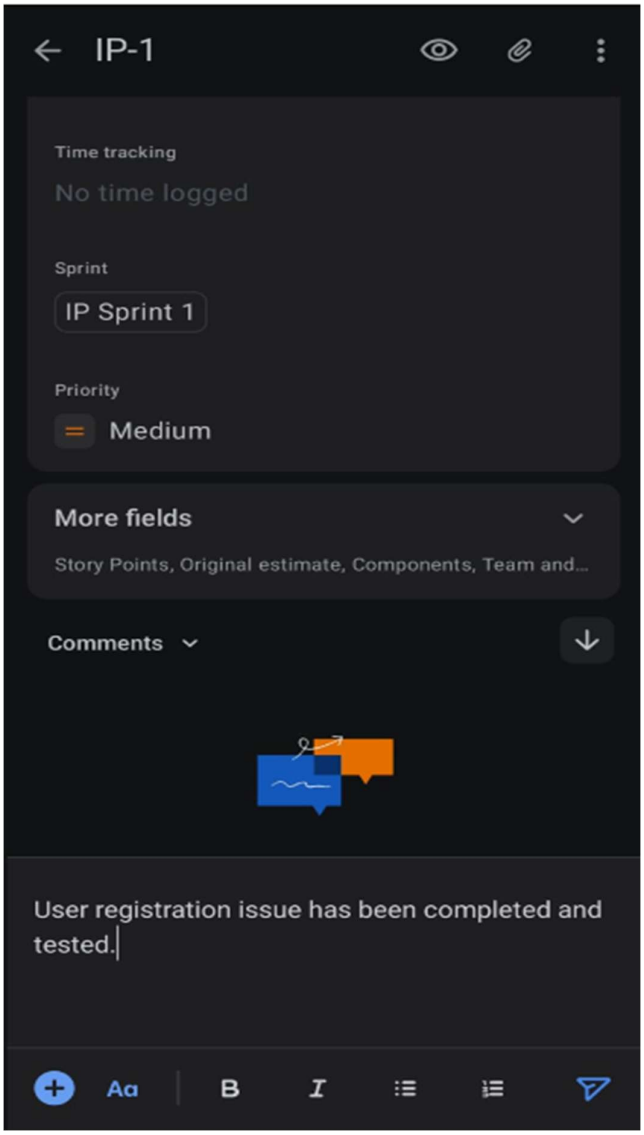


Step 7: Scroll down to the Comments section.



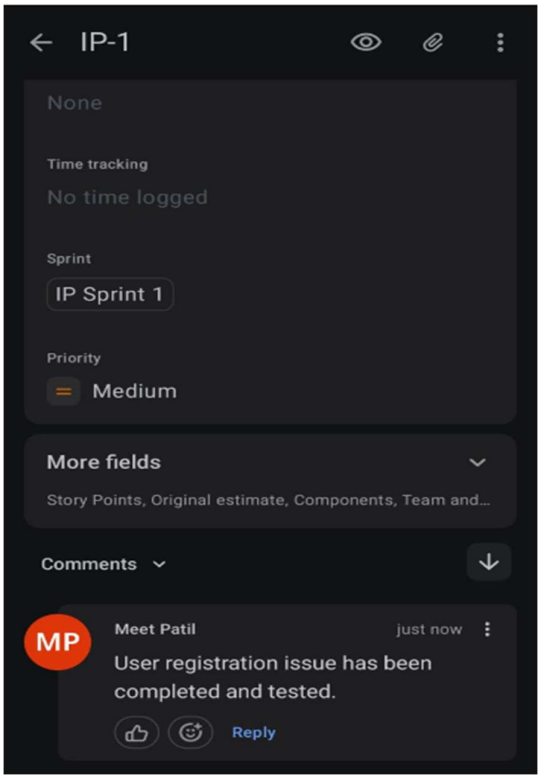
Step 8: Tap Add a comment...

Step 9: Enter this comment:

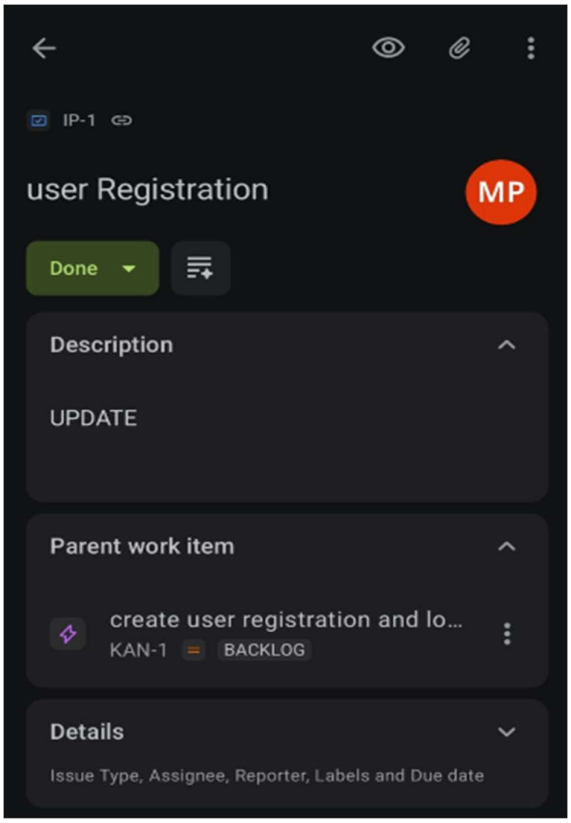


Step 10: Tap the Send (➤) button on your keyboard or the comment submission button.

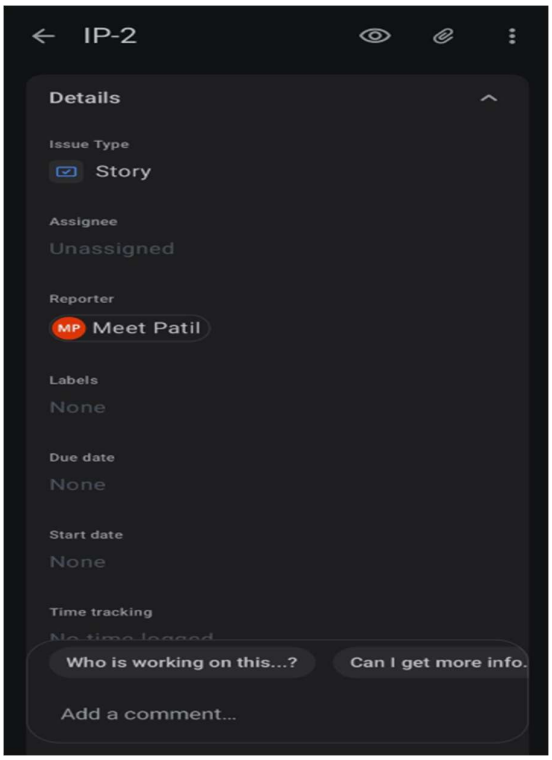
Step 11: Check that your comment appears under Comments.



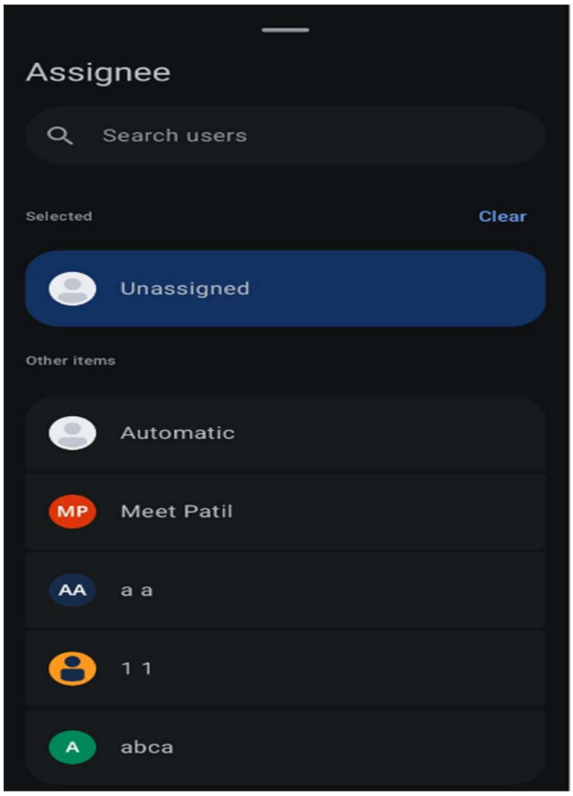
Step 12: Open the Details section.



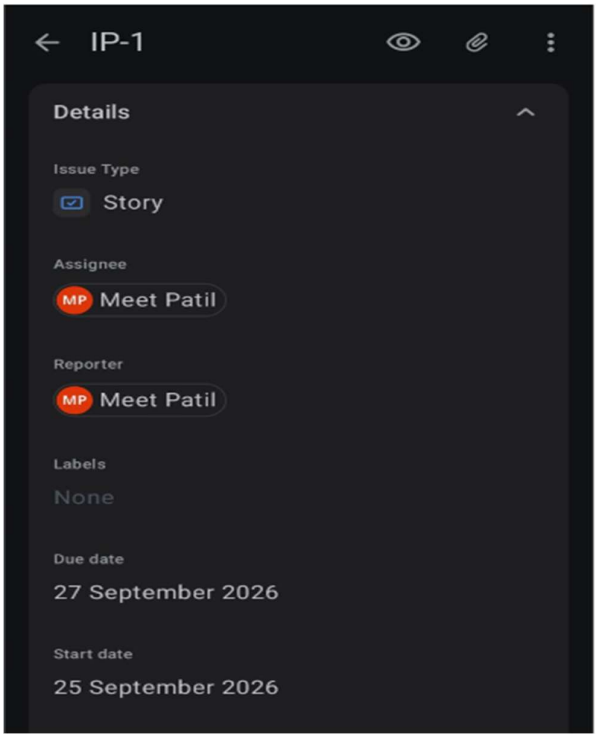
Step 13: Tap the Assignee field.



Step 14: Select your name or another available user.

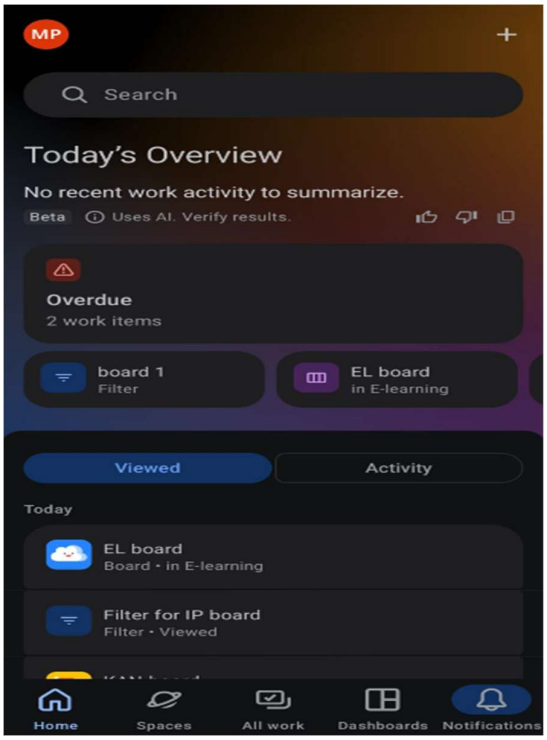


Step 15: Save the changes, if a Save button appears.

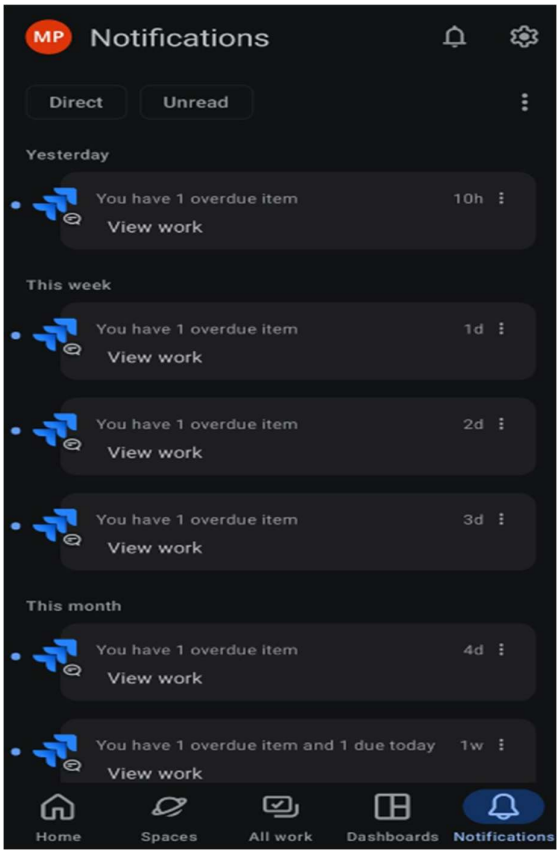


C. Receive notifications and manage real-time updates

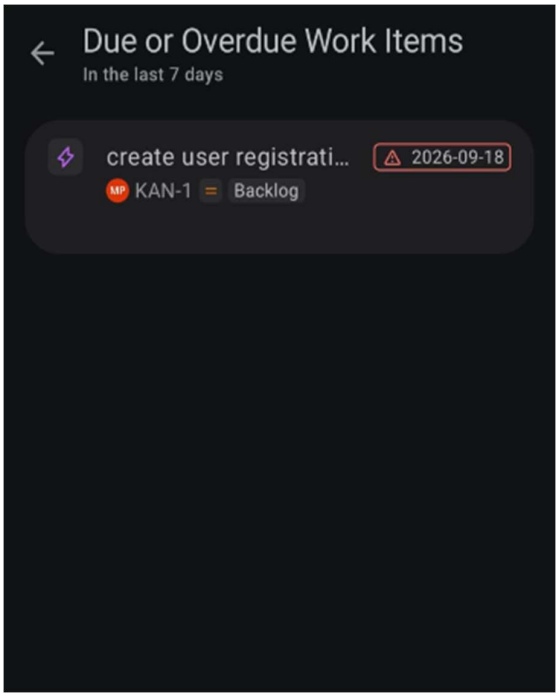
Step 1: Open the Jira Mobile App.



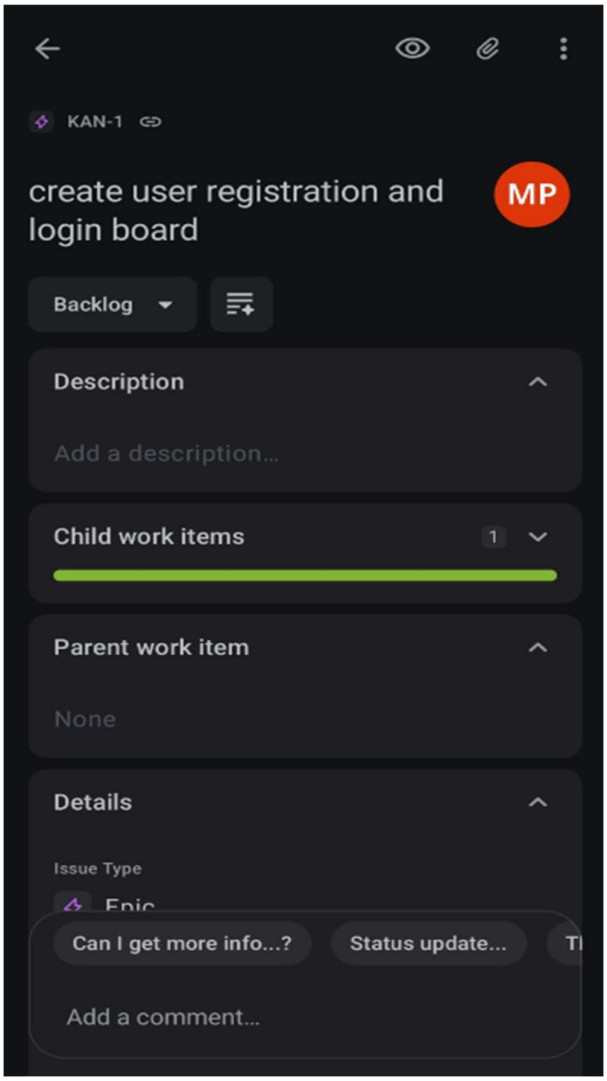
Step 2: Tap Notifications at the bottom of the screen.



Step 3: Tap View work on any notification.

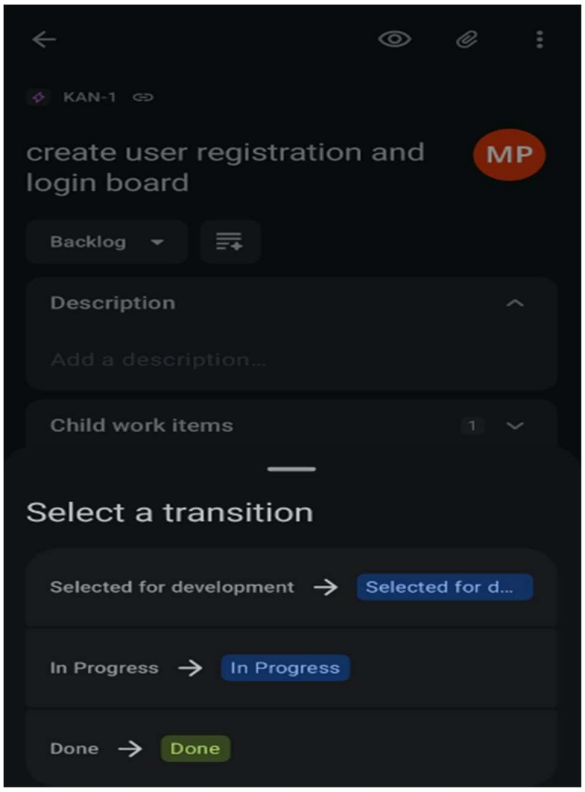


Step 4: Tap the create user registration issue.

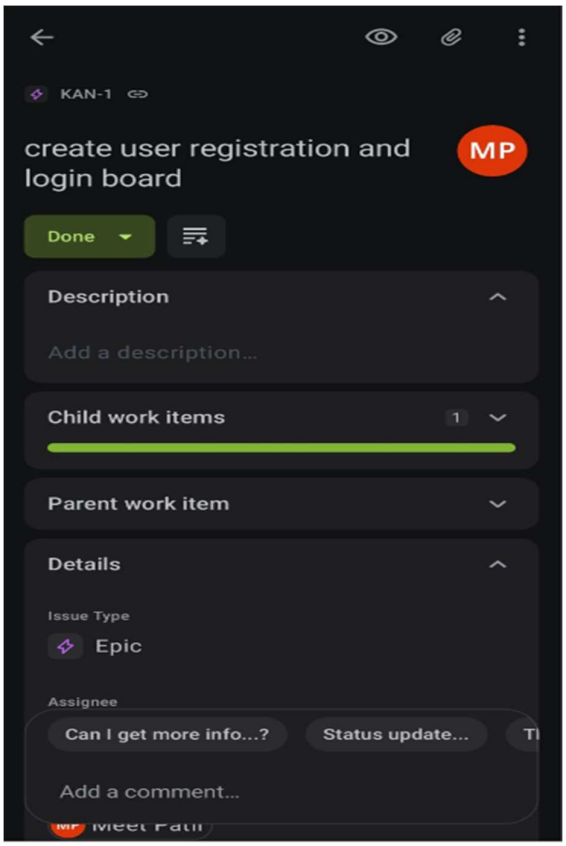


Step 5: Check the issue details, including the Backlog status and overdue date.

Step 6: Tap the Back arrow at the top-left.



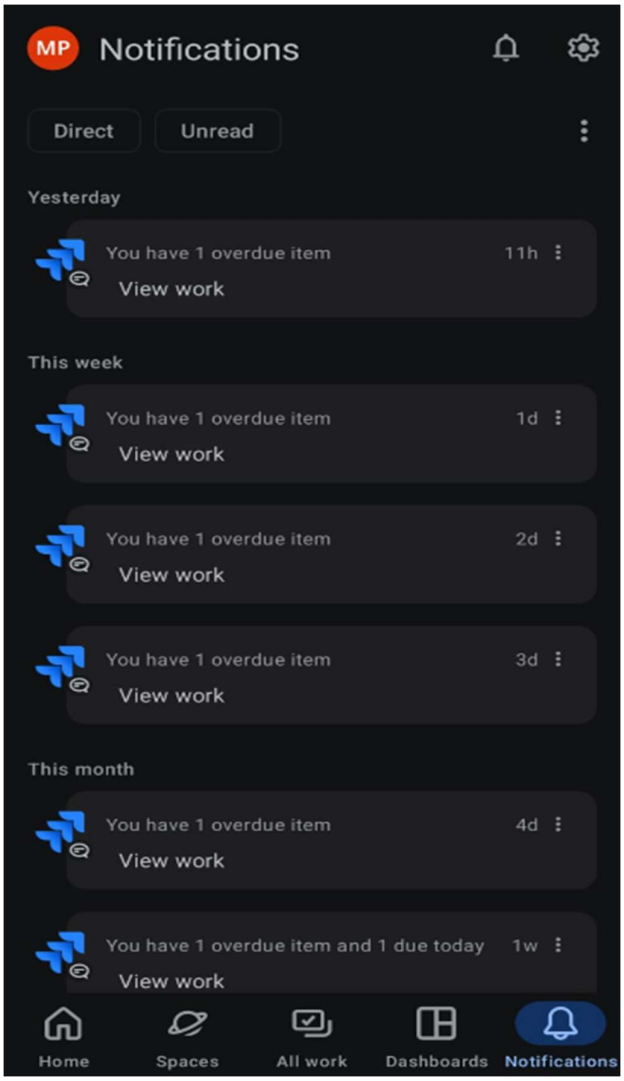
Step 7: Tap Done → Done.



Step 8: Tap the Back arrow (←) at the top-left.

Step 9: Return to the Notifications section.

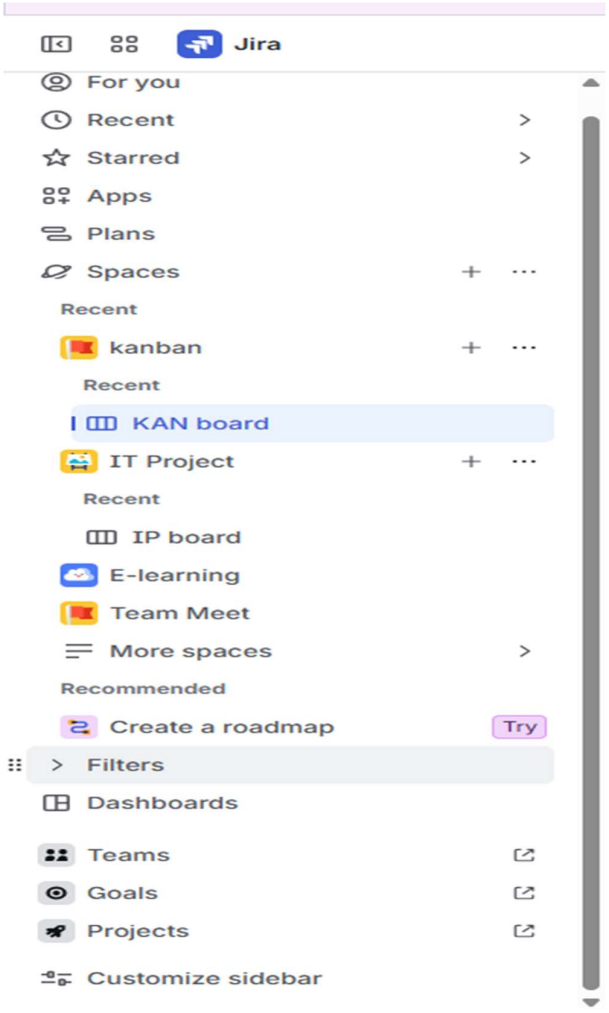
Step 10: Check the notifications and updated work items.



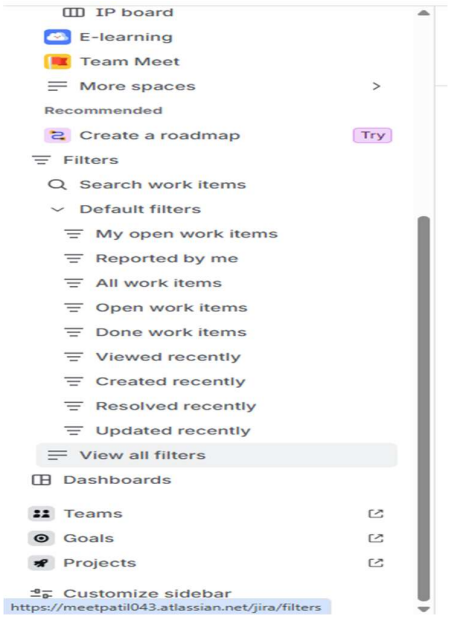
Practical No. 8
JQL & Filters

A. Write basic JQL queries (Status, Assignee, Priority).
B. Create and save filters.

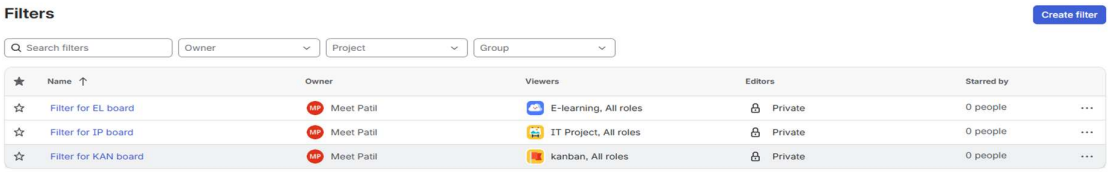
Step 1: Click on Filters.



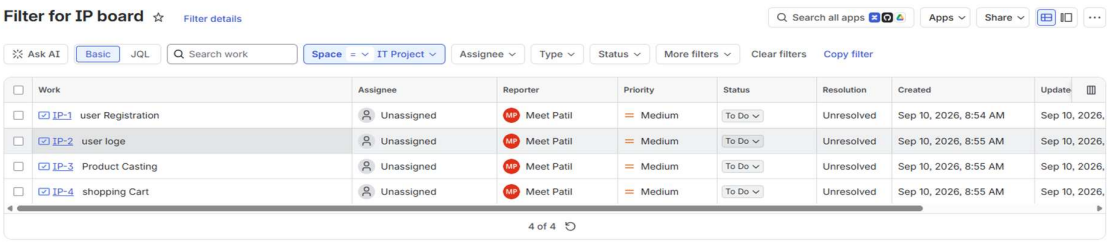
Step 2: Click on View all filters.



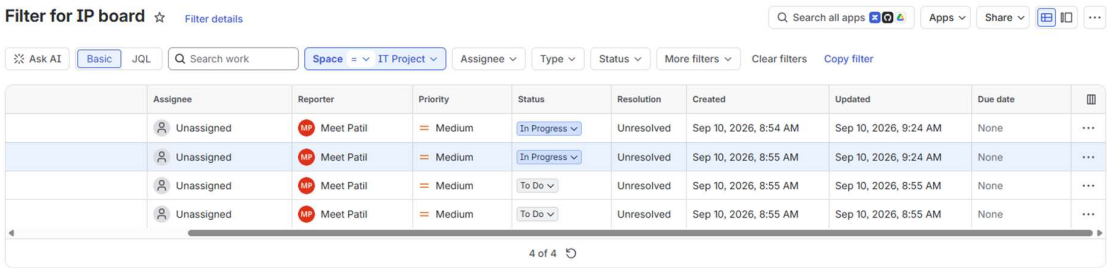
Step 3: Select the One Project.



Step 4: Click on IP-1.

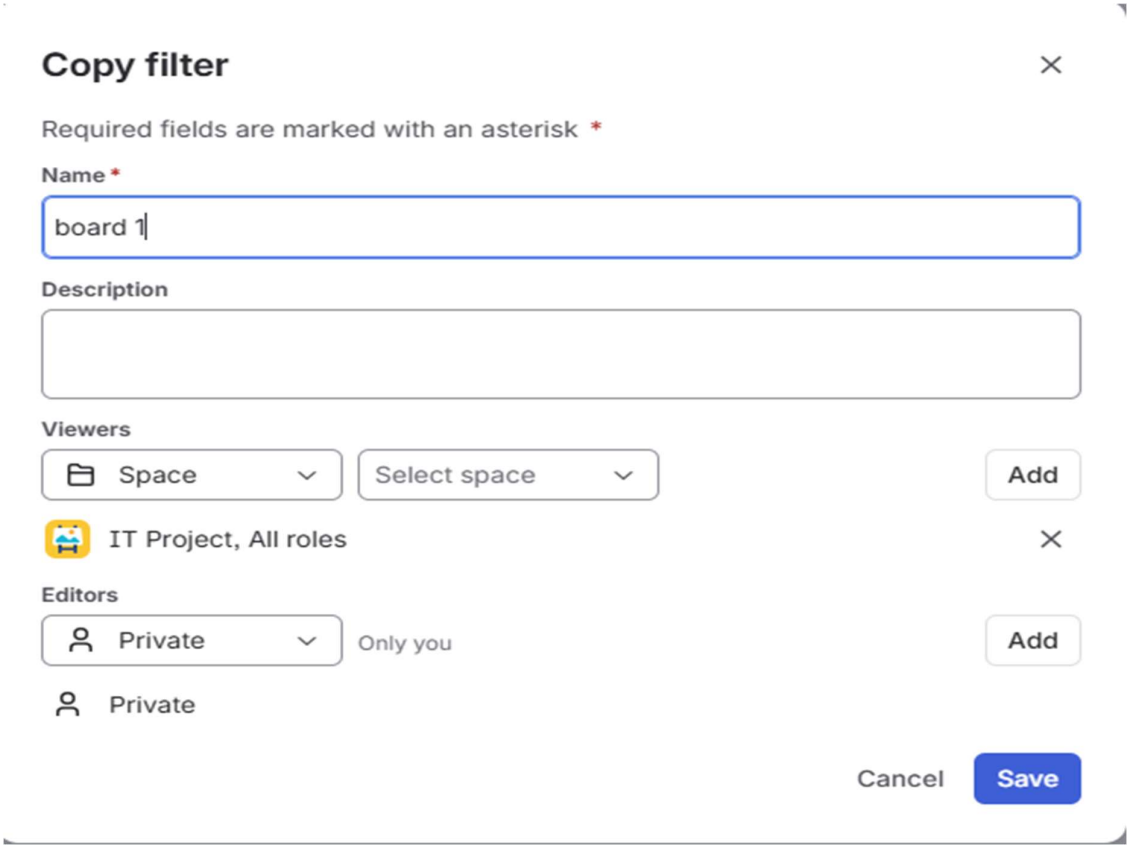


Step 5: Set the required Status.

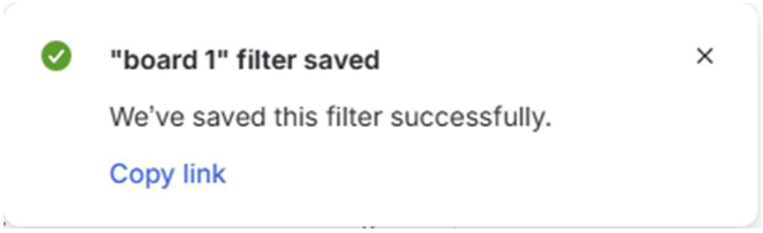


Step 6: Click on Copy Filter.

Step 7: Enter the Filter Name.



Step 8: Click on Save.

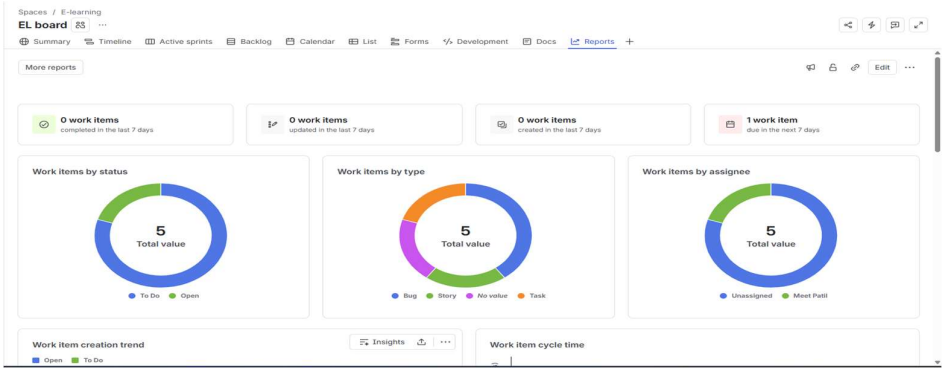


Practical No. 9
Reports & Dashboards

A. Generate Burndown Chart and Velocity Chart.

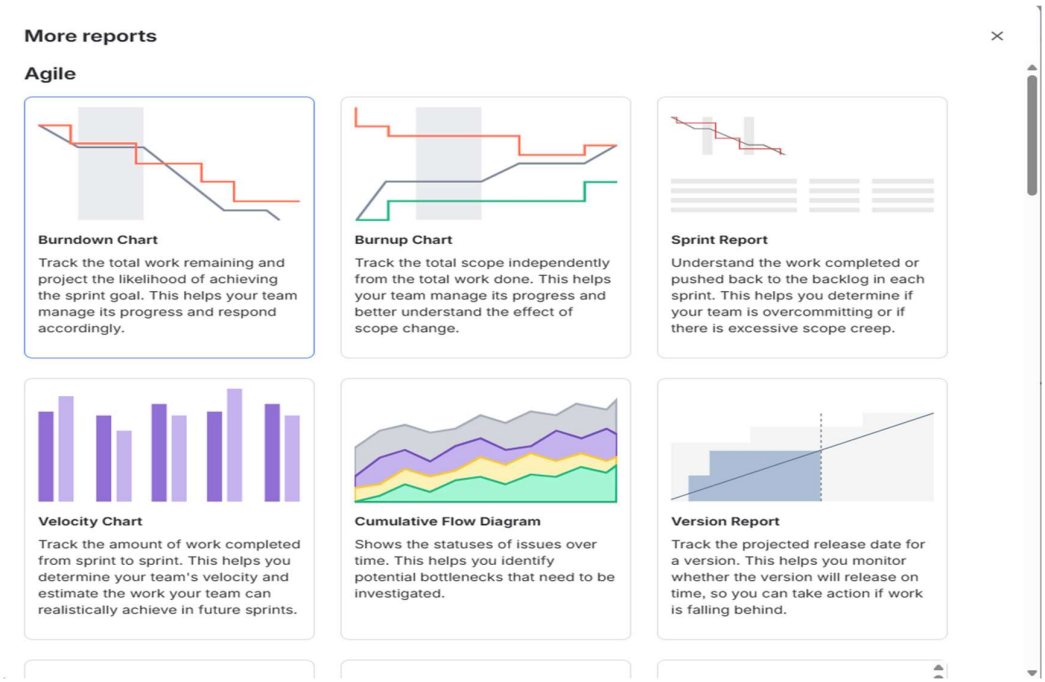
Step 1: Open the Board.

Step 2: Click on Reports. → Click on More Reports.



Step 3: Select Burndown Chart.

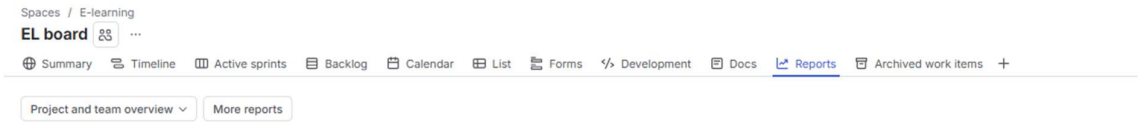
Step 4: View the Burndown Chart.



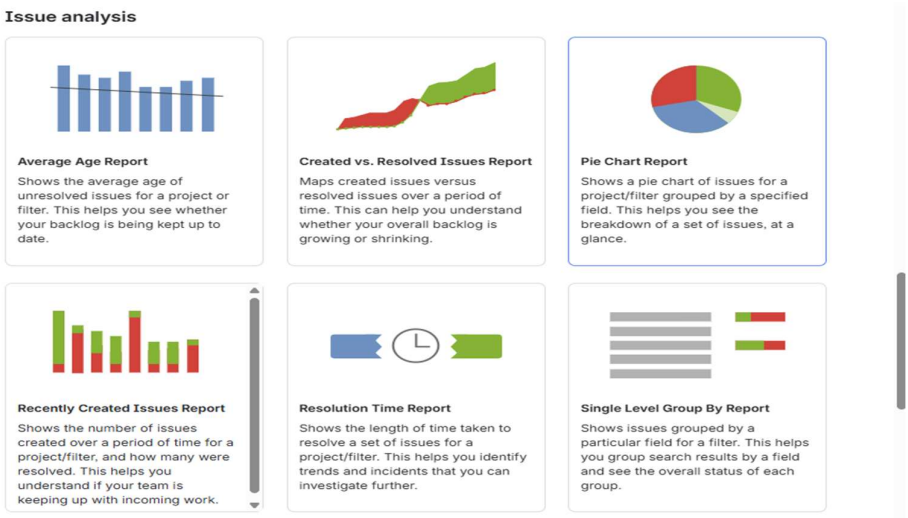
B. Create Pie Chart / Issue Statistics Report.

Step 1: Open the Jira Board.

Step 2: Click on Reports. → Click on More Reports.

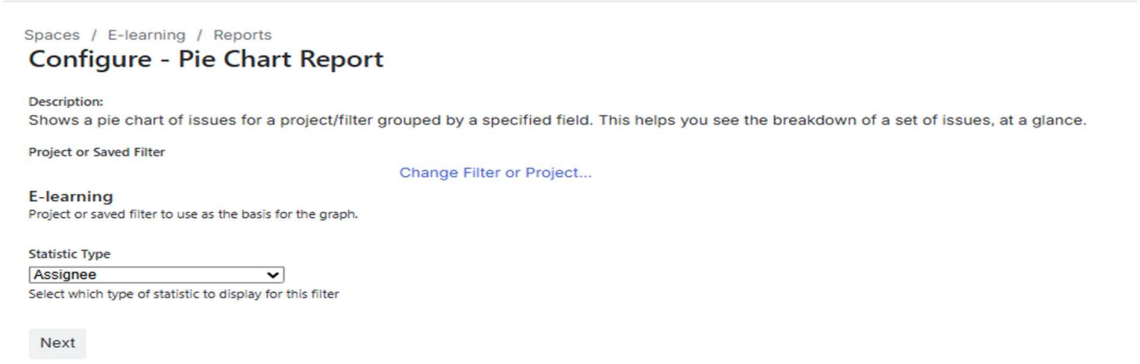


Step 3: In Issue Analysis Select Pie Chart Report.

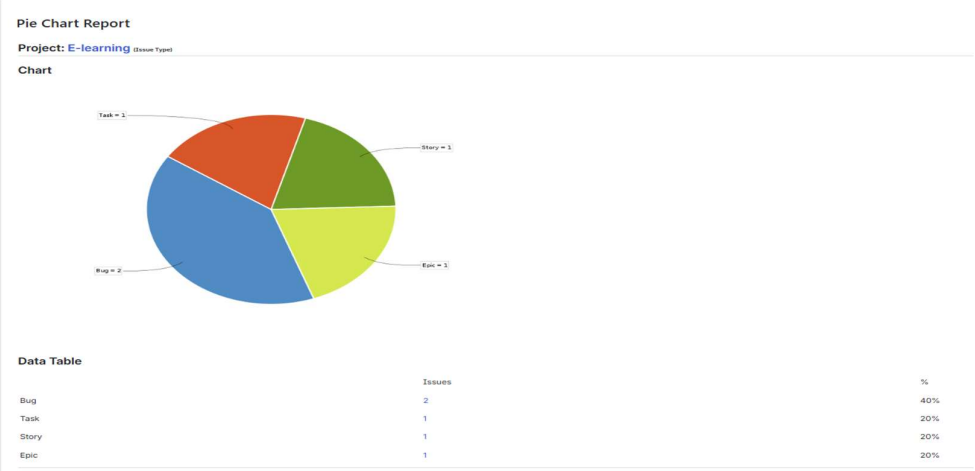


Step 4: Select the required Project.

Step 5: Select the required Statistic Type.



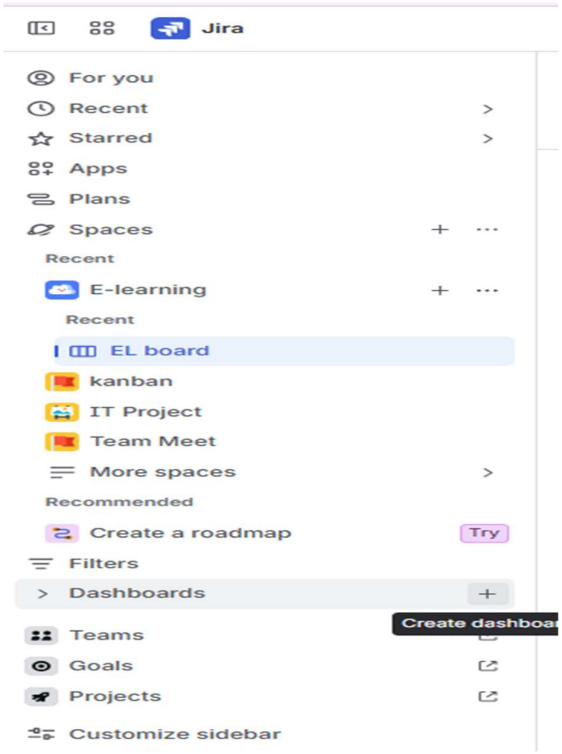
Step 6: View the generated Report.



C. Create a Custom Dashboard with gadgets.

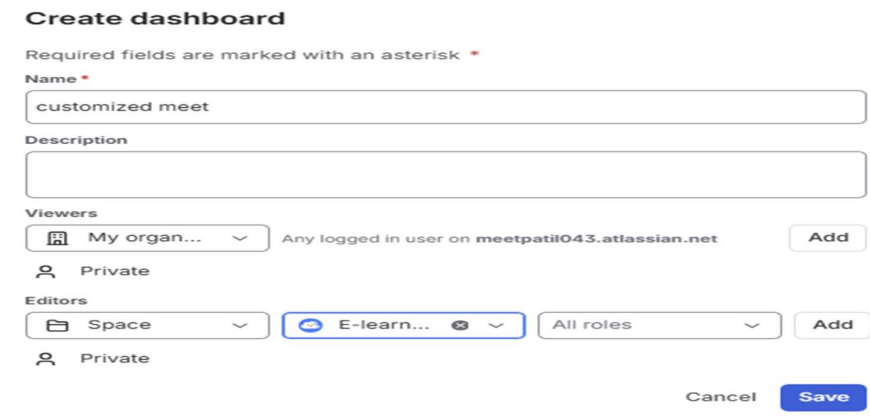
Step 1: Open the Jira Dashboard.

Step 2: Click on Create Dashboard.

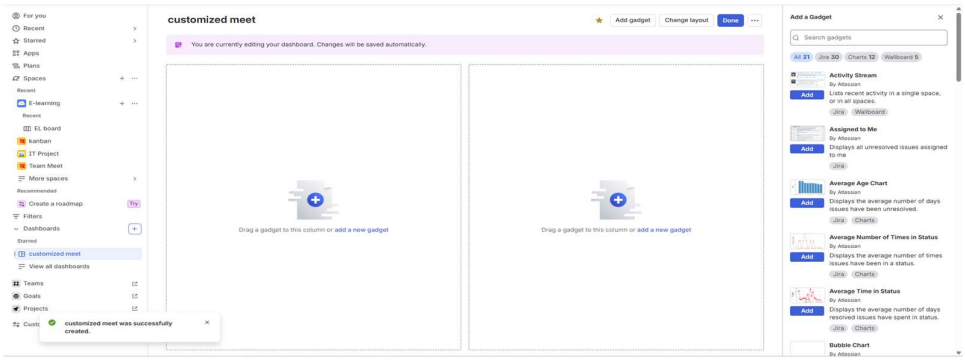


Step 3: Enter the Dashboard Name.

Step 4: Click on Save.



Step 5: Click on Add Gadget.

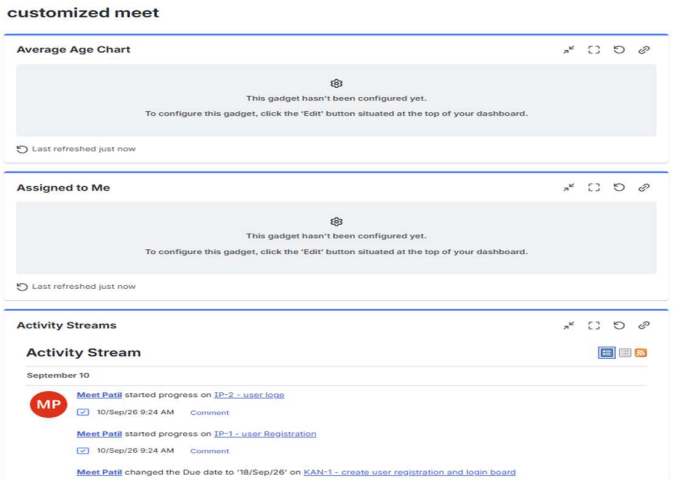


Step 6: Select the required Gadget.

Step 7: Configure the gadget settings.

Step 8: Click on Save.

Step 9: View the created Custom Dashboard.

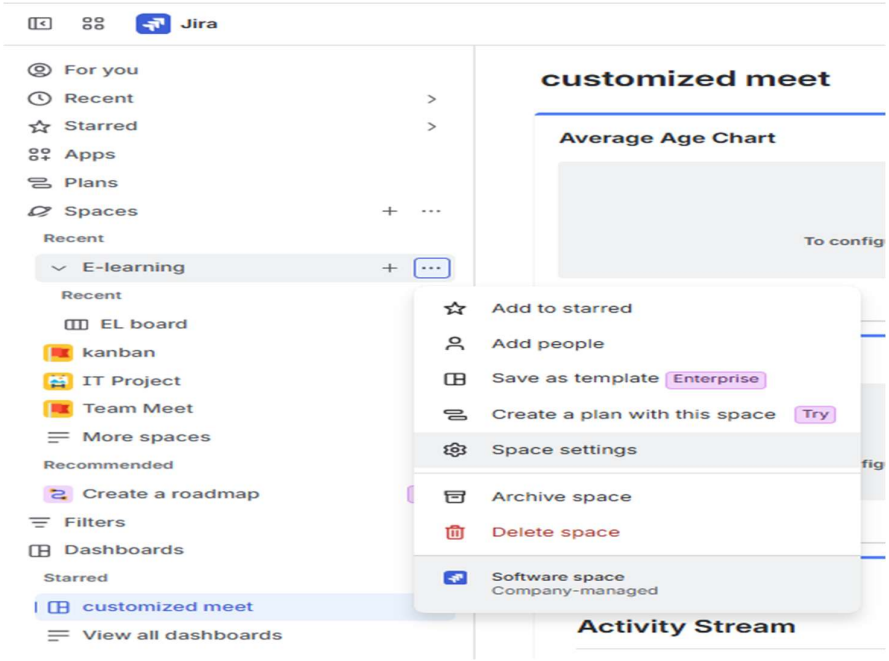


Practical No 10.
Permission & Security Management

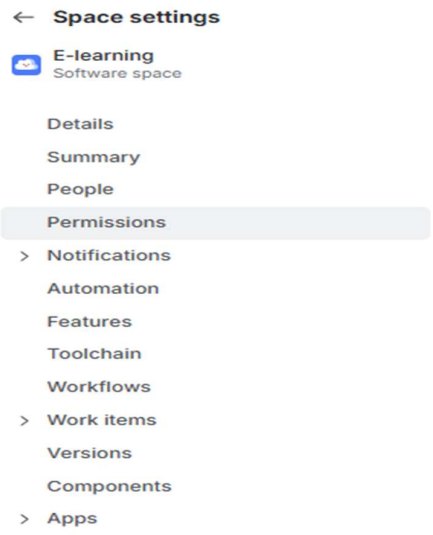
A. View and modify Project Permission Scheme.

Step 1: Open E-learning → Click on the three dots (...).

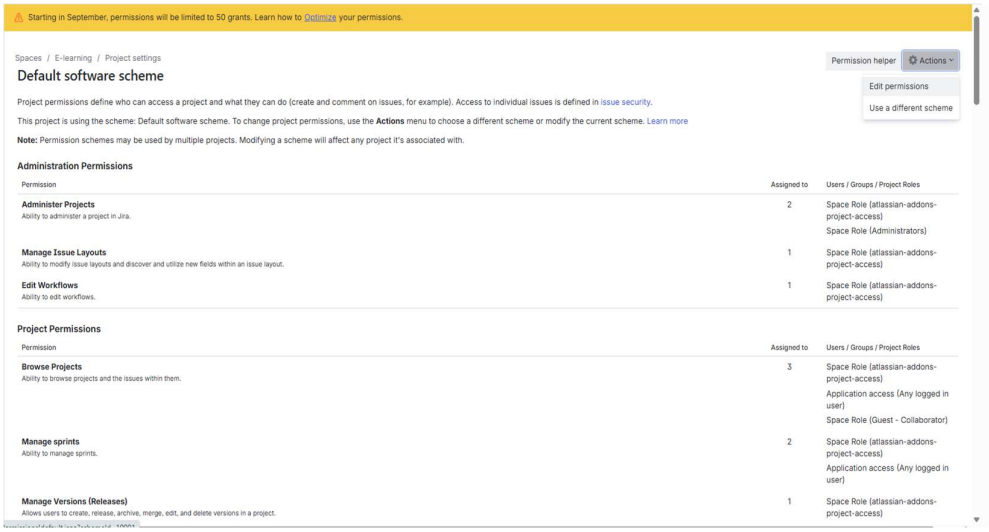
Step 2: Click on Space settings.



Step 3: Click on Permissions.



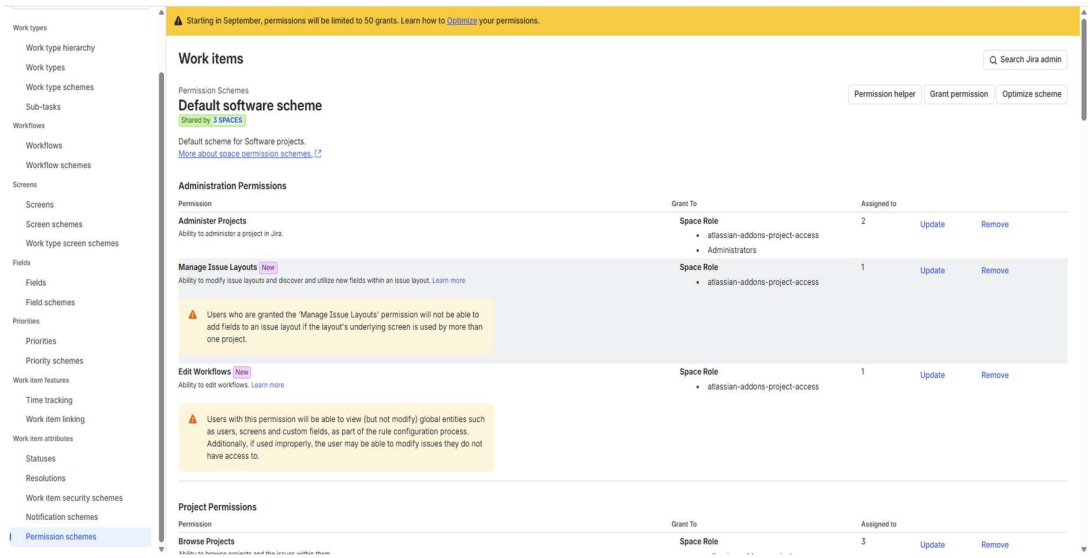
Step 4: Click on Actions or Edit Permissions.



Step 5: Select the required Permission.

Step 6: Add the required User, Group, or Project Role.

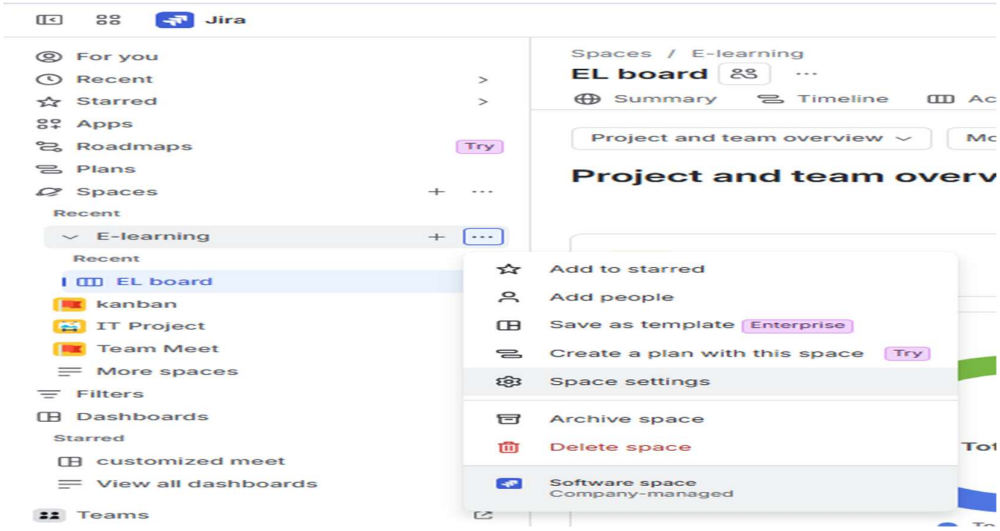
Step 7: Click on Save or Update.



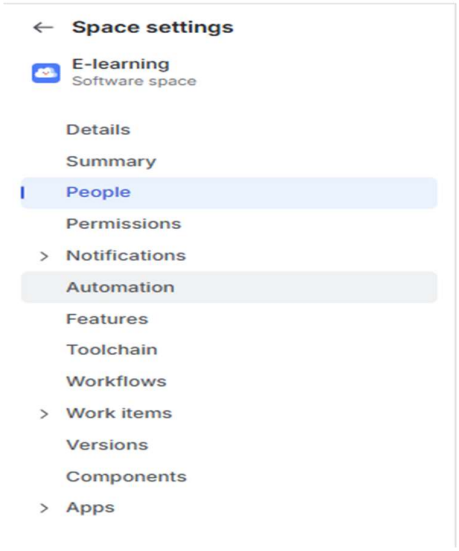
B. Configure Project Roles and assign permissions.

Step 1: Open E-learning → Click on the three dots (...).

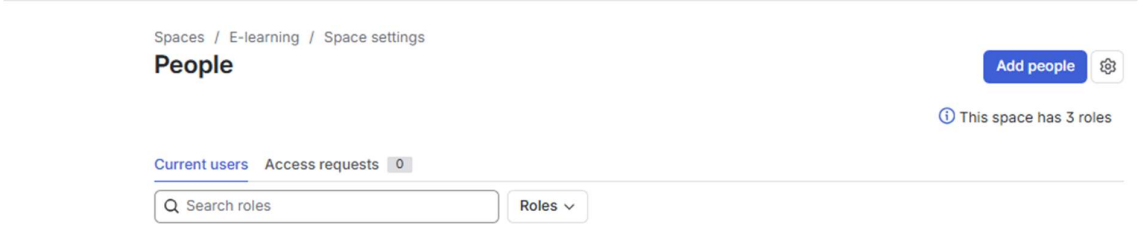
Step 2: Click on Space settings.



Step 3: Click on People or Project Roles.



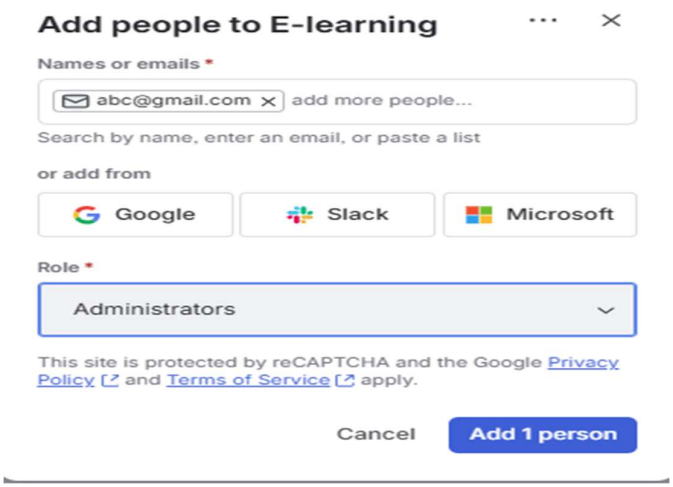
Step 4: Click on Add People.



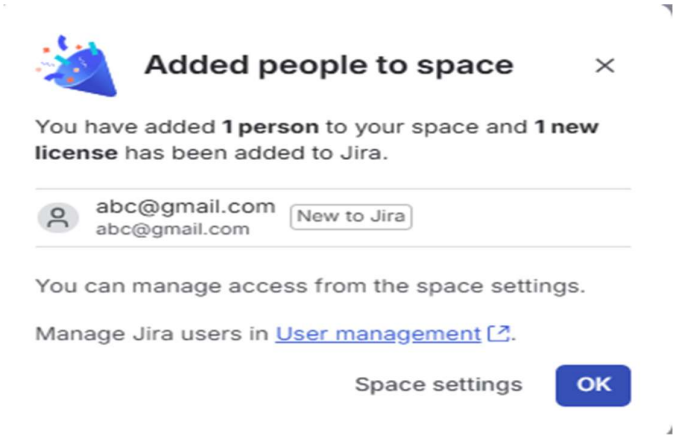
Step 5: Enter the User Name or Email Address.

Step 6: Select the required Project Role.

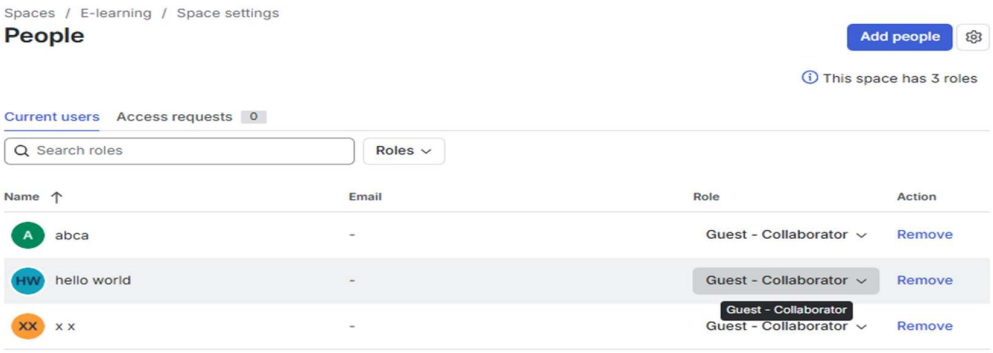
Step 7: Assign the role. → Click Add Person.



Step 8: Click on OK.



Step 9: Save the changes.

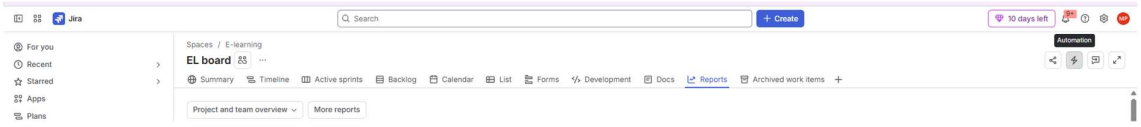


Practical No. 11
Jira Automation

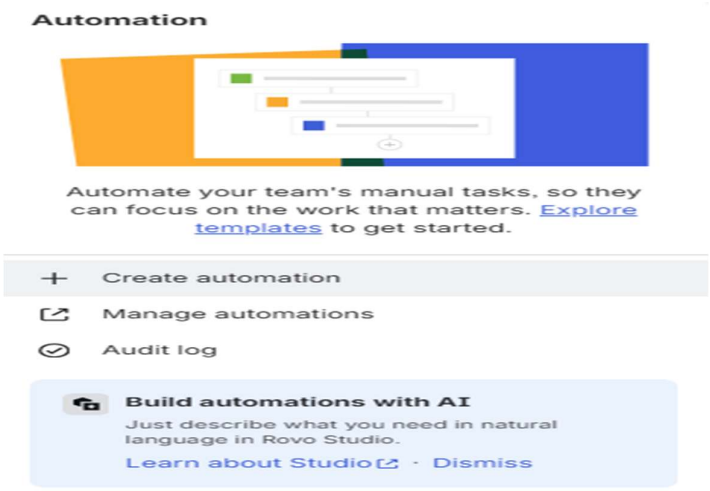
A. Create an Automation Rule (Trigger → Condition → Action).

Step 1: Open the Jira Project.

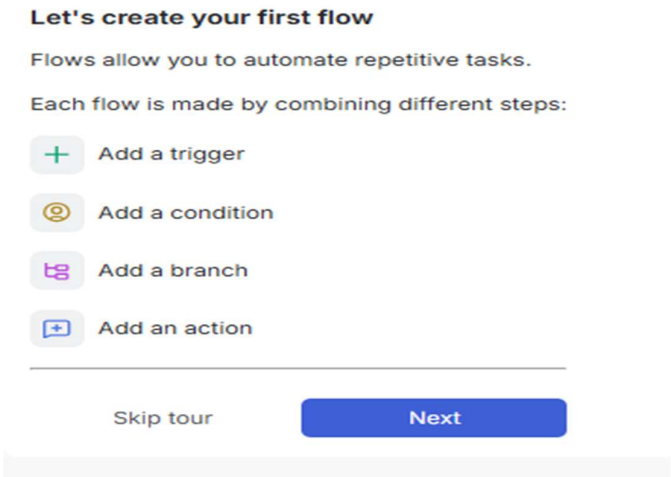
Step 2: Click on Automation.



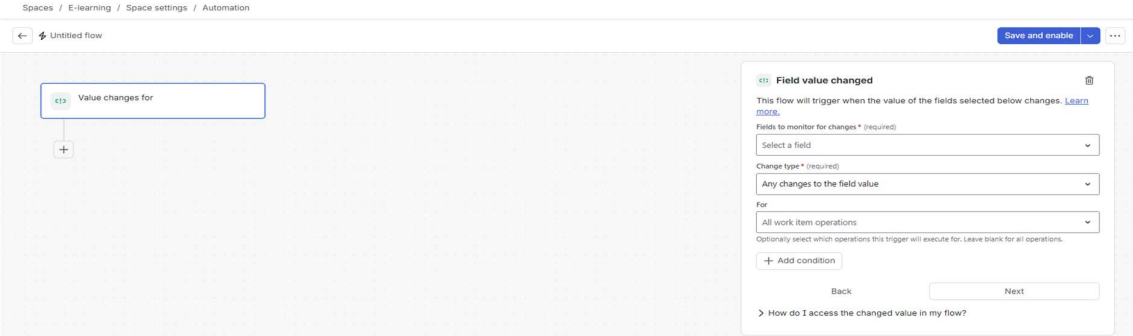
Step 3: Click on Create Automation.



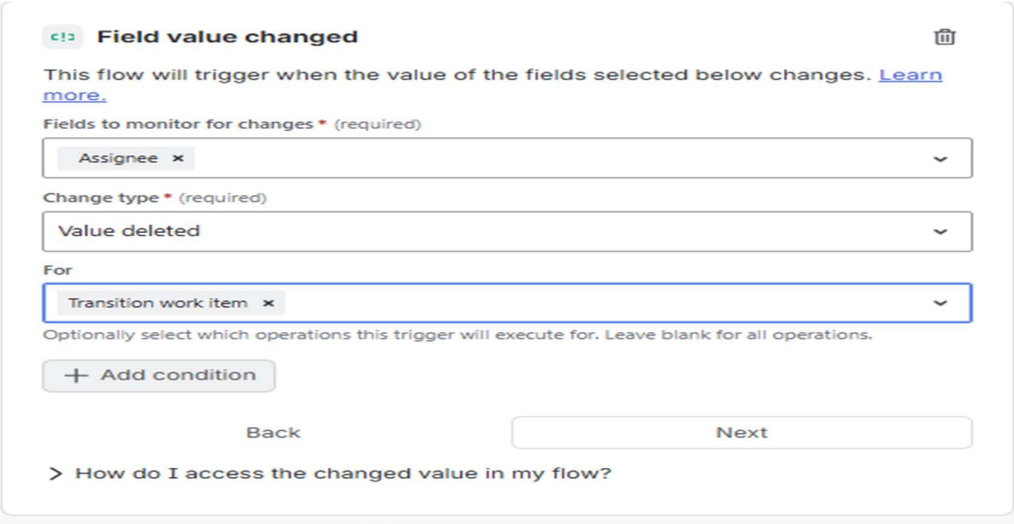
Step 4: Select the required Trigger.



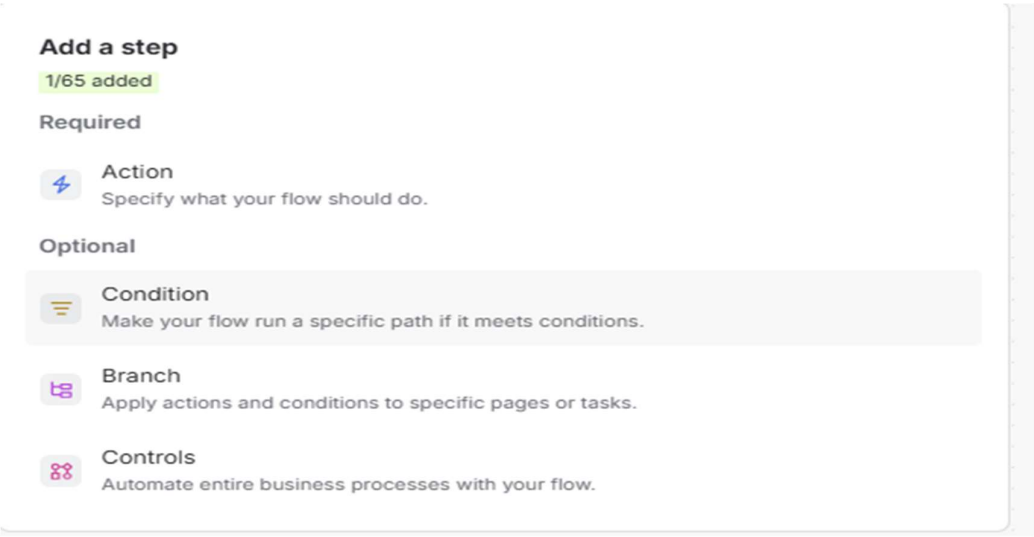
Step 5: Click Next.



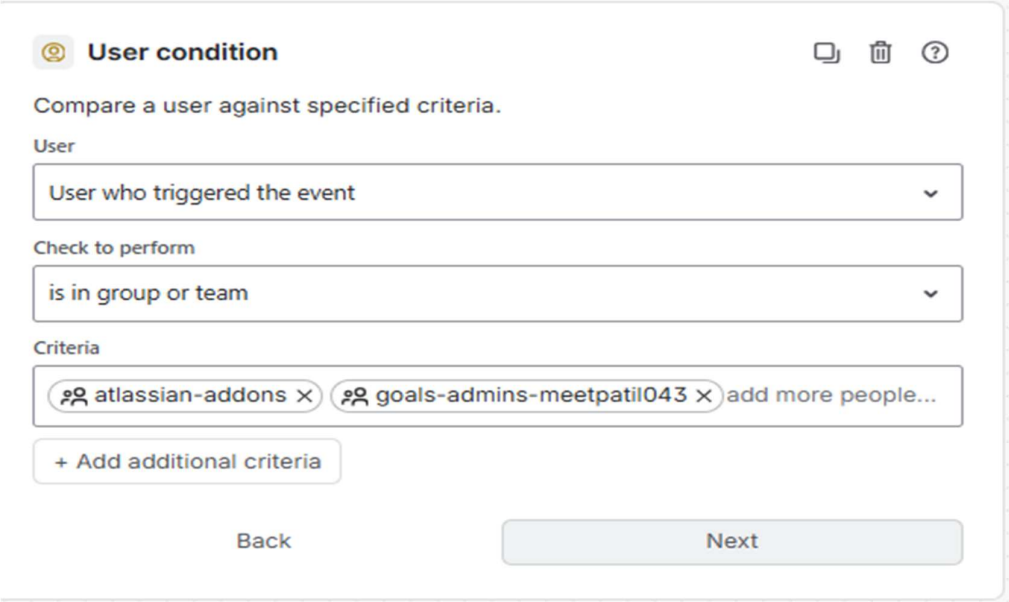
Step 6: Enter the required Fields and Click Next.



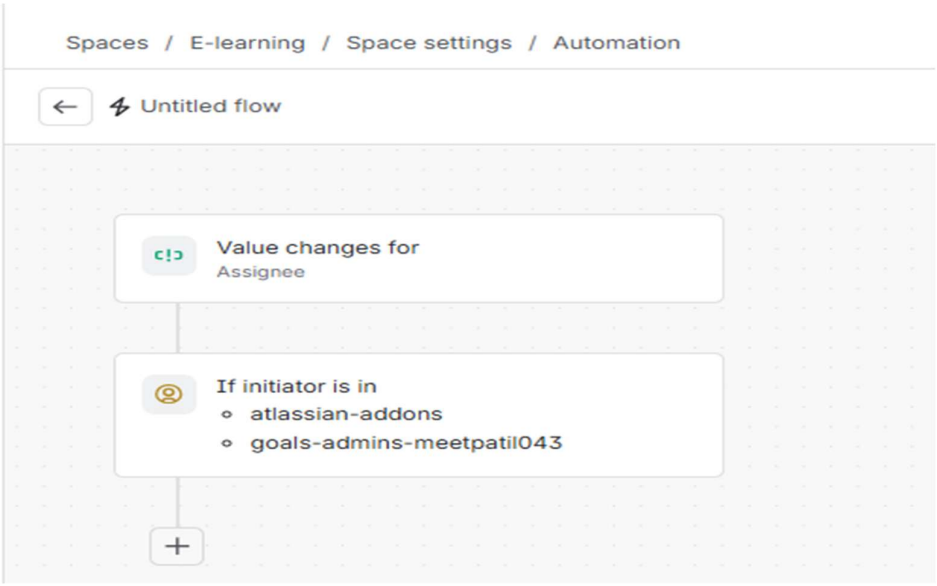
Step 7: Add a Condition



Step 8: Select the User Conditions.



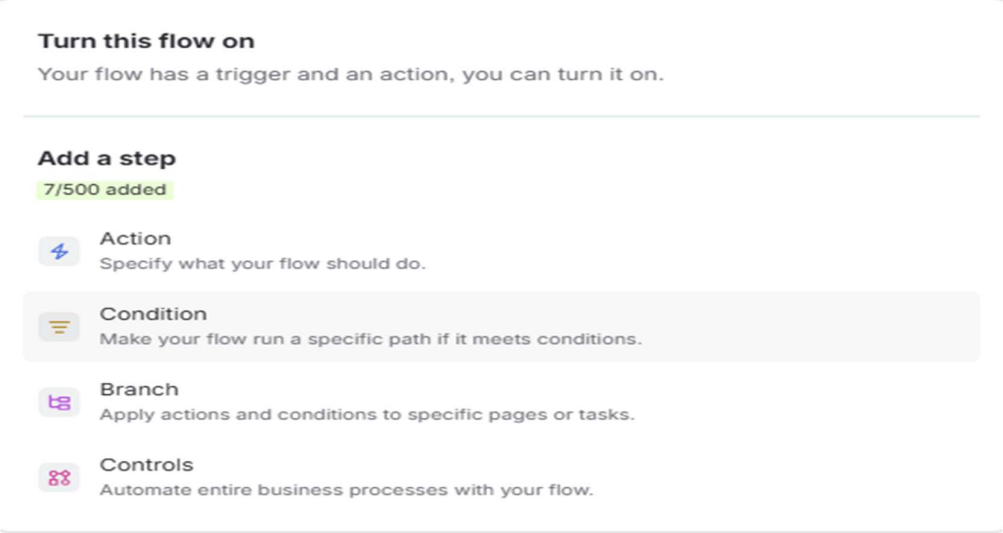
Step 9: Click on Next.



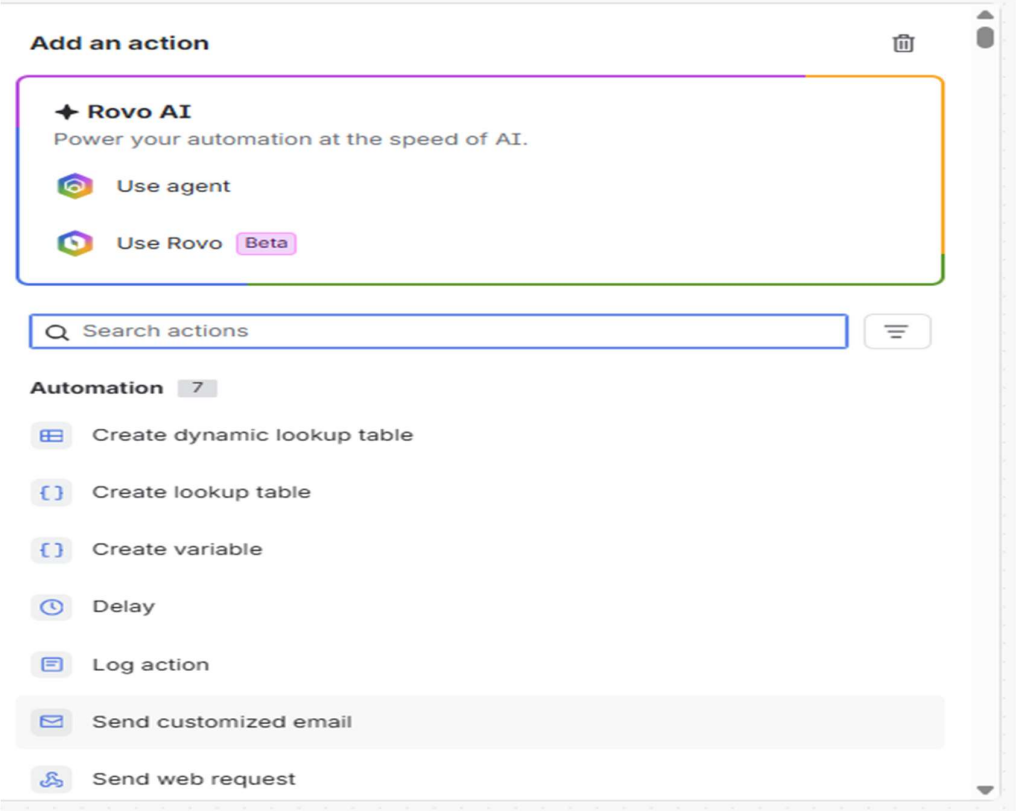
B. Configure rule (e.g., When issue is moved to Done → send email / auto-assign).

C. Test and enable automation rules.

Step 1: Add a Condition.

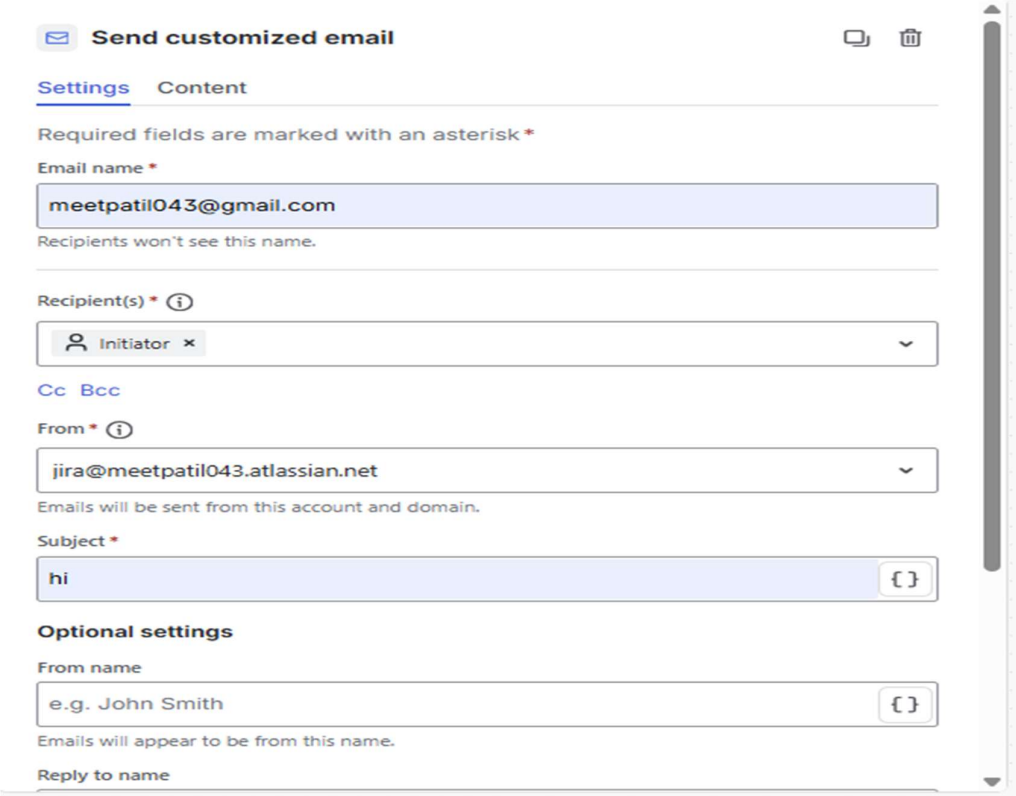


Step 2: In Automation Select the Send Customized Email.



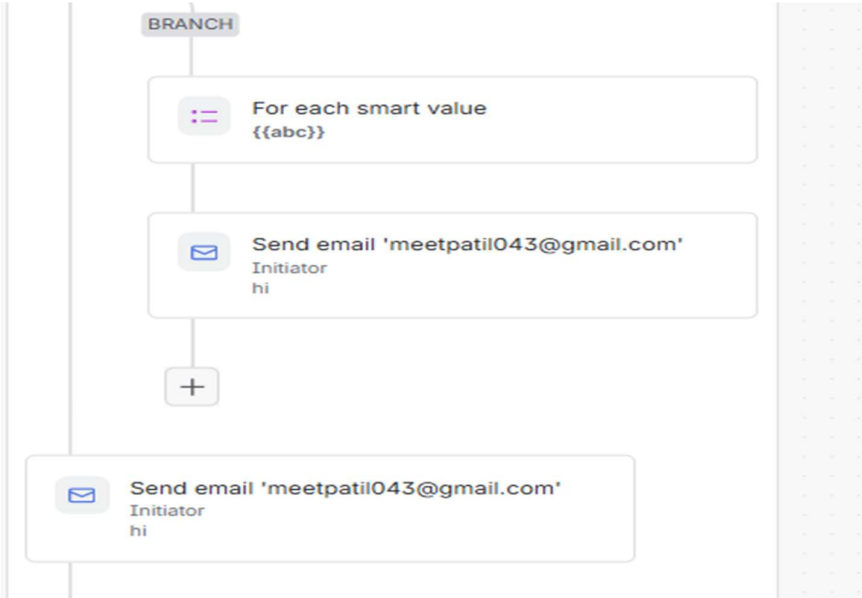
Step 3: In Settings Configure the details.

Step 4: Click on Save.



Step 5: Enable the Automation Rule.

Step 6: Save the changes.



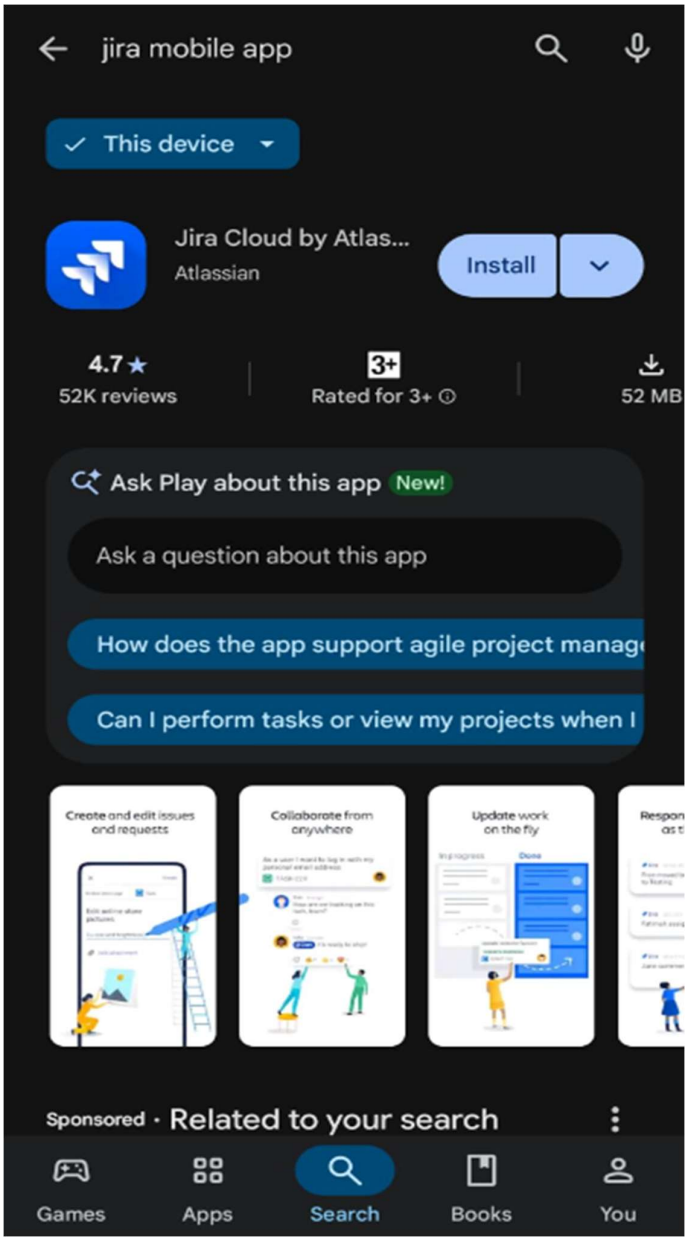
Practical No 12.
Introduction to Jira Mobile App

A. Install and configure the Jira Mobile App.

Step 1: Open the Google Play Store on your mobile phone.

Step 2: Search for Jira Cloud by Atlassian.

Step 3: Click on Install.



Step 4: Open the Jira Mobile App.



Step 5: Click on Log in.

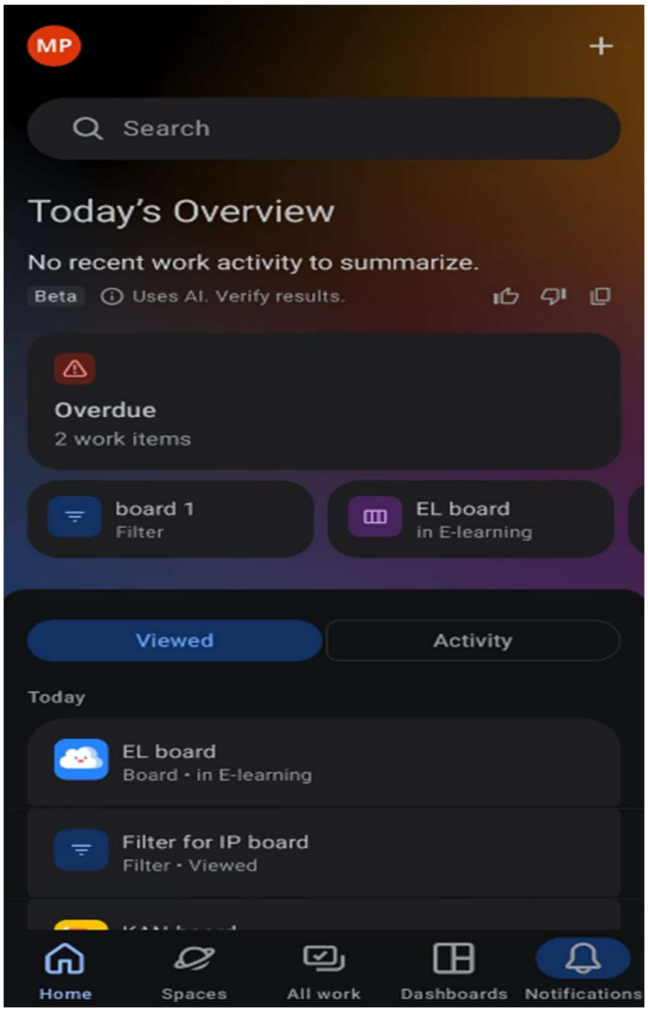
Step 6: Enter your Email Address.





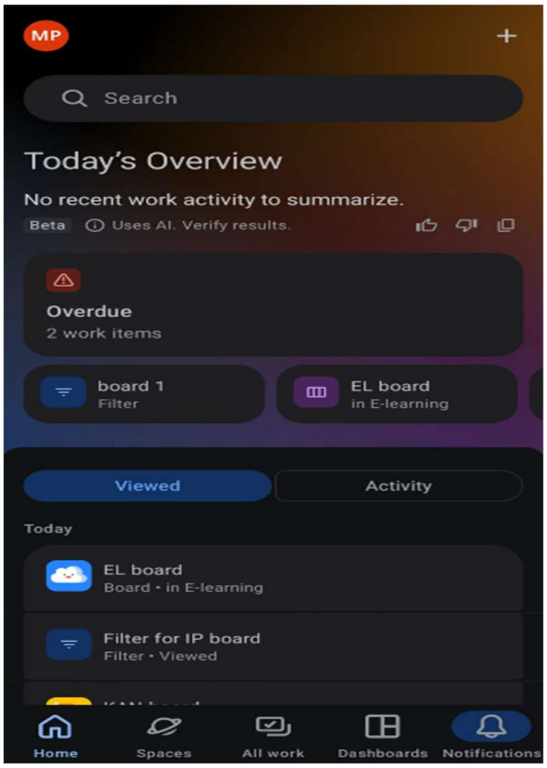
Step 7: Select the required Jira Workspace/Project.

Step 8: The Jira Mobile App is successfully configured.



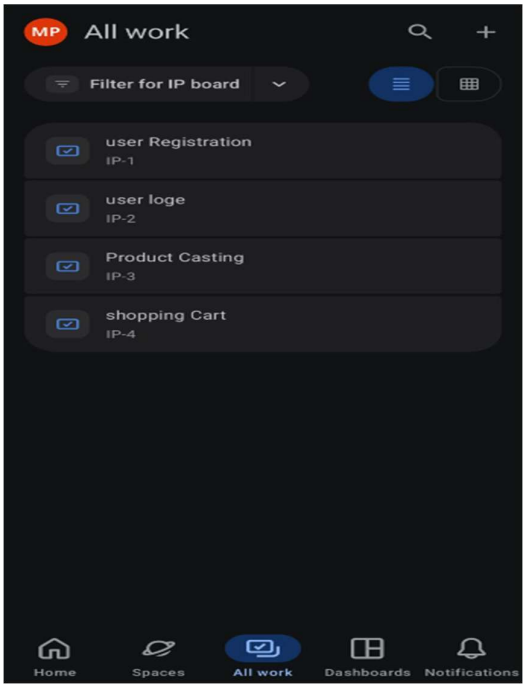
B. View and update issues from mobile (Status, Comment, Assign).

Step 1: Open the Jira mobile app.

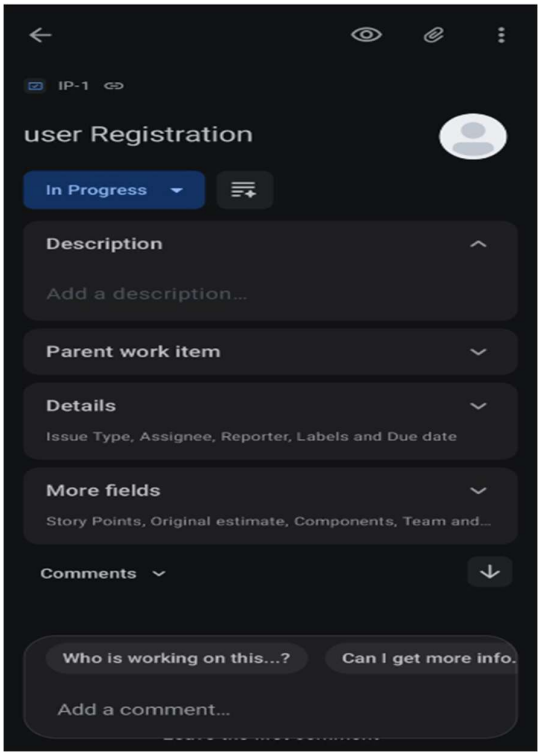


Step 2: Tap All work at the bottom.

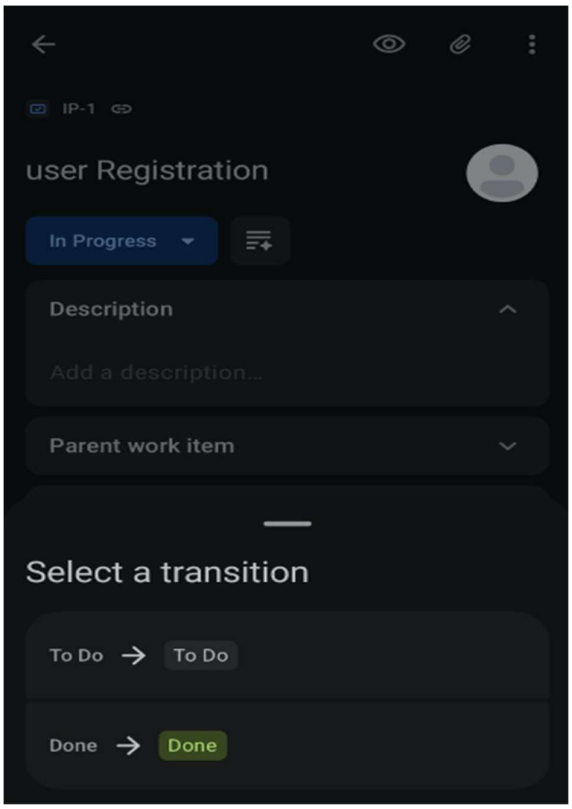
Step 3: Select Projects and choose your project.



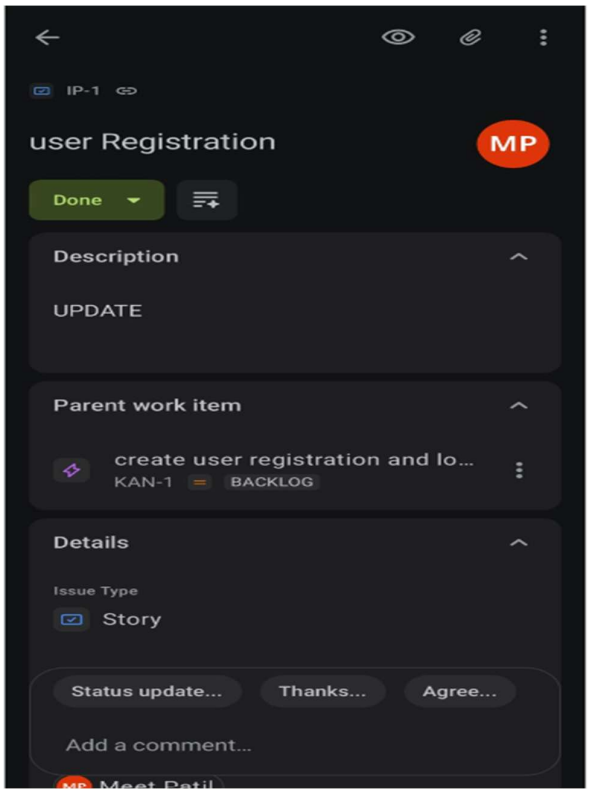
Step 4: Click the "In Progress".



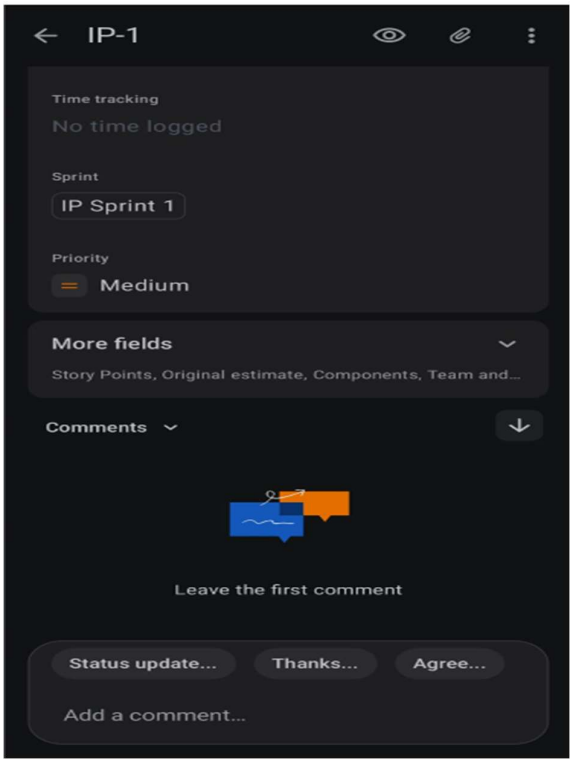
Step 5: Select another status, such as Done.



Step 6: Update the issue.

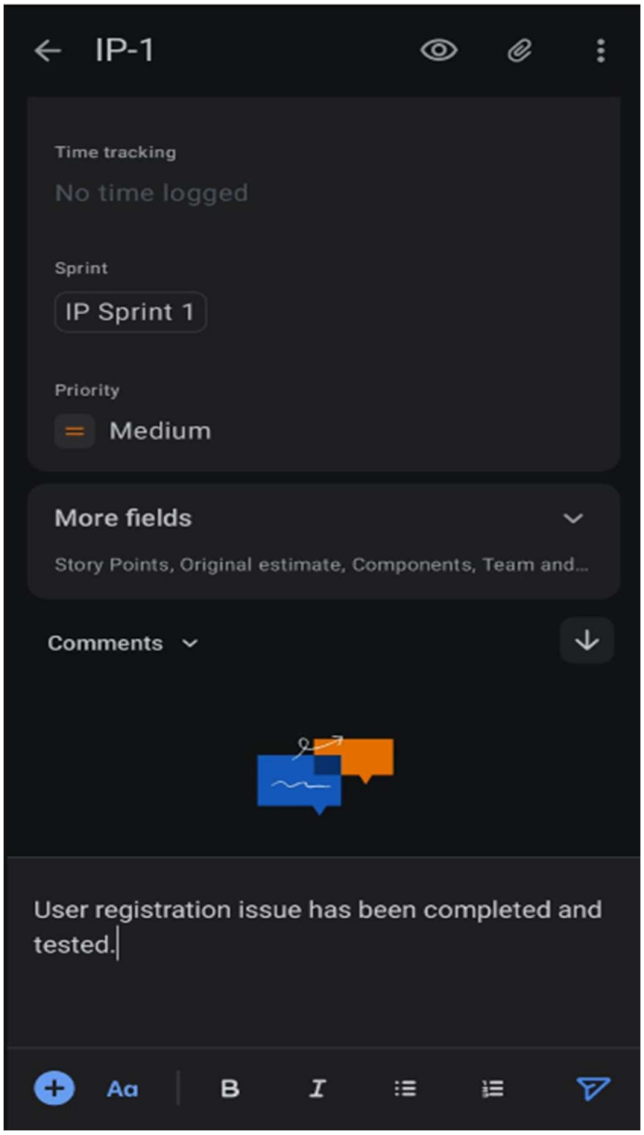


Step 7: Scroll down to the Comments section.



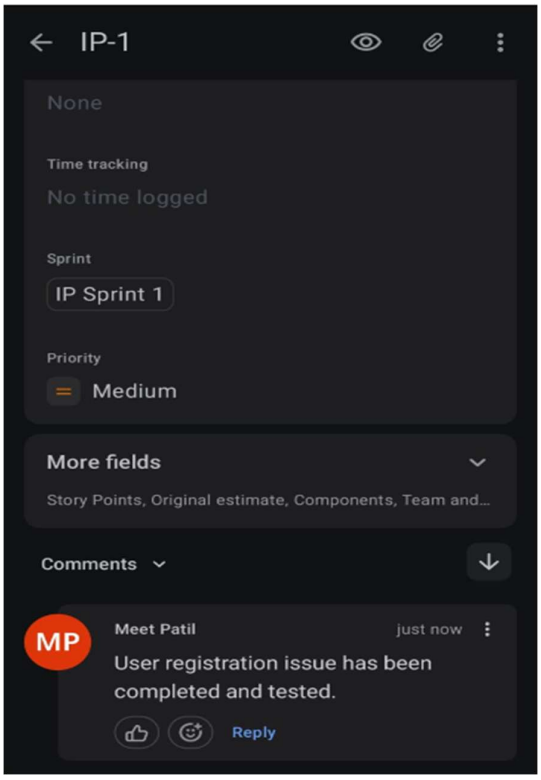
Step 8: Tap Add a comment...

Step 9: Enter this comment:

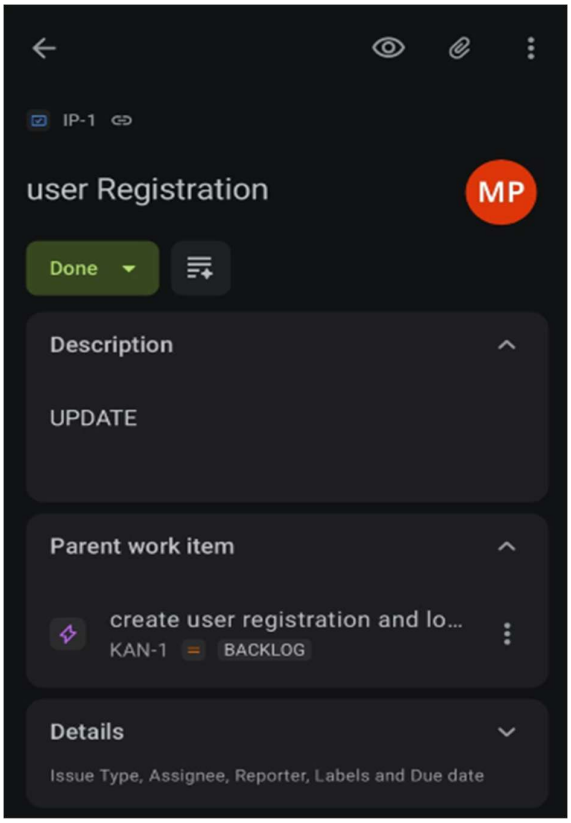


Step 10: Tap the Send (➤) button on your keyboard or the comment submission button.

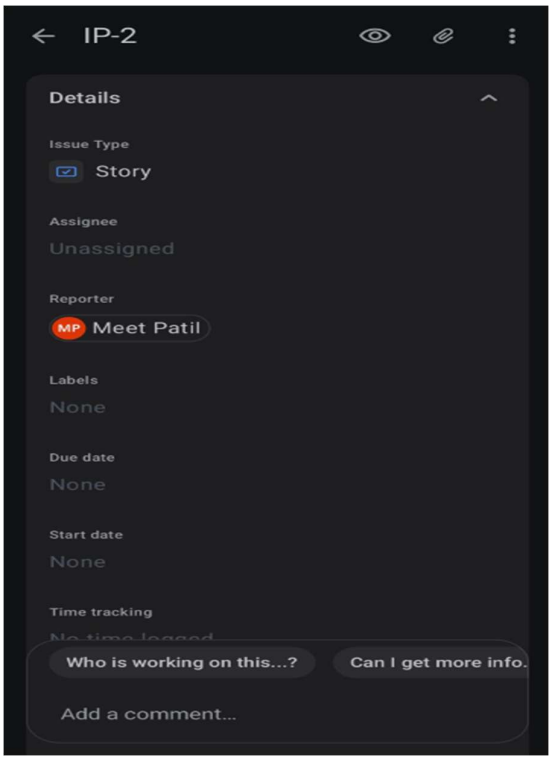
Step 11: Check that your comment appears under Comments.



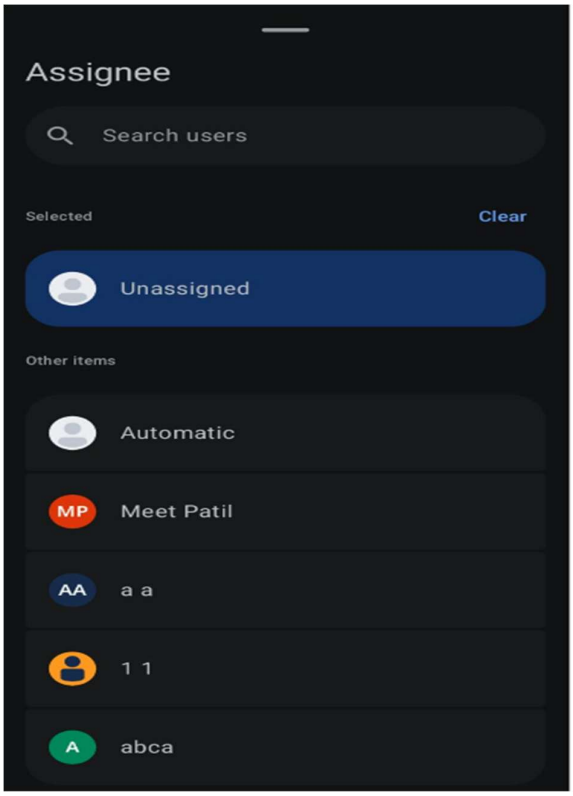
Step 12: Open the Details section.



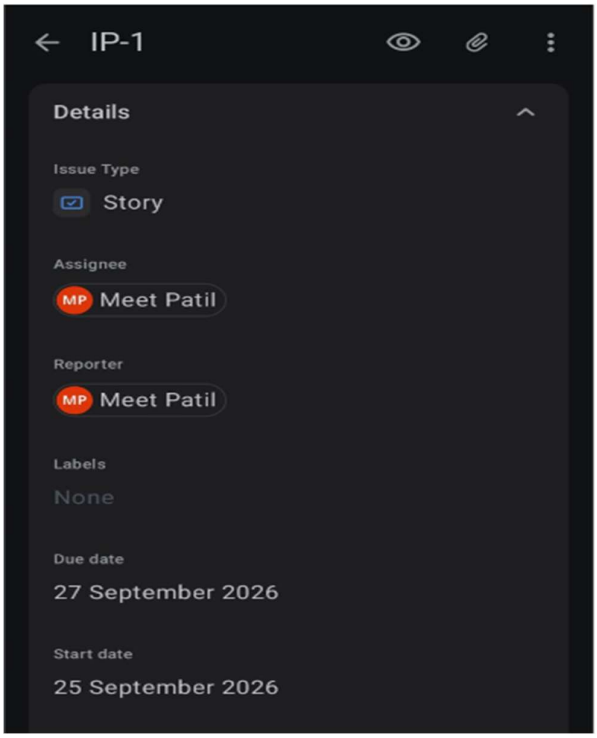
Step 13: Tap the Assignee field.



Step 14: Select your name or another available user.

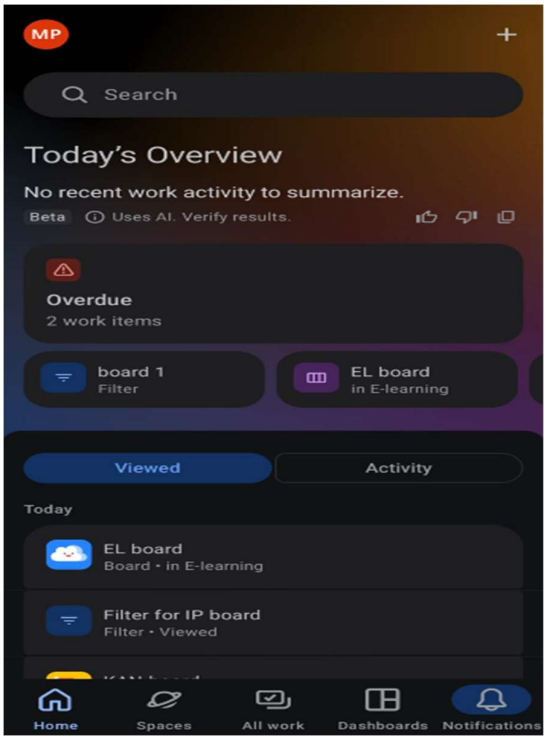


Step 15: Save the changes, if a Save button appears.

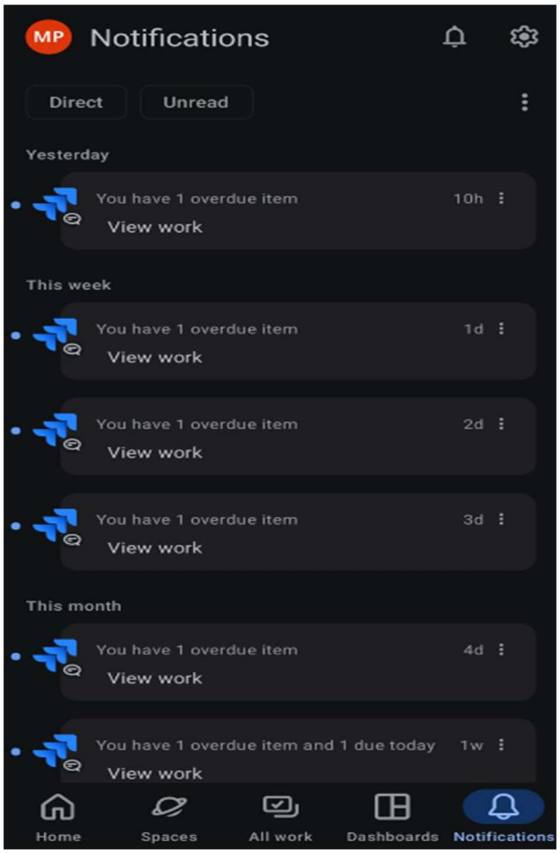


C. Receive notifications and manage real-time updates

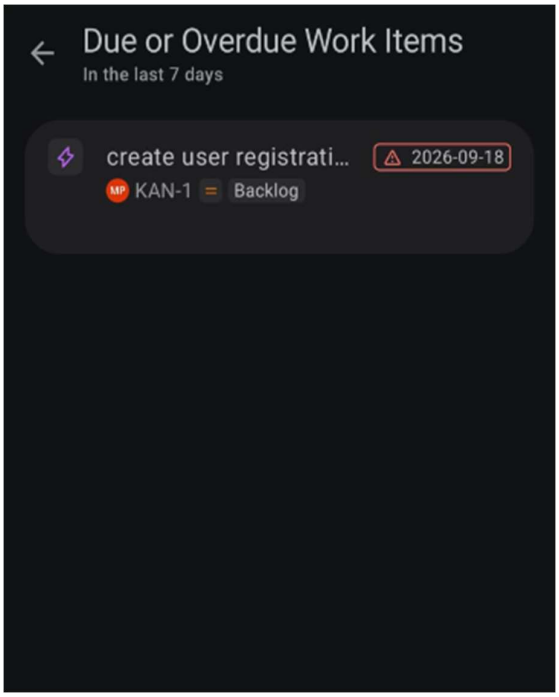
Step 1: Open the Jira Mobile App.



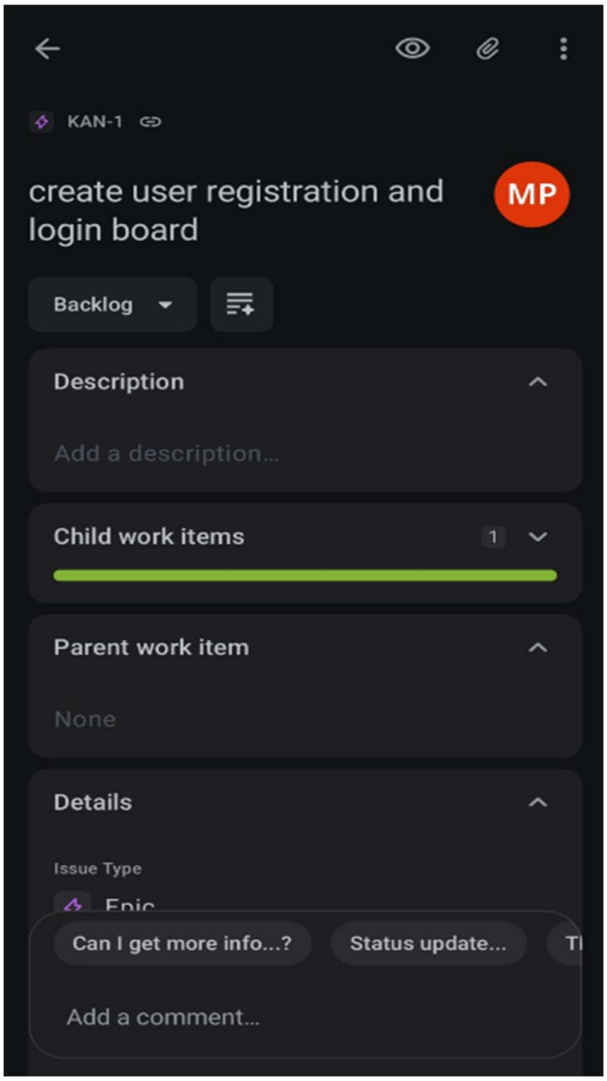
Step 2: Tap Notifications at the bottom of the screen.



Step 3: Tap View work on any notification.

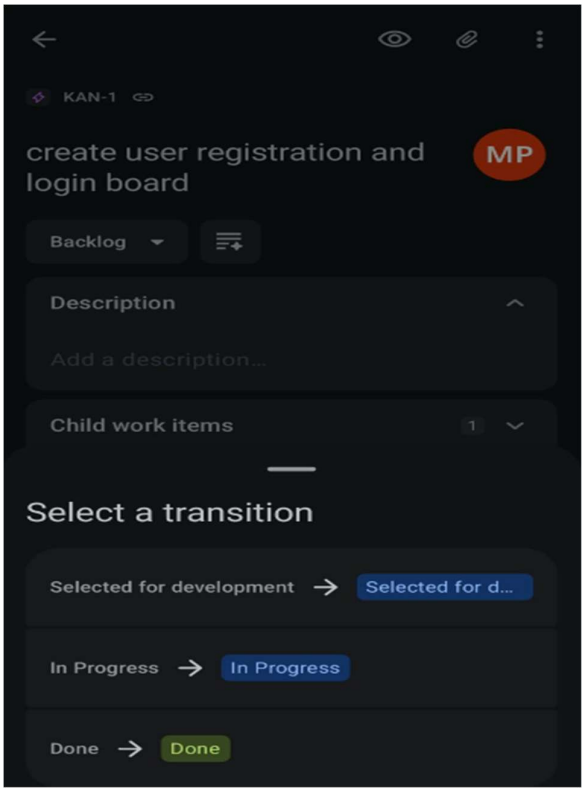


Step 4: Tap the create user registration issue.

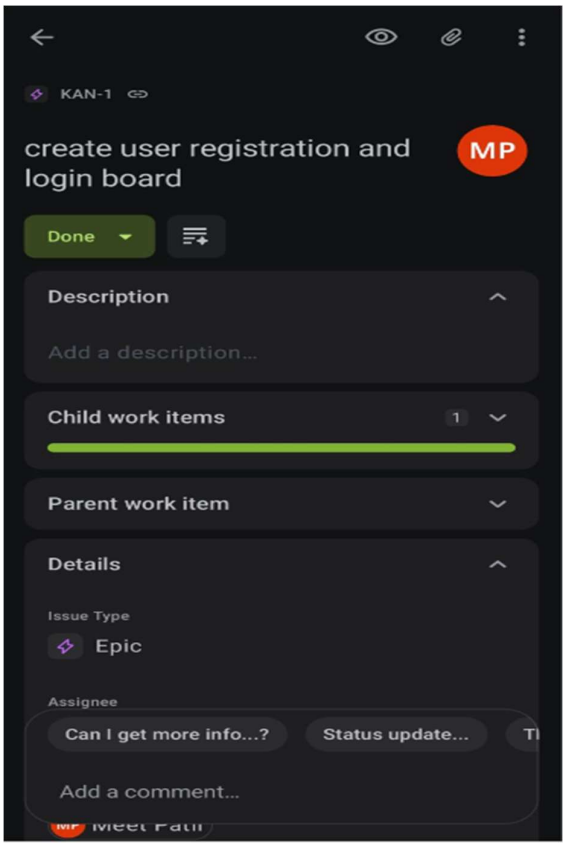


Step 5: Check the issue details, including the Backlog status and overdue date.

Step 6: Tap the Back arrow at the top-left.



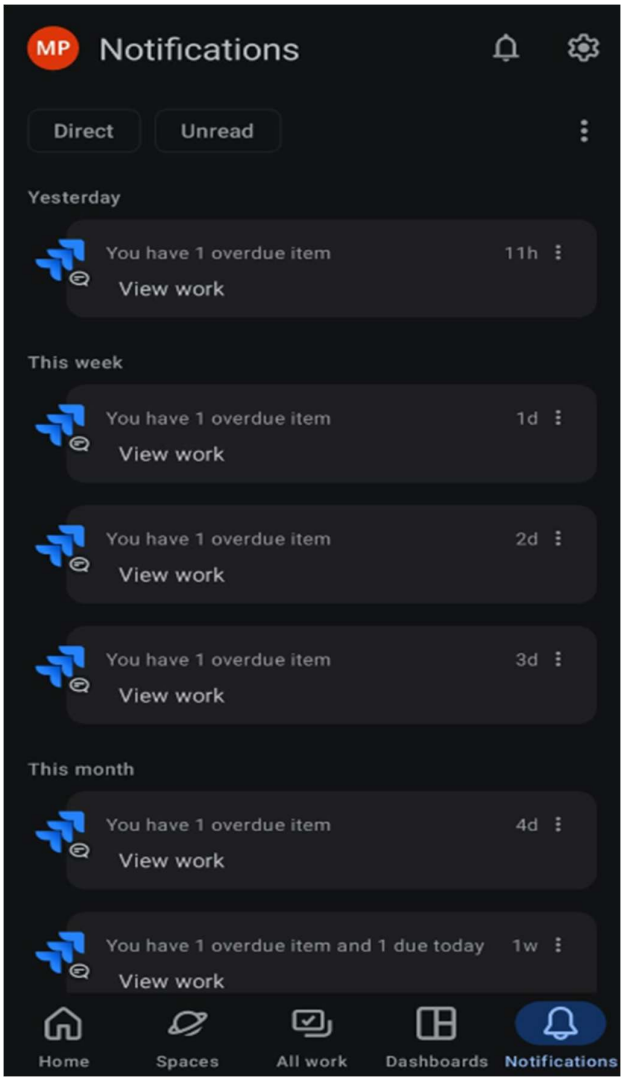
Step 7: Tap Done → Done.



Step 8: Tap the Back arrow (←) at the top-left.

Step 9: Return to the Notifications section.

Step 10: Check the notifications and updated work items.



Problem Solving by Searching(Informed Search)

Practical No.3(A)

Aim:- Given an initial configuration of the 8-puzzle and a goal configuration, write a Python program to find the solution using Greedy Best-First Search with the Manhattan Distance heuristic.

Code:-

```
import heapq

GOAL_STATE = (1, 2, 3,
              4, 5, 6,
              7, 8, 0)

GOAL_POSITIONS = {tile: (i // 3, i % 3) for i, tile in enumerate(GOAL_STATE)}


def get_manhattan_distance(state):
    distance = 0
    for i, tile in enumerate(state):
        if tile != 0:  # Skip the blank tile
            current_row, current_col = i // 3, i % 3
            goal_row, goal_col = GOAL_POSITIONS[tile]
            distance += abs(current_row - goal_row) + abs(current_col - goal_col)
    return distance


def get_neighbors(state):
    neighbors = []
    blank_idx = state.index(0)
    row, col = blank_idx // 3, blank_idx % 3

    moves = [(-1, 0), (1, 0), (0, -1), (0, 1)]  # Up, Down, Left, Right

    for dr, dc in moves:
        new_row, new_col = row + dr, col + dc
        if 0 <= new_row < 3 and 0 <= new_col < 3:
            new_blank_idx = new_row * 3 + new_col

            # Convert to list to swap tiles, then back to tuple
```

```
            new_state = list(state)
            new_state[blank_idx], new_state[new_blank_idx] = new_state[new_blank_idx],
new_state[blank_idx]
            neighbors.append(tuple(new_state))

    return neighbors


def greedy_best_first_search(start_state):
    open_list = []
    heapq.heappush(open_list, (get_manhattan_distance(start_state), start_state, [start_state]))

    visited = set()
    visited.add(start_state)

    while open_list:
        _, current_state, path = heapq.heappop(open_list)

        # Check if goal is reached
        if current_state == GOAL_STATE:
            return path

        for neighbor in get_neighbors(current_state):
            if neighbor not in visited:
                visited.add(neighbor)
                h_value = get_manhattan_distance(neighbor)
                heapq.heappush(open_list, (h_value, neighbor, path + [neighbor]))

    return None


def print_state(state):
    for i in range(0, 9, 3):
        print(list(state[i:i + 3]))
    print()


if __name__ == "__main__":
    initial_configuration = (1, 2, 3,
                             0, 4, 6,
                             7, 5, 8)

    print("Initial State:")
    print_state(initial_configuration)
```

```
solution_path = greedy_best_first_search(initial_configuration)

if solution_path:
    print(f"Solution found in {len(solution_path) - 1} moves!\n")

    for step, state in enumerate(solution_path):
        print(f"Step {step}:")
        print_state(state)
else:
    print("No solution could be found.")
```

Output:-

```
Initial State:
[1, 2, 3]
[0, 4, 6]
[7, 5, 8]

Solution found in 3 moves!

Step 0:
[1, 2, 3]
[0, 4, 6]
[7, 5, 8]

Step 1:
[1, 2, 3]
[4, 0, 6]
[7, 5, 8]

Step 2:
[1, 2, 3]
[4, 5, 6]
[7, 0, 8]

Step 3:
[1, 2, 3]
[4, 5, 6]
[7, 8, 0]
```

Practical No.3(B)

Aim:- Given a weighted graph representing citie and distances between them. Write a python program to find the shortest path from Arad to Bucharest using A* search with heuristic function(straight-line distance to destination).

Code:-

```
import heapq

# 1. Graph representation extracted directly from the map image:
romania_map = {
   'Arad': {'Zerind': 75, 'Sibiu': 140, 'Timisoara': 118},
   'Zerind': {'Arad': 75, 'Oradea': 71},
   'Oradea': {'Zerind': 71, 'Sibiu': 151},
   'Sibiu': {'Arad': 140, 'Oradea': 151, 'Fagaras': 99, 'Rimnicu Vilcea': 80},
   'Timisoara': {'Arad': 118, 'Lugoj': 111},
   'Lugoj': {'Timisoara': 111, 'Mehadia': 70},
   'Mehadia': {'Lugoj': 70, 'Drobeta': 75},
   'Drobeta': {'Mehadia': 75, 'Craiova': 120},
   'Craiova': {'Drobeta': 120, 'Rimnicu Vilcea': 146, 'Pitesti': 138},
   'Rimnicu Vilcea': {'Sibiu': 80, 'Craiova': 146, 'Pitesti': 97},
   'Fagaras': {'Sibiu': 99, 'Bucharest': 211},
   'Pitesti': {'Rimnicu Vilcea': 97, 'Craiova': 138, 'Bucharest': 101},
   'Bucharest': {'Fagaras': 211, 'Pitesti': 101, 'Giurgiu': 90, 'Urziceni': 85},
   'Giurgiu': {'Bucharest': 90},
   'Urziceni': {'Bucharest': 85, 'Vaslui': 142, 'Hirsova': 98},
   'Hirsova': {'Urziceni': 98, 'Eforie': 86},
   'Eforie': {'Hirsova': 86},
   'Vaslui': {'Urziceni': 142, 'Iasi': 92},
   'Iasi': {'Vaslui': 92, 'Neamt': 87},
   'Neamt': {'Iasi': 87}
}



# 2. Straight-line distance heuristic h(n) to Bucharest
straight_line_to_bucharest = {
   'Arad': 366, 'Bucharest': 0, 'Craiova': 160, 'Drobeta': 242, 'Eforie': 161,
   'Fagaras': 176, 'Giurgiu': 77, 'Hirsova': 151, 'Iasi': 226, 'Lugoj': 244,
   'Mehadia': 241, 'Neamt': 234, 'Oradea': 380, 'Pitesti': 100, 'Rimnicu Vilcea': 193,
```

```
    'Sibiu': 253, 'Timisoara': 329, 'Urziceni': 80, 'Vaslui': 199, 'Zerind': 374
}



def a_star_search(graph, heuristics, start, goal):
    """Solves the shortest path problem using A* Search."""
    # Priority queue stores tuples of: (f_score, counter, g_score, current_city, path)
    counter = 0
    pq = []

    g_score = {start: 0}
    f_score = heuristics[start]

    # Push initial node
    heapq.heappush(pq, (f_score, counter, 0, start, [start]))

    while pq:
        f, _, current_g, current_city, path = heapq.heappop(pq)

        # If goal is reached, return the path and distance
        if current_city == goal:
            return path, current_g

        # Explore neighbors
        for neighbor, distance in graph[current_city].items():
            tentative_g = current_g + distance

            # If a shorter path to neighbor is found
            if neighbor not in g_score or tentative_g < g_score[neighbor]:
                g_score[neighbor] = tentative_g
                h = heuristics[neighbor]
                f_neighbor = tentative_g + h

                counter += 1
                heapq.heappush(pq, (f_neighbor, counter, tentative_g, neighbor, path + [neighbor]))

    return None, float('inf')
```

```
# --- Run Search ---
if __name__ == "__main__":
    start = 'Arad'
    goal = 'Bucharest'

    path, distance = a_star_search(romania_map, straight_line_to_bucharest, start, goal)

    print(f"Shortest Path: {' -> '.join(path)}")
    print(f"Total Distance: {distance} km")
```

Output:-

```
Shortest Path: Arad -> Sibiu -> Rimnicu Vilcea -> Pitesti -> Bucharest
Total Distance: 418 km
```

Practical No.4 (Baye's Rule Application)

Scenario:-

A medical test is used to detect a particular disease.
The probability that a randomly selected person has the disease is 1%.
If a person has the disease, the test returns positive with probability 99%.
If a person does not have the disease, the test still returns positive with probability 5% (false positive rate).
A person takes the test and receives a positive result.

Aim:- Write a Python program to compute the probability that the person actually hasthe disease given that the test result is positive.

Code:-

```
def calculate_disease_probability():

    prob_disease=0.01
    prob_no_disease=0.99

    true_positive_rate=0.99
    false_positive_rate=0.05

    #1.Calculate the total probability of the positive test result P(T+)
prob_positive_test=(true_positive_rate*prob_disease)+(false_positive_rate*prob_no_disease)

    #2.Apply Baye's Theorem to calculate P(D|T+)
    prob_disease_given_positive=(true_positive_rate*prob_disease)/prob_positive_test

    print(f"Total probability of testing positive P(T+):{prob_positive_test:.4f}")
    print(f"Probability of actually having the disease given a positive test P(D|T+):
{prob_disease_given_positive:.4f}")
    print(f"In Percentage:{prob_disease_given_positive * 100:.2f}%")

if __name__=="__main__":
    calculate_disease_probability()
```

Output:-

```
Total probability of testing positive P(T+):0.0594
Probability of actually having the disease given a positive test P(D|T+): 0.1667
In Percentage:16.67%
```

PRACTICAL NO.5

Aim:- Using a small dataset (e.g., weather or spam dataset), implement a Naive Bayes classifier using scikit-learn.

Code:-

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB

#1. Create a tiny dataset(Text, and Label: 1 for spam, 0 for ham)
corpus=[
    "Get rich quick! Click here now!", #Spam
    "Hey, are we still meeting for lunch today?", #Ham
    "Free cash prize! Claim your reward.", #Spam
    "Can you send me the project report?", #Ham
    "Urgent: Account suspended. Verification required.", #Spam
    "Don't forget to buy milk on your way home.", #Ham
    ]
labels=[1,0,1,0,1,0]

#2. Convert text into numerical vectors(Bag of words)
vectorizer=CountVectorizer()
X=vectorizer.fit_transform(corpus)
y=labels

#3. Train the Naive Bayes model(MultinomialNB is great for text)
model= MultinomialNB()
model.fit(X,y)

#4. Test the model with brand new emails
test_emails=[
    "Hey buddy, let's grab a coffee later",
    "CONGRATULATIONS! You won a free vacation! Click now.",
    ]

#Convert test text using the same vectorizer
X_test=vectorizer.transform(test_emails)
predictions=model.predict(X_test)
```

```
#5. Output the results
for email,pred in zip(test_emails,predictions):
    status="SPAM 🚨" if pred==1 else "HAM(Safe)✔"
    print(f"Email:'{email}'--> Predicted as: {status}")
```

Output:-

Email:'Hey buddy, let's grab a coffee later'--> Predicted as: HAM(Safe)✔
Email:'CONGRATULATIONS! You won a free vacation! Click now.'--> Predicted as: SPAM 🚨

PRACTICAL NO.6

Aim:- Using a simple example (Burglary–Alarm), create a Bayesian Network using pgmpy.

Code:-

```
import pgmpy
from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import VariableElimination

model=DiscreteBayesianNetwork([('Burglary','Alarm'),('Earthquake','Alarm')])
cpd_burglary=TabularCPD(variable='Burglary',variable_card=2,values=[[0.99],[0.01]])

#Prior Probability of Earthquake: P(Earthquake)
cpd_earthquake=TabularCPD(variable='Earthquake',variable_card=2,values=[[0.998],[0.002]
])

#B=0,E=0),(B=0,E=1),(B=1,E=0),(B=1,E=1)
cpd_alarm=TabularCPD(
    variable='Alarm',
    variable_card=2,
    values=[
        [0.999,0.71,0.06,0.05], #P(Alarm=False)
        [0.001,0.29,0.94,0.95] #P(Alarm=True)
        ],
    evidence=['Burglary','Earthquake'],
    evidence_card=[2,2]
)

#3. Add CPDs to the model and validate
model.add_cpds(cpd_burglary,cpd_earthquake,cpd_alarm)
assert model.check_model(),"Model defination is invalid!"

#4. Perform Exact inference using variable elimination
infer=VariableElimination(model)

#Compute posterior probability: P(Burglary | Alarm = True)
posterior=infer.query(variables=['Burglary'],evidence={'Alarm':1})
```

```
#Display the output
print("Posterior Probability distribution P(Burglary| Alarm = True):")
print(posterior)
```

Output:-

```
Posterior Probability distribution P(Burglary| Alarm = True):
+-------------+-----------------+
| Burglary    |   phi(Burglary) |
+=============+=================+
| Burglary(0) |          0.1425 |
+-------------+-----------------+
| Burglary(1) |          0.8575 |
+-------------+-----------------+
```

4.)Aim:- Implement k-fold Cross-Valuation on a Classification Model.

Code:-

```
from sklearn.datasets import load_iris
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import KFold,cross_val_score

iris=load_iris()
X,y=iris.data,iris.target
model=RandomForestClassifier(random_state=42)
KF=KFold(n_splits=5,shuffle=True,random_state=42)
scores=cross_val_score(model ,X,y,cv=KF,scoring="accuracy")
print("----------5-Fold Cross-Validation Results----------")
for fold_idx, scores in enumerate(scores,1):
    print(f"Fold{fold_idx} Accuracy:{scores*100:.2f}%")

print("-"*38)
print(f"Mean Accuracy :   {scores.mean()*100:.2f}%")
print(f"Std Deviatoin:      {scores.std()*100:.2f}%")
```

Output:-

```
----------5-Fold Cross-Validation Results----------
Fold1 Accuracy:100.00%
Fold2 Accuracy:96.67%
Fold3 Accuracy:93.33%
Fold4 Accuracy:93.33%
Fold5 Accuracy:96.67%
--------------------------------------
Mean Accuracy :   96.67%
Std Deviation:      0.00%
```

5.)Aim:- Implement a Decision Tree Classifier using Iris on any relevant dataset**.**

Code:-

```
from sklearn.datasets import load_iris
from sklearn.metrics import accuracy_score
from sklearn.tree import DecisionTreeClassifier,export_text
from sklearn.model_selection import train_test_split

#1.Load the Iris dataset
iris=load_iris()
X,y=iris.data, iris.target

#2.Split into Traininig and Testing sets(80% train,20%test)
X_train,X_test,y_train,y_test=train_test_split(
X,y,test_size=0.2, random_state=42)

#3.Initialize and train the Decision Tree Classifier
#We set max_depth to keep the tree simple and avopid overfitting
model=DecisionTreeClassifier(max_depth=3,random_state=42)
model.fit(X_train,y_train)

#4.Evaluate the Model.
y_pred=model.predict(X_test)
accuracy=accuracy_score(y_test,y_pred)
print(f"Training Accuracy: {accuracy*100:.2f}%\n")

#5.Display the decision rules in text format.
print("---------Learned Decision tree Rules----------")
tree_rules=export_text(model, feature_names=list(iris.feature_names))
print(tree_rules)
```

Output:-

```
Training Accuracy: 100.00%

---------Learned Decision tree Rules------------
|--- petal length (cm) <= 2.45
|   |--- class: 0
|--- petal length (cm) >  2.45
|   |--- petal length (cm) <= 4.75
|   |   |--- petal width (cm) <= 1.65
|   |   |   |--- class: 1
|   |   |--- petal width (cm) >  1.65
|   |   |   |--- class: 2
|   |--- petal length (cm) >  4.75
|   |   |--- petal width (cm) <= 1.75
|   |   |   |--- class: 1
|   |   |--- petal width (cm) >  1.75
|   |   |   |--- class: 2
```

6.)Aim:- Implement Decision tree regression on regression dataset.

Code:-

```
import pandas as pd
from sklearn.datasets import load_diabetes
from sklearn.metrics import mean_squared_error,r2_score
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeRegressor,export_text

#1.Load the regression dataset.
diabetes=load_diabetes()
X=pd.DataFrame(diabetes.data,columns=diabetes.feature_names)
y=diabetes.target

#2.Split into training and testing sets(80% train,20% test)
X_train,X_test,y_train,y_test=train_test_split(
X,y,test_size=0.2,random_state=42)

#3.Initialize and train the Decision Tree Regression.
regressor=DecisionTreeRegressor(max_depth=3, random_state=42)
regressor.fit(X_train,y_train)
y_pred=regressor.predict(X_test)
mse=mean_squared_error(y_test,y_pred)
r2=r2_score(y_test,y_pred)

print("----------Evaluation Metrics--------")
print(f"Mean Squared Error(MSE):{mse:.2f}")
print(f"R-squared Score(R2):    {r2:.2f}")
print("---------Learned Regression Rules(First few levels)-------")
tree_rules=export_text(regressor,feature_names=list(diabetes.feature_names))
print(tree_rules)
```

Output:-

```
----------Evaluation Metrics--------
Mean Squared Error(MSE):3552.70
R-squared Score(R2):    0.33
---------Learned Regression Rules(First few levels)-------
|--- bmi <= 0.01
|   |--- s5 <= 0.01
|   |   |--- s5 <= -0.04
|   |   |   |--- value: [80.88]
|   |   |--- s5 >  -0.04
|   |   |   |--- value: [109.92]
|   |--- s5 >  0.01
|   |   |--- s4 <= 0.09
|   |   |   |--- value: [159.57]
|   |   |--- s4 >  0.09
|   |   |   |--- value: [256.33]
|--- bmi >  0.01
|   |--- bmi <= 0.07
|   |   |--- s6 <= 0.03
|   |   |   |--- value: [175.80]
|   |   |--- s6 >  0.03
|   |   |   |--- value: [230.52]
|   |--- bmi >  0.07
|   |   |--- s2 <= 0.02
|   |   |   |--- value: [291.22]
|   |   |--- s2 >  0.02
|   |   |   |--- value: [225.75]
```

7.)Aim:- Implement k-Nearest Neighbors (k-NN) classifier.

Code:-

```
from sklearn.datasets import load_iris
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
#1.Load the Iris dataset.
iris=load_iris()
X,y=iris.data, iris.target

#2.split into training and testing sets (80% train,20%test)
X_train,X_test,y_train,y_test=train_test_split(
X,y,test_size=0.2,random_state=42)

#3.Scale featuires
scaler = StandardScaler()
X_train_scaled=scaler.fit_transform(X_train)
X_test_scaled=scaler.transform(X_test)

#4.Initialize and train the k-NN Classifier(using 3 neighbors)
knn=KNeighborsClassifier(n_neighbors=3)
knn.fit(X_train_scaled, y_train)

#5.Make predictions and Evaluate accuracy
y_pred=knn.predict(X_test_scaled)
accuracy=accuracy_score(y_test,y_pred)

print(f"KNN Classifier4 Accuracy (k=3):{accuracy*100:.2f}%")
```

Output:-

```
KNN Classifier4 Accuracy (k=3):100.00%
```

8.)Aim:- Implement multiclass classification using logistic Regression or Decision Tree.

Code:-

```
from sklearn.datasets import load_iris
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics  import classification_report
from sklearn.preprocessing import StandardScaler

#1.Load the Iris dataset.
iris=load_iris()
X,y=iris.data, iris.target

#2.split into training and testing sets (80% train,20%test)
X_train,X_test,y_train,y_test=train_test_split(
X,y,test_size=0.2,random_state=42)

#3.Scale featuires
scaler = StandardScaler()
X_train_scaled=scaler.fit_transform(X_train)
X_test_scaled=scaler.transform(X_test)

#4.Train Multiclass Logistic Regression
model=LogisticRegression(random_state=42)
model.fit(X_train_scaled,y_train)

#5.Evaluate the Model.
y_pred=model.predict(X_test_scaled)

print("------------Multiclass Classification Report----------")
```

```
print(classification_report(y_test,y_pred,target_names=iris.target_names))
```

Output:-

```
------------Multiclass Classification Report-------------------
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        10
  versicolor       1.00      1.00      1.00         9
   virginica       1.00      1.00      1.00        11

    accuracy                           1.00        30
   macro avg       1.00      1.00      1.00        30
weighted avg       1.00      1.00      1.00        30
```

PRACTICAL 8. Ethics and Bias Analysis

Aim:- Select any publicly available dataset (for example, Iris, Adult Income, or a classification dataset of your choice) and analyze it for possible bias and class Imbalance.

Code:-

```
import pandas as pd
from sklearn.datasets import load_iris
iris=load_iris()
data=pd.DataFrame(
    iris.data,
    columns=iris.feature_names
)

data['Class']=iris.target
print("First 5 record")
print(data.head())
print("\nClass Distribution")
print(data['Class'].value_counts())

print("Missing Values:")
print(data.isnull().sum())
```

Output:-

```
First 5 record
   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)  \
0                5.1               3.5                1.4               0.2
1                4.9               3.0                1.4               0.2
2                4.7               3.2                1.3               0.2
3                4.6               3.1                1.5               0.2
4                5.0               3.6                1.4               0.2

   Class
0      0
1      0
2      0
3      0
4      0
```

Class Distribution
Class
0 50
1 50
2 50

Name: count, dtype: int64
Missing Values:
sepal length (cm) 0
sepal width (cm) 0
petal length (cm) 0
petal width (cm) 0
Class 0
dtype: int64

Practical No.1(Vaccum Cleaner)

Aim:- Design a simple rational agent by defining PEAS for vacuum cleaner environment and autonomous taxi. Classify enviornment. Implement a simple table driven agent in python.

Code:-

```
class TableDrivenVacuumAgent:
    def __init__(self):

        self.percept_history=[]
        self.table={
            #Time Step 1:Single percept sequences.
            ((('A','Clean'),),):'Right',
            ((('A','Dirty'),),):'Suck',
            ((('B','Clean'),),):'Left',
            ((('B','Dirty'),),):'Suck',
            #Time Step 2:Two percept sequences(examples of history).
            ((('A','Clean'),),(('B','Dirty'),)):'Suck',
            ((('A','Clean'),),(('B','Clean'),)):'Right',
            ((('B','Dirty'),),(('A','Clean'),)):'Right',
            ((('B','Dirty'),),(('B','Clean'),)):'Left',

            #Time Step 3:Three percept sequences
            ((('A','Dirty'),),(('A','Clean'),),(('B','Dirty'),)):'Suck',
            ((('A','Dirty'),),(('A','Clean'),),(('B','Clean'),)):'Left',
            }

    def act(self,current_percept):
        self.percept_history.append(current_percept)
        #Convert history list to tuple of tuples so it can be used as dictionary key.
        history_key=tuple(self.percept_history)
        action=self.table.get(history_key,'Right')
        return action
```

```
if __name__=="__main__":
    agent=TableDrivenVacuumAgent()
    #Step 1:Agent starts in room A, which is dirty.
    percept_1=('A','Dirty')
    action_1=agent.act(percept_1)
    print(f"Percept:{percept_1}->Action:{action_1}")

    #Step 2:Agent sucked the dirt, so the room A is now cleaned.
    percept_2=('A','Clean')
    action_2=agent.act(percept_2)
    print(f"Perecpt:{percept_2}->Action:{action_2}")

    #Step 3:Agent moved Right and finds Room B is dirty.
    percept_3=('B','Dirty')
    action_3=agent.act(percept_3)
    print(f"Percept:{percept_3}->Action:{action_3}")
```

Output:-

```
Percept:('A', 'Dirty')->Action:Right
Perecpt:('A', 'Clean')->Action:Right
Percept:('B', 'Dirty')->Action:Right
```

Practical No.1(Taxi Driver)

Code:-

```
class TableDrivenTaxiAgent:
    def __init__(self):
        self.percept_history=[]
        #Percept Format:(Tarffic Signal,Obstacle)
        self.table={
            #Time Step 1: Single percept sequence
            (('Red','No'),):'Stop',
            (('Green','No'),):'Move Forward',
            (('Yellow','No'),):'Slow Down',
            (('Green','Yes'),):'Brake',

            #Time Step 2: Two percept sequence
            (('Red','No'),
             ('Green','No')):'Move Forward',
            (('Green','No'),
             ('Green','Yes')):'Brake',
            (('Green','Yes'),
             ('Green','No')):'Move Forward',

            #Time Step 3: Three percept sequenece
            (('Red','No'),
             ('Green','No'),
             ('Green','Yes')):'Brake',
            (('Green','No'),
             ('Green','Yes'),
             ('Green','No')):'Move Forward'
            }

    def act(self,current_percept):
        #Store current percept
```

```
        self.percept_history.append(current_percept)
        #Convert history to tuple
        history_key=tuple(self.percept_history)

        action=self.table.get(history_key,'Wait')
        return action

if __name__=="__main__":
    agent=TableDrivenTaxiAgent()
    percept_1=('Red','No')
    action_1=agent.act(percept_1)
    print(f"Percept:{percept_1}->Action:{action_1}")

    #Step 2
    percept_2=('Green','No')
    action_2=agent.act(percept_2)
    print(f"Percept:{percept_2}->Action:{action_2}")

    #Step 3
    percept_3=('Green','Yes')
    action_3=agent.act(percept_3)
    print(f"Percept:{percept_3}->Action:{action_3}")
```

Output:-

```
Percept:('Red', 'No')->Action:Stop
Percept:('Green', 'No')->Action:Move Forward
Percept:('Green', 'Yes')->Action:Brake
```

Prroblem Solving by Searching(Uninformed Search)

Practical No.2(A)

Aim:- Given an initial configuration of the 8-puzzle and a goal configuration. Write a python program to find the shortest sequence of moves to reach the goal state using Breadth-First Search(BFS).

Code:-

```
from collections import deque

MOVES = [(-1, 0), (1, 0), (0, -1), (0, 1)]


def get_neighbors(state):

    neighbors = []
    # Find the index of the empty tile(0)
    idx = state.index(0)
    row, col = idx // 3, idx % 3

    for dr, dc in MOVES:
        new_row, new_col = row + dr, col + dc

        # Check if the move stays within the 3x3 grid
        if 0 <= new_row < 3 and 0 <= new_col < 3:
            new_idx = new_row * 3 + new_col

            # Create a new state by swapping the empty tile with the target tile
            new_state = list(state)
            new_state[idx], new_state[new_idx] = (
                new_state[new_idx],
                new_state[idx],
            )
            neighbors.append(tuple(new_state))

    return neighbors


def solve_8_puzzle(start, goal):

    # Create lists to tuple because tuples are hashable(can be put in sets)
    start_tuple = tuple(start)
```

```
    goal_tuple = tuple(goal)

    if start_tuple == goal_tuple:
        return [start_tuple]

    # Queue stores tuples of (current_state,path_taken_to_reach_it)
    queue = deque([(start_tuple, [start_tuple])])
    visited = {start_tuple}

    while queue:
        current_state, path = queue.popleft()

        # Check all possible valid moves from the current state
        for neighbor in get_neighbors(current_state):
            if neighbor == goal_tuple:
                return path + [neighbor]

            if neighbor not in visited:
                visited.add(neighbor)
                queue.append((neighbor, path + [neighbor]))

    return None  # Return None if the goal state is unreachable


def print_board(state):

    for i in range(0, 9, 3):
        print(list(state[i:i + 3]))
    print()


# 0 represents the empty space
initial_state = [1, 2, 3, 4, 0, 5, 7, 8, 6]

goal_state = [1, 2, 3, 4, 5, 6, 7, 8, 0]

print("Solving the puzzle...\n")
solution_path = solve_8_puzzle(initial_state, goal_state)

if solution_path:
    print(f"Success! Shortest path found in {len(solution_path) - 1} moves.\n")
    for step, state in enumerate(solution_path):
        print(f"Step {step}:")
        print_board(state)
```

```
else:
    print("No solution exists for this configuration.")
```

Output:-

```
Solving the puzzle...

Success! Shortest path found in 2 moves.

Step 0:
[1, 2, 3]
[4, 0, 5]
[7, 8, 6]

Step 1:
[1, 2, 3]
[4, 5, 0]
[7, 8, 6]

Step 2:
[1, 2, 3]
[4, 5, 6]
[7, 8, 0]
```

Practical No.2(B)

Aim:- Given two water jugs of 4 litres and 3 litres capacity, write a Python program to obtain exactly 2 litres in one jug using Depth-First Search (DFS). [Vary capacity of jugs]

Code:-

```
J1_CAP = 4
J2_CAP = 3
TARGET = 2

def get_next_states(state):
    j1, j2 = state
    moves = []

    #1.Fill a jug completely
    moves.append((J1_CAP, j2))   #Fill Jug1
    moves.append((j1, J2_CAP))   #Fill Jug2

    #2.Empty a jug completely
    moves.append((0, j2))        #Empty Jug1
    moves.append((j1, 0))        #Empty Jug2

    #3.Pour from Jug1 to Jug2
    pour_to_j2 = min(j1, J2_CAP - j2)
    moves.append((j1 - pour_to_j2, j2 + pour_to_j2))

    #4.Pour from Jug2 to Jug1
    pour_to_j1 = min(j2, J1_CAP - j1)
    moves.append((j1 + pour_to_j1, j2 - pour_to_j1))

    return list(set(moves))

def solve_water_jug_dfs():
    stack = [((0, 0), [(0, 0)])]
    visited = {(0, 0)}

    while stack:
        (j1, j2), path = stack.pop()   #Pop the latest element (LIFO behaviour)

        #Check if either jug has reached the target quantity
        if j1 == TARGET or j2 == TARGET:
            return path
```

```
        for next_state in get_next_states((j1, j2)):
            if next_state not in visited:
                visited.add(next_state)
                stack.append((next_state, path + [next_state]))

    return None

solution = solve_water_jug_dfs()

if solution:
    print(f"Solution found using DFS ({len(solution)-1} steps):")
    for step, (j1, j2) in enumerate(solution):
        print(f"Step {step}: Jug1={j1}L, Jug2={j2}L")
else:
    print(f"It is impossible to get exactly {TARGET}L with these jugs.")
```

Output:-

```
Solution found using DFS (4 steps):
Step 0: Jug1=0L, Jug2=0L
Step 1: Jug1=0L, Jug2=3L
Step 2: Jug1=3L, Jug2=0L
Step 3: Jug1=3L, Jug2=3L
Step 4: Jug1=4L, Jug2=2L
```